Oneflare Grand Plaza

QA Automation Test Manual

Playwright (Python) + pytest regression suite and CI/CD pipeline
Line-by-line explanation of the syntax used in every file of the repository, and why

Application under test	https://hotel-plaza-preview.vercel.app
Repository	lordfinesse808/Hotel-Oneflare
Source revision	07b0646
Generated	11 Oct 2026
Source of test cases	Oneflare_Bug_Reports_and_Test_Cases_WebSite_v3.xlsx (123 TCs, 51 bugs)

Contents

Contents 2

Part 1 - Syntax primer 5

How to read Part 3 . . . 5

Python essentials used in this repository . . . 5

pytest essentials . . . 5

Playwright essentials . . . 6

Design principles . . . 6

Part 2 - Architecture and execution flow 7

Repository layout . . . 7

CI/CD gates . . . 9

Part 3 - Line-by-line reference 11

README.md . . . 11

requirements.txt . . . 13

pytest.ini . . . 14

ruff.toml . . . 15

.gitignore . . . 16

.env.example . . . 16

config/__init__.py . . . 17

config/settings.py . . . 17

conftest.py . . . 19

utils/__init__.py . . . 24

utils/data_factory.py . . . 24

utils/site_data.py . . . 26

utils/traceability.py . . . 29

pages/__init__.py . . . 33

pages/base_page.py . . . 33

pages/search_widget.py . . . 38
pages/home_page.py . . . 40
pages/rooms_page.py . . . 42
pages/room_detail_page.py . . . 48
pages/booking_page.py . . . 53
pages/auth_modal.py . . . 57
pages/account_page.py . . . 64
pages/manage_booking_page.py . . . 66
pages/branches_page.py . . . 68
pages/gallery_page.py . . . 69
pages/about_page.py . . . 71
pages/contact_page.py . . . 72
tests/__init__.py . . . 74
tests/functional/__init__.py . . . 74
tests/functional/test_navigation.py . . . 74
tests/functional/test_home.py . . . 87
tests/functional/test_rooms.py . . . 92
tests/functional/test_room_details.py . . . 113
tests/functional/test_booking_flow.py . . . 124
tests/functional/test_auth.py . . . 140
tests/functional/test_account.py . . . 152
tests/functional/test_branches.py . . . 157
tests/functional/test_gallery.py . . . 162
tests/functional/test_about_contact.py . . . 167
tests/non_functional/__init__.py . . . 171
tests/non_functional/test_performance.py . . . 172
tests/non_functional/test_accessibility.py . . . 179
tests/non_functional/test_responsive.py . . . 186

tests/non_functional/test_security.py . . . 190
tests/non_functional/test_seo_content.py . . . 197
ci/quality_gate.py . . . 199
ci/check_traceability.py . . . 202
ci/discover.py . . . 205
.github/workflows/ci.yml . . . 213
.github/workflows/discover.yml . . . 221
docs/test_cases.csv . . . 222
Appendix A - Marker reference **223**
Appendix B - Workbook test case to test mapping **224**

Part 1 - Syntax primer

This manual explains every line of code in the repository. Part 1 introduces the building blocks once; Part 2 shows how the pieces fit; Part 3 walks through each file line by line; the appendices list markers and the mapping from workbook test cases to automated tests.

How to read Part 3

- Each file starts with an overview (purpose and design reasons), followed by a table with one row per non-blank line: line number, the code, and an explanation of the syntax and why it is written that way.
- Blank lines are omitted: they carry no code and only follow PEP 8 spacing (two blank lines between top-level definitions, one between methods).
- Repeated idioms (e.g. expect(...).to_be_visible()) are explained each time they appear, so any page can be read on its own.

Python essentials used in this repository

- **import / from ... import ...** - loads code from another module. Imports are grouped standard library, third-party, then project (enforced by ruff rule I).
- **def name(params):** - defines a function; the indented block is its body. Indentation (4 spaces) is syntax in Python, not style.
- **class Child(Parent):** - defines a class that inherits everything from Parent. **__init__** is the constructor and **self** is the instance.
- **super().__init__(page)** - runs the parent constructor first so inherited attributes (header, footer...) exist.
- **Type hints** (page: Page, -> int, str | None) document expected types; Python does not enforce them at runtime but editors and readers benefit.
- **f-strings** f"Total {amount}" insert values into text. **Raw strings** r"\d+" keep backslashes literally - used for regular expressions.
- **Regular expressions** (module re): \d digit, + one or more, * zero or more, ? optional, (...) capture group, ^/$ start/end, | alternative, re.I ignore case. Used wherever page text contains dynamic numbers (prices, counts, dates).
- **Comprehensions** [x for x in items if cond] build lists/sets/dicts in one expression.
- **lambda** args: expression - a tiny inline function, used as predicates (wait_for_count(lambda c: c > 0)) and event callbacks.
- **with** - context manager: guarantees cleanup (closing files, pools). **try/except** - handles an expected error instead of crashing.
- **@decorator** - wraps the next function/class: @pytest.fixture, @pytest.mark.*, @dataclass, @property, @classmethod.
- **yield** in a fixture - code before yield is setup, code after is teardown; one function controls the whole lifecycle.
- **assert condition, message** - fails the test when false; pytest shows the compared values.

pytest essentials

- **Test discovery**: files test_*.py, functions test_*. testpaths in pytest.ini limits the search to tests/.
- **Fixtures**: functions decorated with @pytest.fixture; a test asks for one by naming it as a parameter (dependency injection). Scope 'function' = fresh per test (isolation); 'session' = once per run (expensive setup such as sign-in).
- **conftest.py**: fixtures and hooks visible to every test below its folder, no import needed.

- **Markers**: @pytest.mark.name labels tests; select with -m (pytest -m smoke). Custom markers are registered in pytest.ini; --strict-markers turns typos into errors.
- **parametrize**: one test function, many cases - each value is reported separately with its own id.
- **skip / skipif / xfail**: skip = not run (with reason); xfail = run, but a failure is expected (known bug). strict=False: an unexpected pass is reported as XPASS instead of failing.
- **Hooks**: functions named pytest_* (pytest_collection_modifyitems, pytest_addoption, pytest_sessionfinish...) that pytest calls at fixed points; used to auto-tag tests and write reports.
- **Plugins used**: pytest-playwright (browser fixtures, --browser, traces), pytest-xdist (-n parallel workers), pytest-rerunfailures (--reruns), pytest-html (HTML report).

Playwright essentials

- **Browser -> Context -> Page**: a context is an isolated profile (cookies, storage); a page is a tab. Each test gets a new context, so tests never share state.
- **Locators** describe how to find elements and are resolved lazily on every action, so they never go stale. Priority used here: get_by_role (role + accessible name, like a screen reader) > get_by_text / get_by_label > CSS/XPath only when nothing semantic exists.
- **Auto-waiting**: click/fill wait until the element is attached, visible, stable, enabled and not covered. No manual sleeps are needed for actions.
- **Web-first assertions**: expect(locator).to_be_visible() / to_have_text() / to_have_url() retry until the condition holds or 5 s pass. Plain assert is used only for values already computed in Python.
- **Scoping**: main.get_by_role(...) searches inside only; filter(has=..., has_text=...) narrows to one card. Inside has=, use page-level locators: they are evaluated relative to each candidate element.
- **evaluate()** runs JavaScript in the page for measurements the API does not offer (performance timing, computed style, geometry).
- **page.request / APIRequestContext**: HTTP calls without rendering - fast checks of status codes, headers and API payloads, and live test data.
- **Tracing**: on failure a trace.zip (DOM snapshots, network, console, timeline) is kept; open with 'playwright show-trace'.

Design principles

- **Page Object Model**: locators and actions live in pages/; tests contain only steps and assertions. A UI change is fixed once in the page object.
- **Data from the system under test**: branch and room ids come from the public API at run time, never hard-coded.
- **Traceability**: every test carries @pytest.mark.tc with workbook ids; CI fails if a test is untagged.
- **Known defects are tracked, not hidden**: @pytest.mark.defect makes a test xfail while the bug is open and XPASS when it is fixed.
- **Safe by default**: tests that create live records or generate load are opt-in via flags.
- **Determinism**: timezone Africa/Lagos and locale en-NG are fixed so 'today' and formatting are stable; one automatic rerun maximum for network noise.

Part 2 - Architecture and execution flow

1. Configure	config/settings.py reads .env (local) or CI secrets/variables. pytest.ini fixes reports, traces and markers.
2. Collect	pytest finds tests/**/test_*.py. conftest.py auto-marks folders; utils/traceability.py turns defect/live_write/needs_account markers into xfail/skip.
3. Prepare data	The session fixture site calls /api/branches and /api/rooms once, so tests use live ids.
4. Drive the UI	Each test gets an isolated browser context; page objects (pages/) open pages and expose locators/actions.
5. Assert	Web-first expect() assertions retry until the UI settles; plain assert checks computed values.
6. Report	JUnit XML + HTML report + traces for failures + traceability.csv/md in reports/.
7. Gate	ci/quality_gate.py enforces pass-rate / minimum tests / duration; ci.yml chains the gates and the final quality-gate job.

Repository layout

`README.md`	Entry point for humans: setup, how to run each gate, how workbook statuses map to markers, the CI gates and the repository settings (secrets, branch protection) needed to finish the setup. Markdown is documentation, not code, so it is summarised here rather than explained line by line.
`requirements.txt`	Pins the Python packages the suite needs. CI and every developer install exactly this list (pip install -r requirements.txt), so runs are reproducible. '>=' sets a minimum version with known-good features.
`pytest.ini`	pytest's configuration. Centralising options here means every run - local or CI - produces the same reports, traces and marker checks without long command lines.
`ruff.toml`	Configuration of ruff, the linter/formatter used as CI gate 1. Consistent style and early detection of errors (unused imports, undefined names, risky patterns) keep the suite maintainable.
`.gitignore`	Files git must never commit: generated reports, caches, virtual environments and .env (which holds credentials).
`.env.example`	Template for the local .env file. It documents every supported variable with safe placeholder values; real secrets live only in the developer's .env (gitignored) or in GitHub Actions secrets.
`config/__init__.py`	Empty file that makes config a Python package so 'from config.settings import ...' works.
`config/settings.py`	Single source of configuration. Every value reads an environment variable with a default, so the same code runs locally (.env), in CI (secrets/variables) and against other environments without edits. Feature flags (ALLOW_LIVE_WRITES, ALLOW_LOAD_TESTS) keep tests that write real data or create load OFF by default.
`conftest.py`	pytest loads conftest.py automatically. It holds shared fixtures (browser context defaults, live API data, page objects, mobile context, signed-in session, runtime monitor), auto-applies folder markers and registers the traceability plugin. Fixtures give each test a clean, isolated starting point with no copy-pasted setup.
`utils/__init__.py`	Makes utils a package.
`utils/data_factory.py`	Test-data builders. Generated data (unique emails via uuid) guarantees tests never collide with each other or with earlier runs; frozen dataclasses make shared data immutable.

utils/site_data.py	Reads branches and rooms from the site's public API at session start. Database ids differ per environment and change over time; reading them live removes hard-coded ids, the most common cause of brittle hotel-booking tests.
utils/traceability.py	A small pytest plugin that turns the QA workbook into executable policy: tc() links tests to TC ids, defect() converts known open bugs into non-strict xfail (tracked, not hidden; XPASS = retest), live_write/needs_account gate tests on flags/secrets, and a CSV/Markdown traceability report is written after every run. Works with parallel workers (xdist) because ids travel in report.user_properties.
pages/__init__.py	Makes pages a package.
pages/base_page.py	Base class of the Page Object Model. Header, footer, mobile menu and navigation helpers are defined once and inherited by every page, so a header change is fixed in one place. open() waits for network idle because Next.js ignores clicks before hydration.
pages/search_widget.py	Component object for the availability search bar that appears on both Home and Rooms. Modelling a reusable component once (composition) avoids duplicating the same locators in two page objects.
pages/home_page.py	Page Object for the Home page: hero, search widget, locations, 'Free for your dates', gallery and amenities sections.
pages/rooms_page.py	Page Object for the Rooms list: search bar, filter panel (room type, bed, amenities, price slider), result heading and cards, mobile filter drawer. wait_for_count() polls because results re-render asynchronously after each filter change.
pages/room_detail_page.py	Page Object for a room detail page: facts, steppers, dates, price breakdown and the not-found state. naira() converts '₦56,250' to an int so totals can be compared numerically.
pages/booking_page.py	Page Object for the booking wizard (guest details -> review -> confirmed), including every validation message, so tests read like the workbook steps.
pages/auth_modal.py	Page Object for the sign-in / create-account dialog (role choice, sign-in form, forgot password, registration). open_sign_in() retries because a click before hydration is ignored.
pages/account_page.py	Page Object for /account (signed-out prompt and signed-in view).
pages/manage_booking_page.py	Page Object for /manage-booking: lookup by booking reference + email.
pages/branches_page.py	Page Object for /branches; card() scopes searches to a single branch card.
pages/gallery_page.py	Page Object for /gallery: category buttons, location filter and photos.
pages/about_page.py	Page Object for /about.
pages/contact_page.py	Page Object for /contact: branch tabs, info panel, map and message form.
tests/__init__.py	Makes tests a package (lets pytest use unique module names).
tests/functional/__init__.py	Package marker.
tests/functional/test_navigation.py	Global chrome: header links, footer links (HTTP status), footer content, mobile menu, header search, policy pages, branded 404 and SEO titles/descriptions. Workbook: TC-010, 016, 017, 019, 043, 044, 060, 061, 069, 083, 086, 087.
tests/functional/test_home.py	Home page sections and the home search widget. Workbook: TC-005, 014, 016, 082-085.
tests/functional/test_rooms.py	Rooms listing: branch scoping, cards, counts, capacity, prices, date validation, guests, filters, slider, mobile drawer. The largest module because the workbook's highest-risk area (search & filters) lives here.
tests/functional/test_room_details.py	Room detail: content, breadcrumb, other rooms, steppers, date limits, price arithmetic, offers, currency format, not-found state.
tests/functional/test_booking_flow.py	Booking wizard end to end without confirming (no live writes): validation, consent, review content, totals across steps, edit links, browser back, PII in URLs. Confirm booking runs only with ALLOW_LIVE_WRITES.

tests/functional/test_auth.py	Sign-in dialog, role choice, validation, invalid credentials, brute-force throttling, registration validation, forgot password.
tests/functional/test_account.py	Account page signed-out/signed-in, manage-a-booking lookup, sign out. Signed-in tests use the TEST_EMAIL/TEST_PASSWORD secrets.
tests/functional/test_branches.py	Branch cards, deep links to Rooms and Contact, branch naming/address consistency across pages.
tests/functional/test_gallery.py	Gallery category and location filters, lightbox, content accuracy.
tests/functional/test_about_contact.py	About page content; Contact info panel per branch, form validation and routing (live submit gated).
tests/non_functional/__init__.py	Package marker.
tests/non_functional/test_performance.py	Performance budgets from the browser's own timing APIs: TTFB/DCL/load, LCP, CLS, Slow-4G LCP via CDP throttling, lazy images, skeleton resolution, console/network errors, and an opt-in 50-request concurrency test.
tests/non_functional/test_accessibility.py	Accessibility: axe-core scan (critical/serious = fail), accessible names, keyboard operability, focus visibility, one h1 + landmarks, 44 px tap targets.
tests/non_functional/test_responsive.py	Layout at 375/768/1440/1920 px (no horizontal scroll, header visible, no clipping) and the core flow on emulated phones with touch.
tests/non_functional/test_security.py	HTTPS redirect, HSTS, security headers, mixed content, anonymous access to staff paths, API error hygiene and server-side validation of crafted booking quotes.
tests/non_functional/test_seo_content.py	robots.txt, sitemap.xml, Open Graph tags and a copy review for placeholders/typos.
ci/quality_gate.py	Reads JUnit XML and fails the CI job when the pass rate, minimum test count or duration thresholds are broken. xfail (known defects) are reported separately so they stay visible. Writes a summary to the GitHub run page.
ci/check_traceability.py	Static gate (no browser): every collected test must carry a tc marker and every TC id must exist in docs/test_cases.csv. Prints workbook coverage.
ci/discover.py	Maintenance tool: visits the live site and prints ARIA snapshots of key UI states. Selectors in the page objects were derived from this output; re-run it after UI changes.
.github/workflows/ci.yml	The end-to-end CI/CD pipeline: guard -> lint (gate 1) -> smoke (gate 2) -> regression on 3 browsers (gate 3) + non-functional (gate 4) -> consolidated report -> quality-gate verdict (the one required status check). Triggered by PRs, pushes, nightly schedule, manual dispatch and every successful deployment.
.github/workflows/discover.yml	Manual/auto workflow that runs ci/discover.py on a GitHub runner (which can reach the site) and prints the snapshots in the job log.
docs/test_cases.csv	Export of the workbook's Test Cases sheet (id, module, title, type, priority, linked bug, expected result, status). It is the source of truth for the traceability check.

CI/CD gates

Gate	Job	Rule
1 Static	lint	ruff lint + format; collection succeeds; every test linked to a valid TC
2 Smoke	smoke	100% pass, at least 8 tests (chromium)
3 Regression	regression x chromium/firefox/webkit	>= 95% pass, >= 60 tests, <= 25 min
4 Non-functional	non-functional	100% of enforced checks (perf, a11y, layout, security, SEO)
Report	report	merges artifacts, publishes workbook traceability

Verdict	quality-gate	single required status check for branch protection

Part 3 - Line-by-line reference

README.md

Entry point for humans: setup, how to run each gate, how workbook statuses map to markers, the CI gates and the repository settings (secrets, branch protection) needed to finish the setup. Markdown is documentation, not code, so it is summarised here rather than explained line by line.

Line	Code	Syntax & why
1	`# Oneflare Grand Plaza - Playwright (Python) QA Suite`	**Markdown heading** (# level 1, ## level 2, ### level 3).
3	`Automated regression suite for <https://hotel-plaza-preview.vercel.app>, built from the QA workbook`	Documentation text.
4	`*Oneflare_Bug_Reports_and_Test_Cases_WebSite_v3.xlsx* (123 test cases, 51 bugs).`	Documentation text.
5	`Every workbook test case is linked to at least one automated test (`@pytest.mark.tc("TC-xxx")`).`	Documentation text.
7	`## Setup`	**Markdown heading** (# level 1, ## level 2, ### level 3).
8	`    python -m venv .venv && source .venv/bin/activate`	**Indented block** = rendered as code/command example.
9	`    pip install -r requirements.txt`	**Indented block** = rendered as code/command example.
10	`    playwright install --with-deps chromium # add firefox webkit for cross-browser runs`	**Indented block** = rendered as code/command example.
11	`    cp .env.example .env # then fill TEST_EMAIL / TEST_PASSWORD`	**Indented block** = rendered as code/command example.
13	`## Run`	**Markdown heading** (# level 1, ## level 2, ### level 3).
14	`    pytest -m smoke # critical path (gate 2)`	**Indented block** = rendered as code/command example.
15	`    pytest tests/functional -n 4 # functional regression (gate 3)`	**Indented block** = rendered as code/command example.
16	`    pytest tests/non_functional # perf, a11y, responsive, security, SEO (gate 4)`	**Indented block** = rendered as code/command example.
17	`    pytest --browser firefox --headed # other browser, visible`	**Indented block** = rendered as code/command example.
18	`    pytest --base-url https://other-env # other environment`	**Indented block** = rendered as code/command example.
19	`    pytest --known-defects=fail # retest run: open bugs fail instead of xfail`	**Indented block** = rendered as code/command example.
20	`    ALLOW_LIVE_WRITES=true pytest -m live_write # creates real accounts/bookings/messages`	**Indented block** = rendered as code/command example.
21	`    ALLOW_LOAD_TESTS=true pytest -k concurrent # 50-request load test (TC-054)`	**Indented block** = rendered as code/command example.
22	`    python ci/check_traceability.py # workbook coverage, no browser needed`	**Indented block** = rendered as code/command example.
23	`    python ci/discover.py # dump ARIA snapshots to re-derive selectors`	**Indented block** = rendered as code/command example.
25	`Reports land in `reports/`: `report.html`, `junit.xml`, `traceability.csv|md`, and traces/videos/screenshots`	Documentation text.
26	`of failures in `reports/artifacts/` (open a trace with `playwright show-trace <trace.zip>`).`	Documentation text.

Line	Code	Syntax & why
28	`## Structure`	**Markdown heading** (# level 1, ## level 2, ### level 3).
29	`    pages/ Page Objects (locators + actions, no business assertions)`	**Indented block** = rendered as code/command example.
30	`      base_page.py header, footer, mobile menu, navigation helpers (inherited by all pages)`	**Indented block** = rendered as code/command example.
31	`      search_widget.py Branch / dates / guests search bar shared by Home and Rooms`	**Indented block** = rendered as code/command example.
32	`      home, rooms, room_detail, booking, auth_modal, account, manage_booking,`	**Indented block** = rendered as code/command example.
33	`      branches, gallery, about, contact`	**Indented block** = rendered as code/command example.
34	`    tests/functional/ user-facing behaviour (auto-marked `functional`)`	**Indented block** = rendered as code/command example.
35	`    tests/non_functional/ performance, accessibility, responsive, security, SEO (auto-marked)`	**Indented block** = rendered as code/command example.
36	`    conftest.py fixtures: live API data, page objects, mobile context, signed-in session`	**Indented block** = rendered as code/command example.
37	`    config/settings.py env-overridable settings, flags and budgets (.env locally, secrets in CI)`	**Indented block** = rendered as code/command example.
38	`    utils/site_data.py branches/rooms read from the public API - no hard-coded DB ids`	**Indented block** = rendered as code/command example.
39	`    utils/traceability.py pytest plugin: tc/defect/live_write/needs_account markers + traceability report`	**Indented block** = rendered as code/command example.
40	`    ci/quality_gate.py pass-rate / min-tests / duration gate over junit.xml (+ job summary)`	**Indented block** = rendered as code/command example.
41	`    ci/check_traceability.py every test tagged, every TC id valid, coverage of the workbook`	**Indented block** = rendered as code/command example.
42	`    ci/discover.py live-site ARIA discovery used to build and maintain selectors`	**Indented block** = rendered as code/command example.
43	`    docs/test_cases.csv workbook test cases (source of truth for traceability)`	**Indented block** = rendered as code/command example.
44	`    docs/Test_Manual.pdf line-by-line manual: syntax of every line in the repository and why`	**Indented block** = rendered as code/command example.
45	`    .github/workflows/ ci.yml (pipeline), discover.yml (selector maintenance)`	**Indented block** = rendered as code/command example.
47	`## How workbook status maps to the suite`	**Markdown heading** (# level 1, ## level 2, ### level 3).
48	`| Workbook | Suite |`	**Markdown table row** (| cell | cell |).
49	`|---|---|`	**Markdown table row** (| cell | cell |).
50	`| Pass | normal test, must pass |`	**Markdown table row** (| cell | cell |).
51	`| Fail (open bug) | `@pytest.mark.defect("BUG-xxx")` -> non-strict **xfail**; an **XPASS** means "looks fixed - retest" |`	**Markdown table row** (| cell | cell |).
52	`| Blocked (live data) | `@pytest.mark.live_write` -> skipped unless `ALLOW_LIVE_WRITES=true` |`	**Markdown table row** (| cell | cell |).

Line	Code	Syntax & why
53	`| Needs an account | `@pytest.mark.needs_account` -> uses `TEST_EMAIL` / `TEST_PASSWORD` |`	**Markdown table row** (| cell | cell |).
54	`| Skipped / N/A | explicit `pytest.skip` with the workbook reason, so it stays visible in reports |`	**Markdown table row** (| cell | cell |).
56	`## CI/CD pipeline (`.github/workflows/ci.yml`)`	**Markdown heading** (# level 1, ## level 2, ### level 3).
57	`Triggers: pull requests, pushes to `main`/`cl/**`/`feature/**`/`fix/**`, nightly 05:00 UTC, manual dispatch`	Documentation text.
58	`(choose environment, `known_defects`, `allow_live_writes`), and every successful **deployment_status**`	Documentation text.
59	`(Vercel preview builds are tested at their own URL).`	Documentation text.
61	`| Gate | Job | Rule |`	**Markdown table row** (| cell | cell |).
62	`|---|---|---|`	**Markdown table row** (| cell | cell |).
63	`| 1 Static | `lint` | ruff lint + format clean; collection succeeds; every test linked to a valid TC |`	**Markdown table row** (| cell | cell |).
64	`| 2 Smoke | `smoke` | 100% pass, >= 8 tests (chromium) |`	**Markdown table row** (| cell | cell |).
65	`| 3 Regression | `regression` x chromium/firefox/webkit | >= 95% pass, >= 60 tests, <= 25 min each |`	**Markdown table row** (| cell | cell |).
66	`| 4 Non-functional | `non-functional` | 100% of enforced checks (perf budgets, axe critical/serious = 0, layout, headers) |`	**Markdown table row** (| cell | cell |).
67	`| Report | `report` | merges all reports, publishes workbook traceability to the run summary |`	**Markdown table row** (| cell | cell |).
68	`| Verdict | `quality-gate` | the single required status check for branch protection |`	**Markdown table row** (| cell | cell |).
70	`Hygiene: one automatic rerun maximum (`--reruns 1`); xfail = known defect, tracked not hidden.`	Documentation text.
72	`### Repository settings to finish the setup`	**Markdown heading** (# level 1, ## level 2, ### level 3).
73	`1. **Secrets** (Settings > Secrets and variables > Actions): `TEST_EMAIL`, `TEST_PASSWORD` for the customer`	**Numbered list** item.
74	`   test account. Without them the signed-in tests are reported as skipped.`	Documentation text.
75	`2. **Variables** (optional): `BASE_URL`, `PERF_TTFB_MS`, `PERF_DCL_MS`, `PERF_LOAD_MS`, `PERF_LCP_MS`, `TEST_PHONE`.`	**Numbered list** item.
76	`3. **Branch protection** on `main`: require the `quality-gate` check.`	**Numbered list** item.
77	`4. **Vercel**: keep the GitHub integration on so deployments post `deployment_status` events.`	**Numbered list** item.

requirements.txt

Pins the Python packages the suite needs. CI and every developer install exactly this list (pip install -r requirements.txt), so runs are reproducible. '>=' sets a minimum version with known-good features.

Line	Code	Syntax & why
1	`pytest>=8.0`	Package **pytest** with a minimum version (>=). Why: the test runner itself. The text after # is a comment.
2	`pytest-playwright>=0.5 # gives us the `page`, `browser`, `context` fixtures + --browser flag`	Package **pytest-playwright** with a minimum version (>=). Why: provides page/browser/context fixtures, --browser and tracing options. The text after # is a comment.
3	`pytest-html>=4.1 # human-readable HTML report`	Package **pytest-html** with a minimum version (>=). Why: generates reports/report.html for humans. The text after # is a comment.
4	`pytest-xdist>=3.5 # parallel execution (-n)`	Package **pytest-xdist** with a minimum version (>=). Why: parallel workers (-n) to cut run time. The text after # is a comment.
5	`pytest-rerunfailures>=14.0 # one controlled retry for network flakiness`	Package **pytest-rerunfailures** with a minimum version (>=). Why: --reruns: a single controlled retry for network noise. The text after # is a comment.
6	`axe-playwright-python>=0.1.4 # axe-core accessibility engine`	Package **axe-playwright-python** with a minimum version (>=). Why: axe-core accessibility engine for Playwright. The text after # is a comment.
7	`ruff>=0.5 # linter / formatter used as a CI gate`	Package **ruff** with a minimum version (>=). Why: linter + formatter used by the lint gate. The text after # is a comment.
8	`python-dotenv>=1.0 # loads .env for local runs`	Package **python-dotenv** with a minimum version (>=). Why: loads .env for local runs. The text after # is a comment.

pytest.ini

pytest's configuration. Centralising options here means every run - local or CI - produces the same reports, traces and marker checks without long command lines.

Line	Code	Syntax & why
1	`[pytest]`	**[pytest]** section header: pytest reads its settings from here.
2	`# Where pytest looks for tests`	**Comment** in the INI file.
3	`testpaths = tests`	**testpaths** = tests: folders pytest searches for tests.
4	`# Base URL: pages call page.goto("/rooms") and Playwright prefixes this value.`	**Comment** in the INI file.
5	`# CI can override it with: --base-url https://other-env.example.com`	**Comment** in the INI file.
6	`base_url = https://hotel-plaza-preview.vercel.app`	**base_url** = https://hotel-plaza-preview.vercel.app: default site under test; Playwright joins relative page.goto('/rooms') paths to it. Overridden by --base-url.
7	`addopts =`	**addopts** = : options added to every pytest command line.
8	`    --strict-markers`	**--strict-markers**: unknown markers become errors.
9	`    -ra`	**-ra**: summary lists the reason of every non-passing test.
10	`    --tracing=retain-on-failure`	**--tracing=retain-on-failure**: Playwright trace (DOM snapshots, network, console, timeline) saved only for failures - the fastest way to debug CI failures.
11	`    --screenshot=only-on-failure`	**--screenshot=only-on-failure**: screenshot saved for failed tests.
12	`    --video=retain-on-failure`	**--video=retain-on-failure**: video kept for failed tests.
13	`    --output=reports/artifacts`	**--output=reports/artifacts**: folder for traces/screenshots/videos.

Line	Code	Syntax & why
14	`    --html=reports/report.html`	**--html=reports/report.html**: human-readable HTML report (pytest-html).
15	`    --self-contained-html`	**--self-contained-html**: CSS/JS embedded so the report is one portable file.
16	`    --junitxml=reports/junit.xml`	**--junitxml=reports/junit.xml**: machine-readable results used by the quality gate.
17	`markers =`	**markers** = : registers custom markers; with --strict-markers a typo in a marker name is an error, not a silent no-op.
18	`    smoke: critical-path checks; 100% must pass before merge`	Marker **smoke**: critical-path checks; 100% must pass before merge. critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
19	`    functional: user-facing behaviour tests (auto-applied to tests/functional)`	Marker **functional**: user-facing behaviour tests (auto-applied to tests/functional). auto-applied to tests/functional.
20	`    non_functional: performance, accessibility, responsive, security (auto-applied)`	Marker **non_functional**: performance, accessibility, responsive, security (auto-applied). auto-applied to tests/non_functional.
21	`    perf: performance budget tests`	Marker **perf**: performance budget tests. performance-budget test (filter with -m perf).
22	`    a11y: accessibility tests`	Marker **a11y**: accessibility tests. accessibility test (filter with -m a11y).
23	`    security: transport/header checks`	Marker **security**: transport/header checks. security test (filter with -m security).
24	`    responsive: viewport/layout tests`	Marker **responsive**: viewport/layout tests. layout/viewport test.
25	`    tc(*ids): link a test to test-case IDs in the QA workbook (TC-xxx)`	Marker **tc**: link a test to test-case IDs in the QA workbook (TC-xxx). links the test to workbook test case(s) - drives traceability reports and the coverage check in CI.
26	`    defect(*ids): expected behaviour is broken by these open bugs (BUG-xxx) - runs as non-strict xfail`	Marker **defect**: expected behaviour is broken by these open bugs (BUG-xxx) - runs as non-strict xfail. the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'.
27	`    live_write: creates real records on the shared site - skipped unless ALLOW_LIVE_WRITES=true`	Marker **live_write**: creates real records on the shared site - skipped unless ALLOW_LIVE_WRITES=true. creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
28	`    needs_account: needs TEST_EMAIL / TEST_PASSWORD of an existing customer account`	Marker **needs_account**: needs TEST_EMAIL / TEST_PASSWORD of an existing customer account. needs TEST_EMAIL/TEST_PASSWORD of an existing customer; skipped when the secrets are absent.
29	`    api: HTTP-level checks without a browser page`	Marker **api**: HTTP-level checks without a browser page. HTTP-level check without the browser UI.
30	`    seo: titles, meta, robots, sitemap`	Marker **seo**: titles, meta, robots, sitemap. search-engine/metadata check.
31	`    mobile: runs in a 375px mobile viewport`	Marker **mobile**: runs in a 375px mobile viewport. runs in a mobile viewport.

ruff.toml

Configuration of ruff, the linter/formatter used as CI gate 1. Consistent style and early detection of errors (unused imports, undefined names, risky patterns) keep the suite maintainable.

Line	Code	Syntax & why
1	`line-length = 110`	**line-length**: maximum line length the formatter wraps at. (TOML key = value syntax)
2	`target-version = "py310"`	**target-version**: oldest Python version the code must support (enables matching syntax checks). (TOML key = value syntax)

Line	Code	Syntax & why
3	`[lint]`	**[lint]**: section for lint rules. (TOML key = value syntax)
4	`select = ["E", "F", "I", "B"]`	**select**: rule families: E/F pycodestyle+pyflakes errors, I import sorting, B bugbear (likely bugs). (TOML key = value syntax)
5	`ignore = ["E501", "B018"] # long lines allowed; B018 hits the check/uncheck ternary`	**ignore**: rules switched off: E501 long lines (formatter handles them), B018 useless expression. (TOML key = value syntax)

.gitignore

Files git must never commit: generated reports, caches, virtual environments and .env (which holds credentials).

Line	Code	Syntax & why
1	`reports/`	Pattern **reports/**: git ignores generated reports and failure artifacts (rebuilt every run).
2	`__pycache__/`	Pattern **__pycache__/**: git ignores Python bytecode caches.
3	`.pytest_cache/`	Pattern **.pytest_cache/**: git ignores pytest's cache.
4	`.venv/`	Pattern **.venv/**: git ignores local virtual environment.
5	`.env`	Pattern **.env**: git ignores local secrets - must never be committed.

.env.example

Template for the local .env file. It documents every supported variable with safe placeholder values; real secrets live only in the developer's .env (gitignored) or in GitHub Actions secrets.

Line	Code	Syntax & why
1	`# Copy to .env (never commit .env). Loaded automatically by config/settings.py.`	**Comment** explaining the variables below.
2	`# In CI the same names come from GitHub Actions secrets (TEST_EMAIL, TEST_PASSWORD) and variables.`	**Comment** explaining the variables below.
3	`BASE_URL=https://hotel-plaza-preview.vercel.app`	**BASE_URL**=value: KEY=VALUE pair read by config/settings.py via os.getenv. environment under test.
5	`# Existing customer account used by sign-in / account tests (TC-040, TC-105, TC-107).`	**Comment** explaining the variables below.
6	`TEST_EMAIL=you@maildrop.cc`	**TEST_EMAIL**=value: KEY=VALUE pair read by config/settings.py via os.getenv. email of the existing customer account used by signed-in tests (CI secret).
7	`TEST_PASSWORD=change-me`	**TEST_PASSWORD**=value: KEY=VALUE pair read by config/settings.py via os.getenv. password of that account (CI secret; never committed).
8	`TEST_PHONE=+2348012345678`	**TEST_PHONE**=value: KEY=VALUE pair read by config/settings.py via os.getenv. phone number used in generated test data.
10	`# true = also run tests that create live records (accounts, bookings, contact messages).`	**Comment** explaining the variables below.
11	`ALLOW_LIVE_WRITES=false`	**ALLOW_LIVE_WRITES**=value: KEY=VALUE pair read by config/settings.py via os.getenv. true = also run tests that create real records (accounts, bookings, messages).

Line	Code	Syntax & why
13	`# Performance budgets (ms)`	**Comment** explaining the variables below.
14	`PERF_TTFB_MS=800`	**PERF_TTFB_MS**=value: KEY=VALUE pair read by config/settings.py via os.getenv. budget: time to first byte (ms).
15	`PERF_DCL_MS=3000`	**PERF_DCL_MS**=value: KEY=VALUE pair read by config/settings.py via os.getenv. budget: DOMContentLoaded (ms).
16	`PERF_LOAD_MS=5000`	**PERF_LOAD_MS**=value: KEY=VALUE pair read by config/settings.py via os.getenv. budget: load event (ms).
17	`PERF_LCP_MS=4000`	**PERF_LCP_MS**=value: KEY=VALUE pair read by config/settings.py via os.getenv. budget: Largest Contentful Paint (ms).

config/__init__.py

Empty file that makes config a Python package so 'from config.settings import ...' works.

Empty file. An empty __init__.py marks the folder as a Python package so its modules can be imported with dotted names.

config/settings.py

Single source of configuration. Every value reads an environment variable with a default, so the same code runs locally (.env), in CI (secrets/variables) and against other environments without edits. Feature flags (ALLOW_LIVE_WRITES, ALLOW_LOAD_TESTS) keep tests that write real data or create load OFF by default.

Line	Code	Syntax & why
1	`"""Central configuration. Every value can be overridden by an environment variable,`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
2	`so CI and local runs use the same code with different inputs.`	**Docstring** continuation (documentation text).
4	`Locally, values are read from a `.env` file in the repo root (gitignored, see `.env.example`).`	**Docstring** continuation (documentation text).
5	`In CI they come from GitHub Actions secrets / variables."""`	**Docstring** continuation (documentation text).
7	`import os`	**import os**: loads the module. Operating-system helpers. Used to read environment variables (os.getenv) so CI and local runs share code but take different inputs.
8	`from pathlib import Path`	**from pathlib import Path**: imports specific names so they can be used without the module prefix. Object-oriented file paths (Path) that work on every OS. *Path*: Filesystem path object.
10	`try: # python-dotenv is optional at runtime; CI injects real env vars`	**try**: runs the block and lets the except clause handle expected errors instead of crashing. Inline comment: “python-dotenv is optional at runtime; CI injects real env vars”.
11	`    from dotenv import load_dotenv`	**from dotenv import load_dotenv**: imports specific names so they can be used without the module prefix. python-dotenv: loads KEY=value pairs from a local .env file into environment variables (local runs only; CI uses secrets). *load_dotenv*: Function that reads .env into os.environ.
13	`    load_dotenv(Path(__file__).resolve().parent.parent / ".env", override=False)`	**Method call** executed for its effect. *load_dotenv()* loads .env into the environment. *resolve()* absolute path. *Path(...)* creates a Path object (constructor call).

Line	Code	Syntax & why
14	`except ImportError: # pragma: no cover`	**except**: handles the listed error type raised in the try block (here: an expected timeout or missing optional dependency). Inline comment: “pragma: no cover”.
15	`    pass`	**pass**: a no-op placeholder (Python needs a statement in every block).
18	`def _flag(name: str, default: str = "false") -> bool:`	**def** defines the private helper *_flag* (leading underscore = internal use). Parameters: *name*; *default*. Parameters with '= value' are optional (default used when omitted). Returns bool.
19	`    return os.getenv(name, default).strip().lower() in ("1", "true", "yes", "on")`	**return**: ends the function and hands back *os.getenv(name, default).strip().lower() in ('1', 'true', 'yes', 'on')*. *lower()* lower-cases a string (case-insensitive comparisons). *strip()* removes surrounding whitespace. *getenv()* reads an environment variable with a default.
22	`BASE_URL = os.getenv("BASE_URL", "https://hotel-plaza-preview.vercel.app").rstrip("/")`	Module-level **constant** *BASE_URL* (UPPER_CASE by convention = not changed at runtime). *getenv()* reads an environment variable with a default.
24	`# Default action timeout (click/fill/etc.) in milliseconds.`	**Comment** (# ... ignored by Python): documents intent for the reader.
25	`DEFAULT_TIMEOUT_MS = int(os.getenv("DEFAULT_TIMEOUT_MS", "10000"))`	Module-level **constant** *DEFAULT_TIMEOUT_MS* (UPPER_CASE by convention = not changed at runtime). *int()* converts to integer. *getenv()* reads an environment variable with a default.
27	`# Existing customer account used for sign-in tests (TC-040, TC-105, TC-107).`	**Comment** (# ... ignored by Python): documents intent for the reader.
28	`# Supplied via the TEST_EMAIL / TEST_PASSWORD secrets - never hard-code real credentials.`	**Comment** (# ... ignored by Python): documents intent for the reader.
29	`TEST_EMAIL = os.getenv("TEST_EMAIL", "")`	Module-level **constant** *TEST_EMAIL* (UPPER_CASE by convention = not changed at runtime). *getenv()* reads an environment variable with a default.
30	`TEST_PASSWORD = os.getenv("TEST_PASSWORD") or os.getenv("PASSWORD", "")`	Module-level **constant** *TEST_PASSWORD* (UPPER_CASE by convention = not changed at runtime). **Boolean expression**: 'or' returns the first truthy value - a compact default. *getenv()* reads an environment variable with a default.
31	`TEST_PHONE = os.getenv("TEST_PHONE", "+2348012345678")`	Module-level **constant** *TEST_PHONE* (UPPER_CASE by convention = not changed at runtime). *getenv()* reads an environment variable with a default.
33	`# Tests that CREATE live records in the shared HMS (accounts, bookings, contact messages)`	**Comment** (# ... ignored by Python): documents intent for the reader.
34	`# are skipped unless explicitly enabled. Mirrors the "Blocked" status in the QA workbook.`	**Comment** (# ... ignored by Python): documents intent for the reader.
35	`ALLOW_LIVE_WRITES = _flag("ALLOW_LIVE_WRITES")`	Module-level **constant** *ALLOW_LIVE_WRITES* (UPPER_CASE by convention = not changed at runtime). *_flag()*: project function _flag() in config/settings.py.
36	`# Backwards-compatible alias used by the original booking test.`	**Comment** (# ... ignored by Python): documents intent for the reader.
37	`ALLOW_BOOKING_SUBMIT = _flag("ALLOW_BOOKING_SUBMIT") or ALLOW_LIVE_WRITES`	Module-level **constant** *ALLOW_BOOKING_SUBMIT* (UPPER_CASE by convention = not changed at runtime). **Boolean expression**: 'or' returns the first truthy value - a compact default. *_flag()*: project function _flag() in config/settings.py.
39	`# Concurrent-load test (TC-054) hits the shared preview with 50 parallel requests - opt-in only.`	**Comment** (# ... ignored by Python): documents intent for the reader.
40	`ALLOW_LOAD_TESTS = _flag("ALLOW_LOAD_TESTS")`	Module-level **constant** *ALLOW_LOAD_TESTS* (UPPER_CASE by convention = not changed at runtime). *_flag()*: project function _flag() in config/settings.py.
42	`# Performance budgets (milliseconds). Tune after you have a baseline.`	**Comment** (# ... ignored by Python): documents intent for the reader.
43	`PERF_TTFB_MS = int(os.getenv("PERF_TTFB_MS", "800"))`	Module-level **constant** *PERF_TTFB_MS* (UPPER_CASE by convention = not changed at runtime). *int()* converts to integer. *getenv()* reads an environment variable with a default.

Line	Code	Syntax & why
44	`PERF_DCL_MS = int(os.getenv("PERF_DCL_MS", "3000"))`	Module-level **constant** *PERF_DCL_MS* (UPPER_CASE by convention = not changed at runtime). *int()* converts to integer. *getenv()* reads an environment variable with a default.
45	`PERF_LOAD_MS = int(os.getenv("PERF_LOAD_MS", "5000"))`	Module-level **constant** *PERF_LOAD_MS* (UPPER_CASE by convention = not changed at runtime). *int()* converts to integer. *getenv()* reads an environment variable with a default.
46	`PERF_LCP_MS = int(os.getenv("PERF_LCP_MS", "4000"))`	Module-level **constant** *PERF_LCP_MS* (UPPER_CASE by convention = not changed at runtime). *int()* converts to integer. *getenv()* reads an environment variable with a default.

conftest.py

pytest loads conftest.py automatically. It holds shared fixtures (browser context defaults, live API data, page objects, mobile context, signed-in session, runtime monitor), auto-applies folder markers and registers the traceability plugin. Fixtures give each test a clean, isolated starting point with no copy-pasted setup.

Line	Code	Syntax & why
1	`"""Shared fixtures. pytest discovers this file automatically - no imports needed in tests."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`from dataclasses import dataclass, field`	**from dataclasses import dataclass, field**: imports specific names so they can be used without the module prefix. dataclass decorator generates __init__/__repr__/__eq__ for simple data-holding classes, removing boilerplate. *dataclass*: Decorator that auto-generates constructor and helpers for data classes. *field*: dataclasses.field configures a dataclass attribute (e.g. default_factory for mutable defaults).
5	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`from config.settings import BASE_URL, DEFAULT_TIMEOUT_MS, TEST_EMAIL, TEST_PASSWORD`	**from config.settings import BASE_URL, DEFAULT_TIMEOUT_MS, TEST_EMAIL, TEST_PASSWORD**: imports specific names so they can be used without the module prefix. Project configuration module: every value can be overridden by an environment variable.
8	`from pages.account_page import AccountPage`	**from pages.account_page import AccountPage**: imports specific names so they can be used without the module prefix. Page Object for /account.
9	`from pages.auth_modal import AuthModal`	**from pages.auth_modal import AuthModal**: imports specific names so they can be used without the module prefix. Page Object for the 'Sign in or create an account' dialog.
10	`from pages.home_page import HomePage`	**from pages.home_page import HomePage**: imports specific names so they can be used without the module prefix. Page Object for the Home page.
11	`from pages.rooms_page import RoomsPage`	**from pages.rooms_page import RoomsPage**: imports specific names so they can be used without the module prefix. Page Object for the Rooms listing.
12	`from utils.site_data import SiteData, load_site_data`	**from utils.site_data import SiteData, load_site_data**: imports specific names so they can be used without the module prefix. Reads branches and rooms from the site's public API so tests use real, current ids.
14	`# Workbook traceability (tc/defect/live_write markers + reports/traceability.*)`	**Comment** (# ... ignored by Python): documents intent for the reader.
15	`pytest_plugins = ["utils.traceability"]`	**Assignment**: binds the name *pytest_plugins* to the value on the right. The value is a **list** literal [...].

Line	Code	Syntax & why
17	`MOBILE = {"viewport": {"width": 375, "height": 812}, "is_mobile": True, "has_touch": True}`	Module-level **constant** *MOBILE* (UPPER_CASE by convention = not changed at runtime). The value is a **dict** literal {key: value}.
20	`# ---------- browser/context defaults ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
21	`@pytest.fixture(scope="session")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'session': created once for the whole run (expensive setup shared).
22	`def browser_context_args(browser_context_args):`	**def** defines the fixture *browser_context_args*. Parameters: *browser_context_args*: default options for every new browser context.
23	`    """Extend (not replace) pytest-playwright's defaults for every browser context."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
24	`    return {`	**return**: ends the function and hands back *{**browser_context_args, 'viewport': {'width': 1440, 'height': 900}, ….*
25	`        **browser_context_args,`	**Continuation** of the statement started on line 24 (inside brackets).
26	`        "viewport": {"width": 1440, "height": 900},`	**Continuation** of the statement started on line 24 (inside brackets).
27	`        "locale": "en-NG",`	**Continuation** of the statement started on line 24 (inside brackets).
28	`        "timezone_id": "Africa/Lagos", # keeps 'today' stable: date pickers default to today in WAT`	**Continuation** of the statement started on line 24 (inside brackets). Inline comment: “keeps 'today' stable: date pickers default to today in WAT”.
29	`    }`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
32	`@pytest.fixture(autouse=True) # autouse = applied to every test without being requested`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation). autouse=True: applied to every test automatically.
33	`def _default_timeout(page):`	**def** defines the fixture *_default_timeout*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
34	`    page.set_default_timeout(DEFAULT_TIMEOUT_MS)`	**Method call** executed for its effect. *set_default_timeout()* sets how long actions (click/fill) wait for elements before failing.
37	`# ---------- tagging ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
38	`def pytest_collection_modifyitems(items):`	**def pytest_collection_modifyitems**: a pytest **hook** - pytest calls it automatically at that stage of the run. Parameters: *items*.
39	`    """Auto-apply markers from folder names so you never forget to tag a test."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
40	`    for item in items:`	**for** loop: repeats the indented block for every item, binding it to *item* (iterating *items*).
41	`        path = str(item.fspath)`	**Assignment**: binds the name *path* to the value on the right. *str()* converts to string.
42	`        if "/tests/functional/" in path:`	**if** statement: the indented block runs only when *'/tests/functional/' in path* is true.
43	`            item.add_marker(pytest.mark.functional)`	**Method call** executed for its effect. *add_marker()* adds a marker to a collected test at runtime (auto-tagging, xfail for defects).
44	`        elif "/tests/non_functional/" in path:`	**if** statement: the indented block runs only when *'/tests/non_functional/' in path* is true.

Line	Code	Syntax & why
45	`            item.add_marker(pytest.mark.non_functional)`	**Method call** executed for its effect. *add_marker()* adds a marker to a collected test at runtime (auto-tagging, xfail for defects).
48	`# ---------- test data from the live API ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
49	`@pytest.fixture(scope="session")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'session': created once for the whole run (expensive setup shared).
50	`def api(playwright, base_url):`	**def** defines the fixture *api*. Parameters: *playwright*: the Playwright driver object (devices, request contexts); *base_url*: site under test (pytest.ini or --base-url).
51	`    """Browser-less HTTP client bound to the site under test."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
52	`    ctx = playwright.request.new_context(base_url=(base_url or BASE_URL).rstrip("/"))`	**Assignment**: binds the name *ctx* to the value on the right. *new_context()* creates an isolated browser profile (own cookies/storage). Isolation means tests cannot affect each other.
53	`    yield ctx`	**yield**: hands the value to the test and pauses the fixture; the code after yield runs after the test finishes (teardown). One function = setup + teardown.
54	`    ctx.dispose()`	**Method call** executed for its effect. *dispose()* releases an APIRequestContext.
57	`@pytest.fixture(scope="session")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'session': created once for the whole run (expensive setup shared).
58	`def site(api) -> SiteData:`	**def** defines the fixture *site*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site. Returns SiteData.
59	`    return load_site_data(api)`	**return**: ends the function and hands back *load_site_data(api)*. *load_site_data()*: project function load_site_data() in utils/site_data.py.
62	`# ---------- page-object fixtures ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
63	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
64	`def home(page) -> HomePage:`	**def** defines the fixture *home*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test. Returns HomePage.
65	`    return HomePage(page).open()`	**return**: ends the function and hands back *HomePage(page).open()*. *open()* opens a file (read/write). *HomePage(...)* creates a HomePage object (constructor call).
68	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
69	`def rooms(page, site) -> RoomsPage:`	**def** defines the fixture *rooms*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Returns RoomsPage.
70	`    """Rooms list for the default branch with 1 adult (every branch room holds at least 1)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
71	`    return RoomsPage(page).open(branch_id=site.default_branch.id, adults=1)`	**return**: ends the function and hands back *RoomsPage(page).open(branch_id=site.default_branch.id, adults=1)*. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).

Line	Code	Syntax & why
74	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
75	`def mobile_page(browser, browser_context_args):`	**def** defines the fixture *mobile_page*. Parameters: *browser*: pytest-playwright fixture: the browser process (session-wide) chosen with --browser; *browser_context_args*: default options for every new browser context.
76	`    ctx = browser.new_context(**{**browser_context_args, **MOBILE})`	**Assignment**: binds the name *ctx* to the value on the right. *new_context()* creates an isolated browser profile (own cookies/storage). Isolation means tests cannot affect each other.
77	`    page = ctx.new_page()`	**Assignment**: binds the name *page* to the value on the right. *new_page()* opens a new tab in a context.
78	`    page.set_default_timeout(DEFAULT_TIMEOUT_MS)`	**Method call** executed for its effect. *set_default_timeout()* sets how long actions (click/fill) wait for elements before failing.
79	`    yield page`	**yield**: hands the value to the test and pauses the fixture; the code after yield runs after the test finishes (teardown). One function = setup + teardown.
80	`    ctx.close()`	**Method call** executed for its effect. *close()* closes a page/context/browser and frees resources.
83	`# ---------- authentication ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
84	`@pytest.fixture(scope="session")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'session': created once for the whole run (expensive setup shared).
85	`def auth_state(browser, browser_context_args, tmp_path_factory):`	**def** defines the fixture *auth_state*. Parameters: *browser*: pytest-playwright fixture: the browser process (session-wide) chosen with --browser; *browser_context_args*: default options for every new browser context; *tmp_path_factory*: creates temporary directories for the session.
86	`    """Sign in ONCE per session with the existing test account (TEST_EMAIL/TEST_PASSWORD) and save`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
87	`    cookies/localStorage. Later tests reuse that file => logged in instantly (fast + stable)."""`	**Docstring** continuation (documentation text).
88	`    if not (TEST_EMAIL and TEST_PASSWORD):`	**if** statement: the indented block runs only when *not (TEST_EMAIL and TEST_PASSWORD)* is true.
89	`        pytest.skip("Set TEST_EMAIL and TEST_PASSWORD (GitHub secrets) to run signed-in tests")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
90	`    context = browser.new_context(**browser_context_args)`	**Assignment**: binds the name *context* to the value on the right. *new_context()* creates an isolated browser profile (own cookies/storage). Isolation means tests cannot affect each other.
91	`    page = context.new_page()`	**Assignment**: binds the name *page* to the value on the right. *new_page()* opens a new tab in a context.
92	`    page.set_default_timeout(DEFAULT_TIMEOUT_MS * 2)`	**Method call** executed for its effect. *set_default_timeout()* sets how long actions (click/fill) wait for elements before failing.
93	`    HomePage(page).open()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *open()* opens a file (read/write). *HomePage(...)* creates a HomePage object (constructor call).
94	`    name = AuthModal(page).sign_in(TEST_EMAIL, TEST_PASSWORD)`	**Assignment**: binds the name *name* to the value on the right. *sign_in()*: project method AuthModal.sign_in() in pages/auth_modal.py - Sign in as a customer and return the name shown in the header.. *AuthModal(...)* creates a AuthModal object (constructor call).
95	`    state_file = tmp_path_factory.mktemp("auth") / "state.json"`	**Assignment**: binds the name *state_file* to the value on the right. *mktemp()* creates a unique temporary directory for the session.

Line	Code	Syntax & why
96	`    context.storage_state(path=str(state_file))`	**Method call** executed for its effect. *storage_state()* saves cookies + localStorage to a JSON file so later contexts start already signed in. *str()* converts to string.
97	`    context.close()`	**Method call** executed for its effect. *close()* closes a page/context/browser and frees resources.
98	`    return {"path": str(state_file), "email": TEST_EMAIL, "display_name": name}`	**return**: ends the function and hands back *{'path': str(state_file), 'email': TEST_EMAIL, 'display_name': name}*. *str()* converts to string.
101	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
102	`def auth_page(browser, browser_context_args, auth_state):`	**def** defines the fixture *auth_page*. Parameters: *browser*: pytest-playwright fixture: the browser process (session-wide) chosen with --browser; *browser_context_args*: default options for every new browser context; *auth_state*: project fixture: signs in once per session and saves the session to a file.
103	`    """A Page that is already logged in. Each test gets its own isolated context."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
104	`    context = browser.new_context(storage_state=auth_state["path"], **browser_context_args)`	**Assignment**: binds the name *context* to the value on the right. *new_context()* creates an isolated browser profile (own cookies/storage). Isolation means tests cannot affect each other.
105	`    page = context.new_page()`	**Assignment**: binds the name *page* to the value on the right. *new_page()* opens a new tab in a context.
106	`    page.set_default_timeout(DEFAULT_TIMEOUT_MS)`	**Method call** executed for its effect. *set_default_timeout()* sets how long actions (click/fill) wait for elements before failing.
107	`    yield page # test runs here`	**yield**: hands the value to the test and pauses the fixture; the code after yield runs after the test finishes (teardown). One function = setup + teardown. Inline comment: “test runs here”.
108	`    context.close() # teardown`	**Method call** executed for its effect. *close()* closes a page/context/browser and frees resources. Inline comment: “teardown”.
111	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
112	`def account(auth_page) -> AccountPage:`	**def** defines the fixture *account*. Parameters: *auth_page*: project fixture: a tab that is already signed in (reuses auth_state). Returns AccountPage.
113	`    return AccountPage(auth_page)`	**return**: ends the function and hands back *AccountPage(auth_page)*. *AccountPage(...)* creates a AccountPage object (constructor call).
116	`# ---------- runtime monitoring (used by non-functional tests) ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
117	`@dataclass`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@dataclass**: generates __init__, __repr__ and __eq__ from the annotated fields.
118	`class PageMonitor:`	**class PageMonitor**: defines a new class (a blueprint for objects).
119	`    console_errors: list = field(default_factory=list)`	**Assignment**: binds the name *console_errors* with type annotation *list* (documents the expected type) to the value on the right.
120	`    page_errors: list = field(default_factory=list)`	**Assignment**: binds the name *page_errors* with type annotation *list* (documents the expected type) to the value on the right.
121	`    bad_responses: list = field(default_factory=list)`	**Assignment**: binds the name *bad_responses* with type annotation *list* (documents the expected type) to the value on the right.

Line	Code	Syntax & why
124	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
125	`def monitor(page) -> PageMonitor:`	**def** defines the fixture *monitor*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test. Returns PageMonitor.
126	`    m = PageMonitor()`	**Assignment**: binds the name *m* to the value on the right. *PageMonitor(...)* creates a PageMonitor object (constructor call).
127	`    page.on("console", lambda msg: m.console_errors.append(msg.text) if msg.type == "error" else None)`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).
128	`    page.on("pageerror", lambda exc: m.page_errors.append(str(exc)))`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. *str()* converts to string. **lambda**: inline anonymous function (used as a callback/predicate).
129	`    page.on(`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs.
130	`        "response",`	**Continuation** of the statement started on line 129 (inside brackets).
131	`        lambda r: (`	**Continuation** of the statement started on line 129 (inside brackets). **lambda**: a small anonymous function written inline.
132	`            m.bad_responses.append((r.status, r.url)) if r.status >= 400 and "favicon" not in r.url else None`	**Continuation** of the statement started on line 129 (inside brackets). *append()* adds an item to a list.
133	`        ),`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
134	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
135	`    return m`	**return**: ends the function and hands back *m*.

utils/__init__.py

Makes utils a package.

Empty file. An empty __init__.py marks the folder as a Python package so its modules can be imported with dotted names.

utils/data_factory.py

Test-data builders. Generated data (unique emails via uuid) guarantees tests never collide with each other or with earlier runs; frozen dataclasses make shared data immutable.

Line	Code	Syntax & why
1	`"""Builds unique test data so tests never collide with each other or with old runs."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.

Line	Code	Syntax & why
3	`import uuid`	**import uuid**: loads the module. Generates random unique identifiers so every test run creates unique emails/identifiers and never collides with earlier data.
4	`from dataclasses import dataclass`	**from dataclasses import dataclass**: imports specific names so they can be used without the module prefix. dataclass decorator generates __init__/__repr__/__eq__ for simple data-holding classes, removing boilerplate. *dataclass*: Decorator that auto-generates constructor and helpers for data classes.
6	`from config.settings import TEST_PHONE`	**from config.settings import TEST_PHONE**: imports specific names so they can be used without the module prefix. Project configuration module: every value can be overridden by an environment variable.
9	`@dataclass(frozen=True) # frozen = immutable, so a test cannot accidentally mutate shared data`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@dataclass**: generates __init__, __repr__ and __eq__ from the annotated fields. frozen=True makes instances immutable so shared test data cannot be modified by accident.
10	`class UserData:`	**class UserData**: defines a new class (a blueprint for objects).
11	`    full_name: str`	**Assignment**: binds the name *full_name* with type annotation *str* (documents the expected type) to the value on the right.
12	`    email: str`	**Assignment**: binds the name *email* with type annotation *str* (documents the expected type) to the value on the right.
13	`    phone: str`	**Assignment**: binds the name *phone* with type annotation *str* (documents the expected type) to the value on the right.
14	`    password: str`	**Assignment**: binds the name *password* with type annotation *str* (documents the expected type) to the value on the right.
16	`    @property`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@property**: the method is read like an attribute (obj.name, no parentheses) - computed on access.
17	`    def first_name(self) -> str:`	**def** defines the method *first_name* of class UserData; *self* is the instance. Returns str.
18	`        # Header shows initials + first name (e.g. "JG John"), so tests need this.`	**Comment** (# ... ignored by Python): documents intent for the reader.
19	`        return self.full_name.split()[0]`	**return**: ends the function and hands back *self.full_name.split()[0]*. *split()* splits a string into a list.
21	`    @classmethod`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@classmethod**: receives the class (cls) instead of an instance - used as an alternative constructor (factory).
22	`    def new(cls, first: str = "Qa") -> "UserData":`	**def** defines the method *new* of class UserData; *self* is the instance. Parameters: *cls*; *first*. Parameters with '= value' are optional (default used when omitted). Returns 'UserData'.
23	`        token = uuid.uuid4().hex[:10] # 10 random hex chars => unique email per call`	**Assignment**: binds the name *token* to the value on the right. *uuid4()* random UUID. Inline comment: “10 random hex chars => unique email per call”.
24	`        return cls(`	**return**: ends the function and hands back *cls(full_name=f'{first} Tester{token[:4]}', email=f'qa-{token}@maildr…*.
25	`            full_name=f"{first} Tester{token[:4]}",`	**Continuation** of the statement started on line 24 (inside brackets). Keyword argument(s) full_name (name=value): self-documenting and order-independent. Uses an **f-string** (f"…{expr}…") to build text with values.

Line	Code	Syntax & why
26	`email=f"qa-{token}@maildrop.cc", # maildrop.cc = disposable inbox domain`	**Continuation** of the statement started on line 24 (inside brackets). Keyword argument(s) email (name=value): self-documenting and order-independent. Uses an **f-string** (f"...{expr}...") to build text with values. Inline comment: “maildrop.cc = disposable inbox domain”.
27	`phone=TEST_PHONE,`	**Continuation** of the statement started on line 24 (inside brackets). Keyword argument(s) phone (name=value): self-documenting and order-independent.
28	`password=f"Qa!{token}#Aa1", # 16 chars: the site requires at least 12`	**Continuation** of the statement started on line 24 (inside brackets). Keyword argument(s) password (name=value): self-documenting and order-independent. Uses an **f-string** (f"...{expr}...") to build text with values. Inline comment: “16 chars: the site requires at least 12”.
29	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
32	`@dataclass(frozen=True)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@dataclass**: generates __init__, __repr__ and __eq__ from the annotated fields. frozen=True makes instances immutable so shared test data cannot be modified by accident.
33	`class GuestData:`	**class GuestData**: defines a new class (a blueprint for objects).
34	`full_name: str = "John Dane"`	**Assignment**: binds the name *full_name* with type annotation *str* (documents the expected type) to the value on the right.
35	`phone: str = "+2348012345678"`	**Assignment**: binds the name *phone* with type annotation *str* (documents the expected type) to the value on the right.
36	`email: str = "guest-qa@maildrop.cc"`	**Assignment**: binds the name *email* with type annotation *str* (documents the expected type) to the value on the right.
37	`notes: str = "Automated test booking - please ignore"`	**Assignment**: binds the name *notes* with type annotation *str* (documents the expected type) to the value on the right.

utils/site_data.py

Reads branches and rooms from the site's public API at session start. Database ids differ per environment and change over time; reading them live removes hard-coded ids, the most common cause of brittle hotel-booking tests.

Line	Code	Syntax & why
1	`"""Test data read from the site's own public API, so tests never hard-code DB ids that change per environment.`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`/api/branches -> branch ids, names, addresses`	**Docstring** continuation (documentation text).
4	`/api/rooms -> rooms (branchId == "" for hotel-level rooms, which are the bookable ones today)`	**Docstring** continuation (documentation text).
5	`"""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
7	`from dataclasses import dataclass, field`	**from dataclasses import dataclass, field**: imports specific names so they can be used without the module prefix. dataclass decorator generates __init__/__repr__/__eq__ for simple data-holding classes, removing boilerplate. *dataclass*: Decorator that auto-generates constructor and helpers for data classes. *field*: dataclasses.field configures a dataclass attribute (e.g. default_factory for mutable defaults).

Line	Code	Syntax & why
9	`from playwright.sync_api import APIRequestContext`	**from playwright.sync_api import APIRequestContext**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *APIRequestContext*: Playwright HTTP client (no browser) used for fast API checks and test-data loading.
12	`@dataclass`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@dataclass**: generates __init__, __repr__ and __eq__ from the annotated fields.
13	`class Branch:`	**class Branch**: defines a new class (a blueprint for objects).
14	`    id: str`	**Assignment**: binds the name *id* with type annotation *str* (documents the expected type) to the value on the right.
15	`    name: str`	**Assignment**: binds the name *name* with type annotation *str* (documents the expected type) to the value on the right.
16	`    city: str`	**Assignment**: binds the name *city* with type annotation *str* (documents the expected type) to the value on the right.
17	`    address: str`	**Assignment**: binds the name *address* with type annotation *str* (documents the expected type) to the value on the right.
18	`    raw: dict = field(repr=False, default_factory=dict)`	**Assignment**: binds the name *raw* with type annotation *dict* (documents the expected type) to the value on the right.
21	`@dataclass`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@dataclass**: generates __init__, __repr__ and __eq__ from the annotated fields.
22	`class SiteData:`	**class SiteData**: defines a new class (a blueprint for objects).
23	`    branches: list[Branch]`	**Assignment**: binds the name *branches* with type annotation *list[Branch]* (documents the expected type) to the value on the right.
24	`    rooms: list[dict] # unscoped catalogue (GET /api/rooms?limit=50)`	**Assignment**: binds the name *rooms* with type annotation *list[dict]* (documents the expected type) to the value on the right. Inline comment: “unscoped catalogue (GET /api/rooms?limit=50)”.
26	`    def branch(self, city: str) -> Branch:`	**def** defines the method *branch* of class SiteData; *self* is the instance. Parameters: *city*. Returns Branch.
27	`        return next(b for b in self.branches if b.city.lower() == city.lower())`	**return**: ends the function and hands back *next((b for b in self.branches if b.city.lower() == city.lower()))*. *next()* first item of an iterator. *lower()* lower-cases a string (case-insensitive comparisons).
29	`    @property`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@property**: the method is read like an attribute (obj.name, no parentheses) - computed on access.
30	`    def default_branch(self) -> Branch:`	**def** defines the method *default_branch* of class SiteData; *self* is the instance. Returns Branch.
31	`        return self.branch("Benin City")`	**return**: ends the function and hands back *self.branch('Benin City')*. *branch()*: project method SiteData.branch() in utils/site_data.py.
33	`    @property`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@property**: the method is read like an attribute (obj.name, no parentheses) - computed on access.
34	`    def hotel_rooms(self) -> list[dict]:`	**def** defines the method *hotel_rooms* of class SiteData; *self* is the instance. Returns list[dict].
35	`        """Rooms not tied to a branch. These open the full booking flow."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
36	`        return [r for r in self.rooms if not r.get("branchId")]`	**return**: ends the function and hands back *[r for r in self.rooms if not r.get('branchId')]*. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).

Line	Code	Syntax & why
38	`    @property`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@property**: the method is read like an attribute (obj.name, no parentheses) - computed on access.
39	`    def bookable_room(self) -> dict:`	**def** defines the method *bookable_room* of class SiteData; *self* is the instance. Returns dict.
40	`        avail = [r for r in self.hotel_rooms if r.get("isAvailable", True)]`	**Assignment**: binds the name *avail* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
41	`        return min(avail or self.hotel_rooms or self.rooms, key=lambda r: r["capacity"]["adults"])`	**return**: ends the function and hands back *min(avail or self.hotel_rooms or self.rooms, key=lambda r: r['capacit…*. *min()* smallest value. **lambda**: inline anonymous function (used as a callback/predicate).
44	`def _data(resp):`	**def** defines the private helper *_data* (leading underscore = internal use). Parameters: *resp*.
45	`    assert resp.ok, f"{resp.url} -> HTTP {resp.status}"`	**assert**: fails the test if *resp.ok* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
46	`    body = resp.json()`	**Assignment**: binds the name *body* to the value on the right. *json()* parses a response body (or returns it) as Python dict/list.
47	`    return body.get("data", body)`	**return**: ends the function and hands back *body.get('data', body)*. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
50	`def load_site_data(request: APIRequestContext) -> SiteData:`	**def** defines the function *load_site_data*. Parameters: *request*: pytest request object (introspection of the requesting test). Returns SiteData.
51	`    branches = [`	**Assignment**: binds the name *branches* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...).
52	`        Branch(b["id"], b["name"], b.get("city", ""), b.get("address", ""), b)`	**Continuation** of the statement started on line 51 (inside brackets). *Branch(...)* creates a Branch object (constructor call). *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
53	`        for b in _data(request.get("/api/branches"))`	**Continuation** of the statement started on line 51 (inside brackets). *_data()*: project function _data() in utils/site_data.py. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
54	`    ]`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
55	`    rooms = _data(request.get("/api/rooms?limit=50"))`	**Assignment**: binds the name *rooms* to the value on the right. *_data()*: project function _data() in utils/site_data.py. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
56	`    rooms = rooms.get("items", rooms) if isinstance(rooms, dict) else rooms`	**Assignment**: binds the name *rooms* to the value on the right. **Conditional expression** (a if condition else b). *isinstance()* type check. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
57	`    return SiteData(branches=branches, rooms=rooms)`	**return**: ends the function and hands back *SiteData(branches=branches, rooms=rooms)*. *SiteData(...)* creates a SiteData object (constructor call).
60	`def branch_rooms(request: APIRequestContext, branch_id: str) -> list[dict]:`	**def** defines the function *branch_rooms*. Parameters: *request*: pytest request object (introspection of the requesting test); *branch_id*. Returns list[dict].
61	`    rooms = _data(request.get(f"/api/rooms?branchId={branch_id}&limit=50"))`	**Assignment**: binds the name *rooms* to the value on the right. *_data()*: project function _data() in utils/site_data.py. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get). Uses an **f-string** (f"...{expr}...") to build text with values.
62	`    return rooms.get("items", rooms) if isinstance(rooms, dict) else rooms`	**return**: ends the function and hands back *rooms.get('items', rooms) if isinstance(rooms, dict) else rooms*. *isinstance()* type check. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).

utils/traceability.py

A small pytest plugin that turns the QA workbook into executable policy: tc() links tests to TC ids, defect() converts known open bugs into non-strict xfail (tracked, not hidden; XPASS = retest), live_write/needs_account gate tests on flags/secrets, and a CSV/Markdown traceability report is written after every run. Works with parallel workers (xdist) because ids travel in report.user_properties.

Line	Code	Syntax & why
1	`"""pytest plugin: links tests to the QA workbook (TC-xxx / BUG-xxx) and writes a traceability report.`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`Markers (registered in pytest.ini):`	**Docstring** continuation (documentation text).
4	`    @pytest.mark.tc("TC-004", "TC-005") test covers these workbook test cases`	**Docstring** continuation (documentation text).
5	`    @pytest.mark.defect("BUG-003") test asserts EXPECTED behaviour that a known open bug breaks.`	**Docstring** continuation (documentation text).
6	`                                        It runs as xfail(strict=False) so the pipeline stays green`	**Docstring** continuation (documentation text).
7	`                                        while the bug is open; an XPASS means "bug looks fixed - retest".`	**Docstring** continuation (documentation text).
8	`                                        Run with --known-defects=fail to treat them as real failures.`	**Docstring** continuation (documentation text).
9	`    @pytest.mark.live_write creates real records (account/booking/message) - skipped unless`	**Docstring** continuation (documentation text).
10	`                                        ALLOW_LIVE_WRITES=true (workbook status "Blocked").`	**Docstring** continuation (documentation text).
11	`    @pytest.mark.needs_account needs TEST_EMAIL/TEST_PASSWORD of an existing customer.`	**Docstring** continuation (documentation text).
13	`Output: reports/traceability.csv and reports/traceability.md (one row per test, TC and BUG ids, outcome).`	**Docstring** continuation (documentation text).
14	`Works with pytest-xdist: ids travel in report.user_properties.`	**Docstring** continuation (documentation text).
15	`"""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
17	`import csv`	**import csv**: loads the module. Reads/writes comma-separated files; used for the workbook export and the traceability report.
18	`from collections import defaultdict`	**from collections import defaultdict**: imports specific names so they can be used without the module prefix. defaultdict creates missing dictionary keys automatically - used to group outcomes per test case. *defaultdict*: Dictionary that creates a default value for missing keys.
19	`from pathlib import Path`	**from pathlib import Path**: imports specific names so they can be used without the module prefix. Object-oriented file paths (Path) that work on every OS. *Path*: Filesystem path object.
21	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
23	`from config.settings import ALLOW_LIVE_WRITES, TEST_EMAIL, TEST_PASSWORD`	**from config.settings import ALLOW_LIVE_WRITES, TEST_EMAIL, TEST_PASSWORD**: imports specific names so they can be used without the module prefix. Project configuration module: every value can be overridden by an environment variable.

Line	Code	Syntax & why
25	`REPORT_DIR = Path("reports")`	Module-level **constant** *REPORT_DIR* (UPPER_CASE by convention = not changed at runtime). *Path(...)* creates a Path object (constructor call).
28	`def pytest_addoption(parser):`	**def pytest_addoption**: a pytest **hook** - pytest calls it automatically at that stage of the run. Parameters: *parser*.
29	`    parser.addoption(`	**Method call** executed for its effect. *addoption()* registers a new command-line option.
30	`        "--known-defects",`	**Continuation** of the statement started on line 29 (inside brackets).
31	`        choices=("xfail", "fail"),`	**Continuation** of the statement started on line 29 (inside brackets). Keyword argument(s) choices (name=value): self-documenting and order-independent.
32	`        default="xfail",`	**Continuation** of the statement started on line 29 (inside brackets). Keyword argument(s) default (name=value): self-documenting and order-independent.
33	`        help="xfail (default): known open bugs are tracked as xfail; fail: treat them as failures (retest run).",`	**Continuation** of the statement started on line 29 (inside brackets). Keyword argument(s) help (name=value): self-documenting and order-independent.
34	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
37	`def _ids(item, name):`	**def** defines the private helper *_ids* (leading underscore = internal use). Parameters: *item*; *name*.
38	`    out = []`	**Assignment**: binds the name *out* to the value on the right. The value is a **list** literal [...].
39	`    for m in item.iter_markers(name):`	**for** loop: repeats the indented block for every item, binding it to *m* (iterating *item.iter_markers(name)*). *iter_markers()* iterates every marker with that name on a test.
40	`        out.extend(str(a) for a in m.args)`	**Method call** executed for its effect. *extend()* adds many items to a list. *str()* converts to string.
41	`    return sorted(set(out))`	**return**: ends the function and hands back *sorted(set(out))*. *sorted()* returns a sorted list. *set()* builds a set (unique values).
44	`def pytest_collection_modifyitems(config, items):`	**def pytest_collection_modifyitems**: a pytest **hook** - pytest calls it automatically at that stage of the run. Parameters: *config*; *items*.
45	`    mode = config.getoption("--known-defects")`	**Assignment**: binds the name *mode* to the value on the right. *getoption()* reads a command-line option value.
46	`    for item in items:`	**for** loop: repeats the indented block for every item, binding it to *item* (iterating *items*).
47	`        tcs, bugs = _ids(item, "tc"), _ids(item, "defect")`	**Assignment**: binds the name *(tcs, bugs)* to the value on the right. Tuple unpacking / tuple value. *_ids()*: project function _ids() in utils/traceability.py.
48	`        item.user_properties.append(("tc", ",".join(tcs)))`	**Method call** executed for its effect. *append()* adds an item to a list. *join()* joins strings with a separator.
49	`        item.user_properties.append(("bugs", ",".join(bugs)))`	**Method call** executed for its effect. *append()* adds an item to a list. *join()* joins strings with a separator.
50	`        defect = item.get_closest_marker("defect")`	**Assignment**: binds the name *defect* to the value on the right. *get_closest_marker()* returns the nearest marker with that name, or None.
51	`        if defect and mode == "xfail":`	**if** statement: the indented block runs only when *defect and mode == 'xfail'* is true.
52	`            why = defect.kwargs.get("reason") or ""`	**Assignment**: binds the name *why* to the value on the right. **Boolean expression**: 'or' returns the first truthy value - a compact default. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
53	`            label = ", ".join(bugs) or "unlogged defect"`	**Assignment**: binds the name *label* to the value on the right. **Boolean expression**: 'or' returns the first truthy value - a compact default. *join()* joins strings with a separator.

Line	Code	Syntax & why
54	`            item.add_marker(pytest.mark.xfail(reason=f"Known defect {label} {why}".strip(), strict=False))`	**Method call** executed for its effect. *add_marker()* adds a marker to a collected test at runtime (auto-tagging, xfail for defects). *xfail()* expected failure: the test runs, failure is reported as XFAIL instead of FAILED. *strip()* removes surrounding whitespace. Uses an **f-string** (f"...{expr}...") to build text with values.
55	`        if item.get_closest_marker("live_write") and not ALLOW_LIVE_WRITES:`	**if** statement: the indented block runs only when *item.get_closest_marker('live_write') and (not ALLOW_LIVE_WRITES)* is true. *get_closest_marker()* returns the nearest marker with that name, or None.
56	`            item.add_marker(pytest.mark.skip(reason="Creates live records - set ALLOW_LIVE_WRITES=true"))`	**Method call** executed for its effect. *add_marker()* adds a marker to a collected test at runtime (auto-tagging, xfail for defects). *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
57	`        if item.get_closest_marker("needs_account") and not (TEST_EMAIL and TEST_PASSWORD):`	**if** statement: the indented block runs only when *item.get_closest_marker('needs_account') and (not (TEST_EMAIL and TEST_PASSWORD...* is true. *get_closest_marker()* returns the nearest marker with that name, or None.
58	`            item.add_marker(`	**Method call** executed for its effect. *add_marker()* adds a marker to a collected test at runtime (auto-tagging, xfail for defects).
59	`                pytest.mark.skip(reason="Set TEST_EMAIL and TEST_PASSWORD for an existing account")`	**Continuation** of the statement started on line 58 (inside brackets). Keyword argument(s) reason (name=value): self-documenting and order-independent. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
60	`            )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
63	`class _Collector:`	**class _Collector**: defines a new class (a blueprint for objects).
64	`    def __init__(self):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once.
65	`        self.rows = {}`	Stores the instance attribute *self.rows* on the object (available to every method). The value is a **dict** literal {key: value}.
67	`    @pytest.hookimpl`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.hookimpl**: marks the method as an implementation of a pytest hook.
68	`    def pytest_runtest_logreport(self, report):`	**def pytest_runtest_logreport**: a pytest **hook** - pytest calls it automatically at that stage of the run. Parameters: *report*.
69	`        props = dict(report.user_properties)`	**Assignment**: binds the name *props* to the value on the right. *dict()* builds a dict (copy of a mapping).
70	`        if report.when == "call" or (report.when == "setup" and report.outcome != "passed"):`	**if** statement: the indented block runs only when *report.when == 'call' or (report.when == 'setup' and report.outcome != 'passed')* is true.
71	`            if hasattr(report, "wasxfail"):`	**if** statement: the indented block runs only when *hasattr(report, 'wasxfail')* is true. *hasattr()* checks an attribute exists.
72	`                outcome = "XPASS (bug fixed? retest)" if report.passed else "XFAIL (known defect)"`	**Assignment**: binds the name *outcome* to the value on the right. **Conditional expression** (a if condition else b).
73	`            else:`	**else**: runs when the if-condition was false.
74	`                outcome = report.outcome.upper()`	**Assignment**: binds the name *outcome* to the value on the right.
75	`            self.rows[report.nodeid] = (props.get("tc", ""), props.get("bugs", ""), outcome)`	Stores the instance attribute *self.rows[report.nodeid]* on the object (available to every method). Tuple unpacking / tuple value. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).

Line	Code	Syntax & why
76	`        elif report.when == "teardown" and report.failed and report.nodeid in self.rows:`	**if** statement: the indented block runs only when *report.when == 'teardown' and report.failed and (report.nodeid in self.rows)* is true.
77	`            tc, bugs, _ = self.rows[report.nodeid]`	**Assignment**: binds the name *(tc, bugs, _)* to the value on the right.
78	`            self.rows[report.nodeid] = (tc, bugs, "ERROR (teardown)")`	Stores the instance attribute *self.rows[report.nodeid]* on the object (available to every method). Tuple unpacking / tuple value.
80	`    @pytest.hookimpl`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.hookimpl**: marks the method as an implementation of a pytest hook.
81	`    def pytest_sessionfinish(self, session):`	**def pytest_sessionfinish**: a pytest **hook** - pytest calls it automatically at that stage of the run. Parameters: *session*.
82	`        if hasattr(session.config, "workerinput") or not self.rows: # only the xdist controller writes`	**if** statement: the indented block runs only when *hasattr(session.config, 'workerinput') or not self.rows* is true. *hasattr()* checks an attribute exists. Inline comment: “only the xdist controller writes”.
83	`            return`	**return**: ends the function and hands back *None*.
84	`        REPORT_DIR.mkdir(exist_ok=True)`	**Method call** executed for its effect. *mkdir()* creates a directory.
85	`        with open(REPORT_DIR / "traceability.csv", "w", newline="") as f:`	**with** statement (context manager): sets up a resource and guarantees it is closed/cleaned up when the block ends, even on error - *open(REPORT_DIR / 'traceability.csv', 'w', newline='')*. *open()* opens a file (read/write).
86	`            w = csv.writer(f)`	**Assignment**: binds the name *w* to the value on the right. *writer()* CSV writer.
87	`            w.writerow(["test_case_ids", "linked_bugs", "outcome", "test"])`	**Method call** executed for its effect. *writerow()* writes one CSV row.
88	`            for nodeid, (tc, bugs, outcome) in sorted(self.rows.items(), key=lambda kv: kv[1][0] or "~"):`	**for** loop: repeats the indented block for every item, binding it to *(nodeid, (tc, bugs, outcome))* (iterating *sorted(self.rows.items(), key=lambda kv: kv[1][0] or '~')*). *sorted()* returns a sorted list. *items()* key/value pairs of a dict. **lambda**: inline anonymous function (used as a callback/predicate).
89	`                w.writerow([tc, bugs, outcome, nodeid])`	**Method call** executed for its effect. *writerow()* writes one CSV row.
91	`        per_tc = defaultdict(list)`	**Assignment**: binds the name *per_tc* to the value on the right.
92	`        for tc, _bugs, outcome in self.rows.values():`	**for** loop: repeats the indented block for every item, binding it to *(tc, _bugs, outcome)* (iterating *self.rows.values()*). *values()* values of a dict.
93	`            for t in filter(None, tc.split(",")):`	**for** loop: repeats the indented block for every item, binding it to *t* (iterating *filter(None, tc.split(','))*). *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *split()* splits a string into a list.
94	`                per_tc[t].append(outcome)`	**Method call** executed for its effect. *append()* adds an item to a list.
95	`        totals = defaultdict(int)`	**Assignment**: binds the name *totals* to the value on the right.
96	`        for _tc, _bugs, outcome in self.rows.values():`	**for** loop: repeats the indented block for every item, binding it to *(_tc, _bugs, outcome)* (iterating *self.rows.values()*). *values()* values of a dict.
97	`            totals[outcome.split(" ")[0]] += 1`	**Augmented assignment** =: updates *totals[outcome.split(' ')[0]]* in place. *split()* splits a string into a list.
98	`        lines = [`	**Assignment**: binds the name *lines* to the value on the right. The value is a **list** literal [...].
99	`            "# Traceability report",`	**Continuation** of the statement started on line 98 (inside brackets).
100	`            "",`	**Continuation** of the statement started on line 98 (inside brackets).
101	` "| Outcome	Tests |",`	**Continuation** of the statement started on line 98 (inside brackets).
102	`            "|---|---|",`	**Continuation** of the statement started on line 98 (inside brackets).

Line	Code	Syntax & why
103	` *[f"| {k}	{v} |" for k, v in sorted(totals.items())],`	**Continuation** of the statement started on line 98 (inside brackets). *sorted()* returns a sorted list. *items()* key/value pairs of a dict. Uses an **f-string** (f"...{expr}...") to build text with values.
104	`            "",`	**Continuation** of the statement started on line 98 (inside brackets).
105	`            f"Workbook test cases exercised: **{len(per_tc)}**",`	**Continuation** of the statement started on line 98 (inside brackets). *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
106	`            "",`	**Continuation** of the statement started on line 98 (inside brackets).
107	` "| Test case	Outcomes |",`	**Continuation** of the statement started on line 98 (inside brackets).
108	`            "|---|---|",`	**Continuation** of the statement started on line 98 (inside brackets).
109	` *[f"| {t}	{', '.join(sorted(set(o)))} |" for t, o in sorted(per_tc.items())],`	**Continuation** of the statement started on line 98 (inside brackets). *sorted()* returns a sorted list. *join()* joins strings with a separator. *items()* key/value pairs of a dict. *set()* builds a set (unique values). Uses an **f-string** (f"...{expr}...") to build text with values.
110	`        ]`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
111	`        (REPORT_DIR / "traceability.md").write_text("\n".join(lines) + "\n")`	**Method call** executed for its effect. *write_text()* writes a whole text file. *join()* joins strings with a separator.
114	`def pytest_configure(config):`	**def pytest_configure**: a pytest **hook** - pytest calls it automatically at that stage of the run. Parameters: *config*.
115	`    config.pluginmanager.register(_Collector(), "oneflare-traceability-collector")`	**Method call** executed for its effect. *register()*: project method AuthModal.register() in pages/auth_modal.py - Full happy path: open form -> fill -> submit -> wait until logged in..

pages/__init__.py

Makes pages a package.

Empty file. An empty __init__.py marks the folder as a Python package so its modules can be imported with dotted names.

pages/base_page.py

Base class of the Page Object Model. Header, footer, mobile menu and navigation helpers are defined once and inherited by every page, so a header change is fixed in one place. open() waits for network idle because Next.js ignores clicks before hydration.

Line	Code	Syntax & why
1	`"""BasePage: everything shared by all pages (header, footer, navigation, helpers).`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
2	`Every other page object inherits from this class (OOP inheritance = no duplicated code)."""`	**Docstring** continuation (documentation text).
4	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
5	`import time`	**import time**: loads the module. Standard-library clock functions. time.monotonic() is used for polling deadlines because it never jumps backwards when the system clock changes.

Line	Code	Syntax & why
7	`from playwright.sync_api import Locator, Page, expect`	**from playwright.sync_api import Locator, Page, expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Locator*: Playwright type describing HOW to find element(s); resolved lazily each time it is used. *Page*: Playwright type for one browser tab; used here as a type hint. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
10	`class BasePage:`	**class BasePage**: defines a new class (a blueprint for objects).
11	`    path = "/" # each subclass overrides this with its own URL path`	**Assignment**: binds the name *path* to the value on the right. Inline comment: “each subclass overrides this with its own URL path”.
13	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
14	`        self.page = page # the Playwright Page this object drives`	Stores the instance attribute *self.page* on the object (available to every method). Inline comment: “the Playwright Page this object drives”.
15	`        # Locators are LAZY: nothing is searched until you act on/assert them,`	**Comment** (# ... ignored by Python): documents intent for the reader.
16	`        # so defining them in __init__ is cheap and always uses fresh DOM state.`	**Comment** (# ... ignored by Python): documents intent for the reader.
17	`        self.header = page.get_by_role("banner") # <header> element`	Instance locator *self.header*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Inline comment: “<header> element”.
18	`        self.main = page.get_by_role("main")`	Instance locator *self.main*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
19	`        self.footer = page.get_by_role("contentinfo")`	Instance locator *self.footer*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
20	`        self.logo = self.header.get_by_role("link", name="Oneflare Grand Plaza")`	Instance locator *self.logo*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
21	`        self.sign_in_button = self.header.get_by_role("button", name="Sign in")`	Instance locator *self.sign_in_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
22	`        self.book_now_link = self.header.get_by_role("link", name="Book now")`	Instance locator *self.book_now_link*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
23	`        self.search_box = page.get_by_role("searchbox", name="Search rooms")`	Instance locator *self.search_box*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
24	`        self.menu_toggle = self.header.get_by_role("button", name="Toggle menu")`	Instance locator *self.menu_toggle*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
25	`        self.mobile_menu = page.get_by_role("dialog", name="Navigation menu")`	Instance locator *self.mobile_menu*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
26	`        self.close_menu = self.mobile_menu.get_by_role("button", name="Close menu")`	Instance locator *self.close_menu*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
27	`        self.footer_head_office = self.footer.get_by_text(re.compile(r"^Head office"))`	Instance locator *self.footer_head_office*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
28	`        self.footer_legal = self.footer.get_by_text(re.compile(r"^©"))`	Instance locator *self.footer_legal*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
29	`        self.footer_contact_items = self.footer.get_by_role("list").first.get_by_role("listitem")`	Instance locator *self.footer_contact_items*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
31	`    # ---------- navigation ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
32	`    def open(self, path: str | None = None):`	**def** defines the method *open* of class BasePage; *self* is the instance. Parameters: *path*. Parameters with '= value' are optional (default used when omitted).
33	`        self.page.goto(path or self.path) # relative path + base_url from pytest.ini / --base-url`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted. Inline comment: “relative path + base_url from pytest.ini / --base-url”.

Line	Code	Syntax & why
34	`        self.settle() # Next.js hydrates after load: clicking earlier is silently ignored`	**Method call** executed for its effect. *settle()*: project method BasePage.settle() in pages/base_page.py - Wait until client-side data fetching has finished (Next.js pages render after /api calls).. Inline comment: “Next.js hydrates after load: clicking earlier is silently ignored”.
35	`        return self # returning self allows chaining: HomePage(page).open().do_x()`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()). Inline comment: “returning self allows chaining: HomePage(page).open().do_x()”.
37	`    def settle(self) -> None:`	**def** defines the method *settle* of class BasePage; *self* is the instance. Returns None.
38	`        """Wait until client-side data fetching has finished (Next.js pages render after /api calls)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
39	`        try:`	**try**: runs the block and lets the except clause handle expected errors instead of crashing.
40	`            self.page.wait_for_load_state("networkidle", timeout=10000)`	**Method call** executed for its effect. *wait_for_load_state()* waits for a page lifecycle state ('domcontentloaded', 'load', 'networkidle'). networkidle = no network traffic for 500 ms, i.e. client-side data has arrived.
41	`        except Exception: # noqa: BLE001 - long-polling widgets (maps) may keep the network busy`	**except**: handles the listed error type raised in the try block (here: an expected timeout or missing optional dependency). Inline comment: “noqa: BLE001 - long-polling widgets (maps) may keep the network busy”.
42	`            pass`	**pass**: a no-op placeholder (Python needs a statement in every block).
44	`    def nav_link(self, name: str) -> Locator:`	**def** defines the method *nav_link* of class BasePage; *self* is the instance. Parameters: *name*. Returns Locator.
45	`        # exact=True stops "Rooms" matching e.g. "See rooms at ..."`	**Comment** (# ... ignored by Python): documents intent for the reader.
46	`        return self.header.get_by_role("navigation").get_by_role("link", name=name, exact=True)`	**return**: ends the function and hands back *self.header.get_by_role('navigation').get_by_role('link', name=name, …. get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
48	`    def footer_link(self, name: str) -> Locator:`	**def** defines the method *footer_link* of class BasePage; *self* is the instance. Parameters: *name*. Returns Locator.
49	`        return self.footer.get_by_role("link", name=name, exact=True)`	**return**: ends the function and hands back *self.footer.get_by_role('link', name=name, exact=True)*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
51	`    def go_to(self, name: str) -> None:`	**def** defines the method *go_to* of class BasePage; *self* is the instance. Parameters: *name*. Returns None.
52	`        self.nav_link(name).click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *nav_link()*: project method BasePage.nav_link() in pages/base_page.py.
54	`    def user_menu(self, name: str) -> Locator:`	**def** defines the method *user_menu* of class BasePage; *self* is the instance. Parameters: *name*. Returns Locator.
55	`        # After login the header shows the customer's name/initials instead of "Sign in"`	**Comment** (# ... ignored by Python): documents intent for the reader.

Line	Code	Syntax & why
56	`        return self.header.get_by_role("link", name=re.compile(re.escape(name), re.I))`	**return**: ends the function and hands back *self.header.get_by_role('link', name=re.compile(re.escape(name), re.I)…. get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. *escape()* escapes regex special characters in literal text. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
58	`    def is_on_screen(self, locator: Locator) -> bool:`	**def** defines the method *is_on_screen* of class BasePage; *self* is the instance. Parameters: *locator*. Returns bool.
59	`        """Slide-in drawers stay in the DOM when closed, translated off-screen; Playwright still calls`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
60	`        them 'visible', so check the geometry instead."""`	**Docstring** continuation (documentation text).
61	`        box = locator.bounding_box()`	**Assignment**: binds the name *box* to the value on the right. *bounding_box()* element position/size on screen; used to detect off-screen drawers.
62	`        width = self.page.viewport_size["width"]`	**Assignment**: binds the name *width* to the value on the right.
63	`        return bool(box) and box["x"] < width - 1 and box["x"] + box["width"] > 1`	**return**: ends the function and hands back *bool(box) and box['x'] < width - 1 and (box['x'] + box['width'] > 1)*. *bool()* converts to True/False.
65	`    def expect_on_screen(self, locator: Locator, on: bool = True, timeout_s: float = 5.0) -> None:`	**def** defines the method *expect_on_screen* of class BasePage; *self* is the instance. Parameters: *locator*; *on*; *timeout_s*. Parameters with '= value' are optional (default used when omitted). Returns None.
66	`        deadline = time.monotonic() + timeout_s`	**Assignment**: binds the name *deadline* to the value on the right. *monotonic()* monotonic clock for timeouts.
67	`        while self.is_on_screen(locator) != on and time.monotonic() < deadline:`	**while** loop: repeats while *self.is_on_screen(locator) != on and time.monotonic() < deadline* is true (bounded polling). *is_on_screen()*: project method BasePage.is_on_screen() in pages/base_page.py - Slide-in drawers stay in the DOM when closed, translated off-screen; Playwright still calls. *monotonic()* monotonic clock for timeouts.
68	`            self.page.wait_for_timeout(100)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
69	`        assert self.is_on_screen(locator) == on, f"{locator} on_screen != {on}"`	**assert**: fails the test if *self.is_on_screen(locator) == on* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *is_on_screen()*: project method BasePage.is_on_screen() in pages/base_page.py - Slide-in drawers stay in the DOM when closed, translated off-screen; Playwright still calls. Uses an **f-string** (f"...{expr}...") to build text with values.
71	`    # ---------- shared assertions ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
72	`    def expect_url_contains(self, fragment: str) -> None:`	**def** defines the method *expect_url_contains* of class BasePage; *self* is the instance. Parameters: *fragment*. Returns None.
73	`        expect(self.page).to_have_url(re.compile(re.escape(fragment)))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. *escape()* escapes regex special characters in literal text. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
75	`    def expect_header_visible(self) -> None:`	**def** defines the method *expect_header_visible* of class BasePage; *self* is the instance. Returns None.

Line	Code	Syntax & why
76	`        expect(self.header).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.

pages/search_widget.py

Component object for the availability search bar that appears on both Home and Rooms. Modelling a reusable component once (composition) avoids duplicating the same locators in two page objects.

Line	Code	Syntax & why
1	`"""Availability search bar shared by Home and Rooms (Branch / Check in / Check out / Guests)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`from datetime import date`	**from datetime import date**: imports specific names so they can be used without the module prefix. date/timedelta give calendar arithmetic (today + 7 days) for check-in/check-out dates. *date*: Calendar date type (no time component).
5	`from playwright.sync_api import Page`	**from playwright.sync_api import Page**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint.
8	`class SearchWidget:`	**class SearchWidget**: defines a new class (a blueprint for objects).
9	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
10	`        self.page = page`	Stores the instance attribute *self.page* on the object (available to every method).
11	`        main = page.get_by_role("main")`	**Assignment**: binds the name *main* to the value on the right. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
12	`        self.branch = main.get_by_role("combobox", name="Branch")`	Instance locator *self.branch*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
13	`        self.check_in = main.get_by_role("textbox", name="Check in")`	Instance locator *self.check_in*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
14	`        self.check_out = main.get_by_role("textbox", name="Check out")`	Instance locator *self.check_out*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
15	`        self.guests = main.get_by_role("combobox", name="Guests")`	Instance locator *self.guests*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
16	`        self.check_availability = main.get_by_role("button", name="Check availability")`	Instance locator *self.check_availability*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
18	`    def set_dates(self, check_in: date | str, check_out: date | str) -> "SearchWidget":`	**def** defines the method *set_dates* of class SearchWidget; *self* is the instance. Parameters: *check_in*; *check_out*. Returns 'SearchWidget'.
19	`        self.check_in.fill(str(check_in))`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *str()* converts to string.
20	`        self.check_out.fill(str(check_out))`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *str()* converts to string.
21	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
23	`    def search(self, branch_id: str | None = None, check_in=None, check_out=None, adults: int | None = None):`	**def** defines the method *search* of class SearchWidget; *self* is the instance. Parameters: *branch_id*; *check_in*; *check_out*; *adults*. Parameters with '= value' are optional (default used when omitted).
24	`        if branch_id:`	**if** statement: the indented block runs only when *branch_id* is true.
25	`            self.branch.select_option(branch_id)`	**Method call** executed for its effect. *select_option()* chooses an <option> in a <select> by value or label.
26	`        if check_in:`	**if** statement: the indented block runs only when *check_in* is true.
27	`            self.check_in.fill(str(check_in))`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *str()* converts to string.
28	`        if check_out:`	**if** statement: the indented block runs only when *check_out* is true.
29	`            self.check_out.fill(str(check_out))`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *str()* converts to string.
30	`        if adults:`	**if** statement: the indented block runs only when *adults* is true.
31	`            self.guests.select_option(str(adults))`	**Method call** executed for its effect. *select_option()* chooses an <option> in a <select> by value or label. *str()* converts to string.
32	`        self.check_availability.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
33	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
35	`    def guest_options(self) -> list[str]:`	**def** defines the method *guest_options* of class SearchWidget; *self* is the instance. Returns list[str].
36	`        return self.guests.locator("option").all_inner_texts()`	**return**: ends the function and hands back *self.guests.locator('option').all_inner_texts()*. *all_inner_texts()* returns visible text of every match as a list of strings. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
38	`    def branch_options(self) -> list[str]:`	**def** defines the method *branch_options* of class SearchWidget; *self* is the instance. Returns list[str].

Line	Code	Syntax & why
39	`        return [t.strip() for t in self.branch.locator("option").all_inner_texts()]`	**return**: ends the function and hands back *[t.strip() for t in self.branch.locator('option').all_inner_texts()]*. *strip()* removes surrounding whitespace. *all_inner_texts()* returns visible text of every match as a list of strings. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

pages/home_page.py

Page Object for the Home page: hero, search widget, locations, 'Free for your dates', gallery and amenities sections.

Line	Code	Syntax & why
1	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
3	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
4	`from pages.search_widget import SearchWidget`	**from pages.search_widget import SearchWidget**: imports specific names so they can be used without the module prefix. Component object for the availability search bar used on Home and Rooms.
7	`class HomePage(BasePage):`	**class HomePage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
8	`    path = "/"`	**Assignment**: binds the name *path* to the value on the right.
10	`    def __init__(self, page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
11	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
12	`        self.hero_heading = self.main.get_by_role("heading", level=1)`	Instance locator *self.hero_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
13	`        self.hero_text = self.main.get_by_role("heading", name="Space to work, and to stop working")`	Instance locator *self.hero_text*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
14	`        self.explore_rooms_link = self.main.get_by_role("link", name="Explore rooms")`	Instance locator *self.explore_rooms_link*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
15	`        self.search = SearchWidget(page)`	Stores the instance attribute *self.search* on the object (available to every method). *SearchWidget(...)* creates a SearchWidget object (constructor call).
16	`        # sections`	**Comment** (# ... ignored by Python): documents intent for the reader.
17	`        self.locations_heading = self.main.get_by_role("heading", name="Find us where you need us.")`	Instance locator *self.locations_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
18	`        self.location_cards = (`	Instance locator *self.location_cards*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements.
19	`            self.main.get_by_role("link").filter(has=page.locator("p")).filter(has_text="Road")`	**Continuation** of the statement started on line 18 (inside brackets). Keyword argument(s) has, has_text (name=value): self-documenting and order-independent. *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
20	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
21	`        self.free_rooms_heading = self.main.get_by_role("heading", name="Free for your dates")`	Instance locator *self.free_rooms_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
22	`        self.free_rooms_caption = self.main.get_by_text("Live at ")`	Instance locator *self.free_rooms_caption*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
23	`        self.see_all_rooms = self.main.get_by_role("link", name="See all rooms")`	Instance locator *self.see_all_rooms*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
24	`        self.gallery_heading = self.main.get_by_role("heading", name="The rest of the house")`	Instance locator *self.gallery_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
25	`        self.amenities_heading = self.main.get_by_role("heading", name="Included, not extra.")`	Instance locator *self.amenities_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
26	`        self.view_details_links = self.main.get_by_role("link", name="View details")`	Instance locator *self.view_details_links*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
27	`        self.price_per_night = self.main.get_by_text(" / night")`	Instance locator *self.price_per_night*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
29	`    def expect_loaded(self) -> None:`	**def** defines the method *expect_loaded* of class HomePage; *self* is the instance. Returns None.
30	`        expect(self.hero_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
31	`        expect(self.search.check_availability).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
33	`    def explore_rooms(self) -> None:`	**def** defines the method *explore_rooms* of class HomePage; *self* is the instance. Returns None.
34	`        self.explore_rooms_link.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
36	`    def section_after(self, heading) -> "object":`	**def** defines the method *section_after* of class HomePage; *self* is the instance. Parameters: *heading*. Returns 'object'.
37	`        """The section element that contains a given heading."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
38	`        return self.main.locator("section").filter(has=heading).first`	**return**: ends the function and hands back *self.main.locator('section').filter(has=heading).first*. *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

pages/rooms_page.py

Page Object for the Rooms list: search bar, filter panel (room type, bed, amenities, price slider), result heading and cards, mobile filter drawer. wait_for_count() polls because results re-render asynchronously after each filter change.

Line	Code	Syntax & why
1	`"""Rooms listing: search bar, filter panel (room type / bed / amenities / price), result cards."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`import time`	**import time**: loads the module. Standard-library clock functions. time.monotonic() is used for polling deadlines because it never jumps backwards when the system clock changes.

Line	Code	Syntax & why
6	`from playwright.sync_api import Locator, Page, expect`	**from playwright.sync_api import Locator, Page, expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Locator*: Playwright type describing HOW to find element(s); resolved lazily each time it is used. *Page*: Playwright type for one browser tab; used here as a type hint. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
8	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
9	`from pages.search_widget import SearchWidget`	**from pages.search_widget import SearchWidget**: imports specific names so they can be used without the module prefix. Component object for the availability search bar used on Home and Rooms.
11	`COUNT_RE = re.compile(r"(\d+)\s+room types?\s+at\s+(.+)") # "5 room types at Oneflare Grand Plaza - ..."`	Module-level **constant** *COUNT_RE* (UPPER_CASE by convention = not changed at runtime). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally). Inline comment: “"5 room types at Oneflare Grand Plaza - ..."”.
14	`class RoomsPage(BasePage):`	**class RoomsPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
15	`    path = "/rooms"`	**Assignment**: binds the name *path* to the value on the right.
17	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
18	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
19	`        self.search = SearchWidget(page)`	Stores the instance attribute *self.search* on the object (available to every method). *SearchWidget(...)* creates a SearchWidget object (constructor call).
20	`        self.check_in = self.search.check_in`	Stores the instance attribute *self.check_in* on the object (available to every method).
21	`        self.check_out = self.search.check_out`	Stores the instance attribute *self.check_out* on the object (available to every method).
22	`        self.branch_select = self.search.branch`	Stores the instance attribute *self.branch_select* on the object (available to every method).
23	`        self.guests = self.search.guests`	Stores the instance attribute *self.guests* on the object (available to every method).
24	`        self.filters = self.main.get_by_role("complementary").first # the <aside> filter panel`	Stores the instance attribute *self.filters* on the object (available to every method). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Inline comment: “the <aside> filter panel”.
25	`        self.price_slider = self.main.get_by_role("slider")`	Instance locator *self.price_slider*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
26	`        self.clear_filters = self.main.get_by_role("button", name=re.compile("Clear (all )?filters", re.I))`	Instance locator *self.clear_filters*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
27	`        self.results_heading = self.main.get_by_role("heading", name=COUNT_RE)`	Instance locator *self.results_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
28	`        self.empty_state = self.main.get_by_text(re.compile("No rooms match your filters"))`	Instance locator *self.empty_state*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
29	`        self.view_details_links = self.main.get_by_role("link", name="View details")`	Instance locator *self.view_details_links*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
30	`        self.select_room_links = self.main.get_by_role("link", name="Select room")`	Instance locator *self.select_room_links*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
31	`        # mobile filter drawer`	**Comment** (# ... ignored by Python): documents intent for the reader.
32	`        self.filter_button = self.main.get_by_role("button", name="Filter", exact=True)`	Instance locator *self.filter_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
33	`        self.filter_drawer = page.get_by_role("dialog", name="Filters")`	Instance locator *self.filter_drawer*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
34	`        self.show_rooms_button = self.filter_drawer.get_by_role(`	Instance locator *self.show_rooms_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
35	`            "button", name=re.compile(r"^Show \d+ rooms?")`	**Continuation** of the statement started on line 34 (inside brackets). Keyword argument(s) name (name=value): self-documenting and order-independent. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

Line	Code	Syntax & why
36	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
38	`    # ---------- navigation ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
39	`    def open(self, branch_id: str | None = None, adults: int | None = None, **params):`	**def** defines the method *open* of class RoomsPage; *self* is the instance. Parameters: *branch_id*; *adults*. ****params** collects any extra keyword arguments into a dict. Parameters with '= value' are optional (default used when omitted).
40	`        query = {`	**Assignment**: binds the name *query* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...).
41	`            k: v for k, v in {"branchId": branch_id, "adults": adults, **params}.items() if v is not None`	**Continuation** of the statement started on line 40 (inside brackets). *items()* key/value pairs of a dict.
42	`        }`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
43	`        path = self.path + ("?" + "&".join(f"{k}={v}" for k, v in query.items()) if query else "")`	**Assignment**: binds the name *path* to the value on the right. *join()* joins strings with a separator. *items()* key/value pairs of a dict. Uses an **f-string** (f"...{expr}...") to build text with values.
44	`        super().open(path)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *open()* opens a file (read/write). *super()* calls the parent class (inheritance).
45	`        expect(self.results_heading).to_be_visible(timeout=20000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
46	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
48	`    # ---------- reading state ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
49	`    def room_count(self) -> int:`	**def** defines the method *room_count* of class RoomsPage; *self* is the instance. Returns int.
50	`        expect(self.results_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
51	`        return int(COUNT_RE.search(self.results_heading.inner_text()).group(1))`	**return**: ends the function and hands back *int(COUNT_RE.search(self.results_heading.inner_text()).group(1))*. *int()* converts to integer. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. *inner_text()* visible text of the element (as rendered, respects CSS).
53	`    def heading_branch(self) -> str:`	**def** defines the method *heading_branch* of class RoomsPage; *self* is the instance. Returns str.
54	`        return COUNT_RE.search(self.results_heading.inner_text()).group(2).strip()`	**return**: ends the function and hands back *COUNT_RE.search(self.results_heading.inner_text()).group(2).strip()*. *strip()* removes surrounding whitespace. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. *inner_text()* visible text of the element (as rendered, respects CSS).
56	`    def wait_for_count(self, predicate, timeout_s: float = 6.0) -> int:`	**def** defines the method *wait_for_count* of class RoomsPage; *self* is the instance. Parameters: *predicate*; *timeout_s*. Parameters with '= value' are optional (default used when omitted). Returns int.
57	`        """Poll until predicate(count) is True (results re-render async after filters)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
58	`        deadline = time.monotonic() + timeout_s`	**Assignment**: binds the name *deadline* to the value on the right. *monotonic()* monotonic clock for timeouts.

Line	Code	Syntax & why
59	`        count = self.room_count()`	**Assignment**: binds the name *count* to the value on the right. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
60	`        while not predicate(count) and time.monotonic() < deadline:`	**while** loop: repeats while *not predicate(count) and time.monotonic() < deadline* is true (bounded polling). *monotonic()* monotonic clock for timeouts.
61	`            self.page.wait_for_timeout(200)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
62	`            count = self.room_count()`	**Assignment**: binds the name *count* to the value on the right. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
63	`        return count`	**return**: ends the function and hands back *count*.
65	`    def card_titles(self) -> list[str]:`	**def** defines the method *card_titles* of class RoomsPage; *self* is the instance. Returns list[str].
66	`        """Room-name links (same href as View details, but the visible room name)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
67	`        names = []`	**Assignment**: binds the name *names* to the value on the right. The value is a **list** literal [...].
68	`        for link in self.view_details_links.all():`	**for** loop: repeats the indented block for every item, binding it to *link* (iterating *self.view_details_links.all()*). *all()* returns a list of Locators, one per current match, for iteration.
69	`            href = link.get_attribute("href")`	**Assignment**: binds the name *href* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type).
70	`            title = self.main.locator(f'a[href="{href}"]').first.inner_text().strip()`	**Assignment**: binds the name *title* to the value on the right. *strip()* removes surrounding whitespace. *inner_text()* visible text of the element (as rendered, respects CSS). *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath). Uses an **f-string** (f"...{expr}...") to build text with values.
71	`            names.append(title)`	**Method call** executed for its effect. *append()* adds an item to a list.
72	`        return names`	**return**: ends the function and hands back *names*.
74	`    def card_texts(self) -> list[str]:`	**def** defines the method *card_texts* of class RoomsPage; *self* is the instance. Returns list[str].
75	`        out = []`	**Assignment**: binds the name *out* to the value on the right. The value is a **list** literal [...].
76	`        for link in self.view_details_links.all():`	**for** loop: repeats the indented block for every item, binding it to *link* (iterating *self.view_details_links.all()*). *all()* returns a list of Locators, one per current match, for iteration.
77	`            out.append(`	**Method call** executed for its effect. *append()* adds an item to a list.
78	`                link.evaluate(`	**Continuation** of the statement started on line 77 (inside brackets). *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
79	`                    """a => { let n = a; while (n.parentElement && !/\\/night/.test(n.innerText)) n = n.parentElement;`	**Continuation** of the statement started on line 77 (inside brackets).
80	`                           return n.innerText; }"""`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Rendered text of the element.
81	`                )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
82	`            )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.

Line	Code	Syntax & why
83	`        return out`	**return**: ends the function and hands back *out*.
85	`    # ---------- actions ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
86	`    def set_max_price(self, value: int) -> None:`	**def** defines the method *set_max_price* of class RoomsPage; *self* is the instance. Parameters: *value*. Returns None.
87	`        self.price_slider.fill(str(value)) # fill() works on <input type=range>`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *str()* converts to string. Inline comment: “fill() works on <input type=range>”.
89	`    def slider_bounds(self) -> tuple[int, int]:`	**def** defines the method *slider_bounds* of class RoomsPage; *self* is the instance. Returns tuple[int, int].
90	`        return (`	**return**: ends the function and hands back *(int(float(self.price_slider.get_attribute('min'))), int(float(self.p….*
91	`            int(float(self.price_slider.get_attribute("min"))),`	**Continuation** of the statement started on line 90 (inside brackets). *int()* converts to integer. *float()* converts to float. *get_attribute()* reads an HTML attribute (href, min, max, alt, type).
92	`            int(float(self.price_slider.get_attribute("max"))),`	**Continuation** of the statement started on line 90 (inside brackets). *int()* converts to integer. *float()* converts to float. *get_attribute()* reads an HTML attribute (href, min, max, alt, type).
93	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
95	`    def select_branch(self, value: str) -> None:`	**def** defines the method *select_branch* of class RoomsPage; *self* is the instance. Parameters: *value*. Returns None.
96	`        self.branch_select.select_option(value)`	**Method call** executed for its effect. *select_option()* chooses an <option> in a <select> by value or label.
98	`    def filter_checkbox(self, name: str) -> Locator:`	**def** defines the method *filter_checkbox* of class RoomsPage; *self* is the instance. Parameters: *name*. Returns Locator.
99	`        return self.filters.get_by_role("checkbox", name=name, exact=True)`	**return**: ends the function and hands back *self.filters.get_by_role('checkbox', name=name, exact=True)*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
101	`    def filter_options(self, group: str) -> list[str]:`	**def** defines the method *filter_options* of class RoomsPage; *self* is the instance. Parameters: *group*. Returns list[str].
102	`        """Labels of the checkboxes under a filter heading ('Room type', 'Bed type', 'Amenities')."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
103	`        heading = self.filters.get_by_role("heading", name=group, exact=True)`	**Assignment**: binds the name *heading* to the value on the right. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
104	`        return heading.evaluate(`	**return**: ends the function and hands back *heading.evaluate("h => { const out = []; let n = h.nextElementSibling…. evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
105	`            """h => { const out = []; let n = h.nextElementSibling;`	**Continuation** of the statement started on line 104 (inside brackets).
106	`                    while (n && !/^H[1-6]$/.test(n.tagName)) {`	**JavaScript** line inside a Python string, executed in the browser by evaluate().

Line	Code	Syntax & why
107	`n.querySelectorAll('label').forEach(l => out.push(l.innerText.trim()));`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Selects all matching DOM elements. Rendered text of the element.
108	`if (n.tagName === 'LABEL') out.push(n.innerText.trim());`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Rendered text of the element.
109	`n = n.nextElementSibling; }`	**JavaScript** line inside a Python string, executed in the browser by evaluate().
110	`return out; }"""`	**JavaScript** line inside a Python string, executed in the browser by evaluate().
111	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
113	`def toggle(self, name: str, on: bool = True) -> None:`	**def** defines the method *toggle* of class RoomsPage; *self* is the instance. Parameters: *name*; *on*. Parameters with '= value' are optional (default used when omitted). Returns None.
114	`box = self.filter_checkbox(name)`	**Assignment**: binds the name *box* to the value on the right. *filter_checkbox()*: project method RoomsPage.filter_checkbox() in pages/rooms_page.py.
115	`if on:`	**if** statement: the indented block runs only when *on* is true.
116	`box.check()`	**Method call** executed for its effect. *check()* ticks a checkbox (no-op if already ticked).
117	`else:`	**else**: runs when the if-condition was false.
118	`box.uncheck()`	**Method call** executed for its effect. *uncheck()* unticks a checkbox.
120	`def open_first_room(self):`	**def** defines the method *open_first_room* of class RoomsPage; *self* is the instance.
121	`from pages.room_detail_page import RoomDetailPage # local import avoids circular import`	**from pages.room_detail_page import RoomDetailPage**: imports specific names so they can be used without the module prefix. Page Object for a single room's detail page. Inline comment: “local import avoids circular import”.
123	`self.view_details_links.first.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
124	`return RoomDetailPage(self.page)`	**return**: ends the function and hands back *RoomDetailPage(self.page)*. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).

pages/room_detail_page.py

Page Object for a room detail page: facts, steppers, dates, price breakdown and the not-found state. naira() converts '₦56,250' to an int so totals can be compared numerically.

Line	Code	Syntax & why
1	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
3	`from playwright.sync_api import Page, expect`	**from playwright.sync_api import Page, expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
5	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.

Line	Code	Syntax & why
7	`MONEY_RE = re.compile(r"₦([\d,]+)")`	Module-level **constant** *MONEY_RE* (UPPER_CASE by convention = not changed at runtime). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
10	`def naira(text: str) -> int:`	**def** defines the function *naira*. Parameters: *text*. Returns int.
11	`    """'₦56,250' -> 56250 (first amount in the string)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
12	`    return int(MONEY_RE.search(text).group(1).replace(",", ""))`	**return**: ends the function and hands back *int(MONEY_RE.search(text).group(1).replace(',', ''))*. *int()* converts to integer. *replace()* string replace. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string.
15	`class RoomDetailPage(BasePage):`	**class RoomDetailPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
16	`    path = "/rooms/" # real URL is /rooms/<uuid>?checkIn=..&checkOut=..&branchId=..`	**Assignment**: binds the name *path* to the value on the right. Inline comment: “real URL is /rooms/<uuid>?checkIn=..&checkOut=..&branchId=..”.
18	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
19	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
20	`        self.title = self.main.get_by_role("heading", level=1)`	Instance locator *self.title*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
21	`        self.not_found = self.main.get_by_role("heading", name="Room not found")`	Instance locator *self.not_found*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
22	`        self.back_to_rooms = self.main.get_by_role("link", name="Back to rooms")`	Instance locator *self.back_to_rooms*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
23	`        self.breadcrumb = self.main.get_by_role("navigation", name="Breadcrumb")`	Instance locator *self.breadcrumb*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
24	`        self.check_in = self.main.locator("input[type=date]").nth(0)`	Instance locator *self.check_in*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *nth()* the n-th (0-based) matching element. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

Line	Code	Syntax & why
25	`self.check_out = self.main.locator("input[type=date]").nth(1)`	Instance locator *self.check_out*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *nth()* the n-th (0-based) matching element. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
26	`self.price_per_night = self.main.get_by_text(re.compile(r"^₦[\d,]+ per night$"))`	Instance locator *self.price_per_night*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
27	`self.total = self.main.get_by_text(re.compile(r"Total\s*₦[\d,]+")).first`	Stores the instance attribute *self.total* on the object (available to every method). *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
28	`self.unpaid_notice = self.main.get_by_text("This booking is unpaid – pay at the hotel.")`	Instance locator *self.unpaid_notice*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
29	`self.continue_button = self.main.get_by_role("button", name="Continue to guest details")`	Instance locator *self.continue_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
30	`self.amenities_heading = self.main.get_by_role("heading", name="What is in the room")`	Instance locator *self.amenities_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
31	`self.house_rules_heading = self.main.get_by_role("heading", name="House rules")`	Instance locator *self.house_rules_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
32	`self.policy_link = self.main.get_by_role("link", name=re.compile("Read the full policy"))`	Instance locator *self.policy_link*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
33	`self.other_rooms_heading = self.main.get_by_role("heading", name=re.compile(r"^Other rooms"))`	Instance locator *self.other_rooms_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
34	`# links listed under 'Other rooms' (each link text is 'Name ₦price / night')`	**Comment** (# ... ignored by Python): documents intent for the reader.

Line	Code	Syntax & why
35	`        self.other_room_links = self.main.get_by_role(`	Instance locator *self.other_room_links*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
36	`            "link", name=re.compile(r"₦[\d,]+ \/ night$")`	**Continuation** of the statement started on line 35 (inside brackets). Keyword argument(s) name (name=value): self-documenting and order-independent. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
37	`        ) # "\/": Playwright serialises regex as /.../`	**Continuation** of the statement started on line 35 (inside brackets). Inline comment: “"\/": Playwright serialises regex as /.../”.
38	`        self.validation_error = self.main.get_by_text(`	Instance locator *self.validation_error*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
39	`            re.compile(`	**Continuation** of the statement started on line 38 (inside brackets). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
40	`                r"(check-?out|departure).*(after|later)|at least 1 night|in the past|maximum|cannot", re.I`	**Continuation** of the statement started on line 38 (inside brackets). Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
41	`            )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
42	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
44	`    def open_room(self, room_id: str, **params):`	**def** defines the method *open_room* of class RoomDetailPage; *self* is the instance. Parameters: *room_id*. ****params** collects any extra keyword arguments into a dict.
45	`        query = "&".join(f"{k}={v}" for k, v in params.items() if v is not None)`	**Assignment**: binds the name *query* to the value on the right. *join()* joins strings with a separator. *items()* key/value pairs of a dict. Uses an **f-string** (f"...{expr}...") to build text with values.
46	`        return self.open(f"/rooms/{room_id}" + (f"?{query}" if query else ""))`	**return**: ends the function and hands back *self.open(f'/rooms/{room_id}' + (f'?{query}' if query else ''))*. *open()* opens a file (read/write). Uses an **f-string** (f"...{expr}...") to build text with values.
48	`    def expect_loaded(self) -> None:`	**def** defines the method *expect_loaded* of class RoomDetailPage; *self* is the instance. Returns None.
49	`        expect(self.page).to_have_url(re.compile(r"/rooms/[0-9a-f-]{36}"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
50	`        expect(self.continue_button).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
52	`    def stepper(self, who: str, direction: str):`	**def** defines the method *stepper* of class RoomDetailPage; *self* is the instance. Parameters: *who*; *direction*.
53	`        """who: 'Adults'|'Children'; direction: 'Increase'|'Decrease'."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.

Line	Code	Syntax & why
54	`        return self.main.get_by_role("button", name=f"{direction} {who}")`	**return**: ends the function and hands back *self.main.get_by_role('button', name=f'{direction} {who}')*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Uses an **f-string** (f"...{expr}...") to build text with values.
56	`    def stepper_value(self, who: str) -> int:`	**def** defines the method *stepper_value* of class RoomDetailPage; *self* is the instance. Parameters: *who*. Returns int.
57	`        """Number shown between the -/+ buttons for 'Adults' or 'Children'."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
58	`        text = self.stepper(who, "Decrease").evaluate("b => b.parentElement.innerText")`	**Assignment**: binds the name *text* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry). *stepper()*: project method RoomDetailPage.stepper() in pages/room_detail_page.py - who: 'Adults'|'Children'; direction: 'Increase'|'Decrease'..
59	`        return int(re.search(r"\d+", text).group(0))`	**return**: ends the function and hands back *int(re.search('\\d+', text).group(0))*. *int()* converts to integer. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
61	`    def fact(self, label: str) -> str:`	**def** defines the method *fact* of class RoomDetailPage; *self* is the instance. Parameters: *label*. Returns str.
62	`        """Value shown under a fact label such as 'Bed', 'Room size', 'Guest capacity'."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
63	`        return self.main.get_by_text(label, exact=True).first.evaluate(`	**return**: ends the function and hands back *self.main.get_by_text(label, exact=True).first.evaluate("e => (e.next…*. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry). *get_by_text()* finds elements by their visible text (exact string or regex).
64	`            "e => (e.nextElementSibling && e.nextElementSibling.innerText || '').trim()"`	**Continuation** of the statement started on line 63 (inside brackets).
65	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
67	`    def price_block_text(self) -> str:`	**def** defines the method *price_block_text* of class RoomDetailPage; *self* is the instance. Returns str.
68	`        return self.total.evaluate(`	**return**: ends the function and hands back *self.total.evaluate("e => { let n = e; while (n.parentElement && !/Ro…*. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
69	`            "e => { let n = e; while (n.parentElement && !/Room ·/.test(n.innerText)) n = n.parentElement;"`	**Continuation** of the statement started on line 68 (inside brackets).
70	`            " return n.innerText.slice(n.innerText.indexOf('Room ·')); }"`	**Continuation** of the statement started on line 68 (inside brackets).
71	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
73	`    def total_amount(self) -> int:`	**def** defines the method *total_amount* of class RoomDetailPage; *self* is the instance. Returns int.

Line	Code	Syntax & why
74	`        return naira(re.search(r"Total\s*(₦[\d,]+)", self.price_block_text()).group(1))`	**return**: ends the function and hands back *naira(re.search('Total\\s*(₦[\\d,]+)', self.price_block_text()).group…*. *naira()*: project function naira() in pages/room_detail_page.py - '₦56,250' -> 56250 (first amount in the string).. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. *price_block_text()*: project method RoomDetailPage.price_block_text() in pages/room_detail_page.py. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
76	`    def set_dates(self, check_in: str, check_out: str) -> None:`	**def** defines the method *set_dates* of class RoomDetailPage; *self* is the instance. Parameters: *check_in*; *check_out*. Returns None.
77	`        self.check_in.fill(check_in)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
78	`        self.check_out.fill(check_out)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
80	`    def continue_to_guest_details(self):`	**def** defines the method *continue_to_guest_details* of class RoomDetailPage; *self* is the instance.
81	`        from pages.booking_page import BookingPage`	**from pages.booking_page import BookingPage**: imports specific names so they can be used without the module prefix. Page Object for the booking wizard (/booking).
83	`        self.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
84	`        return BookingPage(self.page)`	**return**: ends the function and hands back *BookingPage(self.page)*. *BookingPage(...)* creates a BookingPage object (constructor call).

pages/booking_page.py

Page Object for the booking wizard (guest details -> review -> confirmed), including every validation message, so tests read like the workbook steps.

Line	Code	Syntax & why
1	`"""Booking wizard: 1 Your stay > 2 Details > 3 Review & book > 4 Confirmed (all on /booking)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
5	`from playwright.sync_api import Page, expect`	**from playwright.sync_api import Page, expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
7	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
8	`from utils.data_factory import GuestData`	**from utils.data_factory import GuestData**: imports specific names so they can be used without the module prefix. Builders for unique test data (users, guests).
11	`class BookingPage(BasePage):`	**class BookingPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
12	`    path = "/booking"`	**Assignment**: binds the name *path* to the value on the right.

Line	Code	Syntax & why
14	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
15	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
16	`        self.steps = self.main.get_by_text(re.compile(r"Your stay.*Details.*Review & book.*Confirmed"))`	Instance locator *self.steps*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
17	`        # step 2 - guest details`	**Comment** (# ... ignored by Python): documents intent for the reader.
18	`        self.heading = self.main.get_by_role("heading", name="Who is staying?")`	Instance locator *self.heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
19	`        self.guest_name = self.main.get_by_role("textbox", name="John Doe")`	Instance locator *self.guest_name*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
20	`        self.guest_email = self.main.get_by_role("textbox", name="alex.morgan@example.com")`	Instance locator *self.guest_email*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
21	`        self.guest_phone = self.main.get_by_role("textbox", name="+234 800 000 0000")`	Instance locator *self.guest_phone*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
22	`        self.notes = self.main.get_by_role("textbox", name=re.compile("Late arrival"))`	Instance locator *self.notes*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
23	`        self.consent = self.main.get_by_role("checkbox", name=re.compile("I agree to this hotel holding"))`	Instance locator *self.consent*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

Line	Code	Syntax & why
24	`self.privacy_link = self.main.get_by_role("link", name="privacy policy")`	Instance locator *self.privacy_link*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
25	`self.back_to_room = self.main.get_by_role("button", name="Back to the room")`	Instance locator *self.back_to_room*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
26	`self.review_button = self.main.get_by_role("button", name="Review my booking")`	Instance locator *self.review_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
27	`self.name_error = self.main.get_by_text("Enter the guest's full name")`	Instance locator *self.name_error*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
28	`self.email_error = self.main.get_by_text("Enter a valid email")`	Instance locator *self.email_error*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
29	`self.phone_error = self.main.get_by_text("Enter a valid phone number")`	Instance locator *self.phone_error*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
30	`self.consent_error = self.main.get_by_text("You must agree before continuing")`	Instance locator *self.consent_error*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
31	`self.name_format_error = self.main.get_by_text(`	Instance locator *self.name_format_error*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
32	`re.compile(r"(only )?letters|valid (full )?name", re.I)`	**Continuation** of the statement started on line 31 (inside brackets). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
33	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
34	`# step 3 - review`	**Comment** (# ... ignored by Python): documents intent for the reader.
35	`self.review_heading = self.main.get_by_role("heading", name="Check it over")`	Instance locator *self.review_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
36	`        self.change_room = self.main.get_by_role("button", name="Change room or dates")`	Instance locator *self.change_room*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
37	`        self.edit_details = self.main.get_by_role("button", name="Edit details")`	Instance locator *self.edit_details*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
38	`        self.back_to_details = self.main.get_by_role("button", name="Back to guest details")`	Instance locator *self.back_to_details*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
39	`        self.confirm_button = self.main.get_by_role("button", name="Confirm booking")`	Instance locator *self.confirm_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
40	`        self.unpaid_review_note = self.main.get_by_text(re.compile("This booking is unpaid"))`	Instance locator *self.unpaid_review_note*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
41	`        # step 4`	**Comment** (# ... ignored by Python): documents intent for the reader.
42	`        self.confirmed_heading = self.main.get_by_role("heading", name=re.compile("confirmed|booked", re.I))`	Instance locator *self.confirmed_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
43	`        self.reference = self.main.get_by_text(re.compile(r"HMS-\d{4}-\d+"))`	Instance locator *self.reference*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
44	`        # sidebar summary`	**Comment** (# ... ignored by Python): documents intent for the reader.
45	`        self.total = self.main.get_by_text(re.compile(r"Total\s*₦[\d,]+")).last`	Stores the instance attribute *self.total* on the object (available to every method). *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
47	`    def expect_loaded(self) -> None:`	**def** defines the method *expect_loaded* of class BookingPage; *self* is the instance. Returns None.
48	`        expect(self.heading).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.

Line	Code	Syntax & why
50	`    def fill_guest(self, guest: GuestData) -> "BookingPage":`	**def** defines the method *fill_guest* of class BookingPage; *self* is the instance. Parameters: *guest*. Returns 'BookingPage'.
51	`        self.guest_name.fill(guest.full_name)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
52	`        self.guest_email.fill(guest.email)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
53	`        self.guest_phone.fill(guest.phone)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
54	`        self.notes.fill(guest.notes)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
55	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
57	`    def accept_consent(self) -> "BookingPage":`	**def** defines the method *accept_consent* of class BookingPage; *self* is the instance. Returns 'BookingPage'.
58	`        self.consent.check()`	**Method call** executed for its effect. *check()* ticks a checkbox (no-op if already ticked).
59	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
61	`    def go_to_review(self, guest: GuestData) -> "BookingPage":`	**def** defines the method *go_to_review* of class BookingPage; *self* is the instance. Parameters: *guest*. Returns 'BookingPage'.
62	`        self.fill_guest(guest).accept_consent()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *accept_consent()*: project method BookingPage.accept_consent() in pages/booking_page.py. *fill_guest()*: project method BookingPage.fill_guest() in pages/booking_page.py.
63	`        self.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
64	`        expect(self.review_heading).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
65	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
67	`    def total_amounts(self) -> list[int]:`	**def** defines the method *total_amounts* of class BookingPage; *self* is the instance. Returns list[int].
68	`        """Every 'Total ₦x' figure on the page (review panel + sidebar)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
69	`        texts = self.main.get_by_text(re.compile(r"Total\s*₦[\d,]+")).all_inner_texts()`	**Assignment**: binds the name *texts* to the value on the right. *all_inner_texts()* returns visible text of every match as a list of strings. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
70	`        return [int(m.replace(",", "")) for t in texts for m in re.findall(r"Total\s*₦([\d,]+)", t)]`	**return**: ends the function and hands back *[int(m.replace(',', '')) for t in texts for m in re.findall('Total\\s…. int()* converts to integer. *replace()* string replace. *findall()* returns every regex match. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

pages/auth_modal.py

Page Object for the sign-in / create-account dialog (role choice, sign-in form, forgot password, registration). open_sign_in() retries because a click before hydration is ignored.

Line	Code	Syntax & why
1	`"""The 'Sign in or create an account' dialog (a modal, not a separate page).`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`Flow: Sign in -> pick role (Customer / Agent) -> Next -> 'Continue as customer' form`	**Docstring** continuation (documentation text).
4	`(identifier + password, Forgot password?, Create an account)."""`	**Docstring** continuation (documentation text).
6	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
8	`from playwright.sync_api import Page, expect`	**from playwright.sync_api import Page, expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
10	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
11	`from utils.data_factory import UserData`	**from utils.data_factory import UserData**: imports specific names so they can be used without the module prefix. Builders for unique test data (users, guests).
14	`class AuthModal(BasePage):`	**class AuthModal(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
15	`def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
16	`super().__init__(page) # run BasePage.__init__ first so header etc. exist`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance). Inline comment: “run BasePage.__init__ first so header etc. exist”.
17	`self.dialog = page.get_by_role("dialog", name="Sign in or create an account")`	Instance locator *self.dialog*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
18	`d = self.dialog`	**Assignment**: binds the name *d* to the value on the right.
19	`self.close_button = d.get_by_role("button", name="Close", exact=True)`	Instance locator *self.close_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
20	`# step 1 - role`	**Comment** (# ... ignored by Python): documents intent for the reader.
21	`self.role_heading = d.get_by_text("How would you like to use the platform?")`	Instance locator *self.role_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).

Line	Code	Syntax & why
22	`self.customer_role = d.get_by_role("button", name=re.compile(r"^Customer"))`	Instance locator *self.customer_role*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
23	`self.agent_role = d.get_by_role("button", name=re.compile(r"^Agent"))`	Instance locator *self.agent_role*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
24	`self.next_button = d.get_by_role("button", name="Next", exact=True)`	Instance locator *self.next_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
25	`# step 2 - sign in`	**Comment** (# ... ignored by Python): documents intent for the reader.
26	`self.signin_heading = d.get_by_role("heading", name="Continue as customer")`	Instance locator *self.signin_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
27	`self.identifier = d.get_by_role("textbox", name="johndoe@gmail.com")`	Instance locator *self.identifier*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
28	`self.login_password = d.get_by_role("textbox", name="Enter password")`	Instance locator *self.login_password*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
29	`self.continue_button = d.get_by_role("button", name=re.compile(r"^Continue"))`	Instance locator *self.continue_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
30	`self.forgot_password = d.get_by_role("button", name="Forgot password?")`	Instance locator *self.forgot_password*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
31	`self.invalid_credentials = d.get_by_text("Invalid credentials")`	Instance locator *self.invalid_credentials*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
32	`self.required_errors = d.get_by_text("Required", exact=True)`	Instance locator *self.required_errors*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
33	`self.identifier_error = d.get_by_text("Enter a valid email or phone number")`	Instance locator *self.identifier_error*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
34	`# forgot password`	**Comment** (# ... ignored by Python): documents intent for the reader.
35	`self.reset_heading = d.get_by_role("heading", name="Reset your password")`	Instance locator *self.reset_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
36	`self.reset_email = d.get_by_role("textbox", name="johndoe@gmail.com")`	Instance locator *self.reset_email*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
37	`self.send_reset_code = d.get_by_role("button", name=re.compile("Send reset code"))`	Instance locator *self.send_reset_code*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
38	`self.back_to_sign_in = d.get_by_role("button", name="Back to sign in")`	Instance locator *self.back_to_sign_in*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
39	`# registration`	**Comment** (# ... ignored by Python): documents intent for the reader.
40	`self.create_account_button = d.get_by_role("button", name="Create an account")`	Instance locator *self.create_account_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
41	`self.register_heading = d.get_by_role("heading", name="Create a customer account")`	Instance locator *self.register_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
42	`        self.full_name = d.get_by_role("textbox", name="John Doe")`	Instance locator *self.full_name*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
43	`        self.email = d.get_by_role("textbox", name="johndoe@gmail.com")`	Instance locator *self.email*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
44	`        self.country_code = d.get_by_role("combobox", name="Country code")`	Instance locator *self.country_code*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
45	`        self.phone = d.get_by_role("textbox", name="8012345678")`	Instance locator *self.phone*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
46	`        self.password = d.get_by_role("textbox", name="Choose a strong password")`	Instance locator *self.password*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
47	`        self.confirm_password = d.get_by_role("textbox", name="Confirm password")`	Instance locator *self.confirm_password*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
48	`        self.toggle_password = d.get_by_role("button", name="Toggle password visibility")`	Instance locator *self.toggle_password*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
49	`        self.submit = d.get_by_role("button", name="Create account →")`	Instance locator *self.submit*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
51	`    # ---------- open ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
52	`    def open_sign_in(self) -> "AuthModal":`	**def** defines the method *open_sign_in* of class AuthModal; *self* is the instance. Returns 'AuthModal'.
53	`        # retry: a click that lands before React hydration does nothing`	**Comment** (# ... ignored by Python): documents intent for the reader.
54	`        for _ in range(5):`	**for** loop: repeats the indented block for every item, binding it to _ (iterating *range(5)*). *range()* sequence of integers.
55	`            self.sign_in_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
56	`            try:`	**try**: runs the block and lets the except clause handle expected errors instead of crashing.

Line	Code	Syntax & why
57	`                self.dialog.wait_for(state="visible", timeout=2000)`	**Method call** executed for its effect. *wait_for()* waits until the element reaches a state (visible/hidden/attached).
58	`                break`	**break**: leaves the loop immediately.
59	`            except Exception: # noqa: BLE001 - not hydrated yet, click again`	**except**: handles the listed error type raised in the try block (here: an expected timeout or missing optional dependency). Inline comment: “noqa: BLE001 - not hydrated yet, click again”.
60	`                continue`	**continue**: skips to the next loop iteration.
61	`        expect(self.dialog).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
62	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
64	`    def open_customer_form(self) -> "AuthModal":`	**def** defines the method *open_customer_form* of class AuthModal; *self* is the instance. Returns 'AuthModal'.
65	`        """Role choice is remembered per browser, so step 1 may be skipped on later visits."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
66	`        self.open_sign_in()`	**Method call** executed for its effect. *open_sign_in()*: project method AuthModal.open_sign_in() in pages/auth_modal.py.
67	`        if self.customer_role.is_visible():`	**if** statement: the indented block runs only when *self.customer_role.is_visible()* is true. *is_visible()* True/False right now (does not wait) - used for branching, never for asserting.
68	`            self.customer_role.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
69	`            self.next_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
70	`        expect(self.signin_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
71	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
73	`    def open_registration_form(self) -> "AuthModal":`	**def** defines the method *open_registration_form* of class AuthModal; *self* is the instance. Returns 'AuthModal'.
74	`        self.open_sign_in()`	**Method call** executed for its effect. *open_sign_in()*: project method AuthModal.open_sign_in() in pages/auth_modal.py.
75	`        self.create_account_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
76	`        expect(self.register_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
77	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
79	`    # ---------- actions ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
80	`    def sign_in(self, identifier: str, password: str) -> str:`	**def** defines the method *sign_in* of class AuthModal; *self* is the instance. Parameters: *identifier*; *password*. Returns str.

Line	Code	Syntax & why
81	`"""Sign in as a customer and return the name shown in the header."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
82	`self.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
83	`self.identifier.fill(identifier)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
84	`self.login_password.fill(password)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
85	`self.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
86	`expect(self.dialog).to_be_hidden(timeout=20000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_hidden()* waits until the element is hidden or detached.
87	`expect(self.sign_in_button).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
88	`return (`	**return**: ends the function and hands back *self.header.get_by_role('link', name=re.compile('\\w')).filter(has_no…*.
89	`self.header.get_by_role("link", name=re.compile(r"\w"))`	**Continuation** of the statement started on line 88 (inside brackets). Keyword argument(s) name (name=value): self-documenting and order-independent. *inner_text()* visible text of the element (as rendered, respects CSS). *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
90	`.filter(`	**Continuation** of the statement started on line 88 (inside brackets).
91	`has_not_text=re.compile(`	**Continuation** of the statement started on line 88 (inside brackets). Keyword argument(s) has_not_text (name=value): self-documenting and order-independent. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
92	`r"^(Home|Rooms|Branches|Gallery|About|Contact|Book now|Oneflare Grand Plaza)$"`	**Continuation** of the statement started on line 88 (inside brackets). Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
93	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
94	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
95	`.last.inner_text()`	**Continuation** of the statement started on line 88 (inside brackets).
96	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
98	`def fill_registration(self, user: UserData, confirm: str | None = None) -> "AuthModal":`	**def** defines the method *fill_registration* of class AuthModal; *self* is the instance. Parameters: *user*; *confirm*. Parameters with '= value' are optional (default used when omitted). Returns 'AuthModal'.

Line	Code	Syntax & why
99	`        self.full_name.fill(user.full_name)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
100	`        self.email.fill(user.email)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
101	`        self.phone.fill(re.sub(r"^\+234", "", user.phone)) # country code is a separate select (+234 default)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *sub()* regex replace. Contains a **regular expression** (r"..." raw string so backslashes are kept literally). Inline comment: “country code is a separate select (+234 default)”.
102	`        self.password.fill(user.password)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
103	`        # `confirm` lets a test pass a DIFFERENT value to check mismatch handling`	**Comment** (# ... ignored by Python): documents intent for the reader.
104	`        self.confirm_password.fill(user.password if confirm is None else confirm)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
105	`        return self`	**return self**: returns the page object itself so calls can be **chained** (Page(page).open().action()).
107	`    def register(self, user: UserData) -> None:`	**def** defines the method *register* of class AuthModal; *self* is the instance. Parameters: *user*. Returns None.
108	`        """Full happy path: open form -> fill -> submit -> wait until logged in."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
109	`        self.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.
110	`        self.fill_registration(user)`	**Method call** executed for its effect. *fill_registration()*: project method AuthModal.fill_registration() in pages/auth_modal.py.
111	`        self.submit.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
112	`        expect(self.dialog).to_be_hidden(timeout=20000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_hidden()* waits until the element is hidden or detached.
113	`        expect(self.user_menu(user.first_name)).to_be_visible() # proof of success`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *user_menu()*: project method BasePage.user_menu() in pages/base_page.py. Inline comment: “proof of success”.

pages/account_page.py

Page Object for /account (signed-out prompt and signed-in view).

Line	Code	Syntax & why
1	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.

Line	Code	Syntax & why
3	`from playwright.sync_api import Page, expect`	**from playwright.sync_api import Page, expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
5	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
8	`class AccountPage(BasePage):`	**class AccountPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
9	`    path = "/account"`	**Assignment**: binds the name *path* to the value on the right.
11	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
12	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
13	`        # signed out`	**Comment** (# ... ignored by Python): documents intent for the reader.
14	`        self.signed_out_heading = self.main.get_by_role("heading", name="Sign in to view your account")`	Instance locator *self.signed_out_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
15	`        self.signed_out_copy = self.main.get_by_text(re.compile("Manage your bookings"))`	Instance locator *self.signed_out_copy*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
16	`        self.signed_out_sign_in = self.main.get_by_role("button", name="Sign in")`	Instance locator *self.signed_out_sign_in*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
17	`        self.find_by_reference = self.main.get_by_role("link", name="Find your booking by reference")`	Instance locator *self.find_by_reference*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
18	`        # signed in`	**Comment** (# ... ignored by Python): documents intent for the reader.
19	`        self.heading = self.main.get_by_role("heading", level=1)`	Instance locator *self.heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
20	`        self.breadcrumb = self.main.get_by_role("navigation", name="Breadcrumb")`	Instance locator *self.breadcrumb*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
21	`        self.booking_history = self.main.get_by_text(`	Instance locator *self.booking_history*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
22	`            re.compile("Booking history|My bookings|Your bookings", re.I)`	**Continuation** of the statement started on line 21 (inside brackets). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
23	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
24	`        self.sign_out = page.get_by_role("button", name=re.compile("Sign out|Log out", re.I))`	Instance locator *self.sign_out*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
26	`    def expect_signed_in(self) -> None:`	**def** defines the method *expect_signed_in* of class AccountPage; *self* is the instance. Returns None.
27	`        expect(self.signed_out_heading).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
28	`        expect(self.heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.

pages/manage_booking_page.py

Page Object for /manage-booking: lookup by booking reference + email.

Line	Code	Syntax & why
1	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
3	`from playwright.sync_api import Page`	**from playwright.sync_api import Page**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint.
5	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
8	`class ManageBookingPage(BasePage):`	**class ManageBookingPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
9	`    path = "/manage-booking"`	**Assignment**: binds the name *path* to the value on the right.

Line	Code	Syntax & why
11	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
12	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
13	`        self.heading = self.main.get_by_role("heading", name="Find your booking")`	Instance locator *self.heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
14	`        self.reference = self.main.get_by_role("textbox", name=re.compile(r"HMS-\d{4}"))`	Instance locator *self.reference*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
15	`        self.email = self.main.get_by_role("textbox", name="johndoe@gmail.com")`	Instance locator *self.email*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
16	`        self.find_button = self.main.get_by_role("button", name="Find my booking")`	Instance locator *self.find_button*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
17	`        self.sign_in_link = self.main.get_by_role("link", name="Sign in to your account")`	Instance locator *self.sign_in_link*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
18	`        self.not_found = self.main.get_by_text(`	Instance locator *self.not_found*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
19	`            re.compile(r"(couldn't|could not|can't|no booking).*(find|found|match)|not found", re.I)`	**Continuation** of the statement started on line 18 (inside brackets). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
20	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
21	`        self.required = self.main.get_by_text(`	Instance locator *self.required*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
22	`            re.compile("Required|Enter (a|your) (valid )?(booking )?reference", re.I)`	**Continuation** of the statement started on line 21 (inside brackets). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

Line	Code	Syntax & why
23	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.

pages/branches_page.py

Page Object for /branches; card() scopes searches to a single branch card.

Line	Code	Syntax & why
1	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
3	`from playwright.sync_api import Locator, Page`	**from playwright.sync_api import Locator, Page**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Locator*: Playwright type describing HOW to find element(s); resolved lazily each time it is used. *Page*: Playwright type for one browser tab; used here as a type hint.
5	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
8	`class BranchesPage(BasePage):`	**class BranchesPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
9	`    path = "/branches"`	**Assignment**: binds the name *path* to the value on the right.
11	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
12	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
13	`        self.heading = self.main.get_by_role("heading", level=1)`	Instance locator *self.heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
14	`        self.branch_headings = self.main.get_by_role("heading", level=2).filter(`	Instance locator *self.branch_headings*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
15	`            has_text="Oneflare Grand Plaza -"`	**Continuation** of the statement started on line 14 (inside brackets). Keyword argument(s) has_text (name=value): self-documenting and order-independent.
16	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.

Line	Code	Syntax & why
17	`        self.see_rooms_links = self.main.get_by_role("link", name=re.compile(r"^See rooms at "))`	Instance locator *self.see_rooms_links*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
18	`        self.contact_links = self.main.get_by_role("link", name="Contact this branch")`	Instance locator *self.contact_links*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
20	`    def card(self, branch_name: str) -> Locator:`	**def** defines the method *card* of class BranchesPage; *self* is the instance. Parameters: *branch_name*. Returns Locator.
21	`        """The smallest element that holds the branch heading AND its 'Contact this branch' link."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
22	`        heading = self.page.get_by_role("heading", name=branch_name, exact=True) # relative to each div`	**Assignment**: binds the name *heading* to the value on the right. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Inline comment: “relative to each div”.
23	`        return (`	**return**: ends the function and hands back *self.main.locator('div, article, section, li').filter(has=heading).fi…*.
24	`            self.main.locator("div, article, section, li")`	**Continuation** of the statement started on line 23 (inside brackets). *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
25	`            .filter(has=heading)`	**Continuation** of the statement started on line 23 (inside brackets). Keyword argument(s) has (name=value): self-documenting and order-independent.
26	`            .filter(has=self.page.get_by_role("link", name="Contact this branch"))`	**Continuation** of the statement started on line 23 (inside brackets). Keyword argument(s) has, name (name=value): self-documenting and order-independent. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
27	`            .last`	**Continuation** of the statement started on line 23 (inside brackets).
28	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
30	`    def see_rooms(self, branch_name: str) -> Locator:`	**def** defines the method *see_rooms* of class BranchesPage; *self* is the instance. Parameters: *branch_name*. Returns Locator.
31	`        return self.main.get_by_role("link", name=f"See rooms at {branch_name}")`	**return**: ends the function and hands back *self.main.get_by_role('link', name=f'See rooms at {branch_name}')*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Uses an **f-string** (f"...{expr}...") to build text with values.

pages/gallery_page.py

Page Object for /gallery: category buttons, location filter and photos.

Line	Code	Syntax & why
1	`from playwright.sync_api import Page`	**from playwright.sync_api import Page**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint.
3	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
6	`class GalleryPage(BasePage):`	**class GalleryPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
7	`    path = "/gallery"`	**Assignment**: binds the name *path* to the value on the right.
9	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
10	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
11	`        self.heading = self.main.get_by_role("heading", level=1)`	Instance locator *self.heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
12	`        self.intro = self.main.get_by_text("Photographs from")`	Instance locator *self.intro*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex).
13	`        self.location_filter = self.main.get_by_role("combobox")`	Instance locator *self.location_filter*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
14	`        # Gallery photos; the CTA background image has an empty alt and is excluded.`	**Comment** (# ... ignored by Python): documents intent for the reader.
15	`        self.photos = self.main.locator("img[alt]:not([alt=''])")`	Instance locator *self.photos*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
17	`    def category(self, name: str):`	**def** defines the method *category* of class GalleryPage; *self* is the instance. Parameters: *name*.
18	`        return self.main.get_by_role("button", name=name, exact=True)`	**return**: ends the function and hands back *self.main.get_by_role('button', name=name, exact=True)*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
20	`    def image(self, alt: str):`	**def** defines the method *image* of class GalleryPage; *self* is the instance. Parameters: *alt*.

Line	Code	Syntax & why
21	`        return self.main.get_by_role("img", name=alt, exact=True)`	**return**: ends the function and hands back *self.main.get_by_role('img', name=alt, exact=True)*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
23	`    def photo_alts(self) -> list[str]:`	**def** defines the method *photo_alts* of class GalleryPage; *self* is the instance. Returns list[str].
24	`        return [a for a in self.photos.evaluate_all("els => els.map(e => e.alt)")]`	**return**: ends the function and hands back *[a for a in self.photos.evaluate_all('els => els.map(e => e.alt)')]*. *evaluate_all()* runs JavaScript on ALL elements matched by a locator at once.

pages/about_page.py

Page Object for /about.

Line	Code	Syntax & why
1	`from playwright.sync_api import Page`	**from playwright.sync_api import Page**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint.
3	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
6	`class AboutPage(BasePage):`	**class AboutPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
7	`    path = "/about"`	**Assignment**: binds the name *path* to the value on the right.
9	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
10	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
11	`        self.heading = self.main.get_by_role("heading", level=1)`	Instance locator *self.heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
12	`        self.principles_heading = self.main.get_by_role("heading", name="What we hold ourselves to")`	Instance locator *self.principles_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
13	`        self.principles = self.main.get_by_role("heading", level=3)`	Instance locator *self.principles*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
14	`        self.locations_heading = self.main.get_by_role("heading", name="The locations")`	Instance locator *self.locations_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
15	`        self.location_links = self.main.get_by_role("link").filter(has_text="Oneflare Grand Plaza -")`	Instance locator *self.location_links*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

pages/contact_page.py

Page Object for /contact: branch tabs, info panel, map and message form.

Line	Code	Syntax & why
1	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
3	`from playwright.sync_api import Page`	**from playwright.sync_api import Page**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *Page*: Playwright type for one browser tab; used here as a type hint.
5	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
8	`class ContactPage(BasePage):`	**class ContactPage(BasePage)**: defines a class that **inherits** from BasePage, so it gets every locator and helper of the parent without repeating code (OOP inheritance).
9	`    path = "/contact"`	**Assignment**: binds the name *path* to the value on the right.
11	`    def __init__(self, page: Page):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
12	`        super().__init__(page)`	**super().__init__(...)**: runs the parent class constructor first so every inherited attribute (header, footer, navigation locators) is set up before this class adds its own. *super()* calls the parent class (inheritance).
13	`        self.heading = self.main.get_by_role("heading", level=1)`	Instance locator *self.heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
14	`        self.map_frame = self.main.locator("iframe")`	Instance locator *self.map_frame*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

Line	Code	Syntax & why
15	`self.info_note = self.main.get_by_text(re.compile("aren't listed for this branch|not listed", re.I))`	Instance locator *self.info_note*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
16	`self.lookup_link = self.main.get_by_role("link", name="look it up with your reference")`	Instance locator *self.lookup_link*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
17	`self.form_heading = self.main.get_by_role("heading", name="Send a message")`	Instance locator *self.form_heading*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
18	`self.routing = self.main.get_by_text(re.compile(r"^Going to "))`	Instance locator *self.routing*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
19	`self.name = self.main.get_by_role("textbox", name="John Doe")`	Instance locator *self.name*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
20	`self.email = self.main.get_by_role("textbox", name="johndoe@gmail.com")`	Instance locator *self.email*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
21	`self.phone = self.main.get_by_role("textbox", name="+234 800 000 0000")`	Instance locator *self.phone*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
22	`self.subject = self.main.get_by_role("textbox", name="General enquiry")`	Instance locator *self.subject*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
23	`self.message = self.main.get_by_role("textbox", name=re.compile("Tell us what you need"))`	Instance locator *self.message*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
24	`self.send = self.main.get_by_role("button", name="Send message")`	Instance locator *self.send*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
25	`        self.privacy_link = self.main.get_by_role("link", name="privacy policy")`	Instance locator *self.privacy_link*. Locators are **lazy**: nothing is searched now; each use re-queries the live DOM, so the page object never holds stale elements. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
27	`    def open_for_branch(self, branch_id: str) -> "ContactPage":`	**def** defines the method *open_for_branch* of class ContactPage; *self* is the instance. Parameters: *branch_id*. Returns 'ContactPage'.
28	`        return self.open(f"{self.path}?branch={branch_id}")`	**return**: ends the function and hands back *self.open(f'{self.path}?branch={branch_id}')*. *open()* opens a file (read/write). Uses an **f-string** (f"...{expr}...") to build text with values.
30	`    def branch_tab(self, city: str):`	**def** defines the method *branch_tab* of class ContactPage; *self* is the instance. Parameters: *city*.
31	`        return self.main.get_by_role("button", name=city, exact=True)`	**return**: ends the function and hands back *self.main.get_by_role('button', name=city, exact=True)*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
33	`    def info_value(self, label: str):`	**def** defines the method *info_value* of class ContactPage; *self* is the instance. Parameters: *label*.
34	`        """Value paragraph that follows a label such as 'Address'/'Phone'/'Email'/'Hours'."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
35	`        return self.main.get_by_text(label, exact=True).locator("xpath=following-sibling::*[1]")`	**return**: ends the function and hands back *self.main.get_by_text(label, exact=True).locator('xpath=following-sib…. locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath). *get_by_text()* finds elements by their visible text (exact string or regex).

tests/__init__.py

Makes tests a package (lets pytest use unique module names).

Empty file. An empty __init__.py marks the folder as a Python package so its modules can be imported with dotted names.

tests/functional/__init__.py

Package marker.

Empty file. An empty __init__.py marks the folder as a Python package so its modules can be imported with dotted names.

tests/functional/test_navigation.py

Global chrome: header links, footer links (HTTP status), footer content, mobile menu, header search, policy pages, branded 404 and SEO titles/descriptions. Workbook: TC-010, 016, 017, 019, 043, 044, 060, 061, 069, 083, 086, 087.

Line	Code	Syntax & why
1	`"""Global header, footer, mobile menu, header search, policy pages and error pages."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.

Line	Code	Syntax & why
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`from datetime import date`	**from datetime import date**: imports specific names so they can be used without the module prefix. date/timedelta give calendar arithmetic (today + 7 days) for check-in/check-out dates. *date*: Calendar date type (no time component).
6	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
9	`from pages.about_page import AboutPage`	**from pages.about_page import AboutPage**: imports specific names so they can be used without the module prefix. Page Object for /about.
10	`from pages.base_page import BasePage`	**from pages.base_page import BasePage**: imports specific names so they can be used without the module prefix. Base Page Object: header/footer/navigation shared by every page.
11	`from pages.branches_page import BranchesPage`	**from pages.branches_page import BranchesPage**: imports specific names so they can be used without the module prefix. Page Object for /branches.
12	`from pages.contact_page import ContactPage`	**from pages.contact_page import ContactPage**: imports specific names so they can be used without the module prefix. Page Object for /contact.
13	`from pages.gallery_page import GalleryPage`	**from pages.gallery_page import GalleryPage**: imports specific names so they can be used without the module prefix. Page Object for /gallery.
14	`from pages.home_page import HomePage`	**from pages.home_page import HomePage**: imports specific names so they can be used without the module prefix. Page Object for the Home page.
15	`from pages.rooms_page import RoomsPage`	**from pages.rooms_page import RoomsPage**: imports specific names so they can be used without the module prefix. Page Object for the Rooms listing.
17	`HEADER_LINKS = [`	Module-level **constant** *HEADER_LINKS* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
18	`("Home", "/"),`	**Continuation** of the statement started on line 17 (inside brackets).
19	`("Rooms", "/rooms"),`	**Continuation** of the statement started on line 17 (inside brackets).
20	`("Branches", "/branches"),`	**Continuation** of the statement started on line 17 (inside brackets).
21	`("Gallery", "/gallery"),`	**Continuation** of the statement started on line 17 (inside brackets).
22	`("About", "/about"),`	**Continuation** of the statement started on line 17 (inside brackets).
23	`("Contact", "/contact"),`	**Continuation** of the statement started on line 17 (inside brackets).
24	`]`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
25	`FOOTER_LINKS = [`	Module-level **constant** *FOOTER_LINKS* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
26	`"Home",`	**Continuation** of the statement started on line 25 (inside brackets).
27	`"Rooms",`	**Continuation** of the statement started on line 25 (inside brackets).

Line	Code	Syntax & why
28	`    "Gallery",`	**Continuation** of the statement started on line 25 (inside brackets).
29	`    "About us",`	**Continuation** of the statement started on line 25 (inside brackets).
30	`    "Contact",`	**Continuation** of the statement started on line 25 (inside brackets).
31	`    "Book now",`	**Continuation** of the statement started on line 25 (inside brackets).
32	`    "Privacy policy",`	**Continuation** of the statement started on line 25 (inside brackets).
33	`    "Terms & conditions",`	**Continuation** of the statement started on line 25 (inside brackets).
34	`    "Cancellation policy",`	**Continuation** of the statement started on line 25 (inside brackets).
35	`    "Manage a booking",`	**Continuation** of the statement started on line 25 (inside brackets).
36	`]`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
37	`POLICY_PAGES = [`	Module-level **constant** *POLICY_PAGES* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
38	`    ("/privacy-policy", "Privacy policy"),`	**Continuation** of the statement started on line 37 (inside brackets).
39	`    ("/terms", "Terms & conditions"),`	**Continuation** of the statement started on line 37 (inside brackets).
40	`    ("/cancellation-policy", "Cancellation policy"),`	**Continuation** of the statement started on line 37 (inside brackets).
41	`]`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
44	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
45	`@pytest.mark.tc("TC-016", "TC-083")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-016 = “Homepage sections display their content”; TC-083 = “Hero section: title, subtitle, image and Explore rooms CTA”.
46	`def test_home_page_loads(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *home page loads*. Parameters: *home*: project fixture: Home page object, already opened.
47	`    home.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
48	`    expect(home.hero_text).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
51	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
52	`@pytest.mark.tc("TC-043")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-043 = “Header and footer links resolve with no 404”.

Line	Code	Syntax & why
53	`@pytest.mark.parametrize("link,path", HEADER_LINKS, ids=[n for n, _ in HEADER_LINKS])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'link,path'; each value becomes its own test (the id appears in the report).
54	`def test_header_navigation(home: HomePage, link, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *header navigation*. Parameters: *home*: project fixture: Home page object, already opened; *link* (value from @parametrize); *path* (value from @parametrize).
55	`    home.go_to(link)`	**Method call** executed for its effect. *go_to()*: project method BasePage.go_to() in pages/base_page.py.
56	`    expect(home.page).to_have_url(re.compile(re.escape(path) + r"(\?.*)?$"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. *escape()* escapes regex special characters in literal text. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
57	`    expect(home.main).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
60	`@pytest.mark.tc("TC-043")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-043 = “Header and footer links resolve with no 404”.
61	`@pytest.mark.parametrize("name", FOOTER_LINKS)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'name'; each value becomes its own test (the id appears in the report).
62	`def test_footer_links_resolve_without_404(page, home: HomePage, name):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *footer links resolve without 404*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *home*: project fixture: Home page object, already opened; *name* (value from @parametrize).
63	`    href = home.footer_link(name).get_attribute("href")`	**Assignment**: binds the name *href* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type). *footer_link()*: project method BasePage.footer_link() in pages/base_page.py.
64	`    resp = page.request.get(href)`	**Assignment**: binds the name *resp* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
65	`    assert resp.status == 200, f"Footer link '{name}' ({href}) returned HTTP {resp.status}"`	**assert**: fails the test if *resp.status == 200* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
68	`@pytest.mark.tc("TC-043")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-043 = “Header and footer links resolve with no 404”.
69	`def test_logo_returns_home(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *logo returns home*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
70	`    gallery = GalleryPage(page).open()`	**Assignment**: binds the name *gallery* to the value on the right. *open()* opens a file (read/write). *GalleryPage(...)* creates a GalleryPage object (constructor call).

Line	Code	Syntax & why
71	`    gallery.logo.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
72	`    expect(page).to_have_url(re.compile(r"/$"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
75	`@pytest.mark.tc("TC-043")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-043 = “Header and footer links resolve with no 404”.
76	`def test_header_and_footer_present_on_every_page(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *header and footer present on every page*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
77	`    for pg in (HomePage, BranchesPage, GalleryPage, AboutPage, ContactPage):`	**for** loop: repeats the indented block for every item, binding it to *pg* (iterating *(HomePage, BranchesPage, GalleryPage, AboutPage, ContactPag…)*.
78	`        obj = pg(page).open()`	**Assignment**: binds the name *obj* to the value on the right. *open()* opens a file (read/write).
79	`        obj.expect_header_visible()`	**Method call** executed for its effect. *expect_header_visible()*: project method BasePage.expect_header_visible() in pages/base_page.py.
80	`        expect(obj.footer_head_office).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
83	`@pytest.mark.tc("TC-010")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-010 = “Footer shows phone, email and address for each branch and head office”.
84	`def test_footer_head_office_has_address(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *footer head office has address*. Parameters: *home*: project fixture: Home page object, already opened.
85	`    expect(home.footer_head_office).to_have_text(re.compile(r"Head office\s*·\s*\S.{5,}"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
88	`@pytest.mark.tc("TC-010")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-010 = “Footer shows phone, email and address for each branch and head office”.
89	`@pytest.mark.defect("BUG-006")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-006 = “Footer contact block shows branch names with empty phone/email rows and "Head office ." with no address”.

Line	Code	Syntax & why
90	`def test_footer_branch_contacts_have_phone_and_email(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *footer branch contacts have phone and email*. Parameters: *home*: project fixture: Home page object, already opened.
91	`    expect(home.footer_contact_items).to_have_count(5)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
92	`    for item in home.footer_contact_items.all():`	**for** loop: repeats the indented block for every item, binding it to *item* (iterating *home.footer_contact_items.all()*). *all()* returns a list of Locators, one per current match, for iteration.
93	`        text = item.inner_text()`	**Assignment**: binds the name *text* to the value on the right. *inner_text()* visible text of the element (as rendered, respects CSS).
94	`        assert re.search(r"\+?\d[\d\s]{6,}", text), f"No phone in footer entry: {text!r}"`	**assert**: fails the test if *re.search('\\+?\\d[\\d\\s]{6,}', text)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *search()* finds the first regex match anywhere in a string. Uses an **f-string** (f"...{expr}...") to build text with values. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
95	`        assert "@" in text, f"No email in footer entry: {text!r}"`	**assert**: fails the test if *'@' in text* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
98	`@pytest.mark.tc("TC-019")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-019 = “Footer branch list is complete on first render and matches Contact tabs”.
99	`def test_footer_branch_list_complete_and_matches_contact_tabs(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *footer branch list complete and matches contact tabs*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
100	`    home = HomePage(page).open()`	**Assignment**: binds the name *home* to the value on the right. *open()* opens a file (read/write). *HomePage(...)* creates a HomePage object (constructor call).
101	`    expect(home.footer_contact_items).to_have_count(len(site.branches))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). *len()* length of a collection.
102	`    footer_cities = [i.inner_text().splitlines()[0].strip() for i in home.footer_contact_items.all()]`	**Assignment**: binds the name *footer_cities* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *strip()* removes surrounding whitespace. *all()* returns a list of Locators, one per current match, for iteration. *splitlines()* splits text into lines. *inner_text()* visible text of the element (as rendered, respects CSS).
103	`    contact = ContactPage(page).open()`	**Assignment**: binds the name *contact* to the value on the right. *open()* opens a file (read/write). *ContactPage(...)* creates a ContactPage object (constructor call).
104	`    for city in footer_cities:`	**for** loop: repeats the indented block for every item, binding it to *city* (iterating *footer_cities*).
105	`        expect(contact.branch_tab(city)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *branch_tab()*: project method ContactPage.branch_tab() in pages/contact_page.py.

Line	Code	Syntax & why
108	`@pytest.mark.tc("TC-087")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-087 = “Footer legal line shows company name, RC number and current year”.
109	`def test_footer_legal_line(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *footer legal line*. Parameters: *home*: project fixture: Home page object, already opened.
110	`    expect(home.footer_legal).to_have_text(re.compile(rf"© {date.today().year} .+ Limited\. RC \d+\."))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. *compile()* pre-compiles a regex pattern for reuse. *today()* today's date. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
113	`@pytest.mark.tc("TC-017")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-017 = “Unauthenticated access to /staff is blocked”.
114	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
115	`def test_footer_does_not_advertise_staff_console(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *footer does not advertise staff console*. Parameters: *home*: project fixture: Home page object, already opened.
116	`    expect(home.footer.get_by_role("link", name=re.compile("staff|HMS console", re.I))).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
119	`@pytest.mark.mobile`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.mobile**: runs in a mobile viewport.
120	`@pytest.mark.tc("TC-086")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-086 = “Mobile hamburger menu opens, lists all links and closes”.
121	`def test_mobile_hamburger_menu(mobile_page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *mobile hamburger menu*. Parameters: *mobile_page*: project fixture: a tab in a 375x812 touch-enabled mobile context.
122	`    base = BasePage(mobile_page).open("/")`	**Assignment**: binds the name *base* to the value on the right. *open()* opens a file (read/write). *BasePage(...)* creates a BasePage object (constructor call).
123	`    base.expect_on_screen(base.mobile_menu, on=False)`	**Method call** executed for its effect. *expect_on_screen()*: project method BasePage.expect_on_screen() in pages/base_page.py.
124	`    base.menu_toggle.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
125	`    base.expect_on_screen(base.mobile_menu, on=True)`	**Method call** executed for its effect. *expect_on_screen()*: project method BasePage.expect_on_screen() in pages/base_page.py.

Line	Code	Syntax & why
126	`    for name, _ in HEADER_LINKS:`	**for** loop: repeats the indented block for every item, binding it to *(name, _)* (iterating *HEADER_LINKS*).
127	`        expect(base.mobile_menu.get_by_role("link", name=name, exact=True)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
128	`    expect(base.mobile_menu.get_by_role("link", name="Book now")).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
129	`    expect(base.mobile_menu.get_by_role("button", name="Sign in")).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
130	`    base.close_menu.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
131	`    base.expect_on_screen(base.mobile_menu, on=False)`	**Method call** executed for its effect. *expect_on_screen()*: project method BasePage.expect_on_screen() in pages/base_page.py.
134	`@pytest.mark.mobile`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.mobile**: runs in a mobile viewport.
135	`@pytest.mark.tc("TC-086")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-086 = “Mobile hamburger menu opens, lists all links and closes”.
136	`def test_mobile_menu_link_navigates(mobile_page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *mobile menu link navigates*. Parameters: *mobile_page*: project fixture: a tab in a 375x812 touch-enabled mobile context.
137	`    base = BasePage(mobile_page).open("/")`	**Assignment**: binds the name *base* to the value on the right. *open()* opens a file (read/write). *BasePage(...)* creates a BasePage object (constructor call).
138	`    base.menu_toggle.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
139	`    base.mobile_menu.get_by_role("link", name="Gallery", exact=True).click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
140	`    expect(mobile_page).to_have_url(re.compile(r"/gallery"))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
143	`@pytest.mark.tc("TC-044")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-044 = “Header search returns relevant results”.
144	`def test_search_box_starts_empty(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *search box starts empty*. Parameters: *home*: project fixture: Home page object, already opened.
145	`    expect(home.search_box).to_be_empty()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_empty()* waits until an input is empty.
148	`@pytest.mark.tc("TC-044")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-044 = “Header search returns relevant results”.
149	`def test_header_search_opens_results_page(page, home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *header search opens results page*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *home*: project fixture: Home page object, already opened.
150	`    home.search_box.fill("Deluxe")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
151	`    home.search_box.press("Enter")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
152	`    expect(page).to_have_url(re.compile(r"/rooms\?.*q=Deluxe"))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
153	`    expect(RoomsPage(page).results_heading).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *RoomsPage(...)* creates a RoomsPage object (constructor call).
156	`@pytest.mark.tc("TC-044")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-044 = “Header search returns relevant results”.
157	`def test_search_query_lists_only_matching_rooms(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *search query lists only matching rooms*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Docstring: “Same query as the header search, for a party the default branch can host (1 adult).”
158	`    """Same query as the header search, for a party the default branch can host (1 adult)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
159	`    rooms = RoomsPage(page).open(branch_id=site.default_branch.id, adults=1, q="Deluxe")`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).

Line	Code	Syntax & why
160	`    titles = rooms.card_titles()`	**Assignment**: binds the name *titles* to the value on the right. *card_titles()*: project method RoomsPage.card_titles() in pages/rooms_page.py - Room-name links (same href as View details, but the visible room name)..
161	`    assert titles and all("Deluxe" in t for t in titles), titles`	**assert**: fails the test if *titles and all(('Deluxe' in t for t in titles))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *all()* returns a list of Locators, one per current match, for iteration.
164	`@pytest.mark.tc("TC-044")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-044 = “Header search returns relevant results”.
165	`@pytest.mark.defect(reason="header search keeps the default 2 adults, so no Benin room ever matches")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: 'header search keeps the default 2 adults, so no Benin room ever matches'.
166	`def test_header_search_from_home_finds_default_branch_room(page, home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *header search from home finds default branch room*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *home*: project fixture: Home page object, already opened.
167	`    home.search_box.fill("Deluxe")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
168	`    home.search_box.press("Enter")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
169	`    expect(page.get_by_role("main").get_by_role("link", name=re.compile("Deluxe")).first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
172	`@pytest.mark.tc("TC-044")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-044 = “Header search returns relevant results”.
173	`def test_header_search_no_match_shows_empty_state(page, home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *header search no match shows empty state*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *home*: project fixture: Home page object, already opened.
174	`    home.search_box.fill("zzzz")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
175	`    home.search_box.press("Enter")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
176	`    expect(page).to_have_url(re.compile(r"q=zzzz"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

Line	Code	Syntax & why
177	`    expect(page.get_by_role("main").get_by_text(re.compile(r'No rooms match "zzzz"'))).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
180	`@pytest.mark.tc("TC-069")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-069 = “Privacy, Terms and Cancellation policy pages load”.
181	`@pytest.mark.parametrize("path,heading", POLICY_PAGES, ids=[p for p, _ in POLICY_PAGES])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path,heading'; each value becomes its own test (the id appears in the report).
182	`def test_policy_pages_load(page, path, heading):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *policy pages load*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize); *heading* (value from @parametrize).
183	`    resp = page.goto(path)`	**Assignment**: binds the name *resp* to the value on the right. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
184	`    assert resp.status == 200`	**assert**: fails the test if *resp.status == 200* is false. pytest rewrites asserts to print the actual values on failure.
185	`    expect(page.get_by_role("main").get_by_role("heading", level=1)).to_have_text(heading)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
188	`@pytest.mark.tc("TC-069")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-069 = “Privacy, Terms and Cancellation policy pages load”.
189	`@pytest.mark.defect("BUG-020")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-020 = “Privacy policy, Terms & conditions and Cancellation policy pages return 404”.
190	`@pytest.mark.parametrize("path", [p for p, _ in POLICY_PAGES])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
191	`def test_policy_pages_publish_full_policy(page, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *policy pages publish full policy*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize). Docstring: “Pages exist now but only hold an interim 'hasn't published ... yet' notice.”

Line	Code	Syntax & why
192	`    """Pages exist now but only hold an interim 'hasn't published ... yet' notice."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
193	`    page.goto(path)`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
194	`    expect(page.get_by_role("main")).not_to_contain_text(re.compile(r"hasn't published", re.I))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_contain_text()* waits until the text does NOT contain the substring / regex. *compile()* pre-compiles a regex pattern for reuse. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
197	`@pytest.mark.tc("TC-060")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-060 = “Unknown URL shows a branded 404 page”.
198	`def test_unknown_url_shows_branded_404(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *unknown url shows branded 404*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
199	`    resp = page.goto("/xyz-not-here")`	**Assignment**: binds the name *resp* to the value on the right. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
200	`    assert resp.status == 404`	**assert**: fails the test if *resp.status == 404* is false. pytest rewrites asserts to print the actual values on failure.
201	`    main = page.get_by_role("main")`	**Assignment**: binds the name *main* to the value on the right. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
202	`    expect(main.get_by_role("heading", name="Page not found")).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
203	`    expect(main.get_by_role("link", name="Back home")).to_have_attribute("href", "/")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
204	`    expect(page.get_by_role("banner")).to_be_visible() # branded: site chrome is kept`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Inline comment: “branded: site chrome is kept”.

Line	Code	Syntax & why
207	`@pytest.mark.tc("TC-061")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-061 = “Page titles and meta descriptions per page”.
208	`@pytest.mark.seo`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.seo**: search-engine/metadata check.
209	`def test_page_titles_are_unique(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *page titles are unique*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
210	`    titles = {}`	**Assignment**: binds the name *titles* to the value on the right. The value is a **dict** literal {key: value}.
211	`    for path in ["/", "/rooms", "/branches", "/gallery", "/about", "/contact", "/manage-booking"]:`	**for** loop: repeats the indented block for every item, binding it to *path* (iterating *['/', '/rooms', '/branches', '/gallery', '/about', '/contac…*).
212	`        page.goto(path)`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
213	`        titles[path] = page.title()`	**Assignment**: binds the name *titles[path]* to the value on the right. *title()* returns document.title (used for SEO checks).
214	`    assert all("Oneflare Grand Plaza" in t for t in titles.values()), titles`	**assert**: fails the test if *all(('Oneflare Grand Plaza' in t for t in titles.values()))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *all()* returns a list of Locators, one per current match, for iteration. *values()* values of a dict.
215	`    assert len(set(titles.values())) == len(titles), f"Duplicate titles: {titles}"`	**assert**: fails the test if *len(set(titles.values())) == len(titles)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *len()* length of a collection. *set()* builds a set (unique values). *values()* values of a dict. Uses an **f-string** (f"...{expr}...") to build text with values.
218	`@pytest.mark.tc("TC-061")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-061 = “Page titles and meta descriptions per page”.
219	`@pytest.mark.seo`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.seo**: search-engine/metadata check.
220	`def test_meta_descriptions_are_unique(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *meta descriptions are unique*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
221	`    descs = {}`	**Assignment**: binds the name *descs* to the value on the right. The value is a **dict** literal {key: value}.
222	`    for path in ["/", "/rooms", "/contact"]:`	**for** loop: repeats the indented block for every item, binding it to *path* (iterating *['/', '/rooms', '/contact']*).
223	`        page.goto(path)`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
224	`        descs[path] = page.locator('meta[name="description"]').get_attribute("content")`	**Assignment**: binds the name *descs[path]* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type). *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
225	`    assert all(d and len(d) > 20 for d in descs.values()), descs`	**assert**: fails the test if *all((d and len(d) > 20 for d in descs.values()))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *all()* returns a list of Locators, one per current match, for iteration. *values()* values of a dict. *len()* length of a collection.

Line	Code	Syntax & why
226	`    assert len(set(descs.values())) == len(descs), f"Duplicate meta descriptions: {descs}"`	**assert**: fails the test if *len(set(descs.values())) == len(descs)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *len()* length of a collection. *set()* builds a set (unique values). *values()* values of a dict. Uses an **f-string** (f"...{expr}...") to build text with values.

tests/functional/test_home.py

Home page sections and the home search widget. Workbook: TC-005, 014, 016, 082-085.

Line	Code	Syntax & why
1	`"""Home page: hero, search widget, locations, 'Free for your dates', gallery and amenities sections."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`from datetime import date, timedelta`	**from datetime import date, timedelta**: imports specific names so they can be used without the module prefix. date/timedelta give calendar arithmetic (today + 7 days) for check-in/check-out dates. *date*: Calendar date type (no time component). *timedelta*: A duration; date + timedelta(days=2) gives a date two days later.
6	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
9	`from pages.home_page import HomePage`	**from pages.home_page import HomePage**: imports specific names so they can be used without the module prefix. Page Object for the Home page.
10	`from pages.rooms_page import RoomsPage`	**from pages.rooms_page import RoomsPage**: imports specific names so they can be used without the module prefix. Page Object for the Rooms listing.
12	`PLACEHOLDER_COPY = re.compile(r"whatever amount|lorem ipsum|placeholder|TODO|Live changing", re.I)`	Module-level **constant** *PLACEHOLDER_COPY* (UPPER_CASE by convention = not changed at runtime). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
15	`@pytest.mark.tc("TC-083")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-083 = “Hero section: title, subtitle, image and Explore rooms CTA”.
16	`def test_hero_copy_image_and_cta(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *hero copy image and cta*. Parameters: *home*: project fixture: Home page object, already opened.
17	`    expect(home.hero_heading).to_have_text("Space to work, and to stop working")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex.

Line	Code	Syntax & why
18	`    hero = home.hero_heading.locator("xpath=..")`	**Assignment**: binds the name *hero* to the value on the right. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
19	`    expect(hero).not_to_contain_text(PLACEHOLDER_COPY)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_contain_text()* waits until the text does NOT contain the substring / regex.
20	`    home.explore_rooms()`	**Method call** executed for its effect. *explore_rooms()*: project method HomePage.explore_rooms() in pages/home_page.py.
21	`    home.expect_url_contains("/rooms")`	**Method call** executed for its effect. *expect_url_contains()*: project method BasePage.expect_url_contains() in pages/base_page.py.
24	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
25	`@pytest.mark.tc("TC-084")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-084 = “Home search widget submits to Rooms with branch, dates and guests”.
26	`def test_home_search_submits_branch_dates_and_guests(page, home: HomePage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *home search submits branch dates and guests*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *home*: project fixture: Home page object, already opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
27	`    owerri = site.branch("Owerri")`	**Assignment**: binds the name *owerri* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
28	`    check_in = date.today() + timedelta(days=7)`	**Assignment**: binds the name *check_in* to the value on the right. *today()* today's date.
29	`    check_out = check_in + timedelta(days=2)`	**Assignment**: binds the name *check_out* to the value on the right.
30	`    home.search.search(branch_id=owerri.id, check_in=check_in, check_out=check_out, adults=3)`	**Method call** executed for its effect. *search()* finds the first regex match anywhere in a string.
31	`    expect(page).to_have_url(`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches.
32	`        re.compile(rf"/rooms\?.*branchId={owerri.id}.*checkIn={check_in}.*checkOut={check_out}.*adults=3")`	**Continuation** of the statement started on line 31 (inside brackets). Keyword argument(s) branchId, checkIn, checkOut, adults (name=value): self-documenting and order-independent. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
33	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
34	`    rooms = RoomsPage(page)`	**Assignment**: binds the name *rooms* to the value on the right. *RoomsPage(...)* creates a RoomsPage object (constructor call).
35	`    expect(rooms.branch_select).to_have_value(owerri.id)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.

Line	Code	Syntax & why
36	`    expect(rooms.check_in).to_have_value(str(check_in))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value. *str()* converts to string.
37	`    expect(rooms.check_out).to_have_value(str(check_out))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value. *str()* converts to string.
38	`    expect(rooms.guests).to_have_value("3")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
41	`@pytest.mark.tc("TC-016")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-016 = “Homepage sections display their content”.
42	`def test_locations_section_lists_every_branch(home: HomePage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *locations section lists every branch*. Parameters: *home*: project fixture: Home page object, already opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
43	`    expect(home.locations_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
44	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
45	`        expect(home.main.get_by_role("link", name=f"{b.city} {b.address}")).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. Uses an **f-string** (f"...{expr}...") to build text with values.
48	`@pytest.mark.tc("TC-016")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-016 = “Homepage sections display their content”.
49	`def test_gallery_section_shows_images(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *gallery section shows images*. Parameters: *home*: project fixture: Home page object, already opened.
50	`    expect(home.gallery_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
51	`    section = home.gallery_heading.locator("xpath=ancestor::*[count(.//img) >= 3][1]")`	**Assignment**: binds the name *section* to the value on the right. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
52	`    expect(section.get_by_role("img").first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
55	`@pytest.mark.tc("TC-016", "TC-085")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-016 = “Homepage sections display their content”; TC-085 = “Amenities section lists the hotel's amenities”.
56	`def test_amenities_section_lists_amenities(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *amenities section lists amenities*. Parameters: *home*: project fixture: Home page object, already opened.
57	`    expect(home.amenities_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
58	`    # paragraphs between the Amenities heading and the next h2 (amenities load asynchronously)`	**Comment** (# ... ignored by Python): documents intent for the reader.
59	`    items = home.amenities_heading.locator("xpath=following::p[following::h2]")`	**Assignment**: binds the name *items* to the value on the right. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
60	`    expect(items.nth(5)).to_be_attached(timeout=15000) # waits until at least 6 lines exist; BUG-010 showed 1`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_attached()* waits until the element exists in the DOM. *nth()* the n-th (0-based) matching element. Inline comment: “waits until at least 6 lines exist; BUG-010 showed 1”.
63	`@pytest.mark.tc("TC-016")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-016 = “Homepage sections display their content”.
64	`def test_free_for_your_dates_shows_room_cards(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *free for your dates shows room cards*. Parameters: *home*: project fixture: Home page object, already opened.
65	`    expect(home.free_rooms_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
66	`    expect(home.view_details_links.first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
69	`@pytest.mark.tc("TC-082")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-082 = “'Free for your dates' shows the selected branch's rooms and prices”.
70	`def test_free_for_your_dates_matches_selected_branch(page, home: HomePage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *free for your dates matches selected branch*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *home*: project fixture: Home page object, already opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Docstring: “Home cards must be the default branch's rooms at the same prices as /rooms (BUG-029).”
71	`    """Home cards must be the default branch's rooms at the same prices as /rooms (BUG-029)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
72	`    branch = site.default_branch`	**Assignment**: binds the name *branch* to the value on the right.

Line	Code	Syntax & why
73	`    expect(home.free_rooms_caption).to_contain_text(branch.name)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
74	`    home_prices = {int(p.replace(",", "")) for p in re.findall(r"₪([\d,]+) / night", home.main.inner_text())}`	**Assignment**: binds the name *home_prices* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *int()* converts to integer. *replace()* string replace. *findall()* returns every regex match. *inner_text()* visible text of the element (as rendered, respects CSS). Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
75	`    rooms = RoomsPage(page).open(branch_id=branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
76	`    list_prices = {int(p.replace(",", "")) for p in re.findall(r"₪([\d,]+) / night", rooms.main.inner_text())}`	**Assignment**: binds the name *list_prices* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *int()* converts to integer. *replace()* string replace. *findall()* returns every regex match. *inner_text()* visible text of the element (as rendered, respects CSS). Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
77	`    assert home_prices and home_prices <= list_prices, f"home {home_prices} vs /rooms {list_prices}"`	**assert**: fails the test if *home_prices and home_prices <= list_prices* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
80	`@pytest.mark.tc("TC-005", "TC-082")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-005 = “No duplicate identical room cards”; TC-082 = “'Free for your dates' shows the selected branch's rooms and prices”.
81	`@pytest.mark.defect("BUG-003")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-003 = “Heading says "5 room types" but list renders repeated identical cards (Standard Room appears 3+ times) all marked "1 room left"”.
82	`def test_free_for_your_dates_has_no_duplicate_cards(home: HomePage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *free for your dates has no duplicate cards*. Parameters: *home*: project fixture: Home page object, already opened.
83	`    expect(home.view_details_links.first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
84	`    cards = []`	**Assignment**: binds the name *cards* to the value on the right. The value is a **list** literal [...].
85	`    for link in home.view_details_links.all():`	**for** loop: repeats the indented block for every item, binding it to *link* (iterating *home.view_details_links.all()*). *all()* returns a list of Locators, one per current match, for iteration.
86	`        cards.append(`	**Method call** executed for its effect. *append()* adds an item to a list.
87	`            link.evaluate(`	**Continuation** of the statement started on line 86 (inside brackets). *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
88	`                "a => { let n = a; while (n.parentElement && !/\\/night/.test(n.innerText)) n = n.parentElement;"`	**Continuation** of the statement started on line 86 (inside brackets).
89	`                " return n.innerText.replace(/\\d+ rooms? left/, '').trim(); }"`	**Continuation** of the statement started on line 86 (inside brackets).

Line	Code	Syntax & why
90	`            )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
91	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
92	`    dupes = {c.splitlines()[0] for c in cards if cards.count(c) > 1}`	**Assignment**: binds the name *dupes* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *splitlines()* splits text into lines. *count()* number of current matches (does not wait).
93	`    assert not dupes, f"Identical room cards repeated on Home: {dupes}"`	**assert**: fails the test if *not dupes* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.

tests/functional/test_rooms.py

Rooms listing: branch scoping, cards, counts, capacity, prices, date validation, guests, filters, slider, mobile drawer. The largest module because the workbook's highest-risk area (search & filters) lives here.

Line	Code	Syntax & why
1	`"""Rooms listing: search bar, branch scoping, cards, filters, price slider, empty states."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`from datetime import date, timedelta`	**from datetime import date, timedelta**: imports specific names so they can be used without the module prefix. date/timedelta give calendar arithmetic (today + 7 days) for check-in/check-out dates. *date*: Calendar date type (no time component). *timedelta*: A duration; date + timedelta(days=2) gives a date two days later.
6	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
9	`from pages.rooms_page import RoomsPage`	**from pages.rooms_page import RoomsPage**: imports specific names so they can be used without the module prefix. Page Object for the Rooms listing.
10	`from utils.site_data import branch_rooms`	**from utils.site_data import branch_rooms**: imports specific names so they can be used without the module prefix. Reads branches and rooms from the site's public API so tests use real, current ids.
12	`TEST_DATA_RE = re.compile(r"\btest\s*room\b|testroom\d*|TestType|test bed|testbed|\bstring\b", re.I)`	Module-level **constant** *TEST_DATA_RE* (UPPER_CASE by convention = not changed at runtime). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
15	`def _capacity(card_text: str) -> int:`	**def** defines the private helper *_capacity* (leading underscore = internal use). Parameters: *card_text*. Returns int.

Line	Code	Syntax & why
16	`    return int(re.search(r"Max (\d+) adults?", card_text).group(1))`	**return**: ends the function and hands back *int(re.search('Max (\\d+) adults?', card_text).group(1))*. *int()* converts to integer. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
19	`# ---------- listing ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
20	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
21	`@pytest.mark.tc("TC-004")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-004 = “Heading count equals number of distinct room types shown”.
22	`def test_rooms_list_shows_results(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *rooms list shows results*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
23	`    assert rooms.room_count() > 0`	**assert**: fails the test if *rooms.room_count() > 0* is false. pytest rewrites asserts to print the actual values on failure. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
24	`    expect(rooms.view_details_links.first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
27	`@pytest.mark.tc("TC-004")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-004 = “Heading count equals number of distinct room types shown”.
28	`def test_heading_count_equals_room_types_shown(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *heading count equals room types shown*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
29	`    titles = rooms.card_titles()`	**Assignment**: binds the name *titles* to the value on the right. *card_titles()*: project method RoomsPage.card_titles() in pages/rooms_page.py - Room-name links (same href as View details, but the visible room name)..
30	`    assert rooms.room_count() == len(titles) == len(set(titles)), (`	**assert**: fails the test if *rooms.room_count() == len(titles) == len(set(titles))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py. *len()* length of a collection. *set()* builds a set (unique values).
31	`        f"heading={rooms.room_count()} cards={titles}"`	**Continuation** of the statement started on line 30 (inside brackets). Keyword argument(s) heading, cards (name=value): self-documenting and order-independent. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py. Uses an **f-string** (f"...{expr}...") to build text with values.
32	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
35	`@pytest.mark.tc("TC-005")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-005 = “No duplicate identical room cards”.

Line	Code	Syntax & why
36	`def test_no_duplicate_identical_room_cards(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no duplicate identical room cards*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
37	`    texts = [re.sub(r"\d+ rooms? left", "", t).strip() for t in rooms.card_texts()]`	**Assignment**: binds the name *texts* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *strip()* removes surrounding whitespace. *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py. *sub()* regex replace. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
38	`    assert len(texts) == len(set(texts)), f"Duplicate cards: {texts}"`	**assert**: fails the test if *len(texts) == len(set(texts))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *len()* length of a collection. *set()* builds a set (unique values). Uses an **f-string** (f"...{expr}...") to build text with values.
41	`@pytest.mark.tc("TC-003")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-003 = “Room name displays without duplicated word”.
42	`@pytest.mark.parametrize("city", ["Benin City", "Lagos"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'city'; each value becomes its own test (the id appears in the report).
43	`def test_room_names_have_no_duplicated_word(page, site, city):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *room names have no duplicated word*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *city* (value from @parametrize).
44	`    rooms = RoomsPage(page).open(branch_id=site.branch(city).id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.
45	`    bad = [t for t in rooms.card_titles() if re.search(r"\b(\w+) \1\b", t, re.I)]`	**Assignment**: binds the name *bad* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *card_titles()*: project method RoomsPage.card_titles() in pages/rooms_page.py - Room-name links (same href as View details, but the visible room name).. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
46	`    assert not bad, f"Duplicated words in titles: {bad}"`	**assert**: fails the test if *not bad* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
49	`@pytest.mark.tc("TC-003")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-003 = “Room name displays without duplicated word”.
50	`@pytest.mark.defect("BUG-002")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-002 = “Room name duplicated: card title reads "Standard Room Room"”.

Line	Code	Syntax & why
51	`def test_room_names_have_no_redundant_room_suffix(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *room names have no redundant room suffix*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Docstring: “Lagos cards read 'Classic47 Room', 'testroom2 Room', 'TestTypee Room' - type + ' Room' concatenated.”
52	`    """Lagos cards read 'Classic47 Room', 'testroom2 Room', 'TestTypee Room' - type + ' Room' concatenated."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
53	`    rooms = RoomsPage(page).open(branch_id=site.branch("Lagos").id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.
54	`    api_names = {r["name"] for r in branch_rooms(page.request, site.branch("Lagos").id)}`	**Assignment**: binds the name *api_names* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *branch_rooms()*: project function branch_rooms() in utils/site_data.py. *branch()*: project method SiteData.branch() in utils/site_data.py.
55	`    bad = [t for t in rooms.card_titles() if t not in api_names]`	**Assignment**: binds the name *bad* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *card_titles()*: project method RoomsPage.card_titles() in pages/rooms_page.py - Room-name links (same href as View details, but the visible room name)..
56	`    assert not bad, f"Card titles that differ from the room names: {bad}"`	**assert**: fails the test if *not bad* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
59	`@pytest.mark.tc("TC-001", "TC-056")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-001 = “Room card shows image for every room type”; TC-056 = “Alt text, labels and colour contrast”.
60	`@pytest.mark.defect("BUG-001")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-001 = “Room images not rendered on Rooms listing cards (blank grey placeholder)”.
61	`def test_every_room_card_shows_a_photo_with_alt(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *every room card shows a photo with alt*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
62	`    for text, link in zip(rooms.card_texts(), rooms.view_details_links.all(), strict=True):`	**for** loop: repeats the indented block for every item, binding it to *(text, link)* (iterating *zip(rooms.card_texts(), rooms.view_details_links.all(), str…)*. *zip()* iterates several sequences in lock-step (strict=True: lengths must match). *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py. *all()* returns a list of Locators, one per current match, for iteration.
63	`        card = link.locator("xpath=ancestor::*[.//img][1]")`	**Assignment**: binds the name *card* to the value on the right. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
64	`        img = card.locator("img").first`	**Assignment**: binds the name *img* to the value on the right. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

Line	Code	Syntax & why
65	`        expect(img, f"no photo on card: {text.splitlines()[0]}").to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *splitlines()* splits text into lines. Uses an **f-string** (f"...{expr}...") to build text with values.
66	`        assert (img.get_attribute("alt") or "").strip(), f"empty alt on {text.splitlines()[0]}"`	**assert**: fails the test if *(img.get_attribute('alt') or '').strip()* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *strip()* removes surrounding whitespace. *get_attribute()* reads an HTML attribute (href, min, max, alt, type). *splitlines()* splits text into lines. Uses an **f-string** (f"...{expr}...") to build text with values.
69	`@pytest.mark.tc("TC-002")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-002 = “Fallback shown when a room image fails to load”.
70	`def test_layout_survives_blocked_images(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *layout survives blocked images*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
71	`    page.route(re.compile(r".*\.(jpg|jpeg|png|webp)(\?.*)?$|/_next/image.*"), lambda r: r.abort())`	**Method call** executed for its effect. *route()* intercepts network requests matching a pattern so a test can abort or stub them (used to simulate failed images). *compile()* pre-compiles a regex pattern for reuse. *abort()* aborts an intercepted request. **lambda**: inline anonymous function (used as a callback/predicate). Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
72	`    rooms = RoomsPage(page).open(branch_id=site.default_branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
73	`    expect(rooms.view_details_links.first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
74	`    overflow = page.evaluate("() => document.documentElement.scrollWidth - window.innerWidth")`	**Assignment**: binds the name *overflow* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
75	`    assert overflow <= 1, f"Layout broke without images (overflow {overflow}px)"`	**assert**: fails the test if *overflow <= 1* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
78	`@pytest.mark.tc("TC-006")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-006 = “"Rooms left" label reflects real inventory”.
79	`def test_rooms_left_label_is_a_positive_inventory_count(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *rooms left label is a positive inventory count*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded. Docstring: “Full check (label == HMS inventory, decrements after booking) needs back-office access: Blocked.”
80	`    """Full check (label == HMS inventory, decrements after booking) needs back-office access: Blocked."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.

Line	Code	Syntax & why
81	`    labels = rooms.main.get_by_text(re.compile(r"^\d+ rooms? left$")).all_inner_texts()`	**Assignment**: binds the name *labels* to the value on the right. *all_inner_texts()* returns visible text of every match as a list of strings. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
82	`    assert labels and all(int(x.split()[0]) >= 1 for x in labels), labels`	**assert**: fails the test if *labels and all((int(x.split()[0]) >= 1 for x in labels))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *all()* returns a list of Locators, one per current match, for iteration. *int()* converts to integer. *split()* splits a string into a list.
85	`@pytest.mark.tc("TC-009")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-009 = “Capacity text uses correct singular/plural”.
86	`def test_capacity_text_singular_and_plural(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *capacity text singular and plural*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
87	`    rooms = RoomsPage(page).open(branch_id=site.branch("Lagos").id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.
88	`    text = rooms.main.inner_text()`	**Assignment**: binds the name *text* to the value on the right. *inner_text()* visible text of the element (as rendered, respects CSS).
89	`    assert not re.search(r"\b1 (adults|children)\b", text), "singular count with plural noun"`	**assert**: fails the test if *not re.search('\\b1 (adults|children)\\b', text)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
90	`    assert not re.search(r"\b([2-9]|\d{2,}) (adult|child)\b(?!ren)", text), "plural count with singular noun"`	**assert**: fails the test if *not re.search('\\b([2-9]|\\d{2,}) (adult|child)\\b(?!ren)', text)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
93	`@pytest.mark.tc("TC-025")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-025 = “Nights and total price computed correctly”.
94	`def test_card_total_equals_nightly_rate_times_nights(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *card total equals nightly rate times nights*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
95	`    check_in = date.today() + timedelta(days=10)`	**Assignment**: binds the name *check_in* to the value on the right. *today()* today's date.
96	`    for nights in (2, 3):`	**for** loop: repeats the indented block for every item, binding it to *nights* (iterating *(2, 3)*).
97	`        rooms = RoomsPage(page).open(`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
98	`            branch_id=site.default_branch.id,`	**Continuation** of the statement started on line 97 (inside brackets). Keyword argument(s) branch_id (name=value): self-documenting and order-independent.

Line	Code	Syntax & why
99	`            adults=1,`	**Continuation** of the statement started on line 97 (inside brackets). Keyword argument(s) adults (name=value): self-documenting and order-independent.
100	`            checkIn=check_in,`	**Continuation** of the statement started on line 97 (inside brackets). Keyword argument(s) checkIn (name=value): self-documenting and order-independent.
101	`            checkOut=check_in + timedelta(days=nights),`	**Continuation** of the statement started on line 97 (inside brackets). Keyword argument(s) checkOut, days (name=value): self-documenting and order-independent.
102	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
103	`        for text in rooms.card_texts():`	**for** loop: repeats the indented block for every item, binding it to *text* (iterating *rooms.card_texts()*). *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py.
104	`            rate = int(re.search(r"₦([\d,]+) / night", text).group(1).replace(",", ""))`	**Assignment**: binds the name *rate* to the value on the right. *int()* converts to integer. *replace()* string replace. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
105	`            m = re.search(rf"₦([\d,]+) for {nights} nights", text)`	**Assignment**: binds the name *m* to the value on the right. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
106	`            assert m, f"no {nights}-night total on card: {text!r}"`	**assert**: fails the test if *m* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
107	`            assert int(m.group(1).replace(",", "")) == rate * nights`	**assert**: fails the test if *int(m.group(1).replace(',', '')) == rate * nights* is false. pytest rewrites asserts to print the actual values on failure. *int()* converts to integer. *replace()* string replace. *group()* returns a captured regex group.
110	`# ---------- search bar ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
111	`@pytest.mark.tc("TC-065", "TC-018")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-065 = “Default dates follow the guest's time zone (WAT)”; TC-018 = “Date format is unambiguous”.
112	`def test_default_dates_are_today_wat_and_two_nights(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *default dates are today wat and two nights*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
113	`    rooms = RoomsPage(page).open()`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
114	`    today = page.evaluate("() => new Date().toLocaleDateString('en-CA', {timeZone: 'Africa/Lagos'})")`	**Assignment**: binds the name *today* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
115	`    expect(rooms.check_in).to_have_value(today)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.

Line	Code	Syntax & why
116	`    expect(rooms.check_out).to_have_value(str(date.fromisoformat(today) + timedelta(days=2)))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value. *str()* converts to string. *fromisoformat()* parses YYYY-MM-DD into a date.
119	`@pytest.mark.tc("TC-018")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-018 = “Date format is unambiguous”.
120	`def test_date_fields_are_native_date_pickers(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *date fields are native date pickers*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded. Docstring: “Native <input type=date> renders in the visitor's locale, so the format is never ambiguous.”
121	`    """Native <input type=date> renders in the visitor's locale, so the format is never ambiguous."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
122	`    expect(rooms.check_in).to_have_attribute("type", "date")`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value.
123	`    expect(rooms.check_out).to_have_attribute("type", "date")`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value.
126	`@pytest.mark.tc("TC-020", "TC-064")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-020 = “Search with past check-in date is rejected”; TC-064 = “Search bar date validation (past, inverted, same day)”.
127	`def test_past_check_in_is_blocked_by_picker(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *past check in is blocked by picker*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
128	`    today = rooms.page.evaluate("() => new Date().toLocaleDateString('en-CA', {timeZone: 'Africa/Lagos'})")`	**Assignment**: binds the name *today* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
129	`    expect(rooms.check_in).to_have_attribute("min", today)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value.
132	`@pytest.mark.tc("TC-020", "TC-064")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-020 = “Search with past check-in date is rejected”; TC-064 = “Search bar date validation (past, inverted, same day)”.
133	`@pytest.mark.defect("BUG-015")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-015 = “Search bar accepts past and inverted dates and still lists rooms as available”.

Line	Code	Syntax & why
134	`def test_typed_past_check_in_is_rejected(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *typed past check in is rejected*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
135	`    rooms = RoomsPage(page).open(branch_id=site.default_branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
136	`    yesterday = date.today() - timedelta(days=1)`	**Assignment**: binds the name *yesterday* to the value on the right. *today()* today's date.
137	`    rooms.search.search(check_in=yesterday, check_out=date.today() + timedelta(days=1))`	**Method call** executed for its effect. *search()* finds the first regex match anywhere in a string. *today()* today's date.
138	`    page.wait_for_timeout(1000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
139	`    assert f"checkIn={yesterday}" not in page.url, "search ran with a past check-in"`	**assert**: fails the test if *f'checkIn={yesterday}' not in page.url* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
142	`@pytest.mark.tc("TC-021", "TC-064")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-021 = “Check-out before or equal to check-in is rejected”; TC-064 = “Search bar date validation (past, inverted, same day)”.
143	`def test_check_out_picker_min_is_after_check_in(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *check out picker min is after check in*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
144	`    check_in = date.today() + timedelta(days=5)`	**Assignment**: binds the name *check_in* to the value on the right. *today()* today's date.
145	`    rooms.check_in.fill(str(check_in))`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *str()* converts to string.
146	`    expect(rooms.check_out).to_have_attribute("min", str(check_in + timedelta(days=1)))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value. *str()* converts to string.
149	`@pytest.mark.tc("TC-021", "TC-064")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-021 = “Check-out before or equal to check-in is rejected”; TC-064 = “Search bar date validation (past, inverted, same day)”.
150	`@pytest.mark.defect("BUG-014", "BUG-015")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-014 = “Room detail accepts a check-out date before check-in and continues to the booking step (charged as 1 night)”; BUG-015 = “Search bar accepts past and inverted dates and still lists rooms as available”.
151	`@pytest.mark.parametrize("offset", [-2, 0], ids=["inverted", "same-day"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'offset'; each value becomes its own test (the id appears in the report).

Line	Code	Syntax & why
152	`def test_check_out_on_or_before_check_in_is_rejected(page, site, offset):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *check out on or before check in is rejected*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *offset* (value from @parametrize).
153	`    rooms = RoomsPage(page).open(branch_id=site.default_branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
154	`    check_in = date.today() + timedelta(days=5)`	**Assignment**: binds the name *check_in* to the value on the right. *today()* today's date.
155	`    rooms.search.search(check_in=check_in, check_out=check_in + timedelta(days=offset))`	**Method call** executed for its effect. *search()* finds the first regex match anywhere in a string.
156	`    page.wait_for_timeout(1000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
157	`    assert f"checkOut={check_in + timedelta(days=offset)}" not in page.url, "search ran with invalid range"`	**assert**: fails the test if *f'checkOut={check_in + timedelta(days=offset)}' not in page.url* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
160	`@pytest.mark.tc("TC-024")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-024 = “Guest count minimum and maximum limits”.
161	`def test_guest_selector_limits(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *guest selector limits*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
162	`    assert rooms.search.guest_options() == ["1 adult"] + [f"{n} adults" for n in range(2, 7)]`	**assert**: fails the test if *rooms.search.guest_options() == ['1 adult'] + [f'{n} adults' for n in range(2, …* is false. pytest rewrites asserts to print the actual values on failure. *guest_options()*: project method SearchWidget.guest_options() in pages/search_widget.py. *range()* sequence of integers. Uses an **f-string** (f"...{expr}...") to build text with values.
165	`@pytest.mark.tc("TC-007", "TC-008", "TC-024")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-007 = “2-adult search excludes rooms with max 1 adult”; TC-008 = “Boundary: guests equal to and above room max capacity”; TC-024 = “Guest count minimum and maximum limits”.
166	`@pytest.mark.parametrize("adults", [1, 2, 3])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'adults'; each value becomes its own test (the id appears in the report).
167	`def test_guest_search_hides_rooms_below_capacity(page, site, adults):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *guest search hides rooms below capacity*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *adults* (value from @parametrize).
168	`    rooms = RoomsPage(page).open(branch_id=site.branch("Lagos").id, adults=adults)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.

Line	Code	Syntax & why
169	`    caps = [_capacity(t) for t in rooms.card_texts()]`	**Assignment**: binds the name *caps* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py.
170	`    assert all(c >= adults for c in caps), f"{adults}-adult search listed rooms with capacity {caps}"`	**assert**: fails the test if *all((c >= adults for c in caps))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *all()* returns a list of Locators, one per current match, for iteration. Uses an **f-string** (f"...{expr}...") to build text with values.
171	`    if not caps:`	**if** statement: the indented block runs only when *not caps* is true.
172	`        expect(rooms.empty_state).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
175	`@pytest.mark.tc("TC-008")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-008 = “Boundary: guests equal to and above room max capacity”.
176	`def test_room_at_exact_capacity_is_listed(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *room at exact capacity is listed*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Docstring: “Equal to max: listed. (Above max: hidden - covered by test_guest_search_hides_rooms_below_capacity.)”
177	`    """Equal to max: listed. (Above max: hidden - covered by test_guest_search_hides_rooms_below_capacity.)"""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
178	`    rooms = RoomsPage(page).open(branch_id=site.default_branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
179	`    assert any(_capacity(t) == 1 for t in rooms.card_texts())`	**assert**: fails the test if *any((_capacity(t) == 1 for t in rooms.card_texts()))* is false. pytest rewrites asserts to print the actual values on failure. *any()* True if any element is truthy. *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py.
182	`@pytest.mark.tc("TC-022")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-022 = “Search returns rooms for chosen branch only”.
183	`@pytest.mark.parametrize("city", ["Benin City", "Ikeja", "Lekki", "Owerri"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'city'; each value becomes its own test (the id appears in the report).
184	`def test_search_returns_rooms_for_chosen_branch_only(page, site, city):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *search returns rooms for chosen branch only*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *city* (value from @parametrize).
185	`    branch = site.branch(city)`	**Assignment**: binds the name *branch* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
186	`    rooms = RoomsPage(page).open(branch_id=branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).

Line	Code	Syntax & why
187	`    assert rooms.heading_branch() == branch.name`	**assert**: fails the test if *rooms.heading_branch() == branch.name* is false. pytest rewrites asserts to print the actual values on failure. *heading_branch()*: project method RoomsPage.heading_branch() in pages/rooms_page.py.
188	`    for text in rooms.card_texts():`	**for** loop: repeats the indented block for every item, binding it to *text* (iterating *rooms.card_texts()*). *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py.
189	`        assert city in text, f"card not from {city}: {text!r}"`	**assert**: fails the test if *city in text* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
192	`@pytest.mark.tc("TC-022", "TC-076")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-022 = “Search returns rooms for chosen branch only”; TC-076 = “Lagos branch lists only Lagos rooms and no test data”.
193	`@pytest.mark.defect("BUG-025")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-025 = “Lagos branch lists the whole hotel catalogue (28 'room types') including test records such as 'test room', 'testroom2', 'TestType' and 'new test room type'”.
194	`def test_lagos_lists_only_lagos_rooms(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *lagos lists only lagos rooms*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
195	`    lagos = site.branch("Lagos")`	**Assignment**: binds the name *lagos* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
196	`    rooms = RoomsPage(page).open(branch_id=lagos.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
197	`    foreign = [t.splitlines()[0] for t in rooms.card_texts() if "Lagos" not in t]`	**Assignment**: binds the name *foreign* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *splitlines()* splits text into lines. *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py.
198	`    assert not foreign, f"Lagos list contains rooms of other/unknown branches: {foreign}"`	**assert**: fails the test if *not foreign* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
201	`@pytest.mark.tc("TC-076", "TC-077")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-076 = “Lagos branch lists only Lagos rooms and no test data”; TC-077 = “No test or placeholder records visible on any public page”.
202	`@pytest.mark.defect("BUG-025")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-025 = “Lagos branch lists the whole hotel catalogue (28 'room types') including test records such as 'test room', 'testroom2', 'TestType' and 'new test room type'”.

Line	Code	Syntax & why
203	`def test_no_test_records_on_public_listing(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no test records on public listing*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
204	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
205	`        text = RoomsPage(page).open(branch_id=b.id, adults=1).main.inner_text()`	**Assignment**: binds the name *text* to the value on the right. *inner_text()* visible text of the element (as rendered, respects CSS). *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
206	`        assert not TEST_DATA_RE.search(text), f"test data on {b.city}: {TEST_DATA_RE.findall(text)}"`	**assert**: fails the test if *not TEST_DATA_RE.search(text)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *search()* finds the first regex match anywhere in a string. *findall()* returns every regex match. Uses an **f-string** (f"...{expr}...") to build text with values.
209	`@pytest.mark.tc("TC-023")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-023 = “No availability message when all rooms booked”.
210	`def test_no_availability_message_when_nothing_fits(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no availability message when nothing fits*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Docstring: “No sold-out dataset exists in preview, so the 'nothing available' state is produced with 6 adults.”
211	`    """No sold-out dataset exists in preview, so the 'nothing available' state is produced with 6 adults."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
212	`    rooms = RoomsPage(page).open(branch_id=site.default_branch.id, adults=6)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
213	`    assert rooms.room_count() == 0`	**assert**: fails the test if *rooms.room_count() == 0* is false. pytest rewrites asserts to print the actual values on failure. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
214	`    expect(rooms.empty_state).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
217	`@pytest.mark.tc("TC-108")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-108 = “Branch choice persists across pages”.
218	`@pytest.mark.defect("BUG-032")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-032 = “Branch selection is not remembered, although the Branches page says it 'stays with you through booking'”.
219	`def test_branch_choice_persists_across_pages(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch choice persists across pages*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
220	`    lagos = site.branch("Lagos")`	**Assignment**: binds the name *lagos* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.

Line	Code	Syntax & why
221	`    rooms = RoomsPage(page).open()`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
222	`    rooms.select_branch(lagos.id)`	**Method call** executed for its effect. *select_branch()*: project method RoomsPage.select_branch() in pages/rooms_page.py.
223	`    rooms.search.check_availability.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
224	`    expect(page).to_have_url(re.compile(lagos.id))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
225	`    page.goto("/")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
226	`    rooms.go_to("Rooms")`	**Method call** executed for its effect. *go_to()*: project method BasePage.go_to() in pages/base_page.py.
227	`    expect(rooms.branch_select).to_have_value(lagos.id)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
230	`# ---------- filters ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
231	`@pytest.mark.tc("TC-026")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-026 = “Room type filter”.
232	`def test_room_type_filter_and_untick_restores(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *room type filter and untick restores*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
233	`    original = rooms.room_count()`	**Assignment**: binds the name *original* to the value on the right. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
234	`    rooms.toggle("Deluxe Room", on=True)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
235	`    assert rooms.wait_for_count(lambda c: c < original) >= 1`	**assert**: fails the test if *rooms.wait_for_count(lambda c: c < original) >= 1* is false. pytest rewrites asserts to print the actual values on failure. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
236	`    assert all("Deluxe" in t for t in rooms.card_titles())`	**assert**: fails the test if *all(('Deluxe' in t for t in rooms.card_titles()))* is false. pytest rewrites asserts to print the actual values on failure. *all()* returns a list of Locators, one per current match, for iteration. *card_titles()*: project method RoomsPage.card_titles() in pages/rooms_page.py - Room-name links (same href as View details, but the visible room name)..
237	`    rooms.toggle("Deluxe Room", on=False)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
238	`    assert rooms.wait_for_count(lambda c: c == original) == original`	**assert**: fails the test if *rooms.wait_for_count(lambda c: c == original) == original* is false. pytest rewrites asserts to print the actual values on failure. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).

Line	Code	Syntax & why
241	`@pytest.mark.tc("TC-027")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-027 = “Bed type filter”.
242	`@pytest.mark.parametrize("bed", ["King", "Queen"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'bed'; each value becomes its own test (the id appears in the report).
243	`def test_bed_type_filter(rooms: RoomsPage, bed):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *bed type filter*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded; *bed* (value from @parametrize).
244	`    rooms.toggle(bed, on=True)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
245	`    count = rooms.wait_for_count(lambda c: c > 0)`	**Assignment**: binds the name *count* to the value on the right. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
246	`    assert count > 0, f"{bed} filter returned 0 rooms"`	**assert**: fails the test if *count > 0* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
247	`    assert all(re.search(rf"\b{bed}\b", t) for t in rooms.card_texts())`	**assert**: fails the test if *all((re.search(f'\\b{bed}\\b', t) for t in rooms.card_texts()))* is false. pytest rewrites asserts to print the actual values on failure. *all()* returns a list of Locators, one per current match, for iteration. *search()* finds the first regex match anywhere in a string. *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
250	`@pytest.mark.tc("TC-028")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-028 = “Multiple amenities filter uses AND logic”.
251	`def test_amenities_filter_uses_and_logic(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *amenities filter uses and logic*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
252	`    rooms = RoomsPage(page).open(branch_id=site.branch("Lagos").id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.
253	`    rooms.toggle("Mini fridge", on=True)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
254	`    one = rooms.wait_for_count(lambda c: True)`	**Assignment**: binds the name *one* to the value on the right. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
255	`    rooms.toggle("Work desk", on=True)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
256	`    both = rooms.wait_for_count(lambda c: c <= one)`	**Assignment**: binds the name *both* to the value on the right. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).

Line	Code	Syntax & why
257	`    assert both <= one, "adding a second amenity must narrow (AND), not widen (OR) the results"`	**assert**: fails the test if *both <= one* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
260	`@pytest.mark.tc("TC-079")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-079 = “Filter options are built from the branch inventory”.
261	`def test_filter_options_come_from_branch_inventory(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *filter options come from branch inventory*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
262	`    titles = set(rooms.card_titles())`	**Assignment**: binds the name *titles* to the value on the right. *set()* builds a set (unique values). *card_titles()*: project method RoomsPage.card_titles() in pages/rooms_page.py - Room-name links (same href as View details, but the visible room name)..
263	`    options = rooms.filter_options("Room type")`	**Assignment**: binds the name *options* to the value on the right. *filter_options()*: project method RoomsPage.filter_options() in pages/rooms_page.py - Labels of the checkboxes under a filter heading ('Room type', 'Bed type', 'Amenities')..
264	`    assert set(options) == titles, f"options {options} vs listed rooms {titles}"`	**assert**: fails the test if *set(options) == titles* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *set()* builds a set (unique values). Uses an **f-string** (f"...{expr}...") to build text with values.
265	`    for opt in options:`	**for** loop: repeats the indented block for every item, binding it to *opt* (iterating *options*).
266	`        rooms.toggle(opt, on=True)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
267	`        assert rooms.wait_for_count(lambda c: c > 0) > 0, f"'{opt}' returns 0 results"`	**assert**: fails the test if *rooms.wait_for_count(lambda c: c > 0) > 0* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate). Uses an **f-string** (f"...{expr}...") to build text with values.
268	`        rooms.toggle(opt, on=False)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
271	`@pytest.mark.tc("TC-029")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-029 = “Combined filters with no matches show empty state”.
272	`def test_incompatible_filters_show_empty_state(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *incompatible filters show empty state*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
273	`    rooms = RoomsPage(page).open(branch_id=site.branch("Lagos").id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.
274	`    rooms.toggle("Presidential", on=True)`	**Method call** executed for its effect. *toggle()*: project method RoomsPage.toggle() in pages/rooms_page.py.
275	`    low, _ = rooms.slider_bounds()`	**Assignment**: binds the name *(low, _)* to the value on the right. *slider_bounds()*: project method RoomsPage.slider_bounds() in pages/rooms_page.py.

Line	Code	Syntax & why
276	`    rooms.set_max_price(low)`	**Method call** executed for its effect. *set_max_price()*: project method RoomsPage.set_max_price() in pages/rooms_page.py.
277	`    assert rooms.wait_for_count(lambda c: c == 0) == 0`	**assert**: fails the test if *rooms.wait_for_count(lambda c: c == 0) == 0* is false. pytest rewrites asserts to print the actual values on failure. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
278	`    expect(rooms.empty_state).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
281	`@pytest.mark.tc("TC-015")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-015 = “Price slider filters list and updates count”.
282	`def test_price_slider_filters_and_updates_count(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *price slider filters and updates count*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
283	`    low, high = rooms.slider_bounds()`	**Assignment**: binds the name *(low, high)* to the value on the right. *slider_bounds()*: project method RoomsPage.slider_bounds() in pages/rooms_page.py.
284	`    at_high = rooms.room_count()`	**Assignment**: binds the name *at_high* to the value on the right. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
285	`    rooms.set_max_price(low)`	**Method call** executed for its effect. *set_max_price()*: project method RoomsPage.set_max_price() in pages/rooms_page.py.
286	`    at_low = rooms.wait_for_count(lambda c: c < at_high)`	**Assignment**: binds the name *at_low* to the value on the right. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
287	`    assert 0 < at_low < at_high`	**assert**: fails the test if *0 < at_low < at_high* is false. pytest rewrites asserts to print the actual values on failure.
288	`    prices = [int(re.search(r"₦([\d,]+) / night", t).group(1).replace(",", "")) for t in rooms.card_texts()]`	**Assignment**: binds the name *prices* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *int()* converts to integer. *replace()* string replace. *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
289	`    assert all(p <= low for p in prices), prices`	**assert**: fails the test if *all((p <= low for p in prices))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *all()* returns a list of Locators, one per current match, for iteration.
292	`@pytest.mark.tc("TC-015")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-015 = “Price slider filters list and updates count”.
293	`@pytest.mark.defect("BUG-009")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-009 = “Price filter has a single handle and no visible effect or result feedback”.

Line	Code	Syntax & why
294	`def test_clear_filters_control_restores_all_rooms(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *clear filters control restores all rooms*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
295	`    original = rooms.room_count()`	**Assignment**: binds the name *original* to the value on the right. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
296	`    rooms.set_max_price(rooms.slider_bounds()[0])`	**Method call** executed for its effect. *set_max_price()*: project method RoomsPage.set_max_price() in pages/rooms_page.py. *slider_bounds()*: project method RoomsPage.slider_bounds() in pages/rooms_page.py.
297	`    rooms.wait_for_count(lambda c: c < original)`	**Method call** executed for its effect. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
298	`    rooms.clear_filters.click(timeout=3000)`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
299	`    assert rooms.wait_for_count(lambda c: c == original) == original`	**assert**: fails the test if *rooms.wait_for_count(lambda c: c == original) == original* is false. pytest rewrites asserts to print the actual values on failure. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
302	`@pytest.mark.tc("TC-078")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-078 = “Price slider range matches real min and max room prices”.
303	`def test_price_slider_range_matches_room_prices(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *price slider range matches room prices*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
304	`    for city in ("Benin City", "Lagos"):`	**for** loop: repeats the indented block for every item, binding it to *city* (iterating *('Benin City', 'Lagos')*).
305	`        rooms = RoomsPage(page).open(branch_id=site.branch(city).id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.
306	`        low, high = rooms.slider_bounds()`	**Assignment**: binds the name *(low, high)* to the value on the right. *slider_bounds()*: project method RoomsPage.slider_bounds() in pages/rooms_page.py.
307	`        prices = [`	**Assignment**: binds the name *prices* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...).
308	`            int(re.search(r"₦([\d,]+) / night", t).group(1).replace(",", "")) for t in rooms.card_texts()`	**Continuation** of the statement started on line 307 (inside brackets). *int()* converts to integer. *replace()* string replace. *card_texts()*: project method RoomsPage.card_texts() in pages/rooms_page.py. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
309	`        ]`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
310	`        assert low <= min(prices) and high >= max(prices), f"{city}: slider {low}-{high} vs prices {prices}"`	**assert**: fails the test if *low <= min(prices) and high >= max(prices)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *min()* smallest value. *max()* largest value. Uses an **f-string** (f"...{expr}...") to build text with values.

Line	Code	Syntax & why
311	`assert rooms.price_slider.input_value() == str(high) or int(rooms.price_slider.input_value()) >= max(`	**assert**: fails the test if *rooms.price_slider.input_value() == str(high) or int(rooms.price_slider.input_v…* is false. pytest rewrites asserts to print the actual values on failure. *input_value()* current value of an input/select. *str()* converts to string. *int()* converts to integer. *max()* largest value.
312	`prices`	**Continuation** of the statement started on line 311 (inside brackets).
313	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
316	`@pytest.mark.tc("TC-030")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-030 = “Amenities list scrolls and all items are selectable”.
317	`def test_amenities_list_scrolls_and_last_item_selectable(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *amenities list scrolls and last item selectable*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
318	`rooms = RoomsPage(page).open(branch_id=site.branch("Lagos").id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call). *branch()*: project method SiteData.branch() in utils/site_data.py.
319	`options = rooms.filter_options("Amenities")`	**Assignment**: binds the name *options* to the value on the right. *filter_options()*: project method RoomsPage.filter_options() in pages/rooms_page.py - Labels of the checkboxes under a filter heading ('Room type', 'Bed type', 'Amenities')..
320	`last = rooms.filter_checkbox(options[-1])`	**Assignment**: binds the name *last* to the value on the right. *filter_checkbox()*: project method RoomsPage.filter_checkbox() in pages/rooms_page.py.
321	`last.scroll_into_view_if_needed()`	**Method call** executed for its effect. *scroll_into_view_if_needed()* scrolls the element into the viewport.
322	`last.check()`	**Method call** executed for its effect. *check()* ticks a checkbox (no-op if already ticked).
323	`expect(last).to_be_checked()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_checked()* waits until a checkbox is ticked.
326	`@pytest.mark.tc("TC-081")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-081 = “Filters survive reload and back navigation”.
327	`@pytest.mark.defect(reason="workbook TC-081 Fail, no bug logged: filters not kept in URL/state")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: 'workbook TC-081 Fail, no bug logged: filters not kept in URL/state'.
328	`def test_filters_survive_reload(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *filters survive reload*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
329	`low, _ = rooms.slider_bounds()`	**Assignment**: binds the name *(low, _)* to the value on the right. *slider_bounds()*: project method RoomsPage.slider_bounds() in pages/rooms_page.py.
330	`rooms.set_max_price(low)`	**Method call** executed for its effect. *set_max_price()*: project method RoomsPage.set_max_price() in pages/rooms_page.py.

Line	Code	Syntax & why
331	`    filtered = rooms.wait_for_count(lambda c: True)`	**Assignment**: binds the name *filtered* to the value on the right. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
332	`    rooms.page.reload()`	**Method call** executed for its effect. *reload()* reloads the current page.
333	`    assert rooms.wait_for_count(lambda c: c == filtered) == filtered, "filters lost on reload"`	**assert**: fails the test if *rooms.wait_for_count(lambda c: c == filtered) == filtered* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).
336	`@pytest.mark.mobile`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.mobile**: runs in a mobile viewport.
337	`@pytest.mark.tc("TC-080")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-080 = “Mobile filter drawer opens, applies and closes”.
338	`def test_mobile_filter_drawer(mobile_page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *mobile filter drawer*. Parameters: *mobile_page*: project fixture: a tab in a 375x812 touch-enabled mobile context; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
339	`    rooms = RoomsPage(mobile_page).open(branch_id=site.default_branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
340	`    original = rooms.room_count()`	**Assignment**: binds the name *original* to the value on the right. *room_count()*: project method RoomsPage.room_count() in pages/rooms_page.py.
341	`    rooms.filter_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
342	`    rooms.expect_on_screen(rooms.filter_drawer, on=True)`	**Method call** executed for its effect. *expect_on_screen()*: project method BasePage.expect_on_screen() in pages/base_page.py.
343	`    rooms.filter_drawer.get_by_role("checkbox", name="Deluxe Room").check()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *check()* ticks a checkbox (no-op if already ticked). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
344	`    expect(rooms.show_rooms_button).to_have_text(re.compile(r"Show 1 rooms?$"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
345	`    rooms.show_rooms_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
346	`    rooms.expect_on_screen(rooms.filter_drawer, on=False)`	**Method call** executed for its effect. *expect_on_screen()*: project method BasePage.expect_on_screen() in pages/base_page.py.
347	`    assert rooms.wait_for_count(lambda c: c < original) == 1`	**assert**: fails the test if *rooms.wait_for_count(lambda c: c < original) == 1* is false. pytest rewrites asserts to print the actual values on failure. *wait_for_count()*: project method RoomsPage.wait_for_count() in pages/rooms_page.py - Poll until predicate(count) is True (results re-render async after filters).. **lambda**: inline anonymous function (used as a callback/predicate).

Line	Code	Syntax & why
350	`@pytest.mark.mobile`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.mobile**: runs in a mobile viewport.
351	`@pytest.mark.tc("TC-080", "TC-009")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-080 = “Mobile filter drawer opens, applies and closes”; TC-009 = “Capacity text uses correct singular/plural”.
352	`@pytest.mark.defect(reason="drawer button reads 'Show 1 rooms' (plural with 1)")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: "drawer button reads 'Show 1 rooms' (plural with 1)".
353	`def test_mobile_filter_button_singular(mobile_page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *mobile filter button singular*. Parameters: *mobile_page*: project fixture: a tab in a 375x812 touch-enabled mobile context; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
354	`    rooms = RoomsPage(mobile_page).open(branch_id=site.default_branch.id, adults=1)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
355	`    rooms.filter_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
356	`    rooms.filter_drawer.get_by_role("checkbox", name="Deluxe Room").check()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *check()* ticks a checkbox (no-op if already ticked). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
357	`    expect(rooms.show_rooms_button).to_have_text("Show 1 room")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex.
360	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
361	`@pytest.mark.tc("TC-031", "TC-049")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-031 = “View details shows full room info and images”; TC-049 = “Core booking flow in Chrome, Firefox, Edge, Safari”.
362	`def test_view_details_opens_room_page_with_dates(rooms: RoomsPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *view details opens room page with dates*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded.
363	`    detail = rooms.open_first_room()`	**Assignment**: binds the name *detail* to the value on the right. *open_first_room()*: project method RoomsPage.open_first_room() in pages/rooms_page.py.
364	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).

Line	Code	Syntax & why
365	`    expect(detail.page).to_have_url(re.compile(r"checkIn=.*checkOut="))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

tests/functional/test_room_details.py

Room detail: content, breadcrumb, other rooms, steppers, date limits, price arithmetic, offers, currency format, not-found state.

Line	Code	Syntax & why
1	`"""Room detail page: content, dates, steppers, pricing, breadcrumb, not-found state."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`from datetime import date, timedelta`	**from datetime import date, timedelta**: imports specific names so they can be used without the module prefix. date/timedelta give calendar arithmetic (today + 7 days) for check-in/check-out dates. *date*: Calendar date type (no time component). *timedelta*: A duration; date + timedelta(days=2) gives a date two days later.
6	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
9	`from pages.room_detail_page import RoomDetailPage, naira`	**from pages.room_detail_page import RoomDetailPage, naira**: imports specific names so they can be used without the module prefix. Page Object for a single room's detail page.
10	`from pages.rooms_page import RoomsPage`	**from pages.rooms_page import RoomsPage**: imports specific names so they can be used without the module prefix. Page Object for the Rooms listing.
12	`FUTURE = date.today() + timedelta(days=30)`	Module-level **constant** *FUTURE* (UPPER_CASE by convention = not changed at runtime). *today()* today's date.
15	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
16	`def hotel_room(page, site) -> RoomDetailPage:`	**def** defines the fixture *hotel_room*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Returns RoomDetailPage.
17	`    """Hotel-level room (full content, full tax breakdown)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
18	`    d = RoomDetailPage(page).open_room(`	**Assignment**: binds the name *d* to the value on the right. *open_room()*: project method RoomDetailPage.open_room() in pages/room_detail_page.py. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).

Line	Code	Syntax & why
19	`        site.bookable_room["id"], checkIn=FUTURE, checkOut=FUTURE + timedelta(days=2)`	**Continuation** of the statement started on line 18 (inside brackets). Keyword argument(s) checkIn, checkOut, days (name=value): self-documenting and order-independent.
20	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
21	`    d.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
22	`    return d`	**return**: ends the function and hands back *d*.
25	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
26	`def branch_room(rooms: RoomsPage) -> RoomDetailPage:`	**def** defines the fixture *branch_room*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded. Returns RoomDetailPage.
27	`    d = rooms.open_first_room()`	**Assignment**: binds the name *d* to the value on the right. *open_first_room()*: project method RoomsPage.open_first_room() in pages/rooms_page.py.
28	`    d.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
29	`    return d`	**return**: ends the function and hands back *d*.
32	`@pytest.mark.tc("TC-031", "TC-088")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-031 = “View details shows full room info and images”; TC-088 = “Room detail shows size, in-room amenities, house rules and cancellation summary”.
33	`def test_view_details_shows_full_room_info(hotel_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *view details shows full room info*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room.
34	`    expect(hotel_room.title).not_to_be_empty()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_be_empty()* waits until an element has content.
35	`    for label in ("Bed", "Room size", "Guest capacity"):`	**for** loop: repeats the indented block for every item, binding it to *label* (iterating *('Bed', 'Room size', 'Guest capacity')*).
36	`        assert hotel_room.fact(label), f"'{label}' is empty"`	**assert**: fails the test if *hotel_room.fact(label)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *fact()*: project method RoomDetailPage.fact() in pages/room_detail_page.py - Value shown under a fact label such as 'Bed', 'Room size', 'Guest capacity'.. Uses an **f-string** (f"…{expr}…") to build text with values.
37	`    expect(hotel_room.amenities_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
38	`    expect(hotel_room.house_rules_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.

Line	Code	Syntax & why
39	`    expect(hotel_room.price_per_night).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
40	`    expect(hotel_room.continue_button).to_be_enabled()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_enabled()* waits until a control is enabled.
43	`@pytest.mark.tc("TC-031", "TC-088")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-031 = “View details shows full room info and images”; TC-088 = “Room detail shows size, in-room amenities, house rules and cancellation summary”.
44	`@pytest.mark.defect("BUG-030")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-030 = “Room detail page has empty sections, a stray separator and a broken breadcrumb”.
45	`def test_branch_room_detail_sections_populated(branch_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch room detail sections populated*. Parameters: *branch_room*: module fixture: room-detail page of the first room in the default branch.
46	`    assert branch_room.fact("Room size") not in ("", "Not listed"), "room size not listed"`	**assert**: fails the test if *branch_room.fact('Room size') not in ('', 'Not listed')* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *fact()*: project method RoomDetailPage.fact() in pages/room_detail_page.py - Value shown under a fact label such as 'Bed', 'Room size', 'Guest capacity'..
47	`    expect(branch_room.amenities_heading).to_be_visible(timeout=3000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
48	`    expect(branch_room.house_rules_heading).to_be_visible(timeout=3000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
51	`@pytest.mark.tc("TC-031", "TC-001")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-031 = “View details shows full room info and images”; TC-001 = “Room card shows image for every room type”.
52	`@pytest.mark.defect("BUG-001")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-001 = “Room images not rendered on Rooms listing cards (blank grey placeholder)”.
53	`def test_room_detail_has_photos(branch_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *room detail has photos*. Parameters: *branch_room*: module fixture: room-detail page of the first room in the default branch.
54	`    expect(branch_room.main.locator("img[alt]:not([alt=''])").first).to_be_visible(timeout=3000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

Line	Code	Syntax & why
57	`@pytest.mark.tc("TC-090")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-090 = “Other rooms block and breadcrumb show the branch and link correctly”.
58	`def test_branch_room_breadcrumb_and_other_rooms_name_the_branch(branch_room: RoomDetailPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch room breadcrumb and other rooms name the branch*. Parameters: *branch_room*: module fixture: room-detail page of the first room in the default branch; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
59	`    branch = site.default_branch.name`	**Assignment**: binds the name *branch* to the value on the right.
60	`    expect(branch_room.breadcrumb.get_by_role("link", name="Rooms")).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
61	`    expect(branch_room.other_rooms_heading).to_have_text(f"Other rooms at {branch}")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. Uses an **f-string** (f"...{expr}...") to build text with values.
62	`    expect(branch_room.main.get_by_text(re.compile(rf"^{re.escape(branch)} · "))).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. *escape()* escapes regex special characters in literal text. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
65	`@pytest.mark.tc("TC-090")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-090 = “Other rooms block and breadcrumb show the branch and link correctly”.
66	`@pytest.mark.defect("BUG-030")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-030 = “Room detail page has empty sections, a stray separator and a broken breadcrumb”.
67	`def test_breadcrumb_includes_branch(branch_room: RoomDetailPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *breadcrumb includes branch*. Parameters: *branch_room*: module fixture: room-detail page of the first room in the default branch; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
68	`    expect(branch_room.breadcrumb).to_contain_text(site.default_branch.city)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.

Line	Code	Syntax & why
71	`@pytest.mark.tc("TC-090")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-090 = “Other rooms block and breadcrumb show the branch and link correctly”.
72	`def test_other_room_link_opens_that_room(branch_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *other room link opens that room*. Parameters: *branch_room*: module fixture: room-detail page of the first room in the default branch.
73	`    link = branch_room.other_room_links.first`	**Assignment**: binds the name *link* to the value on the right.
74	`    href = link.get_attribute("href")`	**Assignment**: binds the name *href* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type).
75	`    link.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
76	`    expect(branch_room.page).to_have_url(re.compile(re.escape(href.split("?")[0])))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. *escape()* escapes regex special characters in literal text. *split()* splits a string into a list. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
79	`@pytest.mark.tc("TC-090")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-090 = “Other rooms block and breadcrumb show the branch and link correctly”.
80	`@pytest.mark.defect("BUG-003")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-003 = “Heading says "5 room types" but list renders repeated identical cards (Standard Room appears 3+ times) all marked "1 room left"”.
81	`def test_other_rooms_has_no_duplicates(branch_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *other rooms has no duplicates*. Parameters: *branch_room*: module fixture: room-detail page of the first room in the default branch. Docstring: “Benin lists 'Standard Room ₦25,000 / night' twice: same room type shown as separate rooms.”
82	`    """Benin lists 'Standard Room ₦25,000 / night' twice: same room type shown as separate rooms."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
83	`    expect(branch_room.other_room_links.first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
84	`    labels = branch_room.other_room_links.all_inner_texts()`	**Assignment**: binds the name *labels* to the value on the right. *all_inner_texts()* returns visible text of every match as a list of strings.
85	`    assert len(labels) == len(set(labels)), f"duplicate 'other rooms': {labels}"`	**assert**: fails the test if *len(labels) == len(set(labels))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *len()* length of a collection. *set()* builds a set (unique values). Uses an **f-string** (f"...{expr}...") to build text with values.
88	`@pytest.mark.tc("TC-013")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-013 = “Select room opens booking step with correct room and dates”.

Line	Code	Syntax & why
89	`def test_dates_are_carried_over(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *dates are carried over*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
90	`    rooms = RoomsPage(page).open(`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
91	`        branch_id=site.default_branch.id, adults=1, checkIn=FUTURE, checkOut=FUTURE + timedelta(days=3)`	**Continuation** of the statement started on line 90 (inside brackets). Keyword argument(s) branch_id, adults, checkIn, checkOut, days (name=value): self-documenting and order-independent.
92	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
93	`    detail = rooms.open_first_room()`	**Assignment**: binds the name *detail* to the value on the right. *open_first_room()*: project method RoomsPage.open_first_room() in pages/rooms_page.py.
94	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
95	`    expect(detail.check_in).to_have_value(str(FUTURE))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value. *str()* converts to string.
96	`    expect(detail.check_out).to_have_value(str(FUTURE + timedelta(days=3)))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value. *str()* converts to string.
99	`@pytest.mark.tc("TC-092")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-092 = “Search guest count carries to room detail and booking”.
100	`def test_search_guest_count_carries_to_detail(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *search guest count carries to detail*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
101	`    lagos = site.branch("Lagos")`	**Assignment**: binds the name *lagos* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
102	`    rooms = RoomsPage(page).open(branch_id=lagos.id, adults=2)`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
103	`    detail = rooms.open_first_room()`	**Assignment**: binds the name *detail* to the value on the right. *open_first_room()*: project method RoomsPage.open_first_room() in pages/rooms_page.py.
104	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
105	`    expect(page).to_have_url(re.compile(r"adults=2"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

Line	Code	Syntax & why
106	`    assert detail.stepper_value("Adults") == 2`	**assert**: fails the test if *detail.stepper_value('Adults') == 2* is false. pytest rewrites asserts to print the actual values on failure. *stepper_value()*: project method RoomDetailPage.stepper_value() in pages/room_detail_page.py - Number shown between the -/+ buttons for 'Adults' or 'Children'..
109	`@pytest.mark.tc("TC-089")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-089 = “Adults/children steppers respect room capacity”.
110	`def test_steppers_respect_capacity(hotel_room: RoomDetailPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *steppers respect capacity*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
111	`    cap = site.bookable_room["capacity"]`	**Assignment**: binds the name *cap* to the value on the right.
112	`    if cap["adults"] == 1:`	**if** statement: the indented block runs only when *cap['adults'] == 1* is true.
113	`        expect(hotel_room.stepper("Adults", "Increase")).to_be_disabled()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_disabled()* waits until a control is disabled. *stepper()*: project method RoomDetailPage.stepper() in pages/room_detail_page.py - who: 'Adults'|'Children'; direction: 'Increase'|'Decrease'..
114	`    expect(hotel_room.stepper("Adults", "Decrease")).to_be_disabled() # never below 1 adult`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_disabled()* waits until a control is disabled. *stepper()*: project method RoomDetailPage.stepper() in pages/room_detail_page.py - who: 'Adults'|'Children'; direction: 'Increase'|'Decrease'.. Inline comment: “never below 1 adult”.
115	`    expect(hotel_room.stepper("Children", "Decrease")).to_be_disabled() # never below 0 children`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_disabled()* waits until a control is disabled. *stepper()*: project method RoomDetailPage.stepper() in pages/room_detail_page.py - who: 'Adults'|'Children'; direction: 'Increase'|'Decrease'.. Inline comment: “never below 0 children”.
116	`    if cap["children"] == 0:`	**if** statement: the indented block runs only when *cap['children'] == 0* is true.
117	`        expect(hotel_room.stepper("Children", "Increase")).to_be_disabled()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_disabled()* waits until a control is disabled. *stepper()*: project method RoomDetailPage.stepper() in pages/room_detail_page.py - who: 'Adults'|'Children'; direction: 'Increase'|'Decrease'..
120	`@pytest.mark.tc("TC-062")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-062 = “Room detail blocks check-out on or before check-in”.
121	`def test_check_out_picker_min_is_day_after_check_in(hotel_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *check out picker min is day after check in*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room.
122	`    expect(hotel_room.check_out).to_have_attribute("min", str(FUTURE + timedelta(days=1)))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value. *str()* converts to string.
125	`@pytest.mark.tc("TC-062")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-062 = “Room detail blocks check-out on or before check-in”.

Line	Code	Syntax & why
126	`@pytest.mark.defect("BUG-014")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-014 = “Room detail accepts a check-out date before check-in and continues to the booking step (charged as 1 night)”.
127	`@pytest.mark.parametrize("offset", [-5, 0], ids=["before", "same-day"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'offset'; each value becomes its own test (the id appears in the report).
128	`def test_continue_blocked_when_check_out_not_after_check_in(hotel_room: RoomDetailPage, offset):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *continue blocked when check out not after check in*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room; *offset* (value from @parametrize).
129	`    hotel_room.check_out.fill(str(FUTURE + timedelta(days=offset)))`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *str()* converts to string.
130	`    hotel_room.page.wait_for_timeout(500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
131	`    expect(hotel_room.continue_button).to_be_disabled(timeout=2000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_disabled()* waits until a control is disabled.
134	`@pytest.mark.tc("TC-063")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-063 = “Room detail blocks past dates and stays over 90 nights”.
135	`def test_stay_limit_is_enforced_by_picker(hotel_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *stay limit is enforced by picker*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room.
136	`    max_out = hotel_room.check_out.get_attribute("max")`	**Assignment**: binds the name *max_out* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type).
137	`    assert max_out, "no maximum check-out date"`	**assert**: fails the test if *max_out* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
138	`    assert (date.fromisoformat(max_out) - FUTURE).days <= 92`	**assert**: fails the test if *(date.fromisoformat(max_out) - FUTURE).days <= 92* is false. pytest rewrites asserts to print the actual values on failure. *fromisoformat()* parses YYYY-MM-DD into a date.
141	`@pytest.mark.tc("TC-063")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-063 = “Room detail blocks past dates and stays over 90 nights”.
142	`@pytest.mark.defect("BUG-014")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-014 = “Room detail accepts a check-out date before check-in and continues to the booking step (charged as 1 night)”.
143	`def test_past_dates_from_url_are_rejected(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *past dates from url are rejected*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).

Line	Code	Syntax & why
144	`    detail = RoomDetailPage(page).open_room(`	**Assignment**: binds the name *detail* to the value on the right. *open_room()*: project method RoomDetailPage.open_room() in pages/room_detail_page.py. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
145	`        site.bookable_room["id"], checkIn="2020-01-01", checkOut="2020-01-03"`	**Continuation** of the statement started on line 144 (inside brackets). Keyword argument(s) checkIn, checkOut (name=value): self-documenting and order-independent.
146	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
147	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
148	`    detail.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
149	`    page.wait_for_timeout(1500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
150	`    assert "/booking" not in page.url, "past stay accepted and taken to /booking"`	**assert**: fails the test if *'/booking' not in page.url* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
153	`@pytest.mark.tc("TC-025")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-025 = “Nights and total price computed correctly”.
154	`def test_total_equals_nights_times_rate_plus_listed_charges(hotel_room: RoomDetailPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *total equals nights times rate plus listed charges*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
155	`    block = hotel_room.price_block_text()`	**Assignment**: binds the name *block* to the value on the right. *price_block_text()*: project method RoomDetailPage.price_block_text() in pages/room_detail_page.py.
156	`    nights = int(re.search(r"Room · (\d+) nights?", block).group(1))`	**Assignment**: binds the name *nights* to the value on the right. *int()* converts to integer. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
157	`    room_total = naira(re.search(r"Room · \d+ nights?\s*(₦[\d,]+)", block).group(1))`	**Assignment**: binds the name *room_total* to the value on the right. *naira()*: project function naira() in pages/room_detail_page.py - '₦56,250' -> 56250 (first amount in the string).. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
158	`    assert room_total == nights * site.bookable_room["price"]`	**assert**: fails the test if *room_total == nights * site.bookable_room['price']* is false. pytest rewrites asserts to print the actual values on failure.
159	`    amounts = [int(a.replace(",", "")) for a in re.findall(r"₦([\d,]+)", block)]`	**Assignment**: binds the name *amounts* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *int()* converts to integer. *replace()* string replace. *findall()* returns every regex match. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
160	`    total = hotel_room.total_amount()`	**Assignment**: binds the name *total* to the value on the right. *total_amount()*: project method RoomDetailPage.total_amount() in pages/room_detail_page.py.
161	`    charges = amounts[: amounts.index(total)] if total in amounts else amounts[:-1]`	**Assignment**: binds the name *charges* to the value on the right. **Conditional expression** (a if condition else b). *index()* position of a substring/item.

Line	Code	Syntax & why
162	`    assert abs(sum(charges) - total) <= len(charges), f"lines {charges} do not add up to {total}"`	**assert**: fails the test if *abs(sum(charges) - total) <= len(charges)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *abs()* absolute value. *len()* length of a collection. *sum()* adds numbers. Uses an **f-string** (f"...{expr}...") to build text with values.
165	`@pytest.mark.tc("TC-068")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-068 = “5-night stay applies 'fifth night on us'”.
166	`@pytest.mark.defect("BUG-019")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-019 = “Advertised 'fifth night on us' offer is not applied to a 5-night stay”.
167	`def test_fifth_night_free_offer_applied(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *fifth night free offer applied*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
168	`    page.goto("/")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
169	`    if not page.get_by_text(re.compile(r"fifth night", re.I)).count():`	**if** statement: the indented block runs only when *not page.get_by_text(re.compile('fifth night', re.I)).count()* is true. *count()* number of current matches (does not wait). *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
170	`        pytest.skip("'fifth night on us' offer is no longer advertised on Home")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
171	`    detail = RoomDetailPage(page).open_room(`	**Assignment**: binds the name *detail* to the value on the right. *open_room()*: project method RoomDetailPage.open_room() in pages/room_detail_page.py. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
172	`        site.bookable_room["id"], checkIn=FUTURE, checkOut=FUTURE + timedelta(days=5)`	**Continuation** of the statement started on line 171 (inside brackets). Keyword argument(s) checkIn, checkOut, days (name=value): self-documenting and order-independent.
173	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
174	`    room_total = naira(re.search(r"Room · 5 nights\s*(₦[\d,]+)", detail.price_block_text()).group(1))`	**Assignment**: binds the name *room_total* to the value on the right. *naira()*: project function naira() in pages/room_detail_page.py - '₦56,250' -> 56250 (first amount in the string).. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. *price_block_text()*: project method RoomDetailPage.price_block_text() in pages/room_detail_page.py. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
175	`    assert room_total == 4 * site.bookable_room["price"]`	**assert**: fails the test if *room_total == 4 * site.bookable_room['price']* is false. pytest rewrites asserts to print the actual values on failure.
178	`@pytest.mark.tc("TC-097")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-097 = “'Pay at the hotel / unpaid' messaging is consistent and clear”.

Line	Code	Syntax & why
179	`def test_unpaid_pay_at_hotel_notice(hotel_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *unpaid pay at hotel notice*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room.
180	`    expect(hotel_room.unpaid_notice).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
183	`@pytest.mark.tc("TC-122")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-122 = “Currency formatting and rounding (Naira)”.
184	`def test_currency_format(hotel_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *currency format*. Parameters: *hotel_room*: module fixture: room-detail page of a bookable hotel-level room.
185	`    amounts = re.findall(r"₦[\d,.]+", hotel_room.main.inner_text())`	**Assignment**: binds the name *amounts* to the value on the right. *findall()* returns every regex match. *inner_text()* visible text of the element (as rendered, respects CSS). Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
186	`    assert amounts`	**assert**: fails the test if *amounts* is false. pytest rewrites asserts to print the actual values on failure.
187	`    bad = [a for a in amounts if not re.fullmatch(r"₦\d{1,3}(,\d{3})*", a)]`	**Assignment**: binds the name *bad* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *fullmatch()* regex must match the whole string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
188	`    assert not bad, f"non N#,##0 amounts: {bad}"`	**assert**: fails the test if *not bad* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
191	`@pytest.mark.tc("TC-091", "TC-052")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-091 = “Invalid room ID shows a not-found state”; TC-052 = “Skeleton loaders always resolve”.
192	`def test_invalid_room_id_shows_not_found(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *invalid room id shows not found*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
193	`    detail = RoomDetailPage(page).open("/rooms/does-not-exist")`	**Assignment**: binds the name *detail* to the value on the right. *open()* opens a file (read/write). *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
194	`    expect(detail.not_found).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
195	`    expect(detail.back_to_rooms).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
198	`@pytest.mark.tc("TC-091", "TC-060")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-091 = “Invalid room ID shows a not-found state”; TC-060 = “Unknown URL shows a branded 404 page”.

Line	Code	Syntax & why
199	`@pytest.mark.defect("BUG-037")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-037 = “Unknown or invalid room URL shows an endless skeleton instead of a 'room not found' page”.
200	`def test_invalid_room_id_returns_http_404(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *invalid room id returns http 404*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
201	`    resp = page.goto("/rooms/does-not-exist")`	**Assignment**: binds the name *resp* to the value on the right. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
202	`    assert resp.status == 404`	**assert**: fails the test if *resp.status == 404* is false. pytest rewrites asserts to print the actual values on failure.
205	`@pytest.mark.tc("TC-013")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-013 = “Select room opens booking step with correct room and dates”.
206	`def test_back_to_rooms_via_nav(branch_room: RoomDetailPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *back to rooms via nav*. Parameters: *branch_room*: module fixture: room-detail page of the first room in the default branch.
207	`    branch_room.go_to("Rooms")`	**Method call** executed for its effect. *go_to()*: project method BasePage.go_to() in pages/base_page.py.
208	`    branch_room.expect_url_contains("/rooms")`	**Method call** executed for its effect. *expect_url_contains()*: project method BasePage.expect_url_contains() in pages/base_page.py.

tests/functional/test_booking_flow.py

Booking wizard end to end without confirming (no live writes): validation, consent, review content, totals across steps, edit links, browser back, PII in URLs. Confirm booking runs only with ALLOW_LIVE_WRITES.

Line	Code	Syntax & why
1	`"""Booking wizard: guest details validation, consent, review step, edit links, confirmation."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`from datetime import date, timedelta`	**from datetime import date, timedelta**: imports specific names so they can be used without the module prefix. date/timedelta give calendar arithmetic (today + 7 days) for check-in/check-out dates. *date*: Calendar date type (no time component). *timedelta*: A duration; date + timedelta(days=2) gives a date two days later.
6	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.

Line	Code	Syntax & why
7	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
9	`from pages.booking_page import BookingPage`	**from pages.booking_page import BookingPage**: imports specific names so they can be used without the module prefix. Page Object for the booking wizard (/booking).
10	`from pages.room_detail_page import RoomDetailPage`	**from pages.room_detail_page import RoomDetailPage**: imports specific names so they can be used without the module prefix. Page Object for a single room's detail page.
11	`from pages.rooms_page import RoomsPage`	**from pages.rooms_page import RoomsPage**: imports specific names so they can be used without the module prefix. Page Object for the Rooms listing.
12	`from utils.data_factory import GuestData`	**from utils.data_factory import GuestData**: imports specific names so they can be used without the module prefix. Builders for unique test data (users, guests).
14	`FUTURE = date.today() + timedelta(days=45)`	Module-level **constant** *FUTURE* (UPPER_CASE by convention = not changed at runtime). *today()* today's date.
17	`def _start(page, room_id: str, **params) -> BookingPage:`	**def** defines the private helper *_start* (leading underscore = internal use). Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *room_id*. ****params** collects any extra keyword arguments into a dict. Returns BookingPage.
18	`    detail = RoomDetailPage(page).open_room(`	**Assignment**: binds the name *detail* to the value on the right. *open_room()*: project method RoomDetailPage.open_room() in pages/room_detail_page.py. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
19	`        room_id, checkIn=FUTURE, checkOut=FUTURE + timedelta(days=2), **params`	**Continuation** of the statement started on line 18 (inside brackets). Keyword argument(s) checkIn, checkOut, days (name=value): self-documenting and order-independent.
20	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
21	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
22	`    booking = detail.continue_to_guest_details()`	**Assignment**: binds the name *booking* to the value on the right. *continue_to_guest_details()*: project method RoomDetailPage.continue_to_guest_details() in pages/room_detail_page.py.
23	`    booking.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
24	`    return booking`	**return**: ends the function and hands back *booking*.
27	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
28	`def guest_step(page, site) -> BookingPage:`	**def** defines the fixture *guest_step*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Returns BookingPage.
29	`    """Hotel-level room -> guest-details step."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.

Line	Code	Syntax & why
30	`    return _start(page, site.bookable_room["id"])`	**return**: ends the function and hands back *_start(page, site.bookable_room['id'])*.
33	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
34	`def branch_guest_step(rooms: RoomsPage) -> BookingPage:`	**def** defines the fixture *branch_guest_step*. Parameters: *rooms*: project fixture: Rooms list for the default branch, 1 adult, already loaded. Returns BookingPage.
35	`    """Default branch -> first room via 'Select room' -> guest-details step."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
36	`    rooms.select_room_links.first.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
37	`    detail = RoomDetailPage(rooms.page)`	**Assignment**: binds the name *detail* to the value on the right. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
38	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
39	`    booking = detail.continue_to_guest_details()`	**Assignment**: binds the name *booking* to the value on the right. *continue_to_guest_details()*: project method RoomDetailPage.continue_to_guest_details() in pages/room_detail_page.py.
40	`    booking.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
41	`    return booking`	**return**: ends the function and hands back *booking*.
44	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
45	`@pytest.mark.tc("TC-013", "TC-049")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-013 = “Select room opens booking step with correct room and dates”; TC-049 = “Core booking flow in Chrome, Firefox, Edge, Safari”.
46	`def test_select_room_opens_booking_step(branch_guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *select room opens booking step*. Parameters: *branch_guest_step*: module fixture: a branch room booked up to the guest-details step via 'Select room'.
47	`    expect(branch_guest_step.heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
48	`    expect(branch_guest_step.total).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
51	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.

Line	Code	Syntax & why
52	`@pytest.mark.tc("TC-013")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-013 = “Select room opens booking step with correct room and dates”.
53	`def test_reach_guest_details_step(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *reach guest details step*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
54	`    expect(guest_step.heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
55	`    expect(guest_step.total).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
58	`@pytest.mark.tc("TC-093")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-093 = “Booking step indicator shows the current step”.
59	`def test_step_indicator(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *step indicator*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
60	`    expect(guest_step.steps.first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
61	`    guest_step.go_to_review(GuestData())`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
62	`    expect(guest_step.main).to_contain_text(re.compile(r"Your stay\s*Details\s*3\s*Review & book"))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
65	`@pytest.mark.tc("TC-032")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-032 = “Mandatory guest fields validated”.
66	`def test_mandatory_guest_fields_validated(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *mandatory guest fields validated*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
67	`    guest_step.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
68	`    for err in (`	**for** loop: repeats the indented block for every item, binding it to *err* (iterating *(guest_step.name_error, guest_step.email_error, guest_step.…)*.
69	`        guest_step.name_error,`	**Continuation** of the statement (inside brackets).
70	`        guest_step.email_error,`	**Continuation** of the statement (inside brackets).
71	`        guest_step.phone_error,`	**Continuation** of the statement (inside brackets).
72	`        guest_step.consent_error,`	**Continuation** of the statement (inside brackets).

Line	Code	Syntax & why
73	`    ):`	**Continuation** of the statement (inside brackets).
74	`        expect(err).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
75	`    expect(guest_step.review_heading).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
78	`@pytest.mark.tc("TC-033")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-033 = “Email and phone format validation”.
79	`def test_email_and_phone_format_validation(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *email and phone format validation*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
80	`    guest_step.guest_name.fill("Ada Obi")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
81	`    guest_step.guest_email.fill("ada@")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
82	`    guest_step.guest_phone.fill("abc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
83	`    guest_step.accept_consent()`	**Method call** executed for its effect. *accept_consent()*: project method BookingPage.accept_consent() in pages/booking_page.py.
84	`    guest_step.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
85	`    # 'ada@' is stopped by the browser (type=email) or by the app message`	**Comment** (# ... ignored by Python): documents intent for the reader.
86	`    assert not guest_step.guest_email.evaluate("e => e.validity.valid") or guest_step.email_error.is_visible()`	**assert**: fails the test if *not guest_step.guest_email.evaluate('e => e.validity.valid') or guest_step.emai…* is false. pytest rewrites asserts to print the actual values on failure. *is_visible()* True/False right now (does not wait) - used for branching, never for asserting. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
87	`    expect(guest_step.review_heading).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
88	`    guest_step.guest_email.fill("ada@example.com")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
89	`    guest_step.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
90	`    expect(guest_step.phone_error).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
93	`@pytest.mark.tc("TC-033")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-033 = “Email and phone format validation”.

Line	Code	Syntax & why
94	`@pytest.mark.parametrize(`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'phone'; each value becomes its own test (the id appears in the report).
95	`    "phone", ["+2348012345678", "08012345678", "08112345678", "07012345678", "09012345678"]`	**Continuation** of the statement (inside brackets).
96	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
97	`def test_nigerian_phone_formats_accepted(guest_step: BookingPage, phone):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *nigerian phone formats accepted*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step; *phone* (value from @parametrize).
98	`    guest_step.guest_phone.fill(phone)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
99	`    guest_step.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
100	`    expect(guest_step.phone_error).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
103	`@pytest.mark.tc("TC-094")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-094 = “Consent checkbox is required and links to a working privacy policy”.
104	`def test_consent_is_required(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *consent is required*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
105	`    guest_step.fill_guest(GuestData())`	**Method call** executed for its effect. *fill_guest()*: project method BookingPage.fill_guest() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
106	`    guest_step.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
107	`    expect(guest_step.consent_error).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
108	`    expect(guest_step.review_heading).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
111	`@pytest.mark.tc("TC-094", "TC-069")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-094 = “Consent checkbox is required and links to a working privacy policy”; TC-069 = “Privacy, Terms and Cancellation policy pages load”.
112	`def test_consent_privacy_link_works(page, guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *consent privacy link works*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *guest_step*: module fixture: hotel room booked up to the guest-details step.

Line	Code	Syntax & why
113	`    href = guest_step.privacy_link.get_attribute("href")`	**Assignment**: binds the name *href* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type).
114	`    assert page.request.get(href).status == 200`	**assert**: fails the test if *page.request.get(href).status == 200* is false. pytest rewrites asserts to print the actual values on failure. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
117	`@pytest.mark.tc("TC-120")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-120 = “Guest name and identifier character validation”.
118	`@pytest.mark.defect("BUG-047")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-047 = “Weak input validation: guest name accepts HTML/script and digits; sign-in identifier not format-checked”.
119	`@pytest.mark.parametrize(`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'name'; each value becomes its own test (the id appears in the report).
120	`    "name", ["<script>alert(1)</script>", "12345", "A"], ids=["script", "digits", "one-char"]`	**Continuation** of the statement (inside brackets). Keyword argument(s) ids (name=value): self-documenting and order-independent.
121	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
122	`def test_guest_name_character_validation(guest_step: BookingPage, name):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *guest name character validation*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step; *name* (value from @parametrize).
123	`    guest_step.fill_guest(GuestData(full_name=name)).accept_consent()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *accept_consent()*: project method BookingPage.accept_consent() in pages/booking_page.py. *fill_guest()*: project method BookingPage.fill_guest() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
124	`    guest_step.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
125	`    expect(guest_step.review_heading).to_have_count(0, timeout=3000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
128	`@pytest.mark.tc("TC-120", "TC-059")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-120 = “Guest name and identifier character validation”; TC-059 = “Booking and guest data not exposed in URL or API”.
129	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
130	`def test_script_in_guest_name_is_rendered_as_text(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *script in guest name is rendered as text*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
131	`    fired = []`	**Assignment**: binds the name *fired* to the value on the right. The value is a **list** literal [...].

Line	Code	Syntax & why
132	`    guest_step.page.on("dialog", lambda d: (fired.append(d.message), d.dismiss()))`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).
133	`    payload = "<img src=x onerror=alert(1)>"`	**Assignment**: binds the name *payload* to the value on the right.
134	`    guest_step.fill_guest(GuestData(full_name=payload)).accept_consent()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *accept_consent()*: project method BookingPage.accept_consent() in pages/booking_page.py. *fill_guest()*: project method BookingPage.fill_guest() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
135	`    guest_step.review_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
136	`    guest_step.page.wait_for_timeout(1500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
137	`    if guest_step.review_heading.is_visible(): # accepted -> must be shown as plain text`	**if** statement: the indented block runs only when *guest_step.review_heading.is_visible()* is true. *is_visible()* True/False right now (does not wait) - used for branching, never for asserting. Inline comment: “accepted -> must be shown as plain text”.
138	`        expect(guest_step.main.get_by_text(payload)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex).
139	`    assert not fired, "script executed from guest name"`	**assert**: fails the test if *not fired* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
142	`@pytest.mark.tc("TC-095", "TC-066")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-095 = “Review step shows stay, guest and price, with working edit links”; TC-066 = “Total and breakdown identical on room detail, guest details and Review”.
143	`def test_review_shows_stay_guest_and_price(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *review shows stay guest and price*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
144	`    guest = GuestData()`	**Assignment**: binds the name *guest* to the value on the right. *GuestData(...)* creates a GuestData object (constructor call).
145	`    guest_step.go_to_review(guest)`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py.
146	`    main = guest_step.main`	**Assignment**: binds the name *main* to the value on the right.
147	`    expect(main.get_by_text(guest.full_name, exact=True)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex).

Line	Code	Syntax & why
148	`    expect(main.get_by_text(re.compile(re.escape(guest.email)))).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. *escape()* escapes regex special characters in literal text. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
149	`    expect(main.get_by_text(re.compile(r"· 2 nights · 1 adult"))).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
150	`    expect(guest_step.confirm_button).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
153	`@pytest.mark.tc("TC-066")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-066 = “Total and breakdown identical on room detail, guest details and Review”.
154	`def test_total_identical_on_detail_details_and_review(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *total identical on detail details and review*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
155	`    detail = RoomDetailPage(page).open_room(`	**Assignment**: binds the name *detail* to the value on the right. *open_room()*: project method RoomDetailPage.open_room() in pages/room_detail_page.py. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
156	`        site.bookable_room["id"], checkIn=FUTURE, checkOut=FUTURE + timedelta(days=2)`	**Continuation** of the statement started on line 155 (inside brackets). Keyword argument(s) checkIn, checkOut, days (name=value): self-documenting and order-independent.
157	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
158	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
159	`    on_detail = detail.total_amount()`	**Assignment**: binds the name *on_detail* to the value on the right. *total_amount()*: project method RoomDetailPage.total_amount() in pages/room_detail_page.py.
160	`    booking = detail.continue_to_guest_details()`	**Assignment**: binds the name *booking* to the value on the right. *continue_to_guest_details()*: project method RoomDetailPage.continue_to_guest_details() in pages/room_detail_page.py.
161	`    booking.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).
162	`    on_details = booking.total_amounts()`	**Assignment**: binds the name *on_details* to the value on the right. *total_amounts()*: project method BookingPage.total_amounts() in pages/booking_page.py - Every 'Total ₦x' figure on the page (review panel + sidebar)..

Line	Code	Syntax & why
163	`    booking.go_to_review(GuestData())`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
164	`    on_review = booking.total_amounts()`	**Assignment**: binds the name *on_review* to the value on the right. *total_amounts()*: project method BookingPage.total_amounts() in pages/booking_page.py - Every 'Total ₦x' figure on the page (review panel + sidebar)..
165	`    assert set(on_details) == {on_detail} and set(on_review) == {on_detail}, (`	**assert**: fails the test if *set(on_details) == {on_detail} and set(on_review) == {on_detail}* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *set()* builds a set (unique values).
166	`        on_detail,`	**Continuation** of the statement started on line 165 (inside brackets).
167	`        on_details,`	**Continuation** of the statement started on line 165 (inside brackets).
168	`        on_review,`	**Continuation** of the statement started on line 165 (inside brackets).
169	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
172	`@pytest.mark.tc("TC-066", "TC-095")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-066 = “Total and breakdown identical on room detail, guest details and Review”; TC-095 = “Review step shows stay, guest and price, with working edit links”.
173	`@pytest.mark.defect("BUG-017")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-017 = “Price total changes between room detail and Review: different taxes and service-charge rate at each step”.
174	`def test_branch_room_review_shows_a_price(branch_guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch room review shows a price*. Parameters: *branch_guest_step*: module fixture: a branch room booked up to the guest-details step via 'Select room'.
175	`    branch_guest_step.go_to_review(GuestData())`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
176	`    expect(branch_guest_step.main.get_by_text(re.compile("Couldn't load a price estimate"))).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
179	`@pytest.mark.tc("TC-066")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-066 = “Total and breakdown identical on room detail, guest details and Review”.
180	`@pytest.mark.defect("BUG-017")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-017 = “Price total changes between room detail and Review: different taxes and service-charge rate at each step”.

Line	Code	Syntax & why
181	`def test_branch_room_total_is_final_not_estimate(branch_guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch room total is final not estimate*. Parameters: *branch_guest_step*: module fixture: a branch room booked up to the guest-details step via 'Select room'.
182	`    expect(branch_guest_step.main.get_by_text(re.compile(r"\(estimate\)")).first).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
185	`@pytest.mark.tc("TC-067")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-067 = “Tax and fee lines are guest-facing, named clearly and calculated correctly”.
186	`def test_tax_lines_are_guest_facing(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *tax lines are guest facing*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
187	`    guest_step.go_to_review(GuestData())`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
188	`    text = guest_step.main.inner_text()`	**Assignment**: binds the name *text* to the value on the right. *inner_text()* visible text of the element (as rendered, respects CSS).
189	`    assert not re.search(r"\bComm\b", text), "internal 'Comm' line shown to guest"`	**assert**: fails the test if *not re.search('\\bComm\\b', text)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
192	`@pytest.mark.tc("TC-067")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-067 = “Tax and fee lines are guest-facing, named clearly and calculated correctly”.
193	`def test_tax_amounts_match_their_rates(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *tax amounts match their rates*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
194	`    detail = RoomDetailPage(page).open_room(`	**Assignment**: binds the name *detail* to the value on the right. *open_room()*: project method RoomDetailPage.open_room() in pages/room_detail_page.py. *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
195	`        site.bookable_room["id"], checkIn=FUTURE, checkOut=FUTURE + timedelta(days=2)`	**Continuation** of the statement started on line 194 (inside brackets). Keyword argument(s) checkIn, checkOut, days (name=value): self-documenting and order-independent.
196	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
197	`    detail.expect_loaded()`	**Method call** executed for its effect. *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism).

Line	Code	Syntax & why
198	`    block = detail.price_block_text()`	**Assignment**: binds the name *block* to the value on the right. *price_block_text()*: project method RoomDetailPage.price_block_text() in pages/room_detail_page.py.
199	`    room = int(re.search(r"Room · \d+ nights?\s*₦([\d,]+)", block).group(1).replace(",", ""))`	**Assignment**: binds the name *room* to the value on the right. *int()* converts to integer. *replace()* string replace. *group()* returns a captured regex group. *search()* finds the first regex match anywhere in a string. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
200	`    for name, rate, amount in re.findall(r"([A-Za-z][A-Za-z ]+?) ([\d.]+)%[^₦]*₦([\d,]+)", block):`	**for** loop: repeats the indented block for every item, binding it to *(name, rate, amount)* (iterating *re.findall('([A-Za-z][A-Za-z]+?) ([\\d.]+)%[^₦]*₦([\\d,]+)…)*. *findall()* returns every regex match. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
201	`        expected = room * float(rate) / 100`	**Assignment**: binds the name *expected* to the value on the right. *float()* converts to float.
202	`        assert abs(int(amount.replace(",", "")) - expected) <= 1, (`	**assert**: fails the test if *abs(int(amount.replace(',', '')) - expected) <= 1* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *abs()* absolute value. *int()* converts to integer. *replace()* string replace.
203	`            f"{name.strip()} {rate}% of {room} != {amount}"`	**Continuation** of the statement started on line 202 (inside brackets). *strip()* removes surrounding whitespace. Uses an **f-string** (f"...{expr}...") to build text with values.
204	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
207	`@pytest.mark.tc("TC-095")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-095 = “Review step shows stay, guest and price, with working edit links”.
208	`@pytest.mark.defect("BUG-049")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-049 = “Editing from the Review step clears the guest details already entered”.
209	`def test_edit_details_keeps_everything_incl_consent(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *edit details keeps everything incl consent*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
210	`    guest = GuestData()`	**Assignment**: binds the name *guest* to the value on the right. *GuestData(...)* creates a GuestData object (constructor call).
211	`    guest_step.go_to_review(guest)`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py.
212	`    guest_step.edit_details.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
213	`    expect(guest_step.guest_name).to_have_value(guest.full_name)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
214	`    expect(guest_step.guest_email).to_have_value(guest.email)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
215	`    expect(guest_step.guest_phone).to_have_value(guest.phone)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.

Line	Code	Syntax & why
216	`    expect(guest_step.consent).to_be_checked()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_checked()* waits until a checkbox is ticked.
219	`@pytest.mark.tc("TC-095")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-095 = “Review step shows stay, guest and price, with working edit links”.
220	`def test_back_to_guest_details_keeps_contact_data(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *back to guest details keeps contact data*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
221	`    guest = GuestData()`	**Assignment**: binds the name *guest* to the value on the right. *GuestData(...)* creates a GuestData object (constructor call).
222	`    guest_step.go_to_review(guest)`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py.
223	`    guest_step.back_to_details.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
224	`    expect(guest_step.guest_name).to_have_value(guest.full_name)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
225	`    expect(guest_step.guest_email).to_have_value(guest.email)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
226	`    expect(guest_step.guest_phone).to_have_value(guest.phone)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
229	`@pytest.mark.tc("TC-095", "TC-014")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-095 = “Review step shows stay, guest and price, with working edit links”; TC-014 = “Selecting a room then browser back preserves search state”.
230	`def test_change_room_or_dates_keeps_dates(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *change room or dates keeps dates*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
231	`    guest_step.go_to_review(GuestData())`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
232	`    guest_step.change_room.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
233	`    expect(guest_step.page).to_have_url(`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches.
234	`        re.compile(rf"/rooms/.*checkIn={FUTURE}.*checkOut={FUTURE + timedelta(days=2)}")`	**Continuation** of the statement started on line 233 (inside brackets). Keyword argument(s) checkIn, checkOut, days (name=value): self-documenting and order-independent. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

Line	Code	Syntax & why
235	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
238	`@pytest.mark.tc("TC-014")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-014 = “Selecting a room then browser back preserves search state”.
239	`@pytest.mark.defect(reason="'Back to the room' lands on the /rooms list instead of the selected room")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: "'Back to the room' lands on the /rooms list instead of the selected room".
240	`def test_back_to_room_returns_to_detail_page(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *back to room returns to detail page*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
241	`    guest_step.back_to_room.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
242	`    expect(guest_step.page).to_have_url(re.compile(r"/rooms/[0-9a-f-]{36}")) `	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
245	`@pytest.mark.tc("TC-014")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-014 = “Selecting a room then browser back preserves search state”.
246	`def test_browser_back_from_room_preserves_search_state(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *browser back from room preserves search state*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
247	`    lekki = site.branch("Lekki")`	**Assignment**: binds the name *lekki* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
248	`    rooms = RoomsPage(page).open(`	**Assignment**: binds the name *rooms* to the value on the right. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
249	`        branch_id=lekki.id, adults=1, checkIn=FUTURE, checkOut=FUTURE + timedelta(days=2)`	**Continuation** of the statement started on line 248 (inside brackets). Keyword argument(s) branch_id, adults, checkIn, checkOut, days (name=value): self-documenting and order-independent.
250	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
251	`    rooms.select_room_links.first.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
252	`    RoomDetailPage(page).expect_loaded()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *expect_loaded()*: project method defined on several page objects (BookingPage, HomePage, RoomDetailPage); Python runs the version of the object it is called on (polymorphism). *RoomDetailPage(...)* creates a RoomDetailPage object (constructor call).
253	`    page.go_back()`	**Method call** executed for its effect. *go_back()* browser Back button.

Line	Code	Syntax & why
254	`    expect(rooms.branch_select).to_have_value(lekki.id)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
255	`    expect(rooms.check_in).to_have_value(str(FUTURE))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value. *str()* converts to string.
256	`    expect(rooms.guests).to_have_value("1")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_value()* waits until an input/select has the value.
259	`@pytest.mark.tc("TC-097")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-097 = "'Pay at the hotel / unpaid' messaging is consistent and clear".
260	`def test_unpaid_messaging_on_review(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *unpaid messaging on review*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
261	`    guest_step.go_to_review(GuestData())`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
262	`    expect(guest_step.unpaid_review_note).to_contain_text("no room is held until you confirm")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
265	`@pytest.mark.tc("TC-099")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-099 = "Direct /booking without a selected room redirects safely".
266	`def test_direct_booking_url_without_room_redirects(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *direct booking url without room redirects*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
267	`    page.goto("/booking")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
268	`    expect(page).to_have_url(re.compile(r"/rooms"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
271	`@pytest.mark.tc("TC-059")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-059 = "Booking and guest data not exposed in URL or API".
272	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
273	`def test_no_guest_pii_in_urls(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no guest pii in urls*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).

Line	Code	Syntax & why
274	`    guest = GuestData()`	**Assignment**: binds the name *guest* to the value on the right. *GuestData(...)* creates a GuestData object (constructor call).
275	`    urls = []`	**Assignment**: binds the name *urls* to the value on the right. The value is a **list** literal [...].
276	`    page.on("request", lambda r: urls.append(r.url))`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).
277	`    booking = _start(page, site.bookable_room["id"])`	**Assignment**: binds the name *booking* to the value on the right.
278	`    booking.go_to_review(guest)`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py.
279	`    leaked = [u for u in urls if any(x in u for x in (guest.email, guest.phone.lstrip("+"), "Dane"))]`	**Assignment**: binds the name *leaked* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *any()* True if any element is truthy. *lstrip()* removes leading characters.
280	`    assert not leaked, f"PII in URLs: {leaked}"`	**assert**: fails the test if *not leaked* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
283	`# ---------- live writes / not automatable here ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
284	`@pytest.mark.live_write`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.live_write**: creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
285	`@pytest.mark.tc("TC-096", "TC-039")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-096 = “Confirm booking (pay at hotel) creates a reservation and reference”; TC-039 = “Confirmation email and reference are correct”.
286	`def test_confirm_booking_creates_reservation(guest_step: BookingPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *confirm booking creates reservation*. Parameters: *guest_step*: module fixture: hotel room booked up to the guest-details step.
287	`    guest_step.go_to_review(GuestData())`	**Method call** executed for its effect. *go_to_review()*: project method BookingPage.go_to_review() in pages/booking_page.py. *GuestData(...)* creates a GuestData object (constructor call).
288	`    guest_step.confirm_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
289	`    expect(guest_step.reference).to_be_visible(timeout=20000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
292	`@pytest.mark.live_write`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.live_write**: creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
293	`@pytest.mark.tc("TC-037")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-037 = “Last room cannot be double booked”.
294	`def test_last_room_cannot_be_double_booked():`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *last room cannot be double booked*.

Line	Code	Syntax & why
295	`    pytest.skip("Needs approval to create two live bookings of the last unit (TC-037)")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
298	`@pytest.mark.tc("TC-038")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-038 = “Room hold expires after session timeout”.
299	`def test_room_hold_expires():`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *room hold expires*.
300	`    pytest.skip("Hold time is not published and idling past it is not practical in CI (TC-038)")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
303	`@pytest.mark.tc("TC-034", "TC-035", "TC-036")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-034 = “Successful payment confirms booking”; TC-035 = “Declined card shows error and keeps room held”; TC-036 = “Gateway timeout or closed payment window”.
304	`def test_online_payment_flows():`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *online payment flows*.
305	`    pytest.skip("Not applicable: pay-at-hotel only, no online payment gateway (TC-034..036)")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).

tests/functional/test_auth.py

Sign-in dialog, role choice, validation, invalid credentials, brute-force throttling, registration validation, forgot password.

Line	Code	Syntax & why
1	`"""Sign-in modal, registration form validation, forgot password, brute-force protection."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`import uuid`	**import uuid**: loads the module. Generates random unique identifiers so every test run creates unique emails/identifiers and never collides with earlier data.
6	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
9	`from pages.auth_modal import AuthModal`	**from pages.auth_modal import AuthModal**: imports specific names so they can be used without the module prefix. Page Object for the 'Sign in or create an account' dialog.
10	`from pages.home_page import HomePage`	**from pages.home_page import HomePage**: imports specific names so they can be used without the module prefix. Page Object for the Home page.

Line	Code	Syntax & why
11	`from utils.data_factory import UserData`	**from utils.data_factory import UserData**: imports specific names so they can be used without the module prefix. Builders for unique test data (users, guests).
14	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
15	`def modal(page) -> AuthModal:`	**def** defines the fixture *modal*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test. Returns AuthModal.
16	`    HomePage(page).open()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *open()* opens a file (read/write). *HomePage(...)* creates a HomePage object (constructor call).
17	`    return AuthModal(page)`	**return**: ends the function and hands back *AuthModal(page)*. *AuthModal(...)* creates a AuthModal object (constructor call).
20	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
21	`@pytest.mark.tc("TC-100")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-100 = “Sign-in modal: role selection (Customer / Agent) and close”.
22	`def test_sign_in_dialog_opens_with_role_choice(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *sign in dialog opens with role choice*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
23	`    modal.open_sign_in()`	**Method call** executed for its effect. *open_sign_in()*: project method AuthModal.open_sign_in() in pages/auth_modal.py.
24	`    expect(modal.customer_role).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
25	`    expect(modal.agent_role).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
26	`    expect(modal.create_account_button).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
29	`@pytest.mark.tc("TC-100")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-100 = “Sign-in modal: role selection (Customer / Agent) and close”.
30	`def test_customer_role_then_next_opens_sign_in_form(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *customer role then next opens sign in form*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
31	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..

Line	Code	Syntax & why
32	`    expect(modal.identifier).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
33	`    expect(modal.login_password).to_have_attribute("type", "password")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value.
36	`@pytest.mark.tc("TC-100")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-100 = “Sign-in modal: role selection (Customer / Agent) and close”.
37	`@pytest.mark.parametrize("how", ["close-button", "escape"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'how'; each value becomes its own test (the id appears in the report).
38	`def test_dialog_closes(modal: AuthModal, how):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *dialog closes*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready; *how* (value from @parametrize).
39	`    modal.open_sign_in()`	**Method call** executed for its effect. *open_sign_in()*: project method AuthModal.open_sign_in() in pages/auth_modal.py.
40	`    if how == "escape":`	**if** statement: the indented block runs only when *how == 'escape'* is true.
41	`        modal.page.keyboard.press("Escape")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
42	`    else:`	**else**: runs when the if-condition was false.
43	`        modal.close_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
44	`    expect(modal.dialog).to_be_hidden()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_hidden()* waits until the element is hidden or detached.
45	`    expect(modal.sign_in_button).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
48	`@pytest.mark.tc("TC-100", "TC-055")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-100 = “Sign-in modal: role selection (Customer / Agent) and close”; TC-055 = “Keyboard navigation of search, filters and Select room”.
49	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
50	`@pytest.mark.defect(reason="focus is not returned to the Sign in trigger after the dialog closes")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: 'focus is not returned to the Sign in trigger after the dialog closes'.
51	`def test_focus_returns_to_trigger_after_close(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *focus returns to trigger after close*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.

Line	Code	Syntax & why
52	`    modal.open_sign_in()`	**Method call** executed for its effect. *open_sign_in()*: project method AuthModal.open_sign_in() in pages/auth_modal.py.
53	`    modal.page.keyboard.press("Escape")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
54	`    expect(modal.dialog).to_be_hidden()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_hidden()* waits until the element is hidden or detached.
55	`    expect(modal.sign_in_button).to_be_focused()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_focused()* waits until the element has keyboard focus.
58	`@pytest.mark.tc("TC-101")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-101 = “Sign-in mandatory field validation”.
59	`def test_sign_in_mandatory_field_validation(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *sign in mandatory field validation*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
60	`    calls = []`	**Assignment**: binds the name *calls* to the value on the right. The value is a **list** literal [...].
61	`    modal.page.on("request", lambda r: calls.append(r.url) if "/api/auth/login" in r.url else None)`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).
62	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
63	`    modal.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
64	`    expect(modal.required_errors).to_have_count(2)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
65	`    assert not calls, "empty form must not call the login API"`	**assert**: fails the test if *not calls* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
68	`@pytest.mark.tc("TC-120", "TC-101")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-120 = “Guest name and identifier character validation”; TC-101 = “Sign-in mandatory field validation”.
69	`def test_sign_in_identifier_format_checked_client_side(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *sign in identifier format checked client side*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
70	`    calls = []`	**Assignment**: binds the name *calls* to the value on the right. The value is a **list** literal [...].
71	`    modal.page.on("request", lambda r: calls.append(r.url) if "/api/auth/login" in r.url else None)`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).

Line	Code	Syntax & why
72	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
73	`    modal.identifier.fill("abc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
74	`    modal.login_password.fill("whatever-password-1")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
75	`    modal.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
76	`    expect(modal.identifier_error).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
77	`    assert not calls, "invalid identifier was sent to /api/auth/login"`	**assert**: fails the test if *not calls* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
80	`@pytest.mark.tc("TC-040")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-040 = “Sign in with valid and invalid credentials”.
81	`def test_invalid_credentials_show_generic_error(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *invalid credentials show generic error*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
82	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
83	`    modal.identifier.fill(f"nobody-{uuid.uuid4().hex[:8]}@maildrop.cc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *uuid4()* random UUID. Uses an **f-string** (f"...{expr}...") to build text with values.
84	`    modal.login_password.fill("wrong-password-123")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
85	`    modal.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
86	`    expect(modal.invalid_credentials).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
87	`    # no account enumeration: wording must not reveal whether the email exists`	**Comment** (# ... ignored by Python): documents intent for the reader.
88	`    expect(modal.dialog).not_to_contain_text(re.compile(r"no account|not registered|does not exist", re.I))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_contain_text()* waits until the text does NOT contain the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

Line	Code	Syntax & why
91	`@pytest.mark.needs_account`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.needs_account**: needs TEST_EMAIL/TEST_PASSWORD of an existing customer; skipped when the secrets are absent.
92	`@pytest.mark.tc("TC-040")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-040 = “Sign in with valid and invalid credentials”.
93	`def test_wrong_password_for_real_account_is_generic(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *wrong password for real account is generic*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
94	`    from config.settings import TEST_EMAIL`	**from config.settings import TEST_EMAIL**: imports specific names so they can be used without the module prefix. Project configuration module: every value can be overridden by an environment variable.
96	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
97	`    modal.identifier.fill(TEST_EMAIL)`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
98	`    modal.login_password.fill("definitely-wrong-password-1")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
99	`    modal.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
100	`    expect(modal.invalid_credentials).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
103	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
104	`@pytest.mark.tc("TC-102")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-102 = “Brute-force protection on sign-in”.
105	`@pytest.mark.defect("BUG-041")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-041 = “No brute-force protection on sign-in: repeated failed logins are never throttled or locked”.
106	`def test_brute_force_is_throttled(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *brute force is throttled*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready. Docstring: “6 wrong passwords for one (non-existent) identifier must trigger a delay, CAPTCHA or lockout.”
107	`    """6 wrong passwords for one (non-existent) identifier must trigger a delay, CAPTCHA or lockout."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
108	`    statuses = []`	**Assignment**: binds the name *statuses* to the value on the right. The value is a **list** literal [...].

Line	Code	Syntax & why
109	`    modal.page.on("response", lambda r: statuses.append(r.status) if "/api/auth/login" in r.url else None)`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).
110	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
111	`    modal.identifier.fill(f"bruteforce-{uuid.uuid4().hex[:8]}@maildrop.cc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *uuid4()* random UUID. Uses an **f-string** (f"...{expr}...") to build text with values.
112	`    for i in range(7):`	**for** loop: repeats the indented block for every item, binding it to *i* (iterating *range(7)*). *range()* sequence of integers.
113	`        modal.login_password.fill(f"wrong-password-{i:03d}")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). Uses an **f-string** (f"...{expr}...") to build text with values.
114	`        modal.continue_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
115	`        modal.page.wait_for_timeout(700)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
116	`    throttled = (`	**Assignment**: binds the name *throttled* to the value on the right. **Boolean expression**: 'or' returns the first truthy value - a compact default.
117	`        any(s == 429 for s in statuses)`	**Continuation** of the statement started on line 116 (inside brackets). *any()* True if any element is truthy.
118	`        or modal.dialog.get_by_text(re.compile(r"too many|try again later|locked|captcha", re.I)).count() > 0`	**Continuation** of the statement started on line 116 (inside brackets). *count()* number of current matches (does not wait). *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
119	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
120	`    assert throttled, f"7 failed logins, no throttling (statuses {statuses})"`	**assert**: fails the test if *throttled* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
123	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
124	`def test_registration_form_fields(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *registration form fields*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
125	`    modal.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.
126	`    for field in (modal.full_name, modal.email, modal.phone, modal.password, modal.confirm_password):`	**for** loop: repeats the indented block for every item, binding it to *field* (iterating *(modal.full_name, modal.email, modal.phone, modal.password,…)*.

Line	Code	Syntax & why
127	`        expect(field).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
128	`    expect(modal.country_code).to_contain_text("+234")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
131	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
132	`def test_empty_registration_is_blocked_with_field_errors(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *empty registration is blocked with field errors*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
133	`    modal.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.
134	`    modal.submit.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
135	`    expect(modal.register_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
136	`    for msg in ("Enter your full name", "Enter a valid email", "At least 12 characters"):`	**for** loop: repeats the indented block for every item, binding it to *msg* (iterating *('Enter your full name', 'Enter a valid email', 'At least 1…*).
137	`        expect(modal.dialog.get_by_text(msg)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex).
140	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
141	`@pytest.mark.defect(`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: "phone error reads 'Invalid input: expected string, received undefined' (raw schema text)".
142	`    reason="phone error reads 'Invalid input: expected string, received undefined' (raw schema text)"`	**Continuation** of the statement (inside brackets). Keyword argument(s) reason (name=value): self-documenting and order-independent.
143	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
144	`def test_empty_phone_error_is_human_readable(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *empty phone error is human readable*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
145	`    modal.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.

Line	Code	Syntax & why
146	`    modal.submit.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
147	`    expect(modal.dialog.get_by_text(re.compile(r"expected string|received undefined"))).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
150	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
151	`def test_password_visibility_toggle(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *password visibility toggle*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
152	`    modal.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.
153	`    modal.password.fill("Abc!12345678")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
154	`    modal.toggle_password.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
155	`    expect(modal.password).to_have_attribute("type", "text") # revealed`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value. Inline comment: “revealed”.
156	`    modal.toggle_password.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
157	`    expect(modal.password).to_have_attribute("type", "password") # hidden again`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_attribute()* waits until an attribute has the value. Inline comment: “hidden again”.
160	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
161	`def test_short_password_rejected(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *short password rejected*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
162	`    user = UserData.new()`	**Assignment**: binds the name *user* to the value on the right. *new()*: project method UserData.new() in utils/data_factory.py.
163	`    modal.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.
164	`    modal.fill_registration(user)`	**Method call** executed for its effect. *fill_registration()*: project method AuthModal.fill_registration() in pages/auth_modal.py.
165	`    modal.password.fill("Short1!")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).

Line	Code	Syntax & why
166	`    modal.confirm_password.fill("Short1!")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
167	`    modal.submit.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
168	`    expect(modal.register_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
169	`    expect(modal.dialog.get_by_text("At least 12 characters")).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex).
172	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
173	`def test_mismatched_passwords_block_registration(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *mismatched passwords block registration*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
174	`    user = UserData.new()`	**Assignment**: binds the name *user* to the value on the right. *new()*: project method UserData.new() in utils/data_factory.py.
175	`    modal.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.
176	`    modal.fill_registration(user, confirm=user.password + "x")`	**Method call** executed for its effect. *fill_registration()*: project method AuthModal.fill_registration() in pages/auth_modal.py.
177	`    modal.submit.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
178	`    expect(modal.register_heading).to_be_visible() # still on the form`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. Inline comment: “still on the form”.
179	`    expect(modal.dialog).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
182	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
183	`def test_invalid_email_blocks_registration(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *invalid email blocks registration*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
184	`    user = UserData.new()`	**Assignment**: binds the name *user* to the value on the right. *new()*: project method UserData.new() in utils/data_factory.py.
185	`    modal.open_registration_form()`	**Method call** executed for its effect. *open_registration_form()*: project method AuthModal.open_registration_form() in pages/auth_modal.py.

Line	Code	Syntax & why
186	`    modal.fill_registration(user)`	**Method call** executed for its effect. *fill_registration()*: project method AuthModal.fill_registration() in pages/auth_modal.py.
187	`    modal.email.fill("not-an-email")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
188	`    modal.submit.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
189	`    expect(modal.register_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
190	`    # blocked either by the browser (type=email) or by the app's own message`	**Comment** (# … ignored by Python): documents intent for the reader.
191	`    native_invalid = not modal.email.evaluate("e => e.validity.valid")`	**Assignment**: binds the name *native_invalid* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
192	`    assert native_invalid or modal.dialog.get_by_text("Enter a valid email").is_visible()`	**assert**: fails the test if *native_invalid or modal.dialog.get_by_text('Enter a valid email').is_visible()* is false. pytest rewrites asserts to print the actual values on failure. *is_visible()* True/False right now (does not wait) - used for branching, never for asserting. *get_by_text()* finds elements by their visible text (exact string or regex).
195	`@pytest.mark.live_write`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.live_write**: creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
196	`@pytest.mark.tc("TC-103")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-103 = “Create an account: validation and success”.
197	`def test_register_new_customer(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *register new customer*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
198	`    user = UserData.new()`	**Assignment**: binds the name *user* to the value on the right. *new()*: project method UserData.new() in utils/data_factory.py.
199	`    modal.register(user)`	**Method call** executed for its effect. *register()*: project method AuthModal.register() in pages/auth_modal.py - Full happy path: open form -> fill -> submit -> wait until logged in..
200	`    expect(modal.sign_in_button).to_have_count(0) # Sign in disappears once logged in`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). Inline comment: “Sign in disappears once logged in”.
203	`@pytest.mark.tc("TC-104")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-104 = “Forgot password flow”.
204	`def test_forgot_password_form(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *forgot password form*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.

Line	Code	Syntax & why
205	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
206	`    modal.forgot_password.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
207	`    expect(modal.reset_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
208	`    expect(modal.send_reset_code).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
209	`    modal.back_to_sign_in.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
210	`    expect(modal.signin_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
213	`@pytest.mark.live_write`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.live_write**: creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
214	`@pytest.mark.tc("TC-104")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-104 = “Forgot password flow”.
215	`def test_forgot_password_unknown_email_is_generic(modal: AuthModal):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *forgot password unknown email is generic*. Parameters: *modal*: module fixture: Home opened and an AuthModal page object ready.
216	`    modal.open_customer_form()`	**Method call** executed for its effect. *open_customer_form()*: project method AuthModal.open_customer_form() in pages/auth_modal.py - Role choice is remembered per browser, so step 1 may be skipped on later visits..
217	`    modal.forgot_password.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
218	`    modal.reset_email.fill(f"nobody-{uuid.uuid4().hex[:8]}@maildrop.cc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs). *uuid4()* random UUID. Uses an **f-string** (f"...{expr}...") to build text with values.
219	`    modal.send_reset_code.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
220	`    expect(modal.dialog).not_to_contain_text(re.compile(r"no account|not registered|does not exist", re.I))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_contain_text()* waits until the text does NOT contain the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

tests/functional/test_account.py

Account page signed-out/signed-in, manage-a-booking lookup, sign out. Signed-in tests use the TEST_EMAIL/TEST_PASSWORD secrets.

Line	Code	Syntax & why
1	`"""Account area (signed out and signed in), manage-a-booking lookup, sign out."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
5	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
6	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
8	`from pages.account_page import AccountPage`	**from pages.account_page import AccountPage**: imports specific names so they can be used without the module prefix. Page Object for /account.
9	`from pages.contact_page import ContactPage`	**from pages.contact_page import ContactPage**: imports specific names so they can be used without the module prefix. Page Object for /contact.
10	`from pages.home_page import HomePage`	**from pages.home_page import HomePage**: imports specific names so they can be used without the module prefix. Page Object for the Home page.
11	`from pages.manage_booking_page import ManageBookingPage`	**from pages.manage_booking_page import ManageBookingPage**: imports specific names so they can be used without the module prefix. Page Object for /manage-booking (lookup by reference).
14	`# ---------- signed out ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
15	`@pytest.mark.tc("TC-105")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-105 = “Account page for a signed-in customer shows bookings and profile”.
16	`def test_signed_out_account_page_prompts_sign_in(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *signed out account page prompts sign in*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
17	`    account = AccountPage(page).open()`	**Assignment**: binds the name *account* to the value on the right. *open()* opens a file (read/write). *AccountPage(...)* creates a AccountPage object (constructor call).
18	`    expect(account.signed_out_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
19	`    expect(account.signed_out_sign_in).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
22	`@pytest.mark.tc("TC-105")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-105 = “Account page for a signed-in customer shows bookings and profile”.

Line	Code	Syntax & why
23	`def test_customer_account_copy_has_no_agent_wording(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *customer account copy has no agent wording*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
24	`    account = AccountPage(page).open()`	**Assignment**: binds the name *account* to the value on the right. *open()* opens a file (read/write). *AccountPage(...)* creates a AccountPage object (constructor call).
25	`    expect(account.main).not_to_contain_text(re.compile("commission", re.I))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_contain_text()* waits until the text does NOT contain the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
28	`@pytest.mark.tc("TC-041", "TC-114")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-041 = “Manage a booking by reference”; TC-114 = “'Look it up with your reference' opens a booking lookup”.
29	`def test_manage_booking_lookup_form(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *manage booking lookup form*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
30	`    manage = ManageBookingPage(page).open()`	**Assignment**: binds the name *manage* to the value on the right. *open()* opens a file (read/write). *ManageBookingPage(...)* creates a ManageBookingPage object (constructor call).
31	`    expect(manage.heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
32	`    expect(manage.reference).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
33	`    expect(manage.email).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
34	`    expect(manage.find_button).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
37	`@pytest.mark.tc("TC-041")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-041 = “Manage a booking by reference”.
38	`def test_manage_booking_requires_fields(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *manage booking requires fields*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
39	`    manage = ManageBookingPage(page).open()`	**Assignment**: binds the name *manage* to the value on the right. *open()* opens a file (read/write). *ManageBookingPage(...)* creates a ManageBookingPage object (constructor call).
40	`    manage.find_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.

Line	Code	Syntax & why
41	`    expect(manage.heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
42	`    expect(page).to_have_url(re.compile(r"/manage-booking"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
45	`@pytest.mark.tc("TC-041")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-041 = “Manage a booking by reference”.
46	`def test_manage_booking_wrong_reference_shows_not_found(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *manage booking wrong reference shows not found*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
47	`    manage = ManageBookingPage(page).open()`	**Assignment**: binds the name *manage* to the value on the right. *open()* opens a file (read/write). *ManageBookingPage(...)* creates a ManageBookingPage object (constructor call).
48	`    manage.reference.fill("HMS-2026-999999")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
49	`    manage.email.fill("nobody-qa@maildrop.cc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
50	`    manage.find_button.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
51	`    expect(manage.not_found).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
54	`@pytest.mark.tc("TC-114")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-114 = “'Look it up with your reference' opens a booking lookup”.
55	`def test_contact_reference_link_opens_lookup(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *contact reference link opens lookup*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
56	`    contact = ContactPage(page).open()`	**Assignment**: binds the name *contact* to the value on the right. *open()* opens a file (read/write). *ContactPage(...)* creates a ContactPage object (constructor call).
57	`    contact.lookup_link.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
58	`    expect(ManageBookingPage(page).heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *ManageBookingPage(...)* creates a ManageBookingPage object (constructor call).

Line	Code	Syntax & why
61	`@pytest.mark.tc("TC-041")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-041 = “Manage a booking by reference”.
62	`def test_footer_manage_a_booking_opens_lookup(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *footer manage a booking opens lookup*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
63	`    home = HomePage(page).open()`	**Assignment**: binds the name *home* to the value on the right. *open()* opens a file (read/write). *HomePage(...)* creates a HomePage object (constructor call).
64	`    home.footer_link("Manage a booking").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *footer_link()*: project method BasePage.footer_link() in pages/base_page.py.
65	`    expect(ManageBookingPage(page).heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *ManageBookingPage(...)* creates a ManageBookingPage object (constructor call).
68	`# ---------- signed in (needs TEST_EMAIL / TEST_PASSWORD) ----------`	**Section comment**: visual divider grouping related code (ignored by Python).
69	`@pytest.mark.smoke`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.smoke**: critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
70	`@pytest.mark.needs_account`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.needs_account**: needs TEST_EMAIL/TEST_PASSWORD of an existing customer; skipped when the secrets are absent.
71	`@pytest.mark.tc("TC-040")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-040 = “Sign in with valid and invalid credentials”.
72	`def test_valid_sign_in_shows_customer_in_header(auth_page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *valid sign in shows customer in header*. Parameters: *auth_page*: project fixture: a tab that is already signed in (reuses auth_state).
73	`    home = HomePage(auth_page).open()`	**Assignment**: binds the name *home* to the value on the right. *open()* opens a file (read/write). *HomePage(...)* creates a HomePage object (constructor call).
74	`    expect(home.sign_in_button).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
77	`@pytest.mark.needs_account`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.needs_account**: needs TEST_EMAIL/TEST_PASSWORD of an existing customer; skipped when the secrets are absent.
78	`@pytest.mark.tc("TC-105")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-105 = “Account page for a signed-in customer shows bookings and profile”.

Line	Code	Syntax & why
79	`def test_signed_in_account_page_shows_profile(auth_page, auth_state):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *signed in account page shows profile*. Parameters: *auth_page*: project fixture: a tab that is already signed in (reuses auth_state); *auth_state*: project fixture: signs in once per session and saves the session to a file.
80	`    account = AccountPage(auth_page).open()`	**Assignment**: binds the name *account* to the value on the right. *open()* opens a file (read/write). *AccountPage(...)* creates a AccountPage object (constructor call).
81	`    account.expect_signed_in()`	**Method call** executed for its effect. *expect_signed_in()*: project method AccountPage.expect_signed_in() in pages/account_page.py.
82	`    expect(account.main).not_to_contain_text(re.compile("commission", re.I))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_contain_text()* waits until the text does NOT contain the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
85	`@pytest.mark.needs_account`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.needs_account**: needs TEST_EMAIL/TEST_PASSWORD of an existing customer; skipped when the secrets are absent.
86	`@pytest.mark.tc("TC-107")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-107 = “Sign out clears the session”.
87	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
88	`def test_sign_out_clears_session(auth_page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *sign out clears session*. Parameters: *auth_page*: project fixture: a tab that is already signed in (reuses auth_state).
89	`    account = AccountPage(auth_page).open()`	**Assignment**: binds the name *account* to the value on the right. *open()* opens a file (read/write). *AccountPage(...)* creates a AccountPage object (constructor call).
90	`    account.expect_signed_in()`	**Method call** executed for its effect. *expect_signed_in()*: project method AccountPage.expect_signed_in() in pages/account_page.py.
91	`    account.sign_out.first.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
92	`    expect(account.header.get_by_role("button", name="Sign in")).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
93	`    me = auth_page.request.get("/api/auth/me").json()`	**Assignment**: binds the name *me* to the value on the right. *json()* parses a response body (or returns it) as Python dict/list. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
94	`    assert me.get("data") is None, f"/api/auth/me still returns a session: {me}"`	**assert**: fails the test if *me.get('data') is None* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get). Uses an **f-string** (f"...{expr}...") to build text with values.

Line	Code	Syntax & why
95	`    auth_page.go_back()`	**Method call** executed for its effect. *go_back()* browser Back button.
96	`    AccountPage(auth_page).open()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *open()* opens a file (read/write). *AccountPage(...)* creates a AccountPage object (constructor call).
97	`    expect(AccountPage(auth_page).signed_out_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *AccountPage(...)* creates a AccountPage object (constructor call).
100	`@pytest.mark.tc("TC-106")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-106 = “Agent role: book for clients and see commissions”.
101	`def test_agent_role_features():`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *agent role features*.
102	`    pytest.skip("Blocked: no agent account in the preview environment (workbook TC-106)")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
105	`@pytest.mark.live_write`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.live_write**: creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
106	`@pytest.mark.tc("TC-042")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-042 = “Cancel booking follows the cancellation policy”.
107	`def test_cancel_booking_follows_policy():`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *cancel booking follows policy*.
108	`    pytest.skip("Blocked: needs an existing booking to cancel and a published cancellation policy (TC-042)")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).

tests/functional/test_branches.py

Branch cards, deep links to Rooms and Contact, branch naming/address consistency across pages.

Line	Code	Syntax & why
1	`"""Branches page: cards, deep links to Rooms and Contact, naming consistency."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
5	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
6	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.

Line	Code	Syntax & why
8	`from pages.about_page import AboutPage`	**from pages.about_page import AboutPage**: imports specific names so they can be used without the module prefix. Page Object for /about.
9	`from pages.branches_page import BranchesPage`	**from pages.branches_page import BranchesPage**: imports specific names so they can be used without the module prefix. Page Object for /branches.
10	`from pages.contact_page import ContactPage`	**from pages.contact_page import ContactPage**: imports specific names so they can be used without the module prefix. Page Object for /contact.
11	`from pages.home_page import HomePage`	**from pages.home_page import HomePage**: imports specific names so they can be used without the module prefix. Page Object for the Home page.
12	`from pages.rooms_page import RoomsPage`	**from pages.rooms_page import RoomsPage**: imports specific names so they can be used without the module prefix. Page Object for the Rooms listing.
14	`NAME_RE = re.compile(r"^Oneflare Grand Plaza - [A-Z][\w ]+ Branch$")`	Module-level **constant** *NAME_RE* (UPPER_CASE by convention = not changed at runtime). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
15	`ADDRESS_RE = re.compile(r"^\d+[A-Za-z]?(–\d+)? [A-Z][\w .'-]{2,60}$")`	Module-level **constant** *ADDRESS_RE* (UPPER_CASE by convention = not changed at runtime). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
18	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
19	`def branches(page) -> BranchesPage:`	**def** defines the fixture *branches*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test. Returns BranchesPage.
20	`    return BranchesPage(page).open()`	**return**: ends the function and hands back *BranchesPage(page).open()*. *open()* opens a file (read/write). *BranchesPage(...)* creates a BranchesPage object (constructor call).
23	`@pytest.mark.tc("TC-071")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-071 = “Branch cards show room types, lowest price, contacts, directions, offers and photo”.
24	`def test_every_branch_has_a_card_with_rooms_and_price(branches: BranchesPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *every branch has a card with rooms and price*. Parameters: *branches*: module fixture: Branches page opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
25	`    expect(branches.branch_headings).to_have_count(len(site.branches))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). *len()* length of a collection.
26	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
27	`        card = branches.card(b.name)`	**Assignment**: binds the name *card* to the value on the right. *card()*: project method BranchesPage.card() in pages/branches_page.py - The smallest element that holds the branch heading AND its 'Contact this branch' link..
28	`        expect(card).to_contain_text(b.address)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.

Line	Code	Syntax & why
29	`        expect(card).to_contain_text(re.compile(r"[1-9]\d* available"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
30	`        expect(card).to_contain_text(re.compile(r"₦[1-9][\d,]* / night"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
33	`@pytest.mark.tc("TC-071")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-071 = “Branch cards show room types, lowest price, contacts, directions, offers and photo”.
34	`@pytest.mark.defect("BUG-022")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-022 = “Branches page shows '0 available' room types and 'From N0 / night' for every branch, with empty phone, email, hours, getting-here and offers”.
35	`def test_branch_cards_show_contacts_hours_and_photo(branches: BranchesPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch cards show contacts hours and photo*. Parameters: *branches*: module fixture: Branches page opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
36	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
37	`        card = branches.card(b.name)`	**Assignment**: binds the name *card* to the value on the right. *card()*: project method BranchesPage.card() in pages/branches_page.py - The smallest element that holds the branch heading AND its 'Contact this branch' link..
38	`        text = card.inner_text()`	**Assignment**: binds the name *text* to the value on the right. *inner_text()* visible text of the element (as rendered, respects CSS).
39	`        assert re.search(r"\+?\d[\d\s]{6,}", text) and "@" in text, f"{b.city}: no phone/email"`	**assert**: fails the test if *re.search('\\+?\\d[\\d\\s]{6,}', text) and '@' in text* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *search()* finds the first regex match anywhere in a string. Uses an **f-string** (f"...{expr}...") to build text with values. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
40	`        expect(card.locator("img[alt]:not([alt=''])").first).to_be_visible(timeout=2000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
43	`@pytest.mark.tc("TC-072")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-072 = “'See rooms at <branch>' opens Rooms filtered to that branch”.
44	`@pytest.mark.parametrize("city", ["Benin City", "Ikeja", "Lagos", "Lekki", "Owerri"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'city'; each value becomes its own test (the id appears in the report).

Line	Code	Syntax & why
45	`def test_see_rooms_opens_rooms_for_that_branch(page, branches: BranchesPage, site, city):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *see rooms opens rooms for that branch*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *branches*: module fixture: Branches page opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *city* (value from @parametrize).
46	`    b = site.branch(city)`	**Assignment**: binds the name *b* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
47	`    branches.see_rooms(b.name).click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *see_rooms()*: project method BranchesPage.see_rooms() in pages/branches_page.py.
48	`    expect(page).to_have_url(re.compile(rf"/rooms\?branchId={b.id}"))`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_url()* waits until the page URL equals/matches. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
49	`    rooms = RoomsPage(page)`	**Assignment**: binds the name *rooms* to the value on the right. *RoomsPage(...)* creates a RoomsPage object (constructor call).
50	`    expect(rooms.results_heading).to_contain_text(b.name)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
53	`@pytest.mark.tc("TC-073")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-073 = "'Contact this branch' opens Contact with that branch tab selected".
54	`@pytest.mark.parametrize("city", ["Lekki", "Owerri"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'city'; each value becomes its own test (the id appears in the report).
55	`def test_contact_this_branch_selects_that_branch(page, branches: BranchesPage, site, city):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *contact this branch selects that branch*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *branches*: module fixture: Branches page opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *city* (value from @parametrize).
56	`    b = site.branch(city)`	**Assignment**: binds the name *b* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
57	`    branches.card(b.name).get_by_role("link", name="Contact this branch").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox…) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers. *card()*: project method BranchesPage.card() in pages/branches_page.py - The smallest element that holds the branch heading AND its 'Contact this branch' link..
58	`    contact = ContactPage(page)`	**Assignment**: binds the name *contact* to the value on the right. *ContactPage(...)* creates a ContactPage object (constructor call).

Line	Code	Syntax & why
59	`    expect(contact.routing).to_contain_text(b.name)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
60	`    expect(contact.main).to_contain_text(b.address)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
63	`@pytest.mark.tc("TC-074", "TC-075")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-074 = “Branch names and addresses are valid and consistent everywhere”; TC-075 = “Lagos branch name and address follow the standard format”.
64	`def test_branch_names_and_addresses_valid(site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch names and addresses valid*. Parameters: *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
65	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
66	`        assert NAME_RE.match(b.name), f"irregular branch name: {b.name!r}"`	**assert**: fails the test if *NAME_RE.match(b.name)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *match()* regex match anchored at the start of the string. Uses an **f-string** (f"...{expr}...") to build text with values.
67	`        assert ADDRESS_RE.match(b.address), f"malformed address: {b.address!r}"`	**assert**: fails the test if *ADDRESS_RE.match(b.address)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *match()* regex match anchored at the start of the string. Uses an **f-string** (f"...{expr}...") to build text with values.
70	`@pytest.mark.tc("TC-074")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-074 = “Branch names and addresses are valid and consistent everywhere”.
71	`def test_branch_names_consistent_across_pages(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch names consistent across pages*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
72	`    pages = {`	**Assignment**: binds the name *pages* to the value on the right. The value is a **dict** literal {key: value}.
73	`        "branches": BranchesPage(page).open().main.inner_text(),`	**Continuation** of the statement started on line 72 (inside brackets). *inner_text()* visible text of the element (as rendered, respects CSS). *open()* opens a file (read/write). *BranchesPage(...)* creates a BranchesPage object (constructor call).
74	`        "about": AboutPage(page).open().main.inner_text(),`	**Continuation** of the statement started on line 72 (inside brackets). *inner_text()* visible text of the element (as rendered, respects CSS). *open()* opens a file (read/write). *AboutPage(...)* creates a AboutPage object (constructor call).
75	`        "rooms selector": "".join(RoomsPage(page).open().search.branch_options()),`	**Continuation** of the statement started on line 72 (inside brackets). *join()* joins strings with a separator. *branch_options()*: project method SearchWidget.branch_options() in pages/search_widget.py. *open()* opens a file (read/write). *RoomsPage(...)* creates a RoomsPage object (constructor call).
76	`    }`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
77	`    home = HomePage(page).open()`	**Assignment**: binds the name *home* to the value on the right. *open()* opens a file (read/write). *HomePage(...)* creates a HomePage object (constructor call).

Line	Code	Syntax & why
78	`    pages["home"] = home.main.inner_text()`	**Assignment**: binds the name *pages['home']* to the value on the right. *inner_text()* visible text of the element (as rendered, respects CSS).
79	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
80	`        for where, text in pages.items():`	**for** loop: repeats the indented block for every item, binding it to *(where, text)* (iterating *pages.items()*). *items()* key/value pairs of a dict.
81	`            needle = b.address if where == "home" else b.name`	**Assignment**: binds the name *needle* to the value on the right. **Conditional expression** (a if condition else b).
82	`            assert needle in text, f"{b.name}: '{needle}' missing on {where}"`	**assert**: fails the test if *needle in text* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
85	`@pytest.mark.tc("TC-075")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-075 = “Lagos branch name and address follow the standard format”.
86	`def test_lagos_branch_naming(site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *lagos branch naming*. Parameters: *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
87	`    lagos = site.branch("Lagos")`	**Assignment**: binds the name *lagos* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
88	`    assert lagos.name == "Oneflare Grand Plaza - Lagos Branch"`	**assert**: fails the test if *lagos.name == 'Oneflare Grand Plaza - Lagos Branch'* is false. pytest rewrites asserts to print the actual values on failure.
89	`    assert not re.match(r"^\d+ \d+ ", lagos.address), f"double house number: {lagos.address}"`	**assert**: fails the test if *not re.match('^\\d+ \\d+ ', lagos.address)* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *match()* regex match anchored at the start of the string. Uses an **f-string** (f"...{expr}...") to build text with values. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

tests/functional/test_gallery.py

Gallery category and location filters, lightbox, content accuracy.

Line	Code	Syntax & why
1	`"""Gallery: category filter, location filter, lightbox, content accuracy."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
5	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
6	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.

Line	Code	Syntax & why
8	`from pages.gallery_page import GalleryPage`	**from pages.gallery_page import GalleryPage**: imports specific names so they can be used without the module prefix. Page Object for /gallery.
11	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
12	`def gallery(page) -> GalleryPage:`	**def** defines the fixture *gallery*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test. Returns GalleryPage.
13	`    g = GalleryPage(page).open()`	**Assignment**: binds the name *g* to the value on the right. *open()* opens a file (read/write). *GalleryPage(...)* creates a GalleryPage object (constructor call).
14	`    expect(g.photos.first).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
15	`    return g`	**return**: ends the function and hands back *g*.
18	`@pytest.mark.tc("TC-109")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-109 = “Gallery category filter”.
19	`def test_gallery_loads(gallery: GalleryPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *gallery loads*. Parameters: *gallery*: module fixture: Gallery page opened with photos loaded.
20	`    expect(gallery.heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
21	`    expect(gallery.location_filter).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
22	`    assert gallery.photos.count() >= 4`	**assert**: fails the test if *gallery.photos.count() >= 4* is false. pytest rewrites asserts to print the actual values on failure. *count()* number of current matches (does not wait).
25	`@pytest.mark.tc("TC-109")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-109 = “Gallery category filter”.
26	`@pytest.mark.parametrize(`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'category,alt'; each value becomes its own test (the id appears in the report).
27	`    "category,alt",`	**Continuation** of the statement (inside brackets).
28	`    [("Gym", "Fitness studio"), ("Restaurant", "The dining room"), ("Swimming Pool", "The pool courtyard")],`	**Continuation** of the statement (inside brackets).
29	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
30	`def test_category_filter_and_all_restores(gallery: GalleryPage, category, alt):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *category filter and all restores*. Parameters: *gallery*: module fixture: Gallery page opened with photos loaded; *category* (value from @parametrize); *alt* (value from @parametrize).

Line	Code	Syntax & why
31	`    total = gallery.photos.count()`	**Assignment**: binds the name *total* to the value on the right. *count()* number of current matches (does not wait).
32	`    gallery.category(category).click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *category()*: project method GalleryPage.category() in pages/gallery_page.py.
33	`    expect(gallery.image(alt)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *image()*: project method GalleryPage.image() in pages/gallery_page.py.
34	`    expect(gallery.photos).not_to_have_count(total)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_have_count()* waits until the count differs from N.
35	`    gallery.category("All").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *category()*: project method GalleryPage.category() in pages/gallery_page.py.
36	`    expect(gallery.photos).to_have_count(total)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
39	`@pytest.mark.tc("TC-110")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-110 = “Gallery location filter”.
40	`def test_location_filter_shows_photos_or_message(gallery: GalleryPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *location filter shows photos or message*. Parameters: *gallery*: module fixture: Gallery page opened with photos loaded; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
41	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
42	`        gallery.location_filter.select_option(label=b.name)`	**Method call** executed for its effect. *select_option()* chooses an <option> in a <select> by value or label.
43	`        no_photos = gallery.main.get_by_text("No photos match this filter yet.")`	**Assignment**: binds the name *no_photos* to the value on the right. *get_by_text()* finds elements by their visible text (exact string or regex).
44	`        expect(gallery.photos.first.or_(no_photos)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *or_()* matches either of two locators (photos OR the empty-state message).
47	`@pytest.mark.tc("TC-110")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-110 = “Gallery location filter”.
48	`@pytest.mark.defect("BUG-034")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-034 = “Gallery location filter shows an empty area with no message for every branch”.

Line	Code	Syntax & why
49	`def test_every_branch_has_photos(gallery: GalleryPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *every branch has photos*. Parameters: *gallery*: module fixture: Gallery page opened with photos loaded; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
50	`    empty = []`	**Assignment**: binds the name *empty* to the value on the right. The value is a **list** literal [...].
51	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
52	`        gallery.location_filter.select_option(label=b.name)`	**Method call** executed for its effect. *select_option()* chooses an <option> in a <select> by value or label.
53	`        gallery.page.wait_for_timeout(500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
54	`        if gallery.photos.count() == 0:`	**if** statement: the indented block runs only when *gallery.photos.count() == 0* is true. *count()* number of current matches (does not wait).
55	`            empty.append(b.city)`	**Method call** executed for its effect. *append()* adds an item to a list.
56	`    assert not empty, f"No photos for: {empty}"`	**assert**: fails the test if *not empty* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
59	`@pytest.mark.tc("TC-111")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-111 = “Photo lightbox opens, navigates and closes”.
60	`@pytest.mark.defect(reason="regression: clicking a photo no longer opens a lightbox (workbook: Pass)")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: 'regression: clicking a photo no longer opens a lightbox (workbook: Pass)'.
61	`def test_photo_lightbox_opens_navigates_and_closes(gallery: GalleryPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *photo lightbox opens navigates and closes*. Parameters: *gallery*: module fixture: Gallery page opened with photos loaded.
62	`    gallery.photos.first.click(force=True)`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
63	`    box = gallery.page.get_by_role("dialog").filter(has=gallery.page.get_by_role("img"))`	**Assignment**: binds the name *box* to the value on the right. *filter()* narrows a locator: has=/has_not= (must contain another locator) or has_text=. Scopes a search to one card/section. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
64	`    expect(box).to_be_visible(timeout=5000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
65	`    first_alt = box.get_by_role("img").first.get_attribute("alt")`	**Assignment**: binds the name *first_alt* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
66	`    gallery.page.keyboard.press("ArrowRight")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.

Line	Code	Syntax & why
67	`    expect(box.get_by_role("img").first).not_to_have_attribute("alt", first_alt)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *not_to_have_attribute()* waits until an attribute no longer has the value. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
68	`    gallery.page.keyboard.press("Escape")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
69	`    expect(box).to_be_hidden()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_hidden()* waits until the element is hidden or detached.
72	`@pytest.mark.tc("TC-112")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-112 = “Gallery copy and images are accurate”.
73	`def test_gallery_intro_counts_match_branches(gallery: GalleryPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *gallery intro counts match branches*. Parameters: *gallery*: module fixture: Gallery page opened with photos loaded; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
74	`    expect(gallery.intro).to_contain_text(f"{len(site.branches)} houses")`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex. *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
77	`@pytest.mark.tc("TC-112")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-112 = “Gallery copy and images are accurate”.
78	`@pytest.mark.defect("BUG-035")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-035 = “Gallery content errors: copy says 'all three houses' (there are 5), duplicate photo, exterior images from cities without a branch, promised rate links missing”.
79	`def test_no_duplicate_photos_and_room_photos_link_to_rates(gallery: GalleryPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no duplicate photos and room photos link to rates*. Parameters: *gallery*: module fixture: Gallery page opened with photos loaded.
80	`    srcs = gallery.photos.evaluate_all("els => els.map(e => e.currentSrc || e.src)")`	**Assignment**: binds the name *srcs* to the value on the right. *evaluate_all()* runs JavaScript on ALL elements matched by a locator at once.
81	`    assert len(srcs) == len(set(srcs)), "duplicate photos"`	**assert**: fails the test if *len(srcs) == len(set(srcs))* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *len()* length of a collection. *set()* builds a set (unique values).
82	`    room_photo = gallery.image("Deluxe Room")`	**Assignment**: binds the name *room_photo* to the value on the right. *image()*: project method GalleryPage.image() in pages/gallery_page.py.
83	`    holder = room_photo.locator("xpath=ancestor::*[.//a or .//p][2]")`	**Assignment**: binds the name *holder* to the value on the right. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

Line	Code	Syntax & why
84	`    expect(holder).to_contain_text(re.compile(r"₦[\d,]+"), timeout=2000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).

tests/functional/test_about_contact.py

About page content; Contact info panel per branch, form validation and routing (live submit gated).

Line	Code	Syntax & why
1	`"""About page and Contact page (info panel, branch tabs, form validation, routing)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
5	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
6	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
8	`from pages.about_page import AboutPage`	**from pages.about_page import AboutPage**: imports specific names so they can be used without the module prefix. Page Object for /about.
9	`from pages.contact_page import ContactPage`	**from pages.contact_page import ContactPage**: imports specific names so they can be used without the module prefix. Page Object for /contact.
12	`@pytest.mark.tc("TC-113", "TC-123")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-113 = “About page shows heading, story and principles”; TC-123 = “Screen-reader landmarks and headings on each page”.
13	`def test_about_page_heading_story_and_principles(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *about page heading story and principles*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
14	`    about = AboutPage(page).open()`	**Assignment**: binds the name *about* to the value on the right. *open()* opens a file (read/write). *AboutPage(...)* creates a AboutPage object (constructor call).
15	`    expect(about.heading).to_have_text(re.compile(r"\S"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
16	`    expect(about.principles_heading).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.

Line	Code	Syntax & why
17	`    assert about.principles.count() >= 3`	**assert**: fails the test if *about.principles.count() >= 3* is false. pytest rewrites asserts to print the actual values on failure. *count()* number of current matches (does not wait).
18	`    expect(about.location_links).to_have_count(len(site.branches))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent). *len()* length of a collection.
21	`@pytest.fixture`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **@pytest.fixture**: turns the function into a fixture; tests receive its return/yield value by naming it as a parameter. Scope 'function': re-created for every test (isolation).
22	`def contact(page) -> ContactPage:`	**def** defines the fixture *contact*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test. Returns ContactPage.
23	`    return ContactPage(page).open()`	**return**: ends the function and hands back *ContactPage(page).open()*. *open()* opens a file (read/write). *ContactPage(...)* creates a ContactPage object (constructor call).
26	`@pytest.mark.tc("TC-011")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-011 = “Contact info panel shows address, hours, phone, email and a map for every branch tab”.
27	`@pytest.mark.parametrize("city", ["Benin City", "Ikeja", "Lagos", "Lekki", "Owerri"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'city'; each value becomes its own test (the id appears in the report).
28	`def test_branch_tab_shows_address_and_map(contact: ContactPage, site, city):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch tab shows address and map*. Parameters: *contact*: module fixture: Contact page opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *city* (value from @parametrize).
29	`    b = site.branch(city)`	**Assignment**: binds the name *b* to the value on the right. *branch()*: project method SiteData.branch() in utils/site_data.py.
30	`    contact.branch_tab(city).click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *branch_tab()*: project method ContactPage.branch_tab() in pages/contact_page.py.
31	`    expect(contact.main).to_contain_text(b.address)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
32	`    expect(contact.map_frame).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
33	`    expect(contact.routing).to_contain_text(b.name)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_contain_text()* waits until the text contains the substring / regex.
36	`@pytest.mark.tc("TC-011")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-011 = “Contact info panel shows address, hours, phone, email and a map for every branch tab”.

Line	Code	Syntax & why
37	`@pytest.mark.defect("BUG-007")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-007 = “Contact page: map area blank and Hours, Phone, Email fields empty for Benin City branch”.
38	`def test_branch_tab_shows_hours_phone_and_email(contact: ContactPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branch tab shows hours phone and email*. Parameters: *contact*: module fixture: Contact page opened.
39	`    expect(contact.info_note).to_have_count(0)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_count()* waits until the locator matches exactly N elements (0 = absent).
40	`    for label in ("Phone", "Email", "Hours"):`	**for** loop: repeats the indented block for every item, binding it to *label* (iterating *('Phone', 'Email', 'Hours')*).
41	`        expect(contact.main.get_by_text(label, exact=True)).to_be_visible(timeout=2000)`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex).
44	`@pytest.mark.tc("TC-045")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-045 = “Contact form mandatory validation and branch routing”.
45	`def test_contact_form_mandatory_validation(contact: ContactPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *contact form mandatory validation*. Parameters: *contact*: module fixture: Contact page opened.
46	`    contact.send.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
47	`    for msg in (`	**for** loop: repeats the indented block for every item, binding it to *msg* (iterating *('Enter your name', 'Enter a valid email', 'Enter a valid p…*).
48	`        "Enter your name",`	**Continuation** of the statement (inside brackets).
49	`        "Enter a valid email",`	**Continuation** of the statement (inside brackets).
50	`        "Enter a valid phone number",`	**Continuation** of the statement (inside brackets).
51	`        "Tell us a little more",`	**Continuation** of the statement (inside brackets).
52	`    ):`	**Continuation** of the statement (inside brackets).
53	`        expect(contact.main.get_by_text(msg)).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(…) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex).
56	`@pytest.mark.tc("TC-045")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-045 = “Contact form mandatory validation and branch routing”.

Line	Code	Syntax & why
57	`def test_routing_heading_follows_selected_tab(contact: ContactPage, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *routing heading follows selected tab*. Parameters: *contact*: module fixture: Contact page opened; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
58	`    for b in site.branches:`	**for** loop: repeats the indented block for every item, binding it to *b* (iterating *site.branches*).
59	`        contact.branch_tab(b.city).click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *branch_tab()*: project method ContactPage.branch_tab() in pages/contact_page.py.
60	`        expect(contact.routing).to_have_text(re.compile(rf"Going to {re.escape(b.name)}"))`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. *compile()* pre-compiles a regex pattern for reuse. *escape()* escapes regex special characters in literal text. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
63	`@pytest.mark.tc("TC-045")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-045 = “Contact form mandatory validation and branch routing”.
64	`def test_contact_privacy_link_works(page, contact: ContactPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *contact privacy link works*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *contact*: module fixture: Contact page opened.
65	`    assert page.request.get(contact.privacy_link.get_attribute("href")).status == 200`	**assert**: fails the test if *page.request.get(contact.privacy_link.get_attribute('href')).status == 200* is false. pytest rewrites asserts to print the actual values on failure. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get). *get_attribute()* reads an HTML attribute (href, min, max, alt, type).
68	`@pytest.mark.live_write`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.live_write**: creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
69	`@pytest.mark.tc("TC-012")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-012 = “Contact form submits with valid data”.
70	`def test_contact_form_submits_valid_message(contact: ContactPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *contact form submits valid message*. Parameters: *contact*: module fixture: Contact page opened.
71	`    contact.name.fill("Ada Obi (QA automation)")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
72	`    contact.email.fill("ada-qa@maildrop.cc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
73	`    contact.phone.fill("+2348012345678")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
74	`    contact.subject.fill("Automated test - please ignore")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
75	`    contact.message.fill("Automated regression test message. Please ignore.")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).

Line	Code	Syntax & why
76	`    contact.send.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
77	`    expect(contact.main.get_by_text(re.compile(r"(sent|received|thank)", re.I))).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_text()* finds elements by their visible text (exact string or regex). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
80	`@pytest.mark.live_write`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.live_write**: creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
81	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
82	`@pytest.mark.tc("TC-057")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-057 = “Contact form resists script injection”.
83	`def test_contact_form_script_injection_not_executed(contact: ContactPage):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *contact form script injection not executed*. Parameters: *contact*: module fixture: Contact page opened.
84	`    fired = []`	**Assignment**: binds the name *fired* to the value on the right. The value is a **list** literal [...].
85	`    contact.page.on("dialog", lambda d: (fired.append(d.message), d.dismiss()))`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).
86	`    contact.name.fill("<script>alert(1)</script>")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
87	`    contact.email.fill("xss-qa@maildrop.cc")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
88	`    contact.phone.fill("+2348012345678")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
89	`    contact.message.fill("<img src=x onerror=alert(1)> automated security test, please ignore")`	**Method call** executed for its effect. *fill()* clears an input and types the value in one step (also works for date and range inputs).
90	`    contact.send.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
91	`    contact.page.wait_for_timeout(2000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
92	`    assert not fired`	**assert**: fails the test if *not fired* is false. pytest rewrites asserts to print the actual values on failure.

tests/non_functional/__init__.py

Package marker.

Empty file. An empty __init__.py marks the folder as a Python package so its modules can be imported with dotted names.

tests/non_functional/test_performance.py

Performance budgets from the browser's own timing APIs: TTFB/DCL/load, LCP, CLS, Slow-4G LCP via CDP throttling, lazy images, skeleton resolution, console/network errors, and an opt-in 50-request concurrency test.

Line	Code	Syntax & why
1	`"""Performance budgets: navigation timing, LCP, CLS, throttled network, skeletons, API latency under load."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
4	`import statistics`	**import statistics**: loads the module. Statistics helpers; quantiles() computes the 95th percentile latency in the load test.
5	`import time`	**import time**: loads the module. Standard-library clock functions. time.monotonic() is used for polling deadlines because it never jumps backwards when the system clock changes.
6	`from concurrent.futures import ThreadPoolExecutor`	**from concurrent.futures import ThreadPoolExecutor**: imports specific names so they can be used without the module prefix. ThreadPoolExecutor runs many HTTP requests in parallel for the concurrency/load test. *ThreadPoolExecutor*: Pool of threads to run calls concurrently.
8	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
9	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
11	`from config.settings import ALLOW_LOAD_TESTS, PERF_DCL_MS, PERF_LCP_MS, PERF_LOAD_MS, PERF_TTFB_MS`	**from config.settings import ALLOW_LOAD_TESTS, PERF_DCL_MS, PERF_LCP_MS, PERF_LOAD_MS, PERF_TTFB_MS**: imports specific names so they can be used without the module prefix. Project configuration module: every value can be overridden by an environment variable.
13	`PAGES = ["/", "/rooms", "/branches", "/gallery", "/about", "/contact"]`	Module-level **constant** *PAGES* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
15	`NAV_TIMING_JS = """() => {`	Module-level **constant** *NAV_TIMING_JS* (UPPER_CASE by convention = not changed at runtime).
16	`  const n = performance.getEntriesByType('navigation')[0];`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Reads the Navigation Timing entry (browser-measured load phases).
17	`  return { ttfb: n.responseStart, dcl: n.domContentLoadedEventEnd, load: n.loadEventEnd };`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): TTFB = time to first byte of the HTML. DOMContentLoaded = HTML parsed. load event finished (images, CSS).
18	`}"""`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Closes the JavaScript block/expression.
20	`LCP_JS = """() => new Promise(resolve => {`	Module-level **constant** *LCP_JS* (UPPER_CASE by convention = not changed at runtime).
21	`  new PerformanceObserver(list => {`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Subscribes to browser performance entries (LCP / layout shifts).
22	`    const e = list.getEntries(); resolve(e[e.length - 1].startTime);`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Resolves the Promise with the measured value.

Line	Code	Syntax & why
23	`  }).observe({ type: 'largest-contentful-paint', buffered: true });`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): LCP: when the biggest visible element painted (Core Web Vital). Closes the JavaScript block/expression.
24	`  setTimeout(() => resolve(null), 5000); // give up after 5s so the test cannot hang`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Safety timeout so the Promise always resolves and the test cannot hang. Resolves the Promise with the measured value.
25	`})"""`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Closes the JavaScript block/expression.
27	`CLS_JS = """() => new Promise(resolve => {`	Module-level **constant** *CLS_JS* (UPPER_CASE by convention = not changed at runtime).
28	`  let cls = 0;`	**JavaScript** line inside a Python string, executed in the browser by evaluate().
29	`  new PerformanceObserver(list => {`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Subscribes to browser performance entries (LCP / layout shifts).
30	`    for (const e of list.getEntries()) if (!e.hadRecentInput) cls += e.value;`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Ignores shifts caused by user input (as the CLS spec does).
31	`  }).observe({ type: 'layout-shift', buffered: true });`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Layout shift entries; summed = CLS (Core Web Vital). Closes the JavaScript block/expression.
32	`  setTimeout(() => resolve(cls), 3000);`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Safety timeout so the Promise always resolves and the test cannot hang. Resolves the Promise with the measured value.
33	`})"""`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Closes the JavaScript block/expression.
35	`# Chrome DevTools "Slow 4G" preset`	**Comment** (# ... ignored by Python): documents intent for the reader.
36	`SLOW_4G = {`	Module-level **constant** *SLOW_4G* (UPPER_CASE by convention = not changed at runtime). The value is a **dict** literal {key: value}.
37	`    "offline": False,`	**Continuation** of the statement started on line 36 (inside brackets).
38	`    "latency": 150,`	**Continuation** of the statement started on line 36 (inside brackets).
39	`    "downloadThroughput": 1.6 * 1024 * 1024 / 8,`	**Continuation** of the statement started on line 36 (inside brackets).
40	`    "uploadThroughput": 750 * 1024 / 8,`	**Continuation** of the statement started on line 36 (inside brackets).
41	`}`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
44	`def _chromium_only(browser_name):`	**def** defines the private helper *_chromium_only* (leading underscore = internal use). Parameters: *browser_name*: name of the browser under test ('chromium', 'firefox', 'webkit').
45	`    if browser_name != "chromium":`	**if** statement: the indented block runs only when *browser_name != 'chromium'* is true.
46	`        pytest.skip("Uses Chromium-only performance APIs / CDP")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
49	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
50	`@pytest.mark.tc("TC-051")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-051 = “Page load on Slow 4G (Nigeria network profile)”.

Line	Code	Syntax & why
51	`@pytest.mark.parametrize("path", PAGES)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
52	`def test_navigation_timing_within_budget(page, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *navigation timing within budget*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize).
53	`    page.goto(path, wait_until="load")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
54	`    t = page.evaluate(NAV_TIMING_JS)`	**Assignment**: binds the name *t* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
55	`    assert t["ttfb"] <= PERF_TTFB_MS, f"{path} TTFB {t['ttfb']:.0f}ms > {PERF_TTFB_MS}ms"`	**assert**: fails the test if *t['ttfb'] <= PERF_TTFB_MS* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
56	`    assert t["dcl"] <= PERF_DCL_MS, f"{path} DCL {t['dcl']:.0f}ms > {PERF_DCL_MS}ms"`	**assert**: fails the test if *t['dcl'] <= PERF_DCL_MS* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
57	`    assert t["load"] <= PERF_LOAD_MS, f"{path} load {t['load']:.0f}ms > {PERF_LOAD_MS}ms"`	**assert**: fails the test if *t['load'] <= PERF_LOAD_MS* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
60	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
61	`@pytest.mark.tc("TC-051")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-051 = “Page load on Slow 4G (Nigeria network profile)”.
62	`@pytest.mark.parametrize("path", ["/", "/rooms"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
63	`def test_largest_contentful_paint_within_budget(page, browser_name, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *largest contentful paint within budget*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *browser_name*: name of the browser under test ('chromium', 'firefox', 'webkit'); *path* (value from @parametrize).
64	`    _chromium_only(browser_name)`	**Method call** executed for its effect.
65	`    page.goto(path, wait_until="load")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
66	`    lcp = page.evaluate(LCP_JS)`	**Assignment**: binds the name *lcp* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
67	`    assert lcp is not None, "LCP not reported"`	**assert**: fails the test if *lcp is not None* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.

Line	Code	Syntax & why
68	`    assert lcp <= PERF_LCP_MS, f"{path} LCP {lcp:.0f}ms > {PERF_LCP_MS}ms"`	**assert**: fails the test if *lcp <= PERF_LCP_MS* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
71	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
72	`@pytest.mark.tc("TC-051")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-051 = “Page load on Slow 4G (Nigeria network profile)”.
73	`@pytest.mark.parametrize("path", ["/", "/rooms"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
74	`def test_lcp_on_slow_4g(page, browser_name, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *lcp on slow 4g*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *browser_name*: name of the browser under test ('chromium', 'firefox', 'webkit'); *path* (value from @parametrize). Docstring: “Nigeria network profile: Slow 4G must still paint the main content within the LCP budget.”
75	`    """Nigeria network profile: Slow 4G must still paint the main content within the LCP budget."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
76	`    _chromium_only(browser_name)`	**Method call** executed for its effect.
77	`    cdp = page.context.new_cdp_session(page)`	**Assignment**: binds the name *cdp* to the value on the right. *new_cdp_session()* opens a Chrome DevTools Protocol session (Chromium only) to emulate network throttling.
78	`    cdp.send("Network.enable")`	**Method call** executed for its effect. *send()* sends a raw CDP command (e.g. Network.emulateNetworkConditions = 'Slow 4G').
79	`    cdp.send("Network.emulateNetworkConditions", SLOW_4G)`	**Method call** executed for its effect. *send()* sends a raw CDP command (e.g. Network.emulateNetworkConditions = 'Slow 4G').
80	`    page.goto(path, wait_until="load", timeout=60000)`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
81	`    lcp = page.evaluate(LCP_JS)`	**Assignment**: binds the name *lcp* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
82	`    assert lcp is not None and lcp <= PERF_LCP_MS, f"{path} LCP on Slow 4G {lcp}ms > {PERF_LCP_MS}ms"`	**assert**: fails the test if *lcp is not None and lcp <= PERF_LCP_MS* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
85	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
86	`@pytest.mark.tc("TC-051")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-051 = “Page load on Slow 4G (Nigeria network profile)”.
87	`def test_offscreen_images_are_lazy_loaded(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *offscreen images are lazy loaded*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.

Line	Code	Syntax & why
88	`    page.goto("/gallery", wait_until="load")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
89	`    loading = page.locator("main img").evaluate_all("els => els.map(e => e.loading)")`	**Assignment**: binds the name *loading* to the value on the right. *evaluate_all()* runs JavaScript on ALL elements matched by a locator at once. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
90	`    assert loading.count("lazy") >= len(loading) - 2, f"images not lazy loaded: {loading}"`	**assert**: fails the test if *loading.count('lazy') >= len(loading) - 2* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *count()* number of current matches (does not wait). *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
93	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
94	`@pytest.mark.tc("TC-053")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-053 = “Cumulative layout shift stays low”.
95	`@pytest.mark.parametrize(`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
96	`    "path", ["/", pytest.param("/rooms", marks=pytest.mark.defect("BUG-013", reason="CLS ~0.16 on /rooms"))]`	**Continuation** of the statement (inside brackets). Keyword argument(s) marks, reason (name=value): self-documenting and order-independent. *param()* a single parametrize value that can carry its own marks (e.g. a defect mark for one page only).
97	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
98	`def test_cumulative_layout_shift(page, browser_name, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *cumulative layout shift*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *browser_name*: name of the browser under test ('chromium', 'firefox', 'webkit'); *path* (value from @parametrize).
99	`    _chromium_only(browser_name)`	**Method call** executed for its effect.
100	`    page.goto(path, wait_until="load")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
101	`    cls = page.evaluate(CLS_JS)`	**Assignment**: binds the name *cls* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
102	`    assert cls <= 0.1, f"{path} CLS {cls:.3f} > 0.1"`	**assert**: fails the test if *cls <= 0.1* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
105	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
106	`@pytest.mark.tc("TC-052")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-052 = “Skeleton loaders always resolve”.

Line	Code	Syntax & why
107	`@pytest.mark.parametrize("path", ["/rooms/does-not-exist", "/rooms"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
108	`def test_skeleton_loaders_always_resolve(page, browser_name, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *skeleton loaders always resolve*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *browser_name*: name of the browser under test ('chromium', 'firefox', 'webkit'); *path* (value from @parametrize).
109	`    _chromium_only(browser_name)`	**Method call** executed for its effect.
110	`    cdp = page.context.new_cdp_session(page)`	**Assignment**: binds the name *cdp* to the value on the right. *new_cdp_session()* opens a Chrome DevTools Protocol session (Chromium only) to emulate network throttling.
111	`    cdp.send("Network.enable")`	**Method call** executed for its effect. *send()* sends a raw CDP command (e.g. Network.emulateNetworkConditions = 'Slow 4G').
112	`    cdp.send("Network.emulateNetworkConditions", SLOW_4G)`	**Method call** executed for its effect. *send()* sends a raw CDP command (e.g. Network.emulateNetworkConditions = 'Slow 4G').
113	`    page.goto(path, timeout=60000)`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
114	`    expect(page.get_by_role("main").get_by_role("heading", level=1)).to_have_text(`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_have_text()* waits until the text equals the string / matches the regex. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
115	`        re.compile(r"\S"), timeout=15000`	**Continuation** of the statement started on line 114 (inside brackets). Keyword argument(s) timeout (name=value): self-documenting and order-independent. *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
116	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
119	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
120	`@pytest.mark.tc("TC-070")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-070 = “No failing API calls or console errors on page load”.
121	`@pytest.mark.parametrize("path", PAGES)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
122	`def test_no_console_errors_or_failed_requests(page, monitor, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no console errors or failed requests*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *monitor*: project fixture: collects console errors, JS exceptions and 4xx/5xx responses; *path* (value from @parametrize).

Line	Code	Syntax & why
123	`    page.goto(path, wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
124	`    assert not monitor.page_errors, f"JS exceptions: {monitor.page_errors}"`	**assert**: fails the test if *not monitor.page_errors* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
125	`    assert not monitor.console_errors, f"Console errors: {monitor.console_errors}"`	**assert**: fails the test if *not monitor.console_errors* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
126	`    assert not monitor.bad_responses, f"4xx/5xx responses: {monitor.bad_responses}"`	**assert**: fails the test if *not monitor.bad_responses* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
129	`@pytest.mark.perf`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.perf**: performance-budget test (filter with -m perf).
130	`@pytest.mark.tc("TC-054")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-054 = “Rooms search response time under concurrent load”.
131	`@pytest.mark.skipif(`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.skipif**: conditional skip. Condition: not ALLOW_LOAD_TESTS.
132	`    not ALLOW_LOAD_TESTS, reason="Load test against shared preview - set ALLOW_LOAD_TESTS=true"`	**Continuation** of the statement (inside brackets). Keyword argument(s) reason (name=value): self-documenting and order-independent.
133	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
134	`def test_rooms_search_p95_under_concurrent_load(api, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *rooms search p95 under concurrent load*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids). Docstring: “50 concurrent availability searches: p95 < 2 s and no 5xx.”
135	`    """50 concurrent availability searches: p95 < 2 s and no 5xx."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
136	`    url = f"/api/rooms?branchId={site.default_branch.id}&limit=50"`	**Assignment**: binds the name *url* to the value on the right. The value is an **f-string**: {expressions} inside are evaluated and inserted.
138	`    def one(_):`	**def** defines the function *one*. Parameters: _.
139	`        start = time.perf_counter()`	**Assignment**: binds the name *start* to the value on the right. *perf_counter()* high-resolution timer for latency measurements.
140	`        r = api.get(url)`	**Assignment**: binds the name *r* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
141	`        return r.status, (time.perf_counter() - start) * 1000`	**return**: ends the function and hands back *(r.status, (time.perf_counter() - start) * 1000)*. *perf_counter()* high-resolution timer for latency measurements.
143	`    with ThreadPoolExecutor(max_workers=50) as pool:`	**with** statement (context manager): sets up a resource and guarantees it is closed/cleaned up when the block ends, even on error - *ThreadPoolExecutor(max_workers=50)*. *ThreadPoolExecutor(...)* creates a ThreadPoolExecutor object (constructor call).

Line	Code	Syntax & why
144	`        results = list(pool.map(one, range(50)))`	**Assignment**: binds the name *results* to the value on the right. *list()* builds a list. *map()* applies a function to every item. *range()* sequence of integers.
145	`    statuses = [s for s, _ in results]`	**Assignment**: binds the name *statuses* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...).
146	`    p95 = statistics.quantiles([d for _, d in results], n=20)[-1]`	**Assignment**: binds the name *p95* to the value on the right. *quantiles()* computes percentiles.
147	`    assert not [s for s in statuses if s >= 500], f"5xx under load: {statuses}"`	**assert**: fails the test if *not [s for s in statuses if s >= 500]* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
148	`    assert p95 < 2000, f"p95 {p95:.0f}ms >= 2000ms"`	**assert**: fails the test if *p95 < 2000* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.

tests/non_functional/test_accessibility.py

Accessibility: axe-core scan (critical/serious = fail), accessible names, keyboard operability, focus visibility, one h1 + landmarks, 44 px tap targets.

Line	Code	Syntax & why
1	`"""Accessibility: axe-core scan, keyboard operation, focus visibility, labels, landmarks/headings."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
4	`from axe_playwright_python.sync_playwright import Axe`	**from axe_playwright_python.sync_playwright import Axe**: imports specific names so they can be used without the module prefix. Wrapper that injects the axe-core accessibility engine into a Playwright page and returns WCAG violations. *Axe*: axe-core runner class; Axe().run(page) returns accessibility violations.
5	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
7	`PAGES = ["/", "/rooms", "/branches", "/gallery", "/about", "/contact", "/manage-booking"]`	Module-level **constant** *PAGES* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
8	`# Known axe findings on the live site (open defects): tracked as xfail so new violations still fail.`	**Comment** (# ... ignored by Python): documents intent for the reader.
9	`AXE_DEFECTS = {`	Module-level **constant** *AXE_DEFECTS* (UPPER_CASE by convention = not changed at runtime). The value is a **dict** literal {key: value}.
10	`    "/": pytest.mark.defect(reason="axe serious: color-contrast below WCAG AA on Home"),`	**Continuation** of the statement started on line 9 (inside brackets). Keyword argument(s) reason (name=value): self-documenting and order-independent.
11	`    "/gallery": pytest.mark.defect(`	**Continuation** of the statement started on line 9 (inside brackets).
12	`        "BUG-045", reason="axe critical: location <select> has no accessible name"`	**Continuation** of the statement started on line 9 (inside brackets). Keyword argument(s) reason (name=value): self-documenting and order-independent.

Line	Code	Syntax & why
13	`    ),`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
14	`}`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
15	`AXE_PAGES = [pytest.param(p, marks=AXE_DEFECTS[p]) if p in AXE_DEFECTS else p for p in PAGES]`	Module-level **constant** *AXE_PAGES* (UPPER_CASE by convention = not changed at runtime). The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *param()* a single parametrize value that can carry its own marks (e.g. a defect mark for one page only).
16	`BLOCKING = {"critical", "serious"} # gate: these impacts fail the build`	Module-level **constant** *BLOCKING* (UPPER_CASE by convention = not changed at runtime). Inline comment: “gate: these impacts fail the build”.
19	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
20	`@pytest.mark.tc("TC-056")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-056 = “Alt text, labels and colour contrast”.
21	`@pytest.mark.parametrize("path", AXE_PAGES)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
22	`def test_no_critical_or_serious_axe_violations(page, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no critical or serious axe violations*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize).
23	`    page.goto(path, wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
24	`    result = Axe().run(page) # injects axe-core and scans the whole document`	**Assignment**: binds the name *result* to the value on the right. *run()* executes axe-core on the page. *Axe(...)* creates a Axe object (constructor call). Inline comment: “injects axe-core and scans the whole document”.
25	`    blocking = [v for v in result.response["violations"] if v["impact"] in BLOCKING]`	**Assignment**: binds the name *blocking* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...).
26	`    summary = "\n".join(`	**Assignment**: binds the name *summary* to the value on the right. *join()* joins strings with a separator.
27	`        f"- [{v['impact']}] {v['id']}: {v['help']} ({len(v['nodes'])} nodes)" for v in blocking`	**Continuation** of the statement started on line 26 (inside brackets). *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
28	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
29	`    assert not blocking, f"{path} has blocking a11y violations:\n{summary}"`	**assert**: fails the test if *not blocking* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
32	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
33	`@pytest.mark.tc("TC-056")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-056 = “Alt text, labels and colour contrast”.

Line	Code	Syntax & why
34	`@pytest.mark.parametrize("path", ["/", "/rooms", "/contact"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
35	`def test_every_form_control_has_an_accessible_name(page, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *every form control has an accessible name*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize).
36	`    page.goto(path, wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
37	`    unnamed = page.evaluate("""() => [...document.querySelectorAll('main input, main select, main textarea, header input')]`	**Assignment**: binds the name *unnamed* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
38	`      .filter(e => e.type !== 'hidden' && e.offsetParent !== null)`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Keeps only elements matching the condition. offsetParent !== null means the element is rendered (not display:none).
39	`      .filter(e => !(e.labels && e.labels.length) && !e.getAttribute('aria-label')`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Keeps only elements matching the condition. Associated <label> elements give inputs an accessible name. aria-label is an accessible name for assistive technology.
40	`                  && !e.getAttribute('aria-labelledby') && !e.getAttribute('placeholder') && !e.title)`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): aria-label is an accessible name for assistive technology.
41	`      .map(e => e.outerHTML.slice(0, 120))""")`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Transforms each element into a small summary object.
42	`    assert not unnamed, f"Unlabelled controls on {path}: {unnamed}"`	**assert**: fails the test if *not unnamed* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
45	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
46	`@pytest.mark.tc("TC-056")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-056 = “Alt text, labels and colour contrast”.
47	`@pytest.mark.defect("BUG-045")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-045 = “Accessibility: header search and price slider have no accessible name, branch select and date fields show no visible focus, CTA image has no alt”.
48	`def test_room_detail_date_inputs_are_labelled(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *room detail date inputs are labelled*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
49	`    page.goto(f"/rooms/{site.bookable_room['id']}", wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted. Uses an **f-string** (f"...{expr}...") to build text with values.

Line	Code	Syntax & why
50	`    for el in page.locator("main input[type=date]").all():`	**for** loop: repeats the indented block for every item, binding it to *el* (iterating *page.locator('main input[type=date]').all()*). *all()* returns a list of Locators, one per current match, for iteration. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
51	`        name = el.evaluate(`	**Assignment**: binds the name *name* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
52	`            "e => (e.labels && e.labels[0] && e.labels[0].innerText) || e.getAttribute('aria-label')"`	**Continuation** of the statement started on line 51 (inside brackets).
53	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
54	`        assert name, "date input without an accessible name"`	**assert**: fails the test if *name* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
57	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
58	`@pytest.mark.tc("TC-055", "TC-100")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-055 = “Keyboard navigation of search, filters and Select room”; TC-100 = “Sign-in modal: role selection (Customer / Agent) and close”.
59	`def test_sign_in_dialog_is_keyboard_operable(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *sign in dialog is keyboard operable*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
60	`    page.goto("/")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
61	`    page.get_by_role("banner").get_by_role("button", name="Sign in").focus()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *focus()* moves keyboard focus to the element. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
62	`    page.keyboard.press("Enter") # keyboard-only activation`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element. Inline comment: “keyboard-only activation”.
63	`    dialog = page.get_by_role("dialog", name="Sign in or create an account")`	**Assignment**: binds the name *dialog* to the value on the right. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
64	`    expect(dialog).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
65	`    page.keyboard.press("Escape") # dialogs should close with Esc`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element. Inline comment: “dialogs should close with Esc”.
66	`    expect(dialog).to_be_hidden()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_hidden()* waits until the element is hidden or detached.

Line	Code	Syntax & why
69	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
70	`@pytest.mark.tc("TC-055")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-055 = “Keyboard navigation of search, filters and Select room”.
71	`def test_keyboard_reaches_search_filters_and_select_room(page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *keyboard reaches search filters and select room*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
72	`    page.goto(f"/rooms?branchId={site.default_branch.id}&adults=1", wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted. Uses an **f-string** (f"...{expr}...") to build text with values.
73	`    reached = set()`	**Assignment**: binds the name *reached* to the value on the right. *set()* builds a set (unique values).
74	`    for _ in range(80):`	**for** loop: repeats the indented block for every item, binding it to _ (iterating *range(80)*). *range()* sequence of integers.
75	`        page.keyboard.press("Tab")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
76	`        info = page.evaluate(`	**Assignment**: binds the name *info* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
77	`            "() => { const e = document.activeElement; "`	**Continuation** of the statement started on line 76 (inside brackets).
78	`            "return (e.getAttribute('aria-label') || e.innerText || e.name || e.type || '').trim(); }"`	**Continuation** of the statement started on line 76 (inside brackets).
79	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
80	`        reached.add(info)`	**Method call** executed for its effect. *add()* adds an item to a set.
81	`        if "Select room" in reached:`	**if** statement: the indented block runs only when *'Select room' in reached* is true.
82	`            break`	**break**: leaves the loop immediately.
83	`    for needed in ("Check availability", "Select room"):`	**for** loop: repeats the indented block for every item, binding it to *needed* (iterating *('Check availability', 'Select room')*).
84	`        assert needed in reached, f"'{needed}' not reachable by Tab; reached {sorted(reached)[:30]}"`	**assert**: fails the test if *needed in reached* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *sorted()* returns a sorted list. Uses an **f-string** (f"...{expr}...") to build text with values.
87	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
88	`@pytest.mark.tc("TC-055")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-055 = “Keyboard navigation of search, filters and Select room”.

Line	Code	Syntax & why
89	`@pytest.mark.defect("BUG-045")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-045 = “Accessibility: header search and price slider have no accessible name, branch select and date fields show no visible focus, CTA image has no alt”.
90	`def test_focus_is_visible_on_search_controls(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *focus is visible on search controls*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
91	`    page.goto("/rooms", wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
92	`    for loc in (page.get_by_role("combobox", name="Branch"), page.get_by_role("textbox", name="Check in")):`	**for** loop: repeats the indented block for every item, binding it to *loc* (iterating *(page.get_by_role('combobox', name='Branch'), page.get_by_r…)*. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
93	`        loc.focus()`	**Method call** executed for its effect. *focus()* moves keyboard focus to the element.
94	`        style = loc.evaluate("""e => { const s = getComputedStyle(e);`	**Assignment**: binds the name *style* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
95	`            return {outline: s.outlineStyle !== 'none' && parseFloat(s.outlineWidth) > 0, shadow: s.boxShadow !== 'none'}; }""")`	**JavaScript** line inside a Python string, executed in the browser by evaluate().
96	`        assert style["outline"] or style["shadow"], f"no visible focus indicator on {loc}"`	**assert**: fails the test if *style['outline'] or style['shadow']* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
99	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
100	`@pytest.mark.tc("TC-123")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-123 = “Screen-reader landmarks and headings on each page”.
101	`@pytest.mark.parametrize("path", PAGES)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
102	`def test_one_meaningful_h1_and_landmarks(page, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *one meaningful h1 and landmarks*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize).
103	`    page.goto(path, wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
104	`    h1s = page.locator("h1").all_inner_texts()`	**Assignment**: binds the name *h1s* to the value on the right. *all_inner_texts()* returns visible text of every match as a list of strings. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).

Line	Code	Syntax & why
105	`    assert len(h1s) == 1 and h1s[0].strip(), f"{path}: h1s = {h1s}"`	**assert**: fails the test if *len(h1s) == 1 and h1s[0].strip()* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *strip()* removes surrounding whitespace. *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
106	`    for role in ("banner", "main", "contentinfo"):`	**for** loop: repeats the indented block for every item, binding it to *role* (iterating *('banner', 'main', 'contentinfo')*).
107	`        expect(page.get_by_role(role).first).to_be_attached()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_attached()* waits until the element exists in the DOM. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
108	`    expect(page.get_by_role("banner").get_by_role("navigation")).to_be_attached()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_attached()* waits until the element exists in the DOM. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
111	`@pytest.mark.a11y`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.a11y**: accessibility test (filter with -m a11y).
112	`@pytest.mark.tc("TC-117")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-117 = “Touch targets at least 44px on mobile”.
113	`@pytest.mark.defect("BUG-039")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-039 = “Mobile tap targets are smaller than 44px (footer links, breadcrumb, room-card titles)”.
114	`def test_touch_targets_at_least_44px_on_mobile(mobile_page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *touch targets at least 44px on mobile*. Parameters: *mobile_page*: project fixture: a tab in a 375x812 touch-enabled mobile context.
115	`    mobile_page.goto("/rooms", wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
116	`    small = mobile_page.evaluate("""() => [...document.querySelectorAll('a, button')]`	**Assignment**: binds the name *small* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
117	`      .filter(e => e.offsetParent !== null)`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Keeps only elements matching the condition. offsetParent !== null means the element is rendered (not display:none).
118	`      .map(e => ({t: (e.innerText || e.getAttribute('aria-label') || '').trim().slice(0, 30),`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Transforms each element into a small summary object. aria-label is an accessible name for assistive technology. Rendered text of the element.
119	`                  h: e.getBoundingClientRect().height}))`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Element size/position on screen (tap-target height, clipping).

Line	Code	Syntax & why
120	`      .filter(x => x.h > 0 && x.h < 44)""")`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Keeps only elements matching the condition.
121	`    assert not small, f"{len(small)} targets under 44px, e.g. {small[:8]}"`	**assert**: fails the test if *not small* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.

tests/non_functional/test_responsive.py

Layout at 375/768/1440/1920 px (no horizontal scroll, header visible, no clipping) and the core flow on emulated phones with touch.

Line	Code	Syntax & why
1	`"""Responsive layout and device compatibility (mobile / tablet / large desktop, emulated phones)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
4	`from playwright.sync_api import expect`	**from playwright.sync_api import expect**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *expect*: Playwright's web-first assertion function. Assertions auto-retry until they pass or time out (5 s default), removing the need for sleeps.
6	`VIEWPORTS = {`	Module-level **constant** *VIEWPORTS* (UPPER_CASE by convention = not changed at runtime). The value is a **dict** literal {key: value}.
7	`    "mobile": {"width": 375, "height": 812},`	**Continuation** of the statement started on line 6 (inside brackets).
8	`    "tablet": {"width": 768, "height": 1024},`	**Continuation** of the statement started on line 6 (inside brackets).
9	`    "desktop": {"width": 1440, "height": 900},`	**Continuation** of the statement started on line 6 (inside brackets).
10	`    "large": {"width": 1920, "height": 1080},`	**Continuation** of the statement started on line 6 (inside brackets).
11	`}`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
12	`PAGES = ["/", "/rooms", "/branches", "/gallery", "/about", "/contact"]`	Module-level **constant** *PAGES* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
13	`TC_BY_VIEWPORT = {"mobile": "TC-046", "tablet": "TC-047", "desktop": "TC-048", "large": "TC-048"}`	Module-level **constant** *TC_BY_VIEWPORT* (UPPER_CASE by convention = not changed at runtime). The value is a **dict** literal {key: value}.
16	`@pytest.mark.responsive`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.responsive**: layout/viewport test.
17	`@pytest.mark.tc("TC-046", "TC-047", "TC-048", "TC-116")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-046 = “Layout at 375px width (mobile)”; TC-047 = “Layout at 768px width (tablet)”; TC-048 = “Layout at 1920px width (large desktop)”; TC-116 = “Branches page at 375px”.
18	`@pytest.mark.parametrize("vp", VIEWPORTS.values(), ids=VIEWPORTS.keys())`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'vp'; each value becomes its own test (the id appears in the report).

Line	Code	Syntax & why
19	`@pytest.mark.parametrize("path", PAGES)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
20	`def test_no_horizontal_scroll(page, vp, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no horizontal scroll*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *vp* (value from @parametrize); *path* (value from @parametrize).
21	`    page.set_viewport_size(vp)`	**Method call** executed for its effect. *set_viewport_size()* resizes the browser window to emulate a device width.
22	`    page.goto(path, wait_until="load")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
23	`    overflow = page.evaluate("() => document.documentElement.scrollWidth - window.innerWidth")`	**Assignment**: binds the name *overflow* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
24	`    assert overflow <= 1, f"{path} overflows horizontally by {overflow}px at {vp['width']}px"`	**assert**: fails the test if *overflow* <= *1* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
27	`@pytest.mark.responsive`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.responsive**: layout/viewport test.
28	`@pytest.mark.tc("TC-046", "TC-047", "TC-048")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-046 = “Layout at 375px width (mobile)”; TC-047 = “Layout at 768px width (tablet)”; TC-048 = “Layout at 1920px width (large desktop)”.
29	`@pytest.mark.parametrize("vp", VIEWPORTS.values(), ids=VIEWPORTS.keys())`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'vp'; each value becomes its own test (the id appears in the report).
30	`def test_header_visible_on_all_viewports(page, vp):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *header visible on all viewports*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *vp* (value from @parametrize).
31	`    page.set_viewport_size(vp)`	**Method call** executed for its effect. *set_viewport_size()* resizes the browser window to emulate a device width.
32	`    page.goto("/")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
33	`    expect(page.get_by_role("banner")).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
36	`@pytest.mark.responsive`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.responsive**: layout/viewport test.

Line	Code	Syntax & why
37	`@pytest.mark.tc("TC-116", "TC-046")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-116 = “Branches page at 375px”; TC-046 = “Layout at 375px width (mobile)”.
38	`def test_branches_cards_not_clipped_at_375(mobile_page, site):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *branches cards not clipped at 375*. Parameters: *mobile_page*: project fixture: a tab in a 375x812 touch-enabled mobile context; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids).
39	`    mobile_page.goto("/branches", wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
40	`    clipped = mobile_page.evaluate("""() => [...document.querySelectorAll('main h2, main a')]`	**Assignment**: binds the name *clipped* to the value on the right. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
41	`      .filter(e => e.offsetParent !== null)`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Keeps only elements matching the condition. offsetParent !== null means the element is rendered (not display:none).
42	`      .filter(e => e.scrollWidth > e.clientWidth + 1 || e.getBoundingClientRect().right > innerWidth + 1)`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Keeps only elements matching the condition. Element size/position on screen (tap-target height, clipping). scrollWidth > clientWidth means content overflows.
43	`      .map(e => e.innerText.slice(0, 40))""")`	**JavaScript** (inside a Python string, executed in the browser by evaluate()): Transforms each element into a small summary object. Rendered text of the element.
44	`    assert not clipped, f"Clipped at 375px: {clipped}"`	**assert**: fails the test if *not clipped* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
47	`@pytest.mark.responsive`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.responsive**: layout/viewport test.
48	`@pytest.mark.tc("TC-048")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-048 = “Layout at 1920px width (large desktop)”.
49	`def test_content_centred_with_readable_width_at_1920(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *content centred with readable width at 1920*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
50	`    page.set_viewport_size(VIEWPORTS["large"])`	**Method call** executed for its effect. *set_viewport_size()* resizes the browser window to emulate a device width.
51	`    page.goto("/rooms", wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
52	`    box = page.get_by_role("main").locator("h1").bounding_box()`	**Assignment**: binds the name *box* to the value on the right. *bounding_box()* element position/size on screen; used to detect off-screen drawers. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
53	`    assert box["x"] > 100, "content hugs the left edge at 1920px (not centred)"`	**assert**: fails the test if *box['x'] > 100* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
54	`    assert box["width"] < 1700`	**assert**: fails the test if *box['width'] < 1700* is false. pytest rewrites asserts to print the actual values on failure.
57	`@pytest.mark.responsive`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.responsive**: layout/viewport test.
58	`@pytest.mark.tc("TC-050", "TC-049")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-050 = “Mobile browsers: Chrome Android and Safari iOS”; TC-049 = “Core booking flow in Chrome, Firefox, Edge, Safari”.
59	`@pytest.mark.parametrize("phone", ["Pixel 7", "iPhone 13"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'phone'; each value becomes its own test (the id appears in the report).
60	`def test_core_flow_on_emulated_phones(playwright, browser, browser_name, base_url, site, phone):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *core flow on emulated phones*. Parameters: *playwright*: the Playwright driver object (devices, request contexts); *browser*: pytest-playwright fixture: the browser process (session-wide) chosen with --browser; *browser_name*: name of the browser under test ('chromium', 'firefox', 'webkit'); *base_url*: site under test (pytest.ini or --base-url); *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *phone* (value from @parametrize). Docstring: “Search -> filter -> open room with touch emulation (real devices remain a manual activity).”
61	`    """Search -> filter -> open room with touch emulation (real devices remain a manual activity)."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
62	`    profile = dict(playwright.devices[phone])`	**Assignment**: binds the name *profile* to the value on the right. *dict()* builds a dict (copy of a mapping).
63	`    if browser_name == "firefox":`	**if** statement: the indented block runs only when *browser_name == 'firefox'* is true.
64	`        profile.pop("is_mobile", None) # Firefox does not support isMobile`	**Method call** executed for its effect. *pop()* removes a key and returns its value. Inline comment: “Firefox does not support isMobile”.
65	`    profile.pop("default_browser_type", None)`	**Method call** executed for its effect. *pop()* removes a key and returns its value.
66	`    ctx = browser.new_context(**profile, base_url=base_url, timezone_id="Africa/Lagos")`	**Assignment**: binds the name *ctx* to the value on the right. *new_context()* creates an isolated browser profile (own cookies/storage). Isolation means tests cannot affect each other.
67	`    page = ctx.new_page()`	**Assignment**: binds the name *page* to the value on the right. *new_page()* opens a new tab in a context.
68	`    try:`	**try**: runs the block and lets the except clause handle expected errors instead of crashing.
69	`        page.goto(f"/rooms?branchId={site.default_branch.id}&adults=1")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted. Uses an **f-string** (f"...{expr}...") to build text with values.
70	`        page.get_by_role("main").get_by_role("button", name="Filter", exact=True).tap()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *tap()* touch tap (requires has_touch=True context) - used in mobile emulation tests. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
71	`        drawer = page.get_by_role("dialog", name="Filters")`	**Assignment**: binds the name *drawer* to the value on the right. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
72	`        expect(drawer).to_be_visible()`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible.
73	`        drawer.get_by_role("button", name="Close filters").tap()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *tap()* touch tap (requires has_touch=True context) - used in mobile emulation tests. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
74	`        page.get_by_role("link", name="Select room").first.tap()`	**Method call** executed for its effect. *tap()* touch tap (requires has_touch=True context) - used in mobile emulation tests. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
75	`        expect(page.get_by_role("button", name="Continue to guest details")).to_be_visible(timeout=15000)`	**Web-first assertion**: expect(locator).matcher(...) re-checks the condition until it holds or times out (5 s default). This waits for asynchronous UI updates without sleeps, the main cure for flaky UI tests. *to_be_visible()* web-first assertion: waits until the element is visible. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
76	`    finally:`	**finally**: always runs (cleanup), whether or not an error occurred.
77	`        ctx.close()`	**Method call** executed for its effect. *close()* closes a page/context/browser and frees resources.

tests/non_functional/test_security.py

HTTPS redirect, HSTS, security headers, mixed content, anonymous access to staff paths, API error hygiene and server-side validation of crafted booking quotes.

Line	Code	Syntax & why
1	`"""Transport security, response headers, access control, API error hygiene, server-side validation."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import json`	**import json**: loads the module. Serialises Python objects to JSON text; used to inspect API payloads.
5	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`HEADERS = [`	Module-level **constant** *HEADERS* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
8	`    "content-security-policy",`	**Continuation** of the statement started on line 7 (inside brackets).
9	`    "x-frame-options",`	**Continuation** of the statement started on line 7 (inside brackets).
10	`    "x-content-type-options",`	**Continuation** of the statement started on line 7 (inside brackets).

Line	Code	Syntax & why
11	`    "referrer-policy",`	**Continuation** of the statement started on line 7 (inside brackets).
12	`    "permissions-policy",`	**Continuation** of the statement started on line 7 (inside brackets).
13	`]`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
14	`PATHS = ["/", "/rooms", "/booking"]`	Module-level **constant** *PATHS* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
17	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
18	`@pytest.mark.tc("TC-058")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-058 = “All pages served over HTTPS with secure headers”.
19	`def test_http_redirects_to_https(playwright, base_url):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *http redirects to https*. Parameters: *playwright*: the Playwright driver object (devices, request contexts); *base_url*: site under test (pytest.ini or --base-url).
20	`    if not base_url.startswith("https://"):`	**if** statement: the indented block runs only when *not base_url.startswith('https://')* is true. *startswith()* prefix test.
21	`        pytest.skip("Base URL is not HTTPS")`	**Method call** executed for its effect. *skip()* skips the test with a visible reason (keeps blocked/not-applicable cases in the report).
22	`    ctx = playwright.request.new_context() # raw HTTP client, no browser`	**Assignment**: binds the name *ctx* to the value on the right. *new_context()* creates an isolated browser profile (own cookies/storage). Isolation means tests cannot affect each other. Inline comment: “raw HTTP client, no browser”.
23	`    resp = ctx.get(base_url.replace("https://", "http://", 1), max_redirects=0)`	**Assignment**: binds the name *resp* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get). *replace()* string replace.
24	`    assert resp.status in (301, 302, 307, 308)`	**assert**: fails the test if *resp.status in (301, 302, 307, 308)* is false. pytest rewrites asserts to print the actual values on failure.
25	`    assert resp.headers["location"].startswith("https://")`	**assert**: fails the test if *resp.headers['location'].startswith('https://')* is false. pytest rewrites asserts to print the actual values on failure. *startswith()* prefix test.
26	`    ctx.dispose()`	**Method call** executed for its effect. *dispose()* releases an APIRequestContext.
29	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
30	`@pytest.mark.tc("TC-058")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-058 = “All pages served over HTTPS with secure headers”.
31	`def test_hsts_header_present(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *hsts header present*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.
32	`    hsts = api.get("/").headers.get("strict-transport-security", "")`	**Assignment**: binds the name *hsts* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).

Line	Code	Syntax & why
33	`    assert "max-age" in hsts and int(hsts.split("max-age=")[1].split(";")[0]) >= 15552000, hsts`	**assert**: fails the test if *'max-age' in hsts and int(hsts.split('max-age=')[1].split(';')[0]) >= 15552000* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *int()* converts to integer. *split()* splits a string into a list.
36	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
37	`@pytest.mark.tc("TC-118", "TC-058")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-118 = “Security response headers present”; TC-058 = “All pages served over HTTPS with secure headers”.
38	`@pytest.mark.parametrize("path", PATHS)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
39	`@pytest.mark.parametrize("header", HEADERS)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'header'; each value becomes its own test (the id appears in the report).
40	`def test_security_headers_present(api, path, header):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *security headers present*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site; *path* (value from @parametrize); *header* (value from @parametrize).
41	`    assert header in api.get(path).headers, f"{header} missing on {path}"`	**assert**: fails the test if *header in api.get(path).headers* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get). Uses an **f-string** (f"...{expr}...") to build text with values.
44	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
45	`@pytest.mark.tc("TC-118")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-118 = “Security response headers present”.
46	`def test_csp_blocks_framing(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *csp blocks framing*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.
47	`    h = api.get("/").headers`	**Assignment**: binds the name *h* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
48	`    assert "frame-ancestors 'self'" in h.get("content-security-policy", "") or h.get("x-frame-options") in (`	**assert**: fails the test if *"frame-ancestors 'self'" in h.get('content-security-policy', '') or h.get('x-fr…* is false. pytest rewrites asserts to print the actual values on failure. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
49	`        "DENY",`	**Continuation** of the statement started on line 48 (inside brackets).
50	`        "SAMEORIGIN",`	**Continuation** of the statement started on line 48 (inside brackets).
51	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
54	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).

Line	Code	Syntax & why
55	`@pytest.mark.tc("TC-058")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-058 = “All pages served over HTTPS with secure headers”.
56	`def test_no_mixed_content(page):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no mixed content*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
57	`    insecure = []`	**Assignment**: binds the name *insecure* to the value on the right. The value is a **list** literal [...].
58	`    page.on("request", lambda r: insecure.append(r.url) if r.url.startswith("http://") else None)`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *startswith()* prefix test. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate).
59	`    for path in ["/", "/rooms", "/gallery", "/contact"]:`	**for** loop: repeats the indented block for every item, binding it to *path* (iterating *['/', '/rooms', '/gallery', '/contact']*).
60	`        page.goto(path, wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
61	`    assert not insecure, f"Mixed content: {insecure}"`	**assert**: fails the test if *not insecure* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
64	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
65	`@pytest.mark.tc("TC-058")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-058 = “All pages served over HTTPS with secure headers”.
66	`@pytest.mark.defect(reason="Server header 'Vercel' is fine; x-powered-by: Next.js discloses the framework")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Reason: "Server header 'Vercel' is fine; x-powered-by: Next.js discloses the framework".
67	`def test_no_technology_disclosure_headers(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no technology disclosure headers*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.
68	`    assert "x-powered-by" not in api.get("/").headers`	**assert**: fails the test if *'x-powered-by' not in api.get('/').headers* is false. pytest rewrites asserts to print the actual values on failure. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
71	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
72	`@pytest.mark.tc("TC-017")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-017 = “Unauthenticated access to /staff is blocked”.
73	`@pytest.mark.parametrize("path", ["/staff", "/staff/dashboard", "/admin"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).

Line	Code	Syntax & why
74	`def test_staff_console_not_reachable_anonymously(api, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *staff console not reachable anonymously*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site; *path* (value from @parametrize).
75	`    resp = api.get(path, max_redirects=0)`	**Assignment**: binds the name *resp* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
76	`    assert resp.status in (301, 302, 307, 308, 401, 403, 404)`	**assert**: fails the test if *resp.status in (301, 302, 307, 308, 401, 403, 404)* is false. pytest rewrites asserts to print the actual values on failure.
77	`    assert "dashboard" not in resp.text().lower() or resp.status >= 400`	**assert**: fails the test if *'dashboard' not in resp.text().lower() or resp.status >= 400* is false. pytest rewrites asserts to print the actual values on failure. *lower()* lower-cases a string (case-insensitive comparisons). *text()* response body as string.
80	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
81	`@pytest.mark.tc("TC-017", "TC-107")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-017 = “Unauthenticated access to /staff is blocked”; TC-107 = “Sign out clears the session”.
82	`def test_auth_me_without_session(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *auth me without session*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.
83	`    body = api.get("/api/auth/me").json()`	**Assignment**: binds the name *body* to the value on the right. *json()* parses a response body (or returns it) as Python dict/list. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
84	`    assert body.get("data") is None`	**assert**: fails the test if *body.get('data') is None* is false. pytest rewrites asserts to print the actual values on failure. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
87	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
88	`@pytest.mark.api`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.api**: HTTP-level check without the browser UI.
89	`@pytest.mark.tc("TC-119")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-119 = “API errors do not leak internals; unknown API routes return JSON 404”.
90	`def test_unknown_api_route_returns_json_404(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *unknown api route returns json 404*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.
91	`    resp = api.get("/api/unknown-route-qa")`	**Assignment**: binds the name *resp* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
92	`    assert resp.status == 404`	**assert**: fails the test if *resp.status == 404* is false. pytest rewrites asserts to print the actual values on failure.
93	`    assert "application/json" in resp.headers.get("content-type", "")`	**assert**: fails the test if *'application/json' in resp.headers.get('content-type', '')* is false. pytest rewrites asserts to print the actual values on failure. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).

Line	Code	Syntax & why
96	`QUOTE_CASES = {`	Module-level **constant** *QUOTE_CASES* (UPPER_CASE by convention = not changed at runtime). The value is a **dict** literal {key: value}.
97	`    "past-dates": {"checkIn": "2020-01-01", "checkOut": "2020-01-03", "adults": 1},`	**Continuation** of the statement started on line 96 (inside brackets).
98	`    "inverted": {"checkIn": "2027-01-10", "checkOut": "2027-01-05", "adults": 1},`	**Continuation** of the statement started on line 96 (inside brackets).
99	`    "over-90-nights": {"checkIn": "2027-01-01", "checkOut": "2027-06-01", "adults": 1},`	**Continuation** of the statement started on line 96 (inside brackets).
100	`    "over-capacity": {"checkIn": "2027-01-10", "checkOut": "2027-01-12", "adults": 9},`	**Continuation** of the statement started on line 96 (inside brackets).
101	`    "negative-children": {"checkIn": "2027-01-10", "checkOut": "2027-01-12", "adults": 1, "children": -1},`	**Continuation** of the statement started on line 96 (inside brackets).
102	`    "bad-room-id": {"roomId": "not-a-uuid", "checkIn": "2027-01-10", "checkOut": "2027-01-12", "adults": 1},`	**Continuation** of the statement started on line 96 (inside brackets).
103	`}`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
106	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
107	`@pytest.mark.api`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.api**: HTTP-level check without the browser UI.
108	`@pytest.mark.tc("TC-098")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-098 = “Server rechecks dates, capacity and room before quoting”.
109	`@pytest.mark.parametrize("case", QUOTE_CASES.values(), ids=QUOTE_CASES.keys())`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'case'; each value becomes its own test (the id appears in the report).
110	`def test_server_rejects_invalid_quotes(api, site, case):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *server rejects invalid quotes*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *case* (value from @parametrize).
111	`    payload = {"roomId": site.bookable_room["id"], **case}`	**Assignment**: binds the name *payload* to the value on the right. The value is a **dict** literal {key: value}.
112	`    resp = api.post("/api/booking/quote", data=payload)`	**Assignment**: binds the name *resp* to the value on the right. *post()* HTTP POST with a JSON body.
113	`    assert resp.status >= 400, f"{payload} accepted with {resp.status}"`	**assert**: fails the test if *resp.status >= 400* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
116	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
117	`@pytest.mark.api`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.api**: HTTP-level check without the browser UI.

Line	Code	Syntax & why
118	`@pytest.mark.tc("TC-119", "TC-098")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-119 = “API errors do not leak internals; unknown API routes return JSON 404”; TC-098 = “Server rechecks dates, capacity and room before quoting”.
119	`@pytest.mark.defect("BUG-042")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-042 = “API errors expose backend internals, and unknown /api routes return a full HTML page”.
120	`@pytest.mark.parametrize("case", ["past-dates", "bad-room-id"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'case'; each value becomes its own test (the id appears in the report).
121	`def test_quote_validation_errors_are_4xx_and_generic(api, site, case):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *quote validation errors are 4xx and generic*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site; *site*: project fixture: branches and rooms read from the live API (no hard-coded ids); *case* (value from @parametrize).
122	`    resp = api.post("/api/booking/quote", data={"roomId": site.bookable_room["id"], **QUOTE_CASES[case]})`	**Assignment**: binds the name *resp* to the value on the right. *post()* HTTP POST with a JSON body.
123	`    assert 400 <= resp.status < 500, f"validation error returned HTTP {resp.status}"`	**assert**: fails the test if *400 <= resp.status < 500* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
124	`    text = resp.text()`	**Assignment**: binds the name *text* to the value on the right. *text()* response body as string.
125	`    assert "Catalog request failed" not in text and "statusCode" not in text, text[:200]`	**assert**: fails the test if *'Catalog request failed' not in text and 'statusCode' not in text* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report.
128	`@pytest.mark.security`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.security**: security test (filter with -m security).
129	`@pytest.mark.api`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.api**: HTTP-level check without the browser UI.
130	`@pytest.mark.tc("TC-077", "TC-121")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-077 = “No test or placeholder records visible on any public page”; TC-121 = “Content and copy review (grammar, placeholders, test contacts)”.
131	`@pytest.mark.defect("BUG-046")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.defect**: the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'. Linked: BUG-046 = “Placeholder and test content on the public site: test email in public API, tagline typo, informal hero copy, single amenity”.
132	`def test_public_api_exposes_no_test_contacts(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *public api exposes no test contacts*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.

Line	Code	Syntax & why
133	`    text = json.dumps(api.get("/api/customization").json()) + json.dumps(api.get("/api/settings").json())`	**Assignment**: binds the name *text* to the value on the right. *dumps()* serialises to JSON text. *json()* parses a response body (or returns it) as Python dict/list. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
134	`    assert "test@" not in text.lower() and "test-image" not in text.lower(), "test data in public API"`	**assert**: fails the test if *'test@' not in text.lower() and 'test-image' not in text.lower()* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. *lower()* lower-cases a string (case-insensitive comparisons).

tests/non_functional/test_seo_content.py

robots.txt, sitemap.xml, Open Graph tags and a copy review for placeholders/typos.

Line	Code	Syntax & why
1	`"""SEO files, Open Graph tags, copy/content review across pages."""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
3	`import re`	**import re**: loads the module. Python's regular-expression module. Used to match text that varies (prices, counts, dates) instead of exact strings, which keeps assertions robust to dynamic data.
5	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
7	`PAGES = ["/", "/rooms", "/branches", "/gallery", "/about", "/contact"]`	Module-level **constant** *PAGES* (UPPER_CASE by convention = not changed at runtime). The value is a **list** literal [...].
8	`COPY_PROBLEMS = re.compile(`	Module-level **constant** *COPY_PROBLEMS* (UPPER_CASE by convention = not changed at runtime). *compile()* pre-compiles a regex pattern for reuse. Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
9	`    r"whatever amount|lorem ipsum|placeholder|TODO|Live changing|\bteh\b|all three houses|test@", re.I`	**Continuation** of the statement started on line 8 (inside brackets). Contains a **regular expression** (r"..." raw string so backslashes are kept literally).
10	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
13	`@pytest.mark.seo`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.seo**: search-engine/metadata check.
14	`@pytest.mark.tc("TC-115")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-115 = “robots.txt, sitemap.xml and Open Graph tags exist”.
15	`@pytest.mark.parametrize("path,ctype", [("/robots.txt", "text/plain"), ("/sitemap.xml", "xml")])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path,ctype'; each value becomes its own test (the id appears in the report).
16	`def test_robots_and_sitemap(api, path, ctype):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *robots and sitemap*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site; *path* (value from @parametrize); *ctype* (value from @parametrize).
17	`    resp = api.get(path)`	**Assignment**: binds the name *resp* to the value on the right. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).

Line	Code	Syntax & why
18	`    assert resp.status == 200 and ctype in resp.headers.get("content-type", "")`	**assert**: fails the test if *resp.status == 200 and ctype in resp.headers.get('content-type', '')* is false. pytest rewrites asserts to print the actual values on failure. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
21	`@pytest.mark.seo`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.seo**: search-engine/metadata check.
22	`@pytest.mark.tc("TC-115")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-115 = “robots.txt, sitemap.xml and Open Graph tags exist”.
23	`def test_robots_points_to_sitemap_and_hides_private_paths(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *robots points to sitemap and hides private paths*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.
24	`    text = api.get("/robots.txt").text()`	**Assignment**: binds the name *text* to the value on the right. *text()* response body as string. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
25	`    assert "Sitemap:" in text and "Disallow: /account" in text and "Disallow: /api/" in text`	**assert**: fails the test if *'Sitemap:' in text and 'Disallow: /account' in text and ('Disallow: /api/' in t…* is false. pytest rewrites asserts to print the actual values on failure.
28	`@pytest.mark.seo`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.seo**: search-engine/metadata check.
29	`@pytest.mark.tc("TC-115")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-115 = “robots.txt, sitemap.xml and Open Graph tags exist”.
30	`def test_sitemap_lists_public_pages(api):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *sitemap lists public pages*. Parameters: *api*: project fixture: browser-less HTTP client bound to the site.
31	`    xml = api.get("/sitemap.xml").text()`	**Assignment**: binds the name *xml* to the value on the right. *text()* response body as string. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
32	`    for path in ("/rooms", "/branches"):`	**for** loop: repeats the indented block for every item, binding it to *path* (iterating *('/rooms', '/branches')*).
33	`        assert f"{path}</loc>" in xml, f"{path} missing from sitemap"`	**assert**: fails the test if *f'{path}</loc>' in xml* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
36	`@pytest.mark.seo`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.seo**: search-engine/metadata check.
37	`@pytest.mark.tc("TC-115")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-115 = “robots.txt, sitemap.xml and Open Graph tags exist”.
38	`@pytest.mark.parametrize("path", ["/", "/rooms", "/contact"])`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
39	`def test_open_graph_tags(page, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *open graph tags*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize).

Line	Code	Syntax & why
40	`    page.goto(path)`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
41	`    for prop in ("og:title", "og:description", "og:image"):`	**for** loop: repeats the indented block for every item, binding it to *prop* (iterating *('og:title', 'og:description', 'og:image')*).
42	`        content = page.locator(f'meta[property="{prop}"]').get_attribute("content")`	**Assignment**: binds the name *content* to the value on the right. *get_attribute()* reads an HTML attribute (href, min, max, alt, type). *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath). Uses an **f-string** (f"...{expr}...") to build text with values.
43	`        assert content, f"{prop} missing on {path}"`	**assert**: fails the test if *content* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.
46	`@pytest.mark.tc("TC-121")`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.tc**: links the test to workbook test case(s) - drives traceability reports and the coverage check in CI. Linked: TC-121 = “Content and copy review (grammar, placeholders, test contacts)”.
47	`@pytest.mark.parametrize("path", PAGES)`	**Decorator** (@): wraps the function/class below and attaches behaviour or metadata to it. **pytest.mark.parametrize**: data-driven: one test per value. Parameter name(s) 'path'; each value becomes its own test (the id appears in the report).
48	`def test_no_placeholder_or_typo_copy(page, path):`	**def** defines a **test function**. pytest collects every function named test_*; the parameters are fixtures that pytest injects by name (dependency injection). Checks: *no placeholder or typo copy*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path* (value from @parametrize).
49	`    page.goto(path, wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
50	`    found = COPY_PROBLEMS.findall(page.get_by_role("main").inner_text())`	**Assignment**: binds the name *found* to the value on the right. *findall()* returns every regex match. *inner_text()* visible text of the element (as rendered, respects CSS). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
51	`    assert not found, f"{path}: {found}"`	**assert**: fails the test if *not found* is false. pytest rewrites asserts to print the actual values on failure. The text after the comma is the failure message shown in the report. Uses an **f-string** (f"...{expr}...") to build text with values.

ci/quality_gate.py

Reads JUnit XML and fails the CI job when the pass rate, minimum test count or duration thresholds are broken. xfail (known defects) are reported separately so they stay visible. Writes a summary to the GitHub run page.

Line	Code	Syntax & why
1	`"""Quality gate: reads one or more pytest JUnit XML files and exits 1 if thresholds are broken.`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
2	`CI treats a non-zero exit code as a failed job, which blocks the merge.`	**Docstring** continuation (documentation text).

Line	Code	Syntax & why
4	`Known defects (tests marked @pytest.mark.defect) are reported as xfail and do not count against the`	**Docstring** continuation (documentation text).
5	`pass rate; they are listed separately so they stay visible. When running in GitHub Actions the`	**Docstring** continuation (documentation text).
6	`summary is also written to the job summary page ($GITHUB_STEP_SUMMARY)."""`	**Docstring** continuation (documentation text).
8	`import argparse`	**import argparse**: loads the module. Builds a command-line interface (flags such as --min-pass-rate) for CI helper scripts.
9	`import os`	**import os**: loads the module. Operating-system helpers. Used to read environment variables (os.getenv) so CI and local runs share code but take different inputs.
10	`import sys`	**import sys**: loads the module. Interpreter access. sys.exit(code) sets the process exit code that CI uses to decide pass/fail.
11	`import xml.etree.ElementTree as ET`	**import xml.etree.ElementTree**: loads the module. Parses the JUnit XML report written by pytest so the quality gate can count passes/failures.
14	`def collect(paths):`	**def** defines the function *collect*. Parameters: *paths*.
15	`    stats = dict(tests=0, failed=0, skipped=0, xfail=0, duration=0.0)`	**Assignment**: binds the name *stats* to the value on the right. *dict()* builds a dict (copy of a mapping).
16	`    failures = []`	**Assignment**: binds the name *failures* to the value on the right. The value is a **list** literal [...].
17	`    for path in paths:`	**for** loop: repeats the indented block for every item, binding it to *path* (iterating *paths*).
18	`        root = ET.parse(path).getroot()`	**Assignment**: binds the name *root* to the value on the right. *getroot()* root element of an XML tree. *parse()* parses an XML file.
19	`        suites = [root] if root.tag == "testsuite" else root.findall("testsuite")`	**Assignment**: binds the name *suites* to the value on the right. **Conditional expression** (a if condition else b). *findall()* returns every regex match.
20	`        for s in suites:`	**for** loop: repeats the indented block for every item, binding it to *s* (iterating *suites*).
21	`            stats["tests"] += int(s.get("tests", 0))`	**Augmented assignment** =: updates *stats['tests']* in place. *int()* converts to integer. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
22	`            stats["failed"] += int(s.get("failures", 0)) + int(s.get("errors", 0))`	**Augmented assignment** =: updates *stats['failed']* in place. *int()* converts to integer. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
23	`            stats["skipped"] += int(s.get("skipped", 0)) # includes xfail outcomes`	**Augmented assignment** =: updates *stats['skipped']* in place. *int()* converts to integer. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get). Inline comment: “includes xfail outcomes”.
24	`            stats["duration"] += float(s.get("time", 0))`	**Augmented assignment** =: updates *stats['duration']* in place. *float()* converts to float. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
25	`            for case in s.iter("testcase"):`	**for** loop: repeats the indented block for every item, binding it to *case* (iterating *s.iter('testcase')*). *iter()* iterates XML elements by tag.
26	`                sk = case.find("skipped")`	**Assignment**: binds the name *sk* to the value on the right. *find()* first child XML element by tag.
27	`                if sk is not None and sk.get("type") == "pytest.xfail":`	**if** statement: the indented block runs only when *sk is not None and sk.get('type') == 'pytest.xfail'* is true. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get).
28	`                    stats["xfail"] += 1`	**Augmented assignment** =: updates *stats['xfail']* in place.
29	`                if case.find("failure") is not None or case.find("error") is not None:`	**if** statement: the indented block runs only when *case.find('failure') is not None or case.find('error') is not None* is true. *find()* first child XML element by tag.

Line	Code	Syntax & why
30	`                    failures.append(f"{case.get('classname')}::{case.get('name')}")`	**Method call** executed for its effect. *append()* adds an item to a list. *get()* HTTP GET (APIRequestContext) or dict lookup with default (dict.get). Uses an **f-string** (f"...{expr}...") to build text with values.
31	`    return stats, failures`	**return**: ends the function and hands back *(stats, failures)*.
34	`def main() -> int:`	**def** defines the function *main*. Returns int.
35	`    p = argparse.ArgumentParser()`	**Assignment**: binds the name *p* to the value on the right. *ArgumentParser()* creates the command-line parser.
36	`    p.add_argument("junit", nargs="+") # path(s) to junit.xml`	**Method call** executed for its effect. *add_argument()* declares one CLI argument. Inline comment: “path(s) to junit.xml”.
37	`    p.add_argument("--min-pass-rate", type=float, default=100.0) # % of executed tests that must pass`	**Method call** executed for its effect. *add_argument()* declares one CLI argument. Inline comment: “% of executed tests that must pass”.
38	`    p.add_argument("--min-tests", type=int, default=1) # guards against an empty/mis-collected suite`	**Method call** executed for its effect. *add_argument()* declares one CLI argument. Inline comment: “guards against an empty/mis-collected suite”.
39	`    p.add_argument("--max-duration", type=float, default=1e9) # seconds, total wall time of tests`	**Method call** executed for its effect. *add_argument()* declares one CLI argument. Inline comment: “seconds, total wall time of tests”.
40	`    p.add_argument("--name", default="Quality gate")`	**Method call** executed for its effect. *add_argument()* declares one CLI argument.
41	`    a = p.parse_args()`	**Assignment**: binds the name *a* to the value on the right. *parse_args()* parses sys.argv into an object.
43	`    st, failures = collect(a.junit)`	**Assignment**: binds the name *(st, failures)* to the value on the right. *collect()*: project function collect() in ci/quality_gate.py.
44	`    executed = st["tests"] - st["skipped"]`	**Assignment**: binds the name *executed* to the value on the right.
45	`    rate = 100.0 if executed == 0 else (executed - st["failed"]) / executed * 100`	**Assignment**: binds the name *rate* to the value on the right. **Conditional expression** (a if condition else b).
47	`    line = (`	**Assignment**: binds the name *line* to the value on the right. The value is an **f-string**: {expressions} inside are evaluated and inserted.
48	`        f"tests={st['tests']} executed={executed} skipped={st['skipped'] - st['xfail']} "`	**Continuation** of the statement started on line 47 (inside brackets). Keyword argument(s) tests, executed, skipped (name=value): self-documenting and order-independent. Uses an **f-string** (f"...{expr}...") to build text with values.
49	`        f"known_defects(xfail)={st['xfail']} failed={st['failed']} pass_rate={rate:.1f}% "`	**Continuation** of the statement started on line 47 (inside brackets). Keyword argument(s) failed, pass_rate (name=value): self-documenting and order-independent. Uses an **f-string** (f"...{expr}...") to build text with values.
50	`        f"duration={st['duration']:.0f}s"`	**Continuation** of the statement started on line 47 (inside brackets). Keyword argument(s) duration (name=value): self-documenting and order-independent. Uses an **f-string** (f"...{expr}...") to build text with values.
51	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
52	`    print(line)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
54	`    problems = []`	**Assignment**: binds the name *problems* to the value on the right. The value is a **list** literal [...].
55	`    if st["tests"] < a.min_tests:`	**if** statement: the indented block runs only when *st['tests'] < a.min_tests* is true.
56	`        problems.append(f"only {st['tests']} tests ran (< {a.min_tests})")`	**Method call** executed for its effect. *append()* adds an item to a list. Uses an **f-string** (f"...{expr}...") to build text with values.

Line	Code	Syntax & why
57	`    if rate < a.min_pass_rate:`	**if** statement: the indented block runs only when *rate < a.min_pass_rate* is true.
58	`        problems.append(f"pass rate {rate:.1f}% < {a.min_pass_rate}%")`	**Method call** executed for its effect. *append()* adds an item to a list. Uses an **f-string** (f"...{expr}...") to build text with values.
59	`    if st["duration"] > a.max_duration:`	**if** statement: the indented block runs only when *st['duration'] > a.max_duration* is true.
60	`        problems.append(f"duration {st['duration']:.0f}s > {a.max_duration:.0f}s")`	**Method call** executed for its effect. *append()* adds an item to a list. Uses an **f-string** (f"...{expr}...") to build text with values.
62	`    for msg in problems:`	**for** loop: repeats the indented block for every item, binding it to *msg* (iterating *problems*).
63	`        print(f"QUALITY GATE FAILED: {msg}")`	**Method call** executed for its effect. *print()* writes to the console / CI log. Uses an **f-string** (f"...{expr}...") to build text with values.
65	`    summary = os.getenv("GITHUB_STEP_SUMMARY")`	**Assignment**: binds the name *summary* to the value on the right. *getenv()* reads an environment variable with a default.
66	`    if summary:`	**if** statement: the indented block runs only when *summary* is true.
67	`        with open(summary, "a") as f:`	**with** statement (context manager): sets up a resource and guarantees it is closed/cleaned up when the block ends, even on error - *open(summary, 'a')*. *open()* opens a file (read/write).
68	`            f.write(f"### {a.name}: {'FAILED' if problems else 'PASSED'}\n\n`{line}`\n\n")`	**Method call** executed for its effect. *write()* writes text to a file. Uses an **f-string** (f"...{expr}...") to build text with values.
69	`            f.write(f"Thresholds: pass rate >= {a.min_pass_rate}%, tests >= {a.min_tests}\n\n")`	**Method call** executed for its effect. *write()* writes text to a file. Uses an **f-string** (f"...{expr}...") to build text with values.
70	`            for msg in problems:`	**for** loop: repeats the indented block for every item, binding it to *msg* (iterating *problems*).
71	`                f.write(f"- :x: {msg}\n")`	**Method call** executed for its effect. *write()* writes text to a file. Uses an **f-string** (f"...{expr}...") to build text with values.
72	`            if failures:`	**if** statement: the indented block runs only when *failures* is true.
73	`                f.write("\n<details><summary>Failed tests</summary>\n\n")`	**Method call** executed for its effect. *write()* writes text to a file.
74	`                f.write("\n".join(f"- `{t}`" for t in failures[:100]) + "\n\n</details>\n\n")`	**Method call** executed for its effect. *write()* writes text to a file. *join()* joins strings with a separator. Uses an **f-string** (f"...{expr}...") to build text with values.
75	`    return 1 if problems else 0`	**return**: ends the function and hands back *1 if problems else 0*.
78	`if __name__ == "__main__":`	**if __name__ == '__main__'**: true only when the file is executed directly (python file.py), not when imported.
79	`    sys.exit(main())`	**Method call** executed for its effect. *exit()* ends the process with an exit code (non-zero = CI failure). *main()*: project function main() in ci/check_traceability.py.

ci/check_traceability.py

Static gate (no browser): every collected test must carry a tc marker and every TC id must exist in docs/test_cases.csv. Prints workbook coverage.

Line	Code	Syntax & why
1	`"""Static traceability check (no browser): every collected test must carry @pytest.mark.tc(...) and every`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.

Line	Code	Syntax & why
2	`TC id it references must exist in docs/test_cases.csv (exported from the QA workbook).`	**Docstring** continuation (documentation text).
3	`Prints workbook coverage: which TC-xxx are automated and which are not (with the reason column).`	**Docstring** continuation (documentation text).
5	`    python ci/check_traceability.py # exit 1 if a test is untagged or references an unknown TC`	**Docstring** continuation (documentation text).
6	`    python ci/check_traceability.py --list # also print the uncovered workbook cases`	**Docstring** continuation (documentation text).
7	`"""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
9	`import csv`	**import csv**: loads the module. Reads/writes comma-separated files; used for the workbook export and the traceability report.
10	`import os`	**import os**: loads the module. Operating-system helpers. Used to read environment variables (os.getenv) so CI and local runs share code but take different inputs.
11	`import sys`	**import sys**: loads the module. Interpreter access. sys.exit(code) sets the process exit code that CI uses to decide pass/fail.
12	`from pathlib import Path`	**from pathlib import Path**: imports specific names so they can be used without the module prefix. Object-oriented file paths (Path) that work on every OS. *Path*: Filesystem path object.
14	`import pytest`	**import pytest**: loads the module. The test framework: discovers test_* functions, injects fixtures, applies markers, reports results.
16	`ROOT = Path(__file__).resolve().parent.parent`	Module-level **constant** *ROOT* (UPPER_CASE by convention = not changed at runtime). *resolve()* absolute path. *Path(...)* creates a Path object (constructor call).
19	`class _Grab:`	**class _Grab**: defines a new class (a blueprint for objects).
20	`    def __init__(self):`	**def __init__**: the constructor, run when the object is created. In a Page Object it receives the Playwright Page and declares every locator once.
21	`        self.items = []`	Stores the instance attribute *self.items* on the object (available to every method). The value is a **list** literal [...].
23	`    def pytest_collection_finish(self, session):`	**def pytest_collection_finish**: a pytest **hook** - pytest calls it automatically at that stage of the run. Parameters: *session*.
24	`        self.items = session.items`	Stores the instance attribute *self.items* on the object (available to every method).
27	`def main() -> int:`	**def** defines the function *main*. Returns int.
28	`    os.chdir(ROOT)`	**Method call** executed for its effect. *chdir()* changes the working directory.
29	`    grab = _Grab()`	**Assignment**: binds the name *grab* to the value on the right.
30	`    rc = pytest.main(`	**Assignment**: binds the name *rc* to the value on the right. *main()*: project function main() in ci/check_traceability.py.
31	`        ["--collect-only", "-q", "-p", "no:cacheprovider", "-o", "addopts="],`	**Continuation** of the statement started on line 30 (inside brackets). Keyword argument(s) addopts (name=value): self-documenting and order-independent.
32	`        plugins=[grab],`	**Continuation** of the statement started on line 30 (inside brackets). Keyword argument(s) plugins (name=value): self-documenting and order-independent.

Line	Code	Syntax & why
33	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
34	`    if rc != 0:`	**if** statement: the indented block runs only when *rc != 0* is true.
35	`        print("collection failed")`	**Method call** executed for its effect. *print()* writes to the console / CI log.
36	`        return 1`	**return**: ends the function and hands back *1*.
38	`    with open(ROOT / "docs" / "test_cases.csv", newline="") as f:`	**with** statement (context manager): sets up a resource and guarantees it is closed/cleaned up when the block ends, even on error - *open(ROOT / 'docs' / 'test_cases.csv', newline='')*. *open()* opens a file (read/write).
39	`        cases = {r["Test Case ID"]: r for r in csv.DictReader(f)}`	**Assignment**: binds the name *cases* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *DictReader()* reads CSV rows as dictionaries keyed by the header.
41	`    untagged, unknown, covered = [], [], set()`	**Assignment**: binds the name *(untagged, unknown, covered)* to the value on the right. Tuple unpacking / tuple value. *set()* builds a set (unique values).
42	`    for item in grab.items:`	**for** loop: repeats the indented block for every item, binding it to *item* (iterating *grab.items*).
43	`        ids = [a for m in item.iter_markers("tc") for a in m.args]`	**Assignment**: binds the name *ids* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *iter_markers()* iterates every marker with that name on a test.
44	`        if not ids:`	**if** statement: the indented block runs only when *not ids* is true.
45	`            untagged.append(item.nodeid)`	**Method call** executed for its effect. *append()* adds an item to a list.
46	`        for tc in ids:`	**for** loop: repeats the indented block for every item, binding it to *tc* (iterating *ids*).
47	`            (covered.add(tc) if tc in cases else unknown.append(f"{item.nodeid} -> {tc}"))`	**Expr** statement. *add()* adds an item to a set. *append()* adds an item to a list. Uses an **f-string** (f"...{expr}...") to build text with values.
49	`    missing = sorted(set(cases) - covered)`	**Assignment**: binds the name *missing* to the value on the right. *sorted()* returns a sorted list. *set()* builds a set (unique values).
50	`    print(`	**Method call** executed for its effect. *print()* writes to the console / CI log.
51	`        f"tests={len(grab.items)} workbook_cases={len(cases)} automated={len(covered)} "`	**Continuation** of the statement started on line 50 (inside brackets). Keyword argument(s) tests, workbook_cases, automated (name=value): self-documenting and order-independent. *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
52	`        f"coverage={len(covered) / len(cases) * 100:.1f}%"`	**Continuation** of the statement started on line 50 (inside brackets). Keyword argument(s) coverage (name=value): self-documenting and order-independent. *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
53	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
54	`    if missing:`	**if** statement: the indented block runs only when *missing* is true.
55	`        print("Not automated (see README 'Not automated' for why):")`	**Method call** executed for its effect. *print()* writes to the console / CI log.
56	`        for tc in missing:`	**for** loop: repeats the indented block for every item, binding it to *tc* (iterating *missing*).
57	`            c = cases[tc]`	**Assignment**: binds the name *c* to the value on the right.
58	`            print(f" {tc} [{c['Type']}] {c['Title']} - automation candidate: {c['Automation Candidate']}")`	**Method call** executed for its effect. *print()* writes to the console / CI log. Uses an **f-string** (f"...{expr}...") to build text with values.

Line	Code	Syntax & why
59	`    for n in untagged:`	**for** loop: repeats the indented block for every item, binding it to *n* (iterating *untagged*).
60	`        print(f"UNTAGGED TEST: {n}")`	**Method call** executed for its effect. *print()* writes to the console / CI log. Uses an **f-string** (f"...{expr}...") to build text with values.
61	`    for n in unknown:`	**for** loop: repeats the indented block for every item, binding it to *n* (iterating *unknown*).
62	`        print(f"UNKNOWN TC ID: {n}")`	**Method call** executed for its effect. *print()* writes to the console / CI log. Uses an **f-string** (f"...{expr}...") to build text with values.
64	`    summary = os.getenv("GITHUB_STEP_SUMMARY")`	**Assignment**: binds the name *summary* to the value on the right. *getenv()* reads an environment variable with a default.
65	`    if summary:`	**if** statement: the indented block runs only when *summary* is true.
66	`        with open(summary, "a") as f:`	**with** statement (context manager): sets up a resource and guarantees it is closed/cleaned up when the block ends, even on error - *open(summary, 'a')*. *open()* opens a file (read/write).
67	`            f.write(`	**Method call** executed for its effect. *write()* writes text to a file.
68	`                f"### Workbook coverage\n\n{len(covered)}/{len(cases)} test cases automated "`	**Continuation** of the statement started on line 67 (inside brackets). *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
69	`                f"({len(covered) / len(cases) * 100:.1f}%) by {len(grab.items)} tests\n\n"`	**Continuation** of the statement started on line 67 (inside brackets). *len()* length of a collection. Uses an **f-string** (f"...{expr}...") to build text with values.
70	`            )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
71	`    return 1 if (untagged or unknown) else 0`	**return**: ends the function and hands back *1 if untagged or unknown else 0*.
74	`if __name__ == "__main__":`	**if __name__ == '__main__'**: true only when the file is executed directly (python file.py), not when imported.
75	`    sys.exit(main())`	**Method call** executed for its effect. *exit()* ends the process with an exit code (non-zero = CI failure). *main()*: project function main() in ci/check_traceability.py.

ci/discover.py

Maintenance tool: visits the live site and prints ARIA snapshots of key UI states. Selectors in the page objects were derived from this output; re-run it after UI changes.

Line	Code	Syntax & why
1	`"""Site discovery: prints ARIA snapshots of key UI states so page-object selectors can be (re)derived`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
2	`after UI changes. Run in CI (where the site is reachable):`	**Docstring** continuation (documentation text).
3	`    python ci/discover.py https://hotel-plaza-preview.vercel.app`	**Docstring** continuation (documentation text).
4	`"""`	**Docstring**: the first string in a module/class/function documents what it does; tools and help() display it.
6	`import sys`	**import sys**: loads the module. Interpreter access. sys.exit(code) sets the process exit code that CI uses to decide pass/fail.

Line	Code	Syntax & why
8	`from playwright.sync_api import sync_playwright`	**from playwright.sync_api import sync_playwright**: imports specific names so they can be used without the module prefix. Playwright's synchronous Python API (Page, Locator, expect). Synchronous code reads top-to-bottom like a manual test script and needs no async/await. *sync_playwright*: Context manager that starts/stops the Playwright driver in standalone scripts.
10	`BASE = (sys.argv[1] if len(sys.argv) > 1 else "https://hotel-plaza-preview.vercel.app").rstrip("/")`	Module-level **constant** *BASE* (UPPER_CASE by convention = not changed at runtime). *len()* length of a collection.
11	`DIALOG = "[role=dialog]:visible"`	Module-level **constant** *DIALOG* (UPPER_CASE by convention = not changed at runtime).
14	`def section(title):`	**def** defines the function *section*. Parameters: *title*.
15	`    print(f"\n{'=' * 20} {title} {'=' * 20}", flush=True)`	**Method call** executed for its effect. *print()* writes to the console / CI log. Uses an **f-string** (f"...{expr}...") to build text with values.
18	`def snap(page, title, locator="main"):`	**def** defines the function *snap*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *title*; *locator*. Parameters with '= value' are optional (default used when omitted).
19	`    section(title)`	**Method call** executed for its effect. *section()*: project function section() in ci/discover.py.
20	`    print("URL:", page.url)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
21	`    try:`	**try**: runs the block and lets the except clause handle expected errors instead of crashing.
22	`        print(page.locator(locator).first.aria_snapshot(timeout=5000))`	**Method call** executed for its effect. *print()* writes to the console / CI log. *aria_snapshot()* returns the accessibility tree of an element as YAML - what a screen reader 'sees'. Used by discovery to derive role-based selectors. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
23	`    except Exception as e: # noqa: BLE001 - discovery must keep going`	**except**: handles the listed error type raised in the try block (here: an expected timeout or missing optional dependency). Inline comment: “noqa: BLE001 - discovery must keep going”.
24	`        print("SNAPSHOT FAILED:", str(e)[:200])`	**Method call** executed for its effect. *print()* writes to the console / CI log. *str()* converts to string.
27	`def safe(fn, label):`	**def** defines the function *safe*. Parameters: *fn*; *label*.
28	`    try:`	**try**: runs the block and lets the except clause handle expected errors instead of crashing.
29	`        return fn()`	**return**: ends the function and hands back *fn()*.
30	`    except Exception as e: # noqa: BLE001`	**except**: handles the listed error type raised in the try block (here: an expected timeout or missing optional dependency). Inline comment: “noqa: BLE001”.
31	`        print(f"!! {label} failed: {str(e)[:200]}")`	**Method call** executed for its effect. *print()* writes to the console / CI log. *str()* converts to string. Uses an **f-string** (f"...{expr}...") to build text with values.
34	`def go(page, path, wait=1500):`	**def** defines the function *go*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test; *path*; *wait*. Parameters with '= value' are optional (default used when omitted).
35	`    page.goto(BASE + path, wait_until="networkidle")`	**Method call** executed for its effect. *goto()* navigates the tab to a URL; a relative path is joined to base_url. Returns the HTTP Response so status codes can be asserted.
36	`    page.wait_for_timeout(wait)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
39	`def sign_in_states(page):`	**def** defines the function *sign_in_states*. Parameters: *page*: pytest-playwright fixture: a fresh browser tab in a fresh, isolated context for this test.
40	`    go(page, "/")`	**Method call** executed for its effect. *go()*: project function go() in ci/discover.py.

Line	Code	Syntax & why
41	`    page.get_by_role("banner").get_by_role("button", name="Sign in").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
42	`    dlg = page.get_by_role("dialog", name="Sign in or create an account")`	**Assignment**: binds the name *dlg* to the value on the right. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
43	`    dlg.get_by_role("button", name="Customer").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
44	`    page.wait_for_timeout(300)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
45	`    snap(page, "SIGN IN customer selected", DIALOG)`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
46	`    dlg.get_by_role("button", name="Next").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
47	`    page.wait_for_timeout(1000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
48	`    snap(page, "SIGN IN form", DIALOG)`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
49	`    section("SIGN IN inputs")`	**Method call** executed for its effect. *section()*: project function section() in ci/discover.py.
50	`    for el in page.locator(f"{DIALOG} input, {DIALOG} button").all():`	**for** loop: repeats the indented block for every item, binding it to *el* (iterating *page.locator(f'{DIALOG} input, {DIALOG} button').all()*). *all()* returns a list of Locators, one per current match, for iteration. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath). Uses an **f-string** (f"...{expr}...") to build text with values.
51	`        safe(`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py.
52	`            lambda el=el: print(el.evaluate('e => e.outerHTML.replace(/class="[^"]*"/, \'\').slice(0, 200)')),`	**Continuation** of the statement started on line 51 (inside brackets). Keyword argument(s) el, class (name=value): self-documenting and order-independent. *print()* writes to the console / CI log. *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry). **lambda**: a small anonymous function written inline.
53	`            "h",`	**Continuation** of the statement started on line 51 (inside brackets).
54	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.

Line	Code	Syntax & why
55	`    buttons = [b for b in dlg.get_by_role("button").all_inner_texts()]`	**Assignment**: binds the name *buttons* to the value on the right. The value is a **comprehension** - builds a collection in one expression (for ... in ... if ...). *all_inner_texts()* returns visible text of every match as a list of strings. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
56	`    print("BUTTONS:", buttons)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
57	`    submit = dlg.locator("button[type=submit]")`	**Assignment**: binds the name *submit* to the value on the right. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
58	`    safe(lambda: submit.click(), "empty submit")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
59	`    page.wait_for_timeout(800)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
60	`    snap(page, "SIGN IN empty submit", DIALOG)`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
61	`    safe(lambda: page.locator("input[name=identifier]").fill("abc"), "id")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *fill()* clears an input and types the value in one step (also works for date and range inputs). *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
62	`    safe(lambda: page.locator("input[name=password]").fill("wrongpassword1"), "pw")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *fill()* clears an input and types the value in one step (also works for date and range inputs). *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
63	`    reqs = []`	**Assignment**: binds the name *reqs* to the value on the right. The value is a **list** literal [...].
64	`    page.on("request", lambda r: reqs.append(f"{r.method} {r.url}") if "/api/auth" in r.url else None)`	**Method call** executed for its effect. *on()* subscribes to a page event (console, request, response, dialog, pageerror); the callback collects evidence while the test runs. *append()* adds an item to a list. **lambda**: inline anonymous function (used as a callback/predicate). Uses an **f-string** (f"...{expr}...") to build text with values.
65	`    safe(lambda: submit.click(), "abc submit")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
66	`    page.wait_for_timeout(2000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
67	`    print("AUTH REQUESTS (identifier=abc):", reqs)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
68	`    snap(page, "SIGN IN identifier abc", DIALOG)`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
69	`    safe(lambda: page.locator("input[name=identifier]").fill("nobody-qa-0001@maildrop.cc"), "id2")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *fill()* clears an input and types the value in one step (also works for date and range inputs). *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
70	`    safe(lambda: submit.click(), "bad creds submit")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered.
71	`    page.wait_for_timeout(2500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
72	`    snap(page, "SIGN IN bad creds", DIALOG)`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.

Line	Code	Syntax & why
73	`    safe(lambda: dlg.get_by_role("button", name="Forgot password?").click(), "forgot")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
74	`    page.wait_for_timeout(800)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
75	`    snap(page, "FORGOT password", DIALOG)`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
76	`    page.keyboard.press("Escape")`	**Method call** executed for its effect. *press()* presses a key (Enter, Escape, Tab) on the element.
77	`    page.wait_for_timeout(500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
78	`    print(`	**Method call** executed for its effect. *print()* writes to the console / CI log.
79	`        "focused after Esc:",`	**Continuation** of the statement started on line 78 (inside brackets).
80	`        page.evaluate("document.activeElement && document.activeElement.outerHTML.slice(0,120)"),`	**Continuation** of the statement started on line 78 (inside brackets). *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry).
81	`    )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
82	`    # role is remembered: reopen`	**Comment** (# ... ignored by Python): documents intent for the reader.
83	`    page.get_by_role("banner").get_by_role("button", name="Sign in").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
84	`    page.wait_for_timeout(800)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
85	`    snap(page, "SIGN IN reopened", DIALOG)`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
88	`def main():`	**def** defines the function *main*.
89	`    with sync_playwright() as p:`	**with** statement (context manager): sets up a resource and guarantees it is closed/cleaned up when the block ends, even on error - *sync_playwright()*.
90	`        browser = p.chromium.launch()`	**Assignment**: binds the name *browser* to the value on the right. *launch()* starts a browser process.
91	`        ctx = browser.new_context(viewport={"width": 1440, "height": 900}, timezone_id="Africa/Lagos")`	**Assignment**: binds the name *ctx* to the value on the right. *new_context()* creates an isolated browser profile (own cookies/storage). Isolation means tests cannot affect each other.
92	`        ctx.set_default_timeout(6000)`	**Method call** executed for its effect. *set_default_timeout()* sets how long actions (click/fill) wait for elements before failing.
93	`        page = ctx.new_page()`	**Assignment**: binds the name *page* to the value on the right. *new_page()* opens a new tab in a context.
94	`        safe(lambda: sign_in_states(page), "sign in states")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *sign_in_states()*: project function sign_in_states() in ci/discover.py.
96	`        go(page, "/gallery")`	**Method call** executed for its effect. *go()*: project function go() in ci/discover.py.

Line	Code	Syntax & why
97	`safe(lambda: page.get_by_role("main").get_by_role("img", name="Suite bathroom").click(), "img click")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
98	`page.wait_for_timeout(1000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
99	`snap(page, "GALLERY after img click", "body")`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
101	`# review step edit links, branch room`	**Comment** (# ... ignored by Python): documents intent for the reader.
102	`go(`	**Method call** executed for its effect. *go()*: project function go() in ci/discover.py.
103	`page,`	**Continuation** of the statement started on line 102 (inside brackets).
104	`"/rooms?branchId=a877424b-2eac-4320-87f9-e6114b8a5fa6&adults=1&checkIn=2026-12-01&checkOut=2026-12-03",`	**Continuation** of the statement started on line 102 (inside brackets). Keyword argument(s) branchId, adults, checkIn, checkOut (name=value): self-documenting and order-independent.
105	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
106	`page.get_by_role("link", name="Select room").first.click()`	**Method call** executed for its effect. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
107	`page.wait_for_timeout(2500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
108	`print("URL after Select room:", page.url)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
109	`page.get_by_role("button", name="Continue to guest details").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
110	`page.wait_for_timeout(2000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
111	`page.get_by_role("textbox", name="John Doe").fill("QA Tester")`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *fill()* clears an input and types the value in one step (also works for date and range inputs). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
112	`page.get_by_role("textbox", name="alex.morgan@example.com").fill("qa@maildrop.cc")`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *fill()* clears an input and types the value in one step (also works for date and range inputs). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
113	`page.get_by_role("textbox", name="+234 800 000 0000").fill("+2348012345678")`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *fill()* clears an input and types the value in one step (also works for date and range inputs). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
114	`page.get_by_role("checkbox").check()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *check()* ticks a checkbox (no-op if already ticked). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
115	`page.get_by_role("button", name="Review my booking").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
116	`page.wait_for_timeout(2500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
117	`snap(page, "REVIEW branch room")`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
118	`page.get_by_role("button", name="Back to guest details").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
119	`page.wait_for_timeout(1000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
120	`print(`	**Method call** executed for its effect. *print()* writes to the console / CI log.
121	`"after Back values:",`	**Continuation** of the statement started on line 120 (inside brackets).
122	`[i.input_value() for i in page.locator("main input:not([type=checkbox])").all()],`	**Continuation** of the statement started on line 120 (inside brackets). Keyword argument(s) type (name=value): self-documenting and order-independent. *input_value()* current value of an input/select. *all()* returns a list of Locators, one per current match, for iteration. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
123	`page.get_by_role("checkbox").is_checked(),`	**Continuation** of the statement started on line 120 (inside brackets). *is_checked()* checkbox state right now. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
124	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
125	`page.get_by_role("checkbox").check()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *check()* ticks a checkbox (no-op if already ticked). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.

Line	Code	Syntax & why
126	`page.get_by_role("button", name="Review my booking").click()`	**Method call** executed for its effect (calls are **chained**: each returns an object the next call acts on). *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
127	`page.wait_for_timeout(2000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
128	`safe(lambda: page.get_by_role("button", name="Change room or dates").click(), "change")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
129	`page.wait_for_timeout(2000)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
130	`print("URL after Change room or dates:", page.url)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
131	`page.go_back()`	**Method call** executed for its effect. *go_back()* browser Back button.
132	`page.wait_for_timeout(1500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
133	`print("URL after browser back:", page.url)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
135	`# 5-night quote and 90+ night limits on the room detail`	**Comment** (# ... ignored by Python): documents intent for the reader.
136	`go(page, "/rooms/965c310d-7b91-46b3-b2aa-3053c19ec203?checkIn=2026-12-01&checkOut=2026-12-06")`	**Method call** executed for its effect. *go()*: project function go() in ci/discover.py.
137	`safe(`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py.
138	`lambda: print(`	**Continuation** of the statement started on line 137 (inside brackets). *print()* writes to the console / CI log. **lambda**: a small anonymous function written inline.
139	`"5 nights:",`	**Continuation** of the statement started on line 137 (inside brackets).
140	`page.get_by_role("main")`	**Continuation** of the statement started on line 137 (inside brackets). *evaluate()* runs JavaScript inside the page (or on an element) and returns the result to Python - used for measurements Playwright has no API for (performance timings, computed styles, geometry). *get_by_text()* finds elements by their visible text (exact string or regex). *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
141	`.get_by_text("Total")`	**Continuation** of the statement started on line 137 (inside brackets).
142	`.first.evaluate("e => e.parentElement.parentElement.innerText"),`	**Continuation** of the statement started on line 137 (inside brackets).
143	`),`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
144	`"5n",`	**Continuation** of the statement started on line 137 (inside brackets).
145	`)`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
146	`safe(`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py.

Line	Code	Syntax & why
147	`            lambda: print("checkout max:", page.locator("main input[type=date]").nth(1).get_attribute("max")),`	**Continuation** of the statement started on line 146 (inside brackets). Keyword argument(s) type (name=value): self-documenting and order-independent. *print()* writes to the console / CI log. *get_attribute()* reads an HTML attribute (href, min, max, alt, type). *nth()* the n-th (0-based) matching element. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath). **lambda**: a small anonymous function written inline.
148	`            "max",`	**Continuation** of the statement started on line 146 (inside brackets).
149	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
150	`        go(page, "/rooms/965c310d-7b91-46b3-b2aa-3053c19ec203?checkIn=2020-01-01&checkOut=2020-01-03")`	**Method call** executed for its effect. *go()*: project function go() in ci/discover.py.
151	`        safe(`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py.
152	`            lambda: print(`	**Continuation** of the statement started on line 151 (inside brackets). *print()* writes to the console / CI log. **lambda**: a small anonymous function written inline.
153	`                "past dates values:", [i.input_value() for i in page.locator("main input[type=date]").all()]`	**Continuation** of the statement started on line 151 (inside brackets). Keyword argument(s) type (name=value): self-documenting and order-independent. *input_value()* current value of an input/select. *all()* returns a list of Locators, one per current match, for iteration. *locator()* finds elements with a CSS or XPath selector - used only where no role/text exists (e.g. input[type=date], ancestor:: XPath).
154	`            ),`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
155	`            "past",`	**Continuation** of the statement started on line 151 (inside brackets).
156	`        )`	Closing bracket of the multi-line expression that starts above. Python allows line breaks inside (), [] and {} without a backslash; ruff format applies this layout.
157	`        safe(lambda: page.get_by_role("button", name="Continue to guest details").click(), "continue past")`	**Method call** executed for its effect. *safe()*: project function safe() in ci/discover.py. *click()* clicks the element. Playwright auto-waits for it to be visible, stable, enabled and not covered. *get_by_role()* finds elements by ARIA role (button, link, heading, textbox...) and accessible name. This mirrors how users and assistive technology perceive the page, so it survives CSS/markup refactors - the most robust selector Playwright offers.
158	`        page.wait_for_timeout(1500)`	**Method call** executed for its effect. *wait_for_timeout()* fixed pause in milliseconds. Used only inside bounded polling loops or to let a debounce settle - never as the main synchronisation.
159	`        print("URL after continue with past dates:", page.url)`	**Method call** executed for its effect. *print()* writes to the console / CI log.
160	`        snap(page, "DETAIL past dates")`	**Method call** executed for its effect. *snap()*: project function snap() in ci/discover.py.
161	`        browser.close()`	**Method call** executed for its effect. *close()* closes a page/context/browser and frees resources.
164	`if __name__ == "__main__":`	**if __name__ == '__main__'**: true only when the file is executed directly (python file.py), not when imported.
165	`    main()`	**Method call** executed for its effect. *main()*: project function main() in ci/check_traceability.py.

.github/workflows/ci.yml

The end-to-end CI/CD pipeline: guard -> lint (gate 1) -> smoke (gate 2) -> regression on 3 browsers (gate 3) + non-functional (gate 4) -> consolidated report -> quality-gate verdict (the one required status check). Triggered by PRs, pushes, nightly schedule, manual dispatch and every successful deployment.

Line	Code	Syntax & why
1	`name: Oneflare QA Pipeline`	**name:** Display name of the workflow/step/job in the GitHub UI. Value: *Oneflare QA Pipeline*.
3	`# End-to-end pipeline:`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
4	`# lint -> smoke -> (regression x3 browsers | non-functional | api/security) -> report -> quality-gate`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
5	`# Runs on every PR, on pushes to main and work branches, nightly, on demand, and automatically after a`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
6	`# successful (Vercel) deployment so every preview build is tested at its own URL.`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
7	`on:`	**on:** Events that start the workflow.
8	`  pull_request:`	**pull_request:** Run for every pull request (the merge gate).
9	`  push:`	**push:** Run on pushes to the listed branches.
10	`    branches: [main, "cl/**", "feature/**", "fix/**"]`	**branches:** Branch filter; glob patterns like cl/** match any sub-branch. Value: *[main, "cl/**", "feature/**", "fix/**"]*.
11	`  schedule:`	**schedule:** Time-based trigger.
12	`    - cron: "0 5 * * *" # nightly 05:00 UTC (06:00 Lagos) full run`	**cron:** Cron expression (minute hour day month weekday, UTC). Value: *"0 5 * * *" # nightly 05:00 UTC (06:00 Lagos) full run*.
13	`  deployment_status: # Vercel GitHub integration posts this after each deploy`	**deployment_status:** Fired when a deployment (e.g. Vercel) reports status - tests every new preview at its own URL. Value: *# Vercel GitHub integration posts this after each deploy*.
14	`  workflow_dispatch:`	**workflow_dispatch:** Allows a manual run from the Actions tab with inputs.
15	`    inputs:`	**inputs:** Parameters a person can set for a manual run.
16	`      base_url:`	**base_url:** Named block *base_url* (a job, input or section); the indented keys below configure it.
17	`        description: "Environment under test"`	**description:** Help text shown next to the input. Value: *"Environment under test"*.
18	`        default: "https://hotel-plaza-preview.vercel.app"`	**default:** Value used when the input is left unchanged. Value: *"https://hotel-plaza-preview.vercel.app"*.
19	`      known_defects:`	**known_defects:** Named block *known_defects* (a job, input or section); the indented keys below configure it.
20	`        description: "xfail = track open bugs as xfail; fail = retest run, open bugs fail the build"`	**description:** Help text shown next to the input. Value: *"xfail = track open bugs as xfail; fail = retest run, open bugs fail the build"*.
21	`        type: choice`	**type:** Input type (choice = dropdown, boolean = checkbox). Value: *choice*.
22	`        options: [xfail, fail]`	**options:** Allowed values for a choice input. Value: *[xfail, fail]*.
23	`        default: xfail`	**default:** Value used when the input is left unchanged. Value: *xfail*.
24	`      allow_live_writes:`	**allow_live_writes:** Named block *allow_live_writes* (a job, input or section); the indented keys below configure it.
25	`        description: "Run tests that create real accounts/bookings/messages (workbook 'Blocked' cases)"`	**description:** Help text shown next to the input. Value: *"Run tests that create real accounts/bookings/messages (workbook 'Blocked' cases)"*.
26	`        type: boolean`	**type:** Input type (choice = dropdown, boolean = checkbox). Value: *boolean*.
27	`        default: false`	**default:** Value used when the input is left unchanged. Value: *false*.
29	`concurrency: # a new push cancels the older run for the same ref`	**concurrency:** Prevents overlapping runs for the same ref. Value: *# a new push cancels the older run for the same ref*.

Line	Code	Syntax & why
30	`  group: ci-${{ github.workflow }}-${{ github.event.deployment.id || github.ref }}`	**group:** Runs with the same group key are serialised. Value: *ci-${{ github.workflow }}-${{ github.event.deployment.id || github.ref }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
31	`  cancel-in-progress: true`	**cancel-in-progress:** A new push cancels the older run - saves runner minutes and gives faster feedback. Value: *true*.
33	`permissions:`	**permissions:** GITHUB_TOKEN scopes; read-only = least privilege.
34	`  contents: read # least privilege`	**contents:** Repository contents permission. Value: *read # least privilege*.
36	`env:`	**env:** Environment variables visible to every job/step below this level.
37	`  # deployment_status -> the URL that was just deployed; manual -> input; otherwise the preview site`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
38	`  BASE_URL: ${{ github.event.deployment_status.environment_url || github.event.deployment_status.target_url || inputs.base_url || vars.BASE_URL || 'https://hotel-plaza-preview.vercel.app' }}`	**BASE_URL:** Environment variable *BASE_URL* made available to the commands. Value: *${{ github.event.deployment_status.environment_url \|\| github.event.deployment_status.target_url \|\| inputs.base_url \|\| vars.BASE_URL \|\| 'https://hotel-plaza-preview.vercel.app' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
39	`  KNOWN_DEFECTS: ${{ inputs.known_defects || 'xfail' }}`	**KNOWN_DEFECTS:** Environment variable *KNOWN_DEFECTS* made available to the commands. Value: *${{ inputs.known_defects \|\| 'xfail' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
40	`  ALLOW_LIVE_WRITES: ${{ inputs.allow_live_writes && 'true' || 'false' }}`	**ALLOW_LIVE_WRITES:** Environment variable *ALLOW_LIVE_WRITES* made available to the commands. Value: *${{ inputs.allow_live_writes && 'true' \|\| 'false' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
41	`  TEST_EMAIL: ${{ secrets.TEST_EMAIL }}`	**TEST_EMAIL:** Environment variable *TEST_EMAIL* made available to the commands. Value: *${{ secrets.TEST_EMAIL }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
42	`  TEST_PASSWORD: ${{ secrets.TEST_PASSWORD }}`	**TEST_PASSWORD:** Environment variable *TEST_PASSWORD* made available to the commands. Value: *${{ secrets.TEST_PASSWORD }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
43	`  TEST_PHONE: ${{ vars.TEST_PHONE || '+2348012345678' }}`	**TEST_PHONE:** Environment variable *TEST_PHONE* made available to the commands. Value: *${{ vars.TEST_PHONE \|\| '+2348012345678' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
44	`  PERF_TTFB_MS: ${{ vars.PERF_TTFB_MS || '800' }}`	**PERF_TTFB_MS:** Environment variable *PERF_TTFB_MS* made available to the commands. Value: *${{ vars.PERF_TTFB_MS \|\| '800' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
45	`  PERF_DCL_MS: ${{ vars.PERF_DCL_MS || '3000' }}`	**PERF_DCL_MS:** Environment variable *PERF_DCL_MS* made available to the commands. Value: *${{ vars.PERF_DCL_MS \|\| '3000' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
46	`  PERF_LOAD_MS: ${{ vars.PERF_LOAD_MS || '5000' }}`	**PERF_LOAD_MS:** Environment variable *PERF_LOAD_MS* made available to the commands. Value: *${{ vars.PERF_LOAD_MS \|\| '5000' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).

Line	Code	Syntax & why
47	`  PERF_LCP_MS: ${{ vars.PERF_LCP_MS || '4000' }}`	**PERF_LCP_MS:** Environment variable *PERF_LCP_MS* made available to the commands. Value: *${{ vars.PERF_LCP_MS \|\| '4000' }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
48	`  PYTHON_VERSION: "3.12"`	**PYTHON_VERSION:** Environment variable *PYTHON_VERSION* made available to the commands. Value: *"3.12"*.
49	`  PYTEST_COMMON: --reruns 1 --reruns-delay 2`	**PYTEST_COMMON:** Environment variable *PYTEST_COMMON* made available to the commands. Value: *--reruns 1 --reruns-delay 2*.
51	`jobs:`	**jobs:** The pipeline stages; jobs run in parallel unless linked with needs.
52	`  # ---------- skip deployments that did not succeed ----------`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
53	`  guard:`	**guard:** Named block *guard* (a job, input or section); the indented keys below configure it.
54	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
55	`    if: github.event_name != 'deployment_status' || github.event.deployment_status.state == 'success'`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *github.event_name != 'deployment_status' || github.event.deployment_status.state == 'success'*.
56	`    steps:`	**steps:** Ordered commands of the job.
57	`      - run: echo "Testing $BASE_URL (event=${{ github.event_name }}, known_defects=$KNOWN_DEFECTS)"`	**Step running a shell command**: prints a message to the log. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
59	`  # ---------- GATE 1: static checks (seconds, no browser) ----------`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
60	`  lint:`	**lint:** Named block *lint* (a job, input or section); the indented keys below configure it.
61	`    needs: guard`	**needs:** Job dependencies: this job waits for, and only runs after success of, the listed jobs (gates). Value: *guard*.
62	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
63	`    timeout-minutes: 10`	**timeout-minutes:** Hard limit so a hung browser cannot block the pipeline. Value: *10*.
64	`    steps:`	**steps:** Ordered commands of the job.
65	`      - uses: actions/checkout@v4`	**Step using an action** (actions/checkout@v4): clones the repository onto the runner. @vN pins a major version for reproducible builds.
66	`      - uses: actions/setup-python@v5`	**Step using an action** (actions/setup-python@v5): installs the requested Python version (with pip cache). @vN pins a major version for reproducible builds.
67	`        with: { python-version: "${{ env.PYTHON_VERSION }}", cache: pip }`	**with:** Inputs for the action. Value: *{ python-version: "${{ env.PYTHON_VERSION }}", cache: pip }*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
68	`      - run: pip install -r requirements.txt`	**Step running a shell command**: installs Python dependencies listed in requirements.txt.
69	`      - run: ruff check . # lint errors fail the build`	**Step running a shell command**: lint: fails on code errors and import-order problems.
70	`      - run: ruff format --check . # formatting drift fails the build`	**Step running a shell command**: fails if any file is not formatted (prevents style drift).
71	`      - name: Test collection succeeds and every test is linked to a workbook TC`	**Step** (list item '-'): its display name in the run log.
72	`        run: |`	**Step running a shell command**: shell command.
73	`          pytest --collect-only -q --base-url "$BASE_URL" -p no:cacheprovider`	**Shell line** of a multi-line run block: collects tests without running them - catches import/syntax/marker errors in seconds. Flags: --base-url sets the environment under test.

Line	Code	Syntax & why
74	`          python ci/check_traceability.py`	**Shell line** of a multi-line run block: verifies every test is linked to a valid workbook TC and prints coverage.
76	`  # ---------- GATE 2: smoke - 100% must pass ----------`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
77	`  smoke:`	**smoke:** Named block *smoke* (a job, input or section); the indented keys below configure it.
78	`    needs: lint`	**needs:** Job dependencies: this job waits for, and only runs after success of, the listed jobs (gates). Value: *lint*.
79	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
80	`    timeout-minutes: 20`	**timeout-minutes:** Hard limit so a hung browser cannot block the pipeline. Value: *20*.
81	`    steps:`	**steps:** Ordered commands of the job.
82	`      - uses: actions/checkout@v4`	**Step using an action** (actions/checkout@v4): clones the repository onto the runner. @vN pins a major version for reproducible builds.
83	`      - uses: actions/setup-python@v5`	**Step using an action** (actions/setup-python@v5): installs the requested Python version (with pip cache). @vN pins a major version for reproducible builds.
84	`        with: { python-version: "${{ env.PYTHON_VERSION }}", cache: pip }`	**with:** Inputs for the action. Value: *{ python-version: "${{ env.PYTHON_VERSION }}", cache: pip }*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
85	`      - run: pip install -r requirements.txt`	**Step running a shell command**: installs Python dependencies listed in requirements.txt.
86	`      - run: playwright install --with-deps chromium`	**Step running a shell command**: downloads the browser build (--with-deps adds the OS libraries it needs).
87	`      - name: Run smoke suite`	**Step** (list item '-'): its display name in the run log.
88	`        run: pytest -m smoke --base-url "$BASE_URL" $PYTEST_COMMON -n 2 --known-defects "$KNOWN_DEFECTS"`	**Step running a shell command**: runs the tests. Flags: -m smoke selects tests marked smoke; -n N runs N parallel workers (pytest-xdist); --base-url sets the environment under test; --known-defects xfail|fail decides how open bugs are reported.
89	`      - name: Enforce gate (100% pass, at least 8 tests)`	**Step** (list item '-'): its display name in the run log.
90	`        if: always()`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always()*.
91	`        run: python ci/quality_gate.py reports/junit.xml --name Smoke --min-pass-rate 100 --min-tests 8`	**Step running a shell command**: applies the gate thresholds to junit.xml; non-zero exit fails the job. Flags: --min-pass-rate is the gate threshold; --min-tests guards against an empty or mis-collected suite.
92	`      - uses: actions/upload-artifact@v4`	**Step using an action** (actions/upload-artifact@v4): stores reports/ (HTML, JUnit, traces, videos) on the run page for download. @vN pins a major version for reproducible builds.
93	`        if: always()`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always()*.
94	`        with: { name: smoke-reports, path: reports/, retention-days: 14 }`	**with:** Inputs for the action. Value: *{ name: smoke-reports, path: reports/, retention-days: 14 }*.
96	`  # ---------- GATE 3: functional regression across 3 browsers (>=95%) ----------`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
97	`  regression:`	**regression:** Named block *regression* (a job, input or section); the indented keys below configure it.
98	`    needs: smoke`	**needs:** Job dependencies: this job waits for, and only runs after success of, the listed jobs (gates). Value: *smoke*.

Line	Code	Syntax & why
99	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
100	`    timeout-minutes: 40`	**timeout-minutes:** Hard limit so a hung browser cannot block the pipeline. Value: *40*.
101	`    strategy:`	**strategy:** Matrix configuration.
102	`      fail-fast: false # let every browser finish so you see all failures`	**fail-fast:** false = let every matrix job finish so all browser failures are visible. Value: *false # let every browser finish so you see all failures*.
103	`      matrix:`	**matrix:** Creates one job per value (here: per browser).
104	`        browser: [chromium, firefox, webkit]`	**browser:** Matrix variable: chromium, firefox, webkit. Value: *[chromium, firefox, webkit]*.
105	`    steps:`	**steps:** Ordered commands of the job.
106	`      - uses: actions/checkout@v4`	**Step using an action** (actions/checkout@v4): clones the repository onto the runner. @vN pins a major version for reproducible builds.
107	`      - uses: actions/setup-python@v5`	**Step using an action** (actions/setup-python@v5): installs the requested Python version (with pip cache). @vN pins a major version for reproducible builds.
108	`        with: { python-version: "${{ env.PYTHON_VERSION }}", cache: pip }`	**with:** Inputs for the action. Value: *{ python-version: "${{ env.PYTHON_VERSION }}", cache: pip }*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
109	`      - run: pip install -r requirements.txt`	**Step running a shell command**: installs Python dependencies listed in requirements.txt.
110	`      - run: playwright install --with-deps ${{ matrix.browser }}`	**Step running a shell command**: downloads the browser build (--with-deps adds the OS libraries it needs). **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
111	`      - name: Run functional suite`	**Step** (list item '-'): its display name in the run log.
112	`        continue-on-error: true # the >=95% gate below decides, not pytest's exit code`	**continue-on-error:** Key *continue-on-error*. Value: *true # the >=95% gate below decides, not pytest's exit code*.
113	`        run: >`	**Step running a shell command**: shell command.
114	`          pytest tests/functional --browser ${{ matrix.browser }} --base-url "$BASE_URL"`	**Shell line** of a multi-line run block: runs the tests. Flags: --browser picks chromium/firefox/webkit; --base-url sets the environment under test. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
115	`          $PYTEST_COMMON -n 4 --known-defects "$KNOWN_DEFECTS"`	**Shell line** of a multi-line run block: shell command. Flags: -n N runs N parallel workers (pytest-xdist); --known-defects xfail|fail decides how open bugs are reported.
116	`      - name: Enforce gate (>=95% pass)`	**Step** (list item '-'): its display name in the run log.
117	`        if: always()`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always()*.
118	`        run: >`	**Step running a shell command**: shell command.
119	`          python ci/quality_gate.py reports/junit.xml --name "Regression (${{ matrix.browser }})"`	**Shell line** of a multi-line run block: applies the gate thresholds to junit.xml; non-zero exit fails the job. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
120	`          --min-pass-rate 95 --min-tests 60 --max-duration 1500`	**Shell line** of a multi-line run block: shell command. Flags: --min-pass-rate is the gate threshold; --min-tests guards against an empty or mis-collected suite; --max-duration caps total runtime.
121	`      - uses: actions/upload-artifact@v4`	**Step using an action** (actions/upload-artifact@v4): stores reports/ (HTML, JUnit, traces, videos) on the run page for download. @vN pins a major version for reproducible builds.

Line	Code	Syntax & why
122	`        if: always()`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always()*.
123	`        with:`	**with:** Inputs for the action.
124	`          name: regression-${{ matrix.browser }}`	**name:** Display name of the workflow/step/job in the GitHub UI. Value: *regression-${{ matrix.browser }}*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
125	`          path: reports/`	**path:** Files/folder to upload or download. Value: *reports/*.
126	`          retention-days: 14`	**retention-days:** How long GitHub keeps the artifact. Value: *14*.
128	`  # ---------- GATE 4: non-functional (perf budgets, axe a11y, responsive, security, SEO) ----------`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
129	`  non-functional:`	**non-functional:** Named block *non-functional* (a job, input or section); the indented keys below configure it.
130	`    needs: smoke`	**needs:** Job dependencies: this job waits for, and only runs after success of, the listed jobs (gates). Value: *smoke*.
131	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
132	`    timeout-minutes: 30`	**timeout-minutes:** Hard limit so a hung browser cannot block the pipeline. Value: *30*.
133	`    steps:`	**steps:** Ordered commands of the job.
134	`      - uses: actions/checkout@v4`	**Step using an action** (actions/checkout@v4): clones the repository onto the runner. @vN pins a major version for reproducible builds.
135	`      - uses: actions/setup-python@v5`	**Step using an action** (actions/setup-python@v5): installs the requested Python version (with pip cache). @vN pins a major version for reproducible builds.
136	`        with: { python-version: "${{ env.PYTHON_VERSION }}", cache: pip }`	**with:** Inputs for the action. Value: *{ python-version: "${{ env.PYTHON_VERSION }}", cache: pip }*. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
137	`      - run: pip install -r requirements.txt`	**Step running a shell command**: installs Python dependencies listed in requirements.txt.
138	`      - run: playwright install --with-deps chromium`	**Step running a shell command**: downloads the browser build (--with-deps adds the OS libraries it needs).
139	`      - name: Run non-functional suite`	**Step** (list item '-'): its display name in the run log.
140	`        run: pytest tests/non_functional --base-url "$BASE_URL" $PYTEST_COMMON -n 2 --known-defects "$KNOWN_DEFECTS"`	**Step running a shell command**: runs the tests. Flags: -n N runs N parallel workers (pytest-xdist); --base-url sets the environment under test; --known-defects xfail|fail decides how open bugs are reported.
141	`      - name: Enforce gate (100% of enforced checks)`	**Step** (list item '-'): its display name in the run log.
142	`        if: always()`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always()*.
143	`        run: python ci/quality_gate.py reports/junit.xml --name Non-functional --min-pass-rate 100 --min-tests 40`	**Step running a shell command**: applies the gate thresholds to junit.xml; non-zero exit fails the job. Flags: --min-pass-rate is the gate threshold; --min-tests guards against an empty or mis-collected suite.
144	`      - uses: actions/upload-artifact@v4`	**Step using an action** (actions/upload-artifact@v4): stores reports/ (HTML, JUnit, traces, videos) on the run page for download. @vN pins a major version for reproducible builds.

Line	Code	Syntax & why
145	`        if: always()`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always()*.
146	`        with: { name: non-functional-reports, path: reports/, retention-days: 14 }`	**with:** Inputs for the action. Value: *{ name: non-functional-reports, path: reports/, retention-days: 14 }*.
148	`  # ---------- consolidated report: traceability vs the QA workbook ----------`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
149	`  report:`	**report:** Named block *report* (a job, input or section); the indented keys below configure it.
150	`    needs: [smoke, regression, non-functional]`	**needs:** Job dependencies: this job waits for, and only runs after success of, the listed jobs (gates). Value: *[smoke, regression, non-functional]*.
151	`    if: always() && needs.smoke.result != 'skipped'`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always() && needs.smoke.result != 'skipped'*.
152	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
153	`    steps:`	**steps:** Ordered commands of the job.
154	`      - uses: actions/checkout@v4`	**Step using an action** (actions/checkout@v4): clones the repository onto the runner. @vN pins a major version for reproducible builds.
155	`      - uses: actions/download-artifact@v4`	**Step using an action** (actions/download-artifact@v4): downloads every job's artifacts into one folder for the consolidated report. @vN pins a major version for reproducible builds.
156	`        with: { path: all-reports }`	**with:** Inputs for the action. Value: *{ path: all-reports }*.
157	`      - name: Publish traceability summary`	**Step** (list item '-'): its display name in the run log.
158	`        run: |`	**Step running a shell command**: shell command.
159	`          for f in all-reports/*/traceability.md; do`	**Shell line** of a multi-line run block: loops over every job's traceability report.
160	`            echo "## $(basename "$(dirname "$f")")" >> "$GITHUB_STEP_SUMMARY"`	**Shell line** of a multi-line run block: prints a message to the log. Flags: writing to $GITHUB_STEP_SUMMARY shows markdown on the run summary page.
161	`            sed -n '1,12p' "$f" \| tail -n +3 >> "$GITHUB_STEP_SUMMARY"`	**Shell line** of a multi-line run block: prints the summary table part of the report. Flags: -n N runs N parallel workers (pytest-xdist); writing to $GITHUB_STEP_SUMMARY shows markdown on the run summary page.
162	`            echo >> "$GITHUB_STEP_SUMMARY"`	**Shell line** of a multi-line run block: prints a message to the log. Flags: writing to $GITHUB_STEP_SUMMARY shows markdown on the run summary page.
163	`          done`	**Shell line** of a multi-line run block: end of the shell loop.
164	`          python ci/quality_gate.py all-reports/*/junit.xml --name "Overall" --min-pass-rate 0 || true`	**Shell line** of a multi-line run block: applies the gate thresholds to junit.xml; non-zero exit fails the job. Flags: --min-pass-rate is the gate threshold; '|| true' keeps the step green; it is informational.
165	`      - uses: actions/upload-artifact@v4`	**Step using an action** (actions/upload-artifact@v4): stores reports/ (HTML, JUnit, traces, videos) on the run page for download. @vN pins a major version for reproducible builds.
166	`        with: { name: qa-report-bundle, path: all-reports/, retention-days: 30 }`	**with:** Inputs for the action. Value: *{ name: qa-report-bundle, path: all-reports/, retention-days: 30 }*.
168	`  # ---------- final verdict: the ONLY check you mark "required" in branch protection ----------`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
169	`  quality-gate:`	**quality-gate:** Named block *quality-gate* (a job, input or section); the indented keys below configure it.

Line	Code	Syntax & why
170	`    needs: [guard, lint, smoke, regression, non-functional, report]`	**needs:** Job dependencies: this job waits for, and only runs after success of, the listed jobs (gates). Value: *[guard, lint, smoke, regression, non-functional, report]*.
171	`    if: always() && needs.guard.result == 'success'`	**if:** Condition expression; always() lets a job run even when earlier jobs failed (reporting). Value: *always() && needs.guard.result == 'success'*.
172	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
173	`    steps:`	**steps:** Ordered commands of the job.
174	`      - name: Fail if any upstream gate failed or was cancelled`	**Step** (list item '-'): its display name in the run log.
175	`        run: |`	**Step running a shell command**: shell command.
176	`          echo '${{ toJSON(needs) }}'`	**Shell line** of a multi-line run block: prints a message to the log. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
177	`          if [ "${{ contains(needs.*.result, 'failure') }}" = "true" ] \|\| [ "${{ contains(needs.*.result, 'cancelled') }}" = "true" ]; then`	**Shell line** of a multi-line run block: shell condition: fail when any upstream gate failed or was cancelled. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).
178	`            echo "Quality gate FAILED"; exit 1`	**Shell line** of a multi-line run block: prints a message to the log.
179	`          fi`	**Shell line** of a multi-line run block: end of the shell if.
180	`          echo "All quality gates passed"`	**Shell line** of a multi-line run block: prints a message to the log.

.github/workflows/discover.yml

Manual/auto workflow that runs ci/discover.py on a GitHub runner (which can reach the site) and prints the snapshots in the job log.

Line	Code	Syntax & why
1	`name: Site discovery (selector maintenance)`	**name:** Display name of the workflow/step/job in the GitHub UI. Value: *Site discovery (selector maintenance)*.
3	`# Manual tool: dumps ARIA snapshots / links / headers of the live site into the job log.`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
4	`# Use it when the UI changes and page-object selectors need re-deriving.`	**YAML comment**: documentation for the reader (ignored by GitHub Actions).
5	`on:`	**on:** Events that start the workflow.
6	`  workflow_dispatch:`	**workflow_dispatch:** Allows a manual run from the Actions tab with inputs.
7	`    inputs:`	**inputs:** Parameters a person can set for a manual run.
8	`      base_url:`	**base_url:** Named block *base_url* (a job, input or section); the indented keys below configure it.
9	`        description: "Environment to inspect"`	**description:** Help text shown next to the input. Value: *"Environment to inspect"*.
10	`        default: "https://hotel-plaza-preview.vercel.app"`	**default:** Value used when the input is left unchanged. Value: *"https://hotel-plaza-preview.vercel.app"*.
11	`  push:`	**push:** Run on pushes to the listed branches.
12	`    paths: ["ci/discover.py"]`	**paths:** Only run when these files change. Value: *["ci/discover.py"]*.
14	`permissions:`	**permissions:** GITHUB_TOKEN scopes; read-only = least privilege.
15	`  contents: read`	**contents:** Repository contents permission. Value: *read*.

Line	Code	Syntax & why
17	`jobs:`	**jobs:** The pipeline stages; jobs run in parallel unless linked with needs.
18	`  discover:`	**discover:** Named block *discover* (a job, input or section); the indented keys below configure it.
19	`    runs-on: ubuntu-latest`	**runs-on:** Runner image (fresh Ubuntu VM per job). Value: *ubuntu-latest*.
20	`    timeout-minutes: 15`	**timeout-minutes:** Hard limit so a hung browser cannot block the pipeline. Value: *15*.
21	`    steps:`	**steps:** Ordered commands of the job.
22	`      - uses: actions/checkout@v4`	**Step using an action** (actions/checkout@v4): clones the repository onto the runner. @vN pins a major version for reproducible builds.
23	`      - uses: actions/setup-python@v5`	**Step using an action** (actions/setup-python@v5): installs the requested Python version (with pip cache). @vN pins a major version for reproducible builds.
24	`        with: { python-version: "3.12", cache: pip }`	**with:** Inputs for the action. Value: *{ python-version: "3.12", cache: pip }*.
25	`      - run: pip install -r requirements.txt`	**Step running a shell command**: installs Python dependencies listed in requirements.txt.
26	`      - run: playwright install --with-deps chromium`	**Step running a shell command**: downloads the browser build (--with-deps adds the OS libraries it needs).
27	`      - run: python ci/discover.py "${{ inputs.base_url || 'https://hotel-plaza-preview.vercel.app' }}"`	**Step running a shell command**: prints ARIA snapshots of the live site for selector maintenance. **${{ }}** is a GitHub Actions expression evaluated before the step runs (secrets.*, vars.*, inputs.*, matrix.*, github.event.*; '||' picks the first non-empty value).

docs/test_cases.csv

Export of the workbook's Test Cases sheet (id, module, title, type, priority, linked bug, expected result, status). It is the source of truth for the traceability check.

Data file (not code). Columns: Test Case ID, Module, Title, Type, Priority, Linked Bug, Expected Result, Status, Automation Candidate, Scope Ref. Each row is one workbook test case; ci/check_traceability.py reads it with csv.DictReader. The full mapping of these cases to tests is in Appendix B.

Appendix A - Marker reference

Marker	Meaning and why
`smoke`	critical-path test: runs in the Smoke gate, which must pass 100% before anything else runs.
`tc`	links the test to workbook test case(s) - drives traceability reports and the coverage check in CI.
`defect`	the test asserts CORRECT behaviour that a known open bug breaks. The traceability plugin turns it into a non-strict xfail: the pipeline stays green while the bug is open, and an XPASS flags 'looks fixed - retest'.
`live_write`	creates real records (account/booking/message). Skipped unless ALLOW_LIVE_WRITES=true, mirroring 'Blocked' in the workbook.
`needs_account`	needs TEST_EMAIL/TEST_PASSWORD of an existing customer; skipped when the secrets are absent.
`parametrize`	data-driven: one test per value.
`perf`	performance-budget test (filter with -m perf).
`a11y`	accessibility test (filter with -m `a11y`).
`security`	security test (filter with -m security).
`responsive`	layout/viewport test.
`mobile`	runs in a mobile viewport.
`api`	HTTP-level check without the browser UI.
`seo`	search-engine/metadata check.
`skipif`	conditional skip.
`functional`	auto-applied to tests/functional.
`non_functional`	auto-applied to tests/non_functional.

Appendix B - Workbook test case to test mapping

TC	Workbook title	Automated test(s)
TC-001	Room card shows image for every room type	`tests/functional/test_room_details.py::test_room_detail_has_photos` `tests/functional/test_rooms.py::test_every_room_card_shows_a_photo_with_alt`
TC-002	Fallback shown when a room image fails to load	`tests/functional/test_rooms.py::test_layout_survives_blocked_images`
TC-003	Room name displays without duplicated word	`tests/functional/test_rooms.py::test_room_names_have_no_duplicated_word` `tests/functional/test_rooms.py::test_room_names_have_no_redundant_room_suffix`
TC-004	Heading count equals number of distinct room types shown	`tests/functional/test_rooms.py::test_rooms_list_shows_results` `tests/functional/test_rooms.py::test_heading_count_equals_room_types_shown`
TC-005	No duplicate identical room cards	`tests/functional/test_home.py::test_free_for_your_dates_has_no_duplicate_cards` `tests/functional/test_rooms.py::test_no_duplicate_identical_room_cards`
TC-006	"Rooms left" label reflects real inventory	`tests/functional/test_rooms.py::test_rooms_left_label_is_a_positive_inventory_count`
TC-007	2-adult search excludes rooms with max 1 adult	`tests/functional/test_rooms.py::test_guest_search_hides_rooms_below_capacity`
TC-008	Boundary: guests equal to and above room max capacity	`tests/functional/test_rooms.py::test_guest_search_hides_rooms_below_capacity` `tests/functional/test_rooms.py::test_room_at_exact_capacity_is_listed`
TC-009	Capacity text uses correct singular/plural	`tests/functional/test_rooms.py::test_capacity_text_singular_and_plural` `tests/functional/test_rooms.py::test_mobile_filter_button_singular`
TC-010	Footer shows phone, email and address for each branch and head office	`tests/functional/test_navigation.py::test_footer_head_office_has_address` `tests/functional/test_navigation.py::test_footer_branch_contacts_have_phone_and_email`
TC-011	Contact info panel shows address, hours, phone, email and a map for every branch tab	`tests/functional/test_about_contact.py::test_branch_tab_shows_address_and_map` `tests/functional/test_about_contact.py::test_branch_tab_shows_hours_phone_and_email`
TC-012	Contact form submits with valid data	`tests/functional/test_about_contact.py::test_contact_form_submits_valid_message`

TC-013	Select room opens booking step with correct room and dates	`tests/functional/test_booking_flow.py::test_select_room_opens_booking_step`
		`tests/functional/test_booking_flow.py::test_reach_guest_details_step`
		`tests/functional/test_room_details.py::test_dates_are_carried_over`
		`tests/functional/test_room_details.py::test_back_to_rooms_via_nav`

TC-014	Selecting a room then browser back preserves search state	`tests/functional/test_booking_flow.py::test_change_room_or_dates_keeps_dates`
		`tests/functional/test_booking_flow.py::test_back_to_room_returns_to_detail_page`
		`tests/functional/test_booking_flow.py::test_browser_back_from_room_preserves_search_state`

TC-015	Price slider filters list and updates count	`tests/functional/test_rooms.py::test_price_slider_filters_and_updates_count` `tests/functional/test_rooms.py::test_clear_filters_control_restores_all_rooms`

TC-016	Homepage sections display their content	`tests/functional/test_home.py::test_locations_section_lists_every_branch`
		`tests/functional/test_home.py::test_gallery_section_shows_images`
		`tests/functional/test_home.py::test_amenities_section_lists_amenities`
		`tests/functional/test_home.py::test_free_for_your_dates_shows_room_cards`
		`tests/functional/test_navigation.py::test_home_page_loads`

TC	Workbook title	Automated test(s)

TC-017	Unauthenticated access to /staff is blocked	`tests/functional/test_navigation.py::test_footer_does_not_advertise_staff_console`
		`tests/non_functional/test_security.py::test_staff_console_not_reachable_anonymously`
		`tests/non_functional/test_security.py::test_auth_me_without_session`

TC-018	Date format is unambiguous	`tests/functional/test_rooms.py::test_default_dates_are_today_wat_and_two_nights` `tests/functional/test_rooms.py::test_date_fields_are_native_date_pickers`
TC-019	Footer branch list is complete on first render and matches Contact tabs	`tests/functional/test_navigation.py::test_footer_branch_list_complete_and_matches_contact_tabs`
TC-020	Search with past check-in date is rejected	`tests/functional/test_rooms.py::test_past_check_in_is_blocked_by_picker` `tests/functional/test_rooms.py::test_typed_past_check_in_is_rejected`
TC-021	Check-out before or equal to check-in is rejected	`tests/functional/test_rooms.py::test_check_out_picker_min_is_after_check_in` `tests/functional/test_rooms.py::test_check_out_on_or_before_check_in_is_rejected`
TC-022	Search returns rooms for chosen branch only	`tests/functional/test_rooms.py::test_search_returns_rooms_for_chosen_branch_only` `tests/functional/test_rooms.py::test_lagos_lists_only_lagos_rooms`
TC-023	No availability message when all rooms booked	`tests/functional/test_rooms.py::test_no_availability_message_when_nothing_fits`
TC-024	Guest count minimum and maximum limits	`tests/functional/test_rooms.py::test_guest_selector_limits` `tests/functional/test_rooms.py::test_guest_search_hides_rooms_below_capacity`
TC-025	Nights and total price computed correctly	`tests/functional/test_room_details.py::test_total_equals_nights_times_rate_plus_listed_charges` `tests/functional/test_rooms.py::test_card_total_equals_nightly_rate_times_nights`
TC-026	Room type filter	`tests/functional/test_rooms.py::test_room_type_filter_and_untick_restores`
TC-027	Bed type filter	`tests/functional/test_rooms.py::test_bed_type_filter`
TC-028	Multiple amenities filter uses AND logic	`tests/functional/test_rooms.py::test_amenities_filter_uses_and_logic`
TC-029	Combined filters with no matches show empty state	`tests/functional/test_rooms.py::test_incompatible_filters_show_empty_state`
TC-030	Amenities list scrolls and all items are selectable	`tests/functional/test_rooms.py::test_amenities_list_scrolls_and_last_item_selectable`

TC-031	View details shows full room info and images	`tests/functional/test_room_details.py::test_view_details_shows_full_room_info`
		`tests/functional/test_room_details.py::test_branch_room_detail_sections_populated`
		`tests/functional/test_room_details.py::test_room_detail_has_photos`
		`tests/functional/test_rooms.py::test_view_details_opens_room_page_with_dates`

TC-032	Mandatory guest fields validated	`tests/functional/test_booking_flow.py::test_mandatory_guest_fields_validated`
TC-033	Email and phone format validation	`tests/functional/test_booking_flow.py::test_email_and_phone_format_validation` `tests/functional/test_booking_flow.py::test_nigerian_phone_formats_accepted`
TC-034	Successful payment confirms booking	`tests/functional/test_booking_flow.py::test_online_payment_flows`
TC-035	Declined card shows error and keeps room held	`tests/functional/test_booking_flow.py::test_online_payment_flows`
TC-036	Gateway timeout or closed payment window	`tests/functional/test_booking_flow.py::test_online_payment_flows`
TC-037	Last room cannot be double booked	`tests/functional/test_booking_flow.py::test_last_room_cannot_be_double_booked`
TC-038	Room hold expires after session timeout	`tests/functional/test_booking_flow.py::test_room_hold_expires`

TC	Workbook title	Automated test(s)
TC-039	Confirmation email and reference are correct	`tests/functional/test_booking_flow.py::test_confirm_booking_creates_reservation`

TC-040	Sign in with valid and invalid credentials	`tests/functional/test_account.py::test_valid_sign_in_shows_customer_in_header`
		`tests/functional/test_auth.py::test_invalid_credentials_show_generic_error`
		`tests/functional/test_auth.py::test_wrong_password_for_real_account_is_generic`

TC-041	Manage a booking by reference	`tests/functional/test_account.py::test_manage_booking_lookup_form`
		`tests/functional/test_account.py::test_manage_booking_requires_fields`
		`tests/functional/test_account.py::test_manage_booking_wrong_reference_shows_not_found`
		`tests/functional/test_account.py::test_footer_manage_a_booking_opens_lookup`

TC-042	Cancel booking follows the cancellation policy	`tests/functional/test_account.py::test_cancel_booking_follows_policy`

TC-043	Header and footer links resolve with no 404	`tests/functional/test_navigation.py::test_header_navigation`
		`tests/functional/test_navigation.py::test_footer_links_resolve_without_404`
		`tests/functional/test_navigation.py::test_logo_returns_home`
		`tests/functional/test_navigation.py::test_header_and_footer_present_on_every_page`

TC-044	Header search returns relevant results	`tests/functional/test_navigation.py::test_search_box_starts_empty`
		`tests/functional/test_navigation.py::test_header_search_opens_results_page`
		`tests/functional/test_navigation.py::test_search_query_lists_only_matching_rooms`
		`tests/functional/test_navigation.py::test_header_search_from_home_finds_default_branch_room`
		`tests/functional/test_navigation.py::test_header_search_no_match_shows_empty_state`

TC-045	Contact form mandatory validation and branch routing	`tests/functional/test_about_contact.py::test_contact_form_mandatory_validation`
		`tests/functional/test_about_contact.py::test_routing_heading_follows_selected_tab`
		`tests/functional/test_about_contact.py::test_contact_privacy_link_works`

TC-046	Layout at 375px width (mobile)	`tests/non_functional/test_responsive.py::test_no_horizontal_scroll`
		`tests/non_functional/test_responsive.py::test_header_visible_on_all_viewports`
		`tests/non_functional/test_responsive.py::test_branches_cards_not_clipped_at_375`

TC-047	Layout at 768px width (tablet)	`tests/non_functional/test_responsive.py::test_no_horizontal_scroll` `tests/non_functional/test_responsive.py::test_header_visible_on_all_viewports`

TC-048	Layout at 1920px width (large desktop)	`tests/non_functional/test_responsive.py::test_no_horizontal_scroll`
		`tests/non_functional/test_responsive.py::test_header_visible_on_all_viewports`
		`tests/non_functional/test_responsive.py::test_content_centred_with_readable_width_at_1920`

TC-049	Core booking flow in Chrome, Firefox, Edge, Safari	`tests/functional/test_booking_flow.py::test_select_room_opens_booking_step`
		`tests/functional/test_rooms.py::test_view_details_opens_room_page_with_dates`
		`tests/non_functional/test_responsive.py::test_core_flow_on_emulated_phones`

TC-050	Mobile browsers: Chrome Android and Safari iOS	`tests/non_functional/test_responsive.py::test_core_flow_on_emulated_phones`

TC-051	Page load on Slow 4G (Nigeria network profile)	`tests/non_functional/test_performance.py::test_navigation_timing_within_budget`
		`tests/non_functional/test_performance.py::test_largest_contentful_paint_within_budget`
		`tests/non_functional/test_performance.py::test_lcp_on_slow_4g`
		`tests/non_functional/test_performance.py::test_offscreen_images_are_lazy_loaded`

TC-052	Skeleton loaders always resolve	`tests/functional/test_room_details.py::test_invalid_room_id_shows_not_found` `tests/non_functional/test_performance.py::test_skeleton_loaders_always_resolve`

TC	Workbook title	Automated test(s)
TC-053	Cumulative layout shift stays low	`tests/non_functional/test_performance.py::test_cumulative_layout_shift`
TC-054	Rooms search response time under concurrent load	`tests/non_functional/test_performance.py::test_rooms_search_p95_under_concurrent_load`

TC-055	Keyboard navigation of search, filters and Select room	`tests/functional/test_auth.py::test_focus_returns_to_trigger_after_close`
		`tests/non_functional/test_accessibility.py::test_sign_in_dialog_is_keyboard_operable`
		`tests/non_functional/test_accessibility.py::test_keyboard_reaches_search_filters_and_select_room`
		`tests/non_functional/test_accessibility.py::test_focus_is_visible_on_search_controls`

TC-056	Alt text, labels and colour contrast	`tests/functional/test_rooms.py::test_every_room_card_shows_a_photo_with_alt`
		`tests/non_functional/test_accessibility.py::test_no_critical_or_serious_axe_violations`
		`tests/non_functional/test_accessibility.py::test_every_form_control_has_an_accessible_name`
		`tests/non_functional/test_accessibility.py::test_room_detail_date_inputs_are_labelled`

TC-057	Contact form resists script injection	`tests/functional/test_about_contact.py::test_contact_form_script_injection_not_executed`

TC-058	All pages served over HTTPS with secure headers	`tests/non_functional/test_security.py::test_http_redirects_to_https`
		`tests/non_functional/test_security.py::test_hsts_header_present`
		`tests/non_functional/test_security.py::test_security_headers_present`
		`tests/non_functional/test_security.py::test_no_mixed_content`
		`tests/non_functional/test_security.py::test_no_technology_disclosure_headers`

TC-059	Booking and guest data not exposed in URL or API	`tests/functional/test_booking_flow.py::test_script_in_guest_name_is_rendered_as_text` `tests/functional/test_booking_flow.py::test_no_guest_pii_in_urls`
TC-060	Unknown URL shows a branded 404 page	`tests/functional/test_navigation.py::test_unknown_url_shows_branded_404` `tests/functional/test_room_details.py::test_invalid_room_id_returns_http_404`
TC-061	Page titles and meta descriptions per page	`tests/functional/test_navigation.py::test_page_titles_are_unique` `tests/functional/test_navigation.py::test_meta_descriptions_are_unique`
TC-062	Room detail blocks check-out on or before check-in	`tests/functional/test_room_details.py::test_check_out_picker_min_is_day_after_check_in` `tests/functional/test_room_details.py::test_continue_blocked_when_check_out_not_after_check_in`
TC-063	Room detail blocks past dates and stays over 90 nights	`tests/functional/test_room_details.py::test_stay_limit_is_enforced_by_picker` `tests/functional/test_room_details.py::test_past_dates_from_url_are_rejected`

TC-064	Search bar date validation (past, inverted, same day)	`tests/functional/test_rooms.py::test_past_check_in_is_blocked_by_picker`
		`tests/functional/test_rooms.py::test_typed_past_check_in_is_rejected`
		`tests/functional/test_rooms.py::test_check_out_picker_min_is_after_check_in`
		`tests/functional/test_rooms.py::test_check_out_on_or_before_check_in_is_rejected`

TC-065	Default dates follow the guest's time zone (WAT)	`tests/functional/test_rooms.py::test_default_dates_are_today_wat_and_two_nights`

TC-066	Total and breakdown identical on room detail, guest details and Review	`tests/functional/test_booking_flow.py::test_review_shows_stay_guest_and_price`
		`tests/functional/test_booking_flow.py::test_total_identical_on_detail_details_and_review`
		`tests/functional/test_booking_flow.py::test_branch_room_review_shows_a_price`
		`tests/functional/test_booking_flow.py::test_branch_room_total_is_final_not_estimate`

TC-067	Tax and fee lines are guest-facing, named clearly and calculated correctly	`tests/functional/test_booking_flow.py::test_tax_lines_are_guest_facing` `tests/functional/test_booking_flow.py::test_tax_amounts_match_their_rates`
TC-068	5-night stay applies 'fifth night on us'	`tests/functional/test_room_details.py::test_fifth_night_free_offer_applied`

TC	Workbook title	Automated test(s)

TC-069	Privacy, Terms and Cancellation policy pages load	`tests/functional/test_booking_flow.py::test_consent_privacy_link_works`
		`tests/functional/test_navigation.py::test_policy_pages_load`
		`tests/functional/test_navigation.py::test_policy_pages_publish_full_policy`

TC-070	No failing API calls or console errors on page load	`tests/non_functional/test_performance.py::test_no_console_errors_or_failed_requests`
TC-071	Branch cards show room types, lowest price, contacts, directions, offers and photo	`tests/functional/test_branches.py::test_every_branch_has_a_card_with_rooms_and_price` `tests/functional/test_branches.py::test_branch_cards_show_contacts_hours_and_photo`
TC-072	'See rooms at <branch>' opens Rooms filtered to that branch	`tests/functional/test_branches.py::test_see_rooms_opens_rooms_for_that_branch`
TC-073	'Contact this branch' opens Contact with that branch tab selected	`tests/functional/test_branches.py::test_contact_this_branch_selects_that_branch`
TC-074	Branch names and addresses are valid and consistent everywhere	`tests/functional/test_branches.py::test_branch_names_and_addresses_valid` `tests/functional/test_branches.py::test_branch_names_consistent_across_pages`
TC-075	Lagos branch name and address follow the standard format	`tests/functional/test_branches.py::test_branch_names_and_addresses_valid` `tests/functional/test_branches.py::test_lagos_branch_naming`
TC-076	Lagos branch lists only Lagos rooms and no test data	`tests/functional/test_rooms.py::test_lagos_lists_only_lagos_rooms` `tests/functional/test_rooms.py::test_no_test_records_on_public_listing`
TC-077	No test or placeholder records visible on any public page	`tests/functional/test_rooms.py::test_no_test_records_on_public_listing` `tests/non_functional/test_security.py::test_public_api_exposes_no_test_contacts`
TC-078	Price slider range matches real min and max room prices	`tests/functional/test_rooms.py::test_price_slider_range_matches_room_prices`
TC-079	Filter options are built from the branch inventory	`tests/functional/test_rooms.py::test_filter_options_come_from_branch_inventory`
TC-080	Mobile filter drawer opens, applies and closes	`tests/functional/test_rooms.py::test_mobile_filter_drawer` `tests/functional/test_rooms.py::test_mobile_filter_button_singular`
TC-081	Filters survive reload and back navigation	`tests/functional/test_rooms.py::test_filters_survive_reload`
TC-082	'Free for your dates' shows the selected branch's rooms and prices	`tests/functional/test_home.py::test_free_for_your_dates_matches_selected_branch` `tests/functional/test_home.py::test_free_for_your_dates_has_no_duplicate_cards`
TC-083	Hero section: title, subtitle, image and Explore rooms CTA	`tests/functional/test_home.py::test_hero_copy_image_and_cta` `tests/functional/test_navigation.py::test_home_page_loads`
TC-084	Home search widget submits to Rooms with branch, dates and guests	`tests/functional/test_home.py::test_home_search_submits_branch_dates_and_guests`
TC-085	Amenities section lists the hotel's amenities	`tests/functional/test_home.py::test_amenities_section_lists_amenities`
TC-086	Mobile hamburger menu opens, lists all links and closes	`tests/functional/test_navigation.py::test_mobile_hamburger_menu` `tests/functional/test_navigation.py::test_mobile_menu_link_navigates`
TC-087	Footer legal line shows company name, RC number and current year	`tests/functional/test_navigation.py::test_footer_legal_line`
TC-088	Room detail shows size, in-room amenities, house rules and cancellation summary	`tests/functional/test_room_details.py::test_view_details_shows_full_room_info` `tests/functional/test_room_details.py::test_branch_room_detail_sections_populated`

TC	Workbook title	Automated test(s)
TC-089	Adults/children steppers respect room capacity	`tests/functional/test_room_details.py::test_steppers_respect_capacity`

TC-090	Other rooms block and breadcrumb show the branch and link correctly	`tests/functional/test_room_details.py::test_branch_room_breadcrumb_and_other_rooms_name_the_branch`
		`tests/functional/test_room_details.py::test_breadcrumb_includes_branch`
		`tests/functional/test_room_details.py::test_other_room_link_opens_that_room`
		`tests/functional/test_room_details.py::test_other_rooms_has_no_duplicates`

TC-091	Invalid room ID shows a not-found state	`tests/functional/test_room_details.py::test_invalid_room_id_shows_not_found` `tests/functional/test_room_details.py::test_invalid_room_id_returns_http_404`
TC-092	Search guest count carries to room detail and booking	`tests/functional/test_room_details.py::test_search_guest_count_carries_to_detail`
TC-093	Booking step indicator shows the current step	`tests/functional/test_booking_flow.py::test_step_indicator`
TC-094	Consent checkbox is required and links to a working privacy policy	`tests/functional/test_booking_flow.py::test_consent_is_required` `tests/functional/test_booking_flow.py::test_consent_privacy_link_works`

TC-095	Review step shows stay, guest and price, with working edit links	`tests/functional/test_booking_flow.py::test_review_shows_stay_guest_and_price`
		`tests/functional/test_booking_flow.py::test_branch_room_review_shows_a_price`
		`tests/functional/test_booking_flow.py::test_edit_details_keeps_everything_incl_consent`
		`tests/functional/test_booking_flow.py::test_back_to_guest_details_keeps_contact_data`
		`tests/functional/test_booking_flow.py::test_change_room_or_dates_keeps_dates`

TC-096	Confirm booking (pay at hotel) creates a reservation and reference	`tests/functional/test_booking_flow.py::test_confirm_booking_creates_reservation`
TC-097	'Pay at the hotel / unpaid' messaging is consistent and clear	`tests/functional/test_booking_flow.py::test_unpaid_messaging_on_review` `tests/functional/test_room_details.py::test_unpaid_pay_at_hotel_notice`
TC-098	Server rechecks dates, capacity and room before quoting	`tests/non_functional/test_security.py::test_server_rejects_invalid_quotes` `tests/non_functional/test_security.py::test_quote_validation_errors_are_4xx_and_generic`
TC-099	Direct /booking without a selected room redirects safely	`tests/functional/test_booking_flow.py::test_direct_booking_url_without_room_redirects`

TC-100	Sign-in modal: role selection (Customer / Agent) and close	`tests/functional/test_auth.py::test_sign_in_dialog_opens_with_role_choice`
		`tests/functional/test_auth.py::test_customer_role_then_next_opens_sign_in_form`
		`tests/functional/test_auth.py::test_dialog_closes`
		`tests/functional/test_auth.py::test_focus_returns_to_trigger_after_close`
		`tests/non_functional/test_accessibility.py::test_sign_in_dialog_is_keyboard_operable`

TC-101	Sign-in mandatory field validation	`tests/functional/test_auth.py::test_sign_in_mandatory_field_validation` `tests/functional/test_auth.py::test_sign_in_identifier_format_checked_client_side`
TC-102	Brute-force protection on sign-in	`tests/functional/test_auth.py::test_brute_force_is_throttled`

TC-103	Create an account: validation and success	`tests/functional/test_auth.py::test_registration_form_fields`
		`tests/functional/test_auth.py::test_empty_registration_is_blocked_with_field_errors`
		`tests/functional/test_auth.py::test_empty_phone_error_is_human_readable`
		`tests/functional/test_auth.py::test_password_visibility_toggle`
		`tests/functional/test_auth.py::test_short_password_rejected`
		`tests/functional/test_auth.py::test_mismatched_passwords_block_registration`
		`tests/functional/test_auth.py::test_invalid_email_blocks_registration`
		`tests/functional/test_auth.py::test_register_new_customer`

TC	Workbook title	Automated test(s)
TC-104	Forgot password flow	`tests/functional/test_auth.py::test_forgot_password_form` `tests/functional/test_auth.py::test_forgot_password_unknown_email_is_generic`

TC-105	Account page for a signed-in customer shows bookings and profile	`tests/functional/test_account.py::test_signed_out_account_page_prompts_sign_in`
		`tests/functional/test_account.py::test_customer_account_copy_has_no_agent_wording`
		`tests/functional/test_account.py::test_signed_in_account_page_shows_profile`

TC-106	Agent role: book for clients and see commissions	`tests/functional/test_account.py::test_agent_role_features`
TC-107	Sign out clears the session	`tests/functional/test_account.py::test_sign_out_clears_session` `tests/non_functional/test_security.py::test_auth_me_without_session`
TC-108	Branch choice persists across pages	`tests/functional/test_rooms.py::test_branch_choice_persists_across_pages`
TC-109	Gallery category filter	`tests/functional/test_gallery.py::test_gallery_loads` `tests/functional/test_gallery.py::test_category_filter_and_all_restores`
TC-110	Gallery location filter	`tests/functional/test_gallery.py::test_location_filter_shows_photos_or_message` `tests/functional/test_gallery.py::test_every_branch_has_photos`
TC-111	Photo lightbox opens, navigates and closes	`tests/functional/test_gallery.py::test_photo_lightbox_opens_navigates_and_closes`
TC-112	Gallery copy and images are accurate	`tests/functional/test_gallery.py::test_gallery_intro_counts_match_branches` `tests/functional/test_gallery.py::test_no_duplicate_photos_and_room_photos_link_to_rates`
TC-113	About page shows heading, story and principles	`tests/functional/test_about_contact.py::test_about_page_heading_story_and_principles`
TC-114	'Look it up with your reference' opens a booking lookup	`tests/functional/test_account.py::test_manage_booking_lookup_form` `tests/functional/test_account.py::test_contact_reference_link_opens_lookup`

TC-115	robots.txt, sitemap.xml and Open Graph tags exist	`tests/non_functional/test_seo_content.py::test_robots_and_sitemap`
		`tests/non_functional/test_seo_content.py::test_robots_points_to_sitemap_and_hides_private_paths`
		`tests/non_functional/test_seo_content.py::test_sitemap_lists_public_pages`
		`tests/non_functional/test_seo_content.py::test_open_graph_tags`

TC-116	Branches page at 375px	`tests/non_functional/test_responsive.py::test_no_horizontal_scroll` `tests/non_functional/test_responsive.py::test_branches_cards_not_clipped_at_375`
TC-117	Touch targets at least 44px on mobile	`tests/non_functional/test_accessibility.py::test_touch_targets_at_least_44px_on_mobile`
TC-118	Security response headers present	`tests/non_functional/test_security.py::test_security_headers_present` `tests/non_functional/test_security.py::test_csp_blocks_framing`
TC-119	API errors do not leak internals; unknown API routes return JSON 404	`tests/non_functional/test_security.py::test_unknown_api_route_returns_json_404` `tests/non_functional/test_security.py::test_quote_validation_errors_are_4xx_and_generic`

TC-120	Guest name and identifier character validation	`tests/functional/test_auth.py::test_sign_in_identifier_format_checked_client_side`
		`tests/functional/test_booking_flow.py::test_guest_name_character_validation`
		`tests/functional/test_booking_flow.py::test_script_in_guest_name_is_rendered_as_text`

TC-121	Content and copy review (grammar, placeholders, test contacts)	`tests/non_functional/test_security.py::test_public_api_exposes_no_test_contacts` `tests/non_functional/test_seo_content.py::test_no_placeholder_or_typo_copy`
TC-122	Currency formatting and rounding (Naira)	`tests/functional/test_room_details.py::test_currency_format`

TC	Workbook title	Automated test(s)
TC-123	Screen-reader landmarks and headings on each page	`tests/functional/test_about_contact.py::test_about_page_heading_story_and_principles` `tests/non_functional/test_accessibility.py::test_one_meaningful_h1_and_landmarks`